基于预测融合与动态决策的最优微网购电及储能协同策略

摘要

微网购电策略受小区负载、光伏发电量、电价变化及预测信息等因素共同影响，合理协调外网购电与储能充放电是降低运行成本的关键。本文以满足负荷需求和储能安全约束为前提，以购电成本最小化为核心目标，构建由**日前计划、日内实时修正**与**动态调整组成的递进式优化框架**，系统研究不同信息条件下微网的购电与储能协同调度问题。

问题一：针对问题一，本文以十分钟级计划购电量及分钟级储能充放电量为决策变量，建立以全天购电费用最小为目标函数的线性规划模型。考虑居民负载与光伏功率在相邻采样点间连续变化，**通过线性插值得到分钟级供需电量**，并设置分钟电量平衡、储能容量、最大充放电功率、充放电效率以及日初与日末储电量一致等约束，从而确定全天的购电及储能调度策略。采用线性规划方法求解，最终得到**全天购电量为 59566.62kWh，购电费用为 35073.87 元**。

问题二：问题二在问题一的基础上引入负载与光伏逐日波动、日前预测误差及 5 倍电价紧急购电等不确定因素。本文建立**基于 7 日周期预测与风险裕度的日前—日内协同调度模型**。首先，利用历史加权基准与七日前同刻偏差预测居民负载，并采用近 7 日同刻均值预测光伏发电；进一步根据历史净负载预测误差的 80% 分位数构造风险裕度。日前阶段通过三阶段线性规划，在 1.05 倍基准成本预算内优先控制日末储能偏差并进一步降低购电费用；最后，日内依据实际供需逐分钟调整储能动作，缺电时依次减充、增放并在必要时紧急购电，富余时则减放、增充，实现经济性与供电可靠性的协调。最终得到**全年计划购电费用 13,199,845.17 元、紧急购电费用 923,041.90 元、总费用 14,122,887.07 元**。

问题三：问题三在问题二基础上引入多时刻光伏预报与调整购电费用规则。本文建立**基于日前初始计划–日内动态路径选择的协同调度模型**。光伏预测采用“近端直接采用最新版预报、远端动态融合多版预报与历史信息”的分段预测策略，其中未来 1～6 h 采用最新原始预报，未来 7～24 h 采用基于历史预测误差动态学习权重的融合结果。每日 0:00 以日初、日末储电量一致为约束制定初始最低费用购电计划；6:00、12:00、18:00 分别构造保留路径与修改路径，综合考虑购电及调整现金费用与期末储能偏差，通过加权评价选择更优路径，并仅执行其中当天尚未发生的调整。最终得到**全年实际总费用 13,610,394.79 元，其中紧急购电费 809,155.62 元、调整净费用 444,516.88 元**。

问题四：问题四在问题二与问题三基础上引入电价波动机制。本文建立**在问题二、三调度框架的基础上引入电价预测与周阶梯储能机制**。当日电价采用前四周同星期、同一时段的历史均值预测，并根据周内价格变化设置分星期的日末储能参考水平，引导储能在低价日前蓄电、高价阶段逐步释放。**对于问题二对应模型**，每日 0:00 优先控制日末储能偏差并进一步最小化预计购电费用; **对于问题三对应模型**，则在 6:00、12:00 和 18:00 结合最新预测动态比较保留与调整方案。最终得到，对于问题二对应模型**全年实际总购电费用为 14,720,354.85 元，紧急购电成本 964,545.36 元**；对于问题三对应模型**全年实际总购电费用为 14,298,962.51 元，紧急购电成本 840,396.67 元**。

本文的创新点在于：一是利用历史误差分位数构造风险裕度，将预测偏差纳入日前供电准备计划；二是综合费用与储能偏差评价日内调整方案，兼顾调整代价与储能需求；三是通过周阶梯储能参考与实际状态的跨日传递，协调周内电量安排与逐日调度。

关键词：微网调度; 线性规划; 储能调度; 预测优化; 动态电价

一、问题背景与重述

1.1 问题背景

光伏等分布式能源的本地消纳与用电需求的协调，是微网运行调控的重要任务。本文研究的微网包含光伏发电、储能设备和小区负载，并通过与外部电网相连补充电能供给。光伏发电受太阳辐射影响具有明显的季节性和日内波动性[1]，储能设备能够在光伏电量富余或外网电价较低时充电，在供电不足或电价较高时放电，实现电能的跨时段转移，为协调供需和降低购电费用提供调节手段。然而，储能的调节能力受到容量、充放电功率及转换效率的限制，其经济性还受电价结构影响[2]，需要与外网购电统筹安排。

在实际运行中，小区负载与光伏发电随时间变化，提前制定的购电计划可能因预测偏差而出现电量不足或富余。同时，光伏预报的日内更新为修正计划提供了新的信息，但调整购电与紧急补购均可能产生额外费用；外网电价的实时波动又进一步增加了购电时机与储能安排的决策难度。因此，如何综合利用历史数据、光伏预报及电价信息，在满足小区负载需求和储能运行约束的前提下，借鉴滚动优化思想[3]，协调日前购电计划、日内调整与储能充放电，降低实际购电总费用，是本文研究的核心问题。

1.2 问题重述

本文以含光伏发电与储能设备、并与外部电网相连的小区微网为研究对象。依据题目给定的储能设备参数，以及附件提供的小区负载、光伏发电实际与预测功率、外网电价等数据，研究不同信息条件与交易规则下的购电及储能调度问题。在满足小区负载需求的前提下，综合考虑储能容量、充放电功率限制及充放电效率，以购电总费用最小化为目标，统筹安排外网购电与储能充放电，实现经济性与稳定性的平衡，并按各问题要求确定相应的计划调整策略。具体需要解决以下四个问题：

问题一：第一问要求在每天电价和小区负载相同、光伏发电功率一天的预测数据已知的条件下，建立每日 0:00 制定当日购电计划的优化模型。在满足小区负载需求、储能运行约束及 0:00 与 24:00 储电量相等的前提下，以全天购电费用最小化为目标，确定当日购电与储能充放电方案。最后按要求给出指定时段的购电量、充放电量、日初日末储电量及全天购电总量与费用，并将完整计划保存至指定文件。

问题二：在各日对应时段电价相同、小区负载与光伏发电功率随时间变化的条件下，引入紧急购电机制：当微网供电不足以满足负载需求时，需按交易时刻电价的 5 倍向外网紧急购电，其余购电费用仍按计划购电量计算。根据上述条件及附件 1 中的电价、附件 2 中的数据，建立每日 0:00 制定当日计划购电策略的数学模型，按规定格式给出表 3 所列日期的购电、储能充放电及紧急购电结果，并将完整购电策略保存至指定文件。

问题三：在问题二的基础上，新增每日指定时点发布的未来 24 小时整点光伏发电功率预报，允许根据更新后的预报调整购电计划。调整购电量低于计划购电量时，减少部分按交易时刻电价的 50% 计收违约费用；高于计划购电量时，增加部分按交易时刻电价的 1.5 倍计费，总购电费用包括计划购电费用、紧急购电费用及调整购电的相关费用。依据附件 1 中的电价、附件 2 中的负载与光伏发电数据及附件 3 中的预报数据，建立微网当日购电策略的数学模型，制定每日 0:00 的初始计划及其他预报时刻的调整策略。按规定格式给出表 3 所列日期的购电、储能充放电及紧急购电结果，将完整购电策略保存至指定文件，并分析是否有必要利用其他时刻的预报调整购电策略。

问题四：进一步考虑外网电价实时波动的情况，建立微网当日购电策略的数学模型。依据附件 2 中的小区负载与光伏发电数据、附件 3 中的光伏发电预报及附件 4 中的电价数据，在波动电价下

重新计算问题二和问题三，并在论文中给出相应的计算结果。按照附件 5 中的模板，将对应问题二、问题三的完整购电策略分别保存至指定文件。

二、问题分析

2.1 问题一的分析

问题一要求在电价与小区负载的日内变化规律每日相同、当日光伏发电功率预测数据已知的条件下，于每日 0:00 制定当日购电计划。由于当日负载、电价及光伏发电预测信息均已给定，因此该问题属于确定信息条件下的单日购电与储能协同调度问题。微网需要结合不同时段的电价与光伏预测出力，利用储能的跨时段调节能力合理安排外网购电，在满足负载需求及储能运行约束的前提下，尽可能降低全天购电费用。

原始负载及光伏功率数据以离散时刻的功率值给出，为刻画采样点之间的功率变化并统一功率与电量的时间尺度，本文考虑对相邻采样点进行线性插值，并以分钟为最小时间单位计算负载及光伏发电电量。在此基础上，考虑采用十分钟购电、区间内均匀供给与分钟级储能调节相结合的时间尺度设置，以刻画购电区间内部的供需变化，为后续建立购电与储能协同优化模型奠定基础。

2.2 问题二的分析

问题二在问题一的基础上进一步考虑负载与光伏发电的逐日波动。每日 0:00 制定购电计划时，无法预知当日真实负载与光伏出力，若实际供电不足，则需按对应时段电价的 5 倍进行紧急购电。因此，该问题由确定信息条件下的单日调度转变为同时包含预测误差与高额缺电成本的逐日动态调度问题。其核心在于如何充分利用历史数据提高日前预测的合理性与稳健性，并在计划购电成本与供电风险之间进行权衡。因此，需要在问题一确定性调度框架的基础上，利用历史信息进行日前预测，并通过日内实时修正应对预测偏差，从而形成兼顾购电经济性与供电可靠性的逐日调度思路。

2.3 问题三的分析

问题三在问题二的基础上进一步引入 0:00、6:00、12:00 和 18:00 多个时刻发布的未来 24 小时光伏功率预报，并允许根据更新后的预报调整既有购电计划。随着日内新预报不断到达，微网可以获得更接近实际运行状态的信息，但调整原有购电计划会产生额外的违约或追加购电成本，因此并非每次获得新预报后都应立即修改计划。该问题的核心由单纯应对预测误差，进一步转变为预报信息价值与计划调整成本之间的动态权衡：既需要利用更新后的预测提高调度方案与实际供需的匹配程度，又要避免频繁调整造成不必要的额外费用。

因此，需要在问题二日前计划与日内修正框架的基础上，引入多时刻预测更新与动态决策机制，此类滚动优化方法在微网调度中已有应用[4]，在每次新预报到达后综合比较维持原计划和调整计划的预期运行效果，据此判断是否有必要修改当天尚未执行的购电安排，从而实现预测信息利用与调整成本控制之间的协调。

2.4 问题四的分析

问题四在前述问题的基础上进一步考虑外网电价随日期和时段实时波动的情况，并要求在波动电价条件下重新求解问题二和问题三。此时，每日购电决策不仅受到负载与光伏预测误差的影响，还需要考虑未来电价变化带来的购电时机选择。由于储能具有跨时段甚至跨日转移电能的能力，不同

日期之间的购电与储能安排也因此产生更强的关联：过早释放储能可能错失后续高价时段的调节价值，而储能保留过多又可能增加当前购电成本。因此，本问的核心由单纯应对供需不确定性进一步扩展为供需预测、电价变化与储能跨时段配置之间的协调优化问题。

因此，需要在问题二、三既有调度框架的基础上进一步利用历史电价信息刻画价格变化规律，并结合储能的跨日调节能力重新安排购电与充放电计划；对于允许日内调整的情形，还需综合考虑最新预测信息、价格变化及调整成本，形成适应波动电价环境的动态调度策略。

2.5 整体思路分析

总体来看，四个问题呈现出由确定信息向动态不确定环境逐步拓展的递进关系。问题一在已知负载、电价及光伏预测的条件下建立基础购电与储能协同调度模型；问题二进一步考虑负载与光伏的逐日波动及预测误差，引入日前预测与日内修正；问题三加入日内多时刻光伏预报和购电调整机制，使调度由静态计划进一步转向动态决策；问题四则继续考虑外网电价波动，在前述框架下综合协调供需预测、电价变化与储能跨时段配置。由此，本文形成了从确定性优化、预测修正、动态调整到波动电价适应的逐层扩展建模体系。



图 1: 思路框架图

三、模型假设

1. 负载与光伏功率近似：假设附录表格中的小区负载和光伏发电功率均为每 10 分钟一次的瞬时检测值，且相邻检测点之间的功率呈线性变化。基于此，对相邻采样点进行线性插值，计算每分钟平均功率，并将该平均值作为对应一分钟内的恒定功率。

2. 时间离散：以一分钟为最小建模时间单位，按分钟计算供需电量平衡，不进一步刻画一分钟内部的功率变化。

3. 电价分段恒定：电价以十分钟为一个计价区间，同一区间内电价保持不变。

4. 购电与充放电均匀分布：每个十分钟区间内，外网购电量、储能充电量和放电量均匀分布，即对应功率在该区间内保持不变。

5. 单向购电：微网仅从外部电网购电，不向外部电网反向售电。

6. 储能效率与功率限制：储能充电效率和放电效率均为 90%，且保持不变。充电与放电各自输入端的功率均不得超过 5000 kW。

7. 允许同时充放电：允许储能设备在同一时间区间内同时充电和放电，二者分别满足对应功率限制。

四、符号说明

符号	说明	单位或取值
A_d	计划日末储能与 B_d 的绝对偏差辅助变量	kWh
B_d	当天日末安全储能参考水平，不是硬下限	kWh
C	全天从外部电网购电的总费用	元
C_d	当日计划购电费用	元
C_d^*	首末计划储能相等时的最低计划购电费用	元
$D_{d,i}$	当前日与历史日的环形日期距离	天
$E_{d,t}$	第 d 天第 t 个分钟区间的实际负载/光伏发电量	kWh
$\bar{E}_{d,t}$	用过去日期计算的负载/光伏加权基准电量	kWh
$\hat{E}_{d,t}$	当天 0:00 形成的分钟负载、光伏预测电量	kWh
H_d	全天紧急购电量	kWh
p_T	第 T 个十分钟区间内的外网购电单价	元/kWh
$r_{d,t}$	当日负载/光伏发电量相对当日历史基准的偏差	–
S_t	第 t 个分钟区间起始时刻储能设备内的电量	kWh
$W_{d,t}$	第 d 日第 t 个分钟区间内未被负载或储能消纳的电量	kWh
$X_{d,T}$	第 d 日第 T 个十分钟区间从外部电网购入的电量	kWh
Y_T	储能因放电而减少的电量	kWh
Z_T	储能因充电而增加的电量	kWh
δ_d	当天携带的固定每日回落量	kWh/天

五、数据分析与预处理

5.1 数据插值与分钟级电量换算

由于附件中的小区负载功率与光伏发电功率以 10 分钟为采样间隔，而后续调度模型采用 1 分钟时间尺度，为获得各分钟边界时刻的功率值并统一功率与电量的计量口径，本文对相邻采样点之间的功率进行线性插值。

设相邻采样时刻为 a 和 b，其中 $b-a=10$ 分钟，插值后的光伏发电功率 $P^{light}(u)$ 和小区负载功率 $P^{need}(u)$ 分别为：

$$P^{light}(u)=P^{light}(a)+\frac{u-a}{b-a}\left[P^{light}(b)-P^{light}(a)\right],\quad a\le u\le b. \tag{1}$$

$$P^{need}(u)=P^{need}(a)+\frac{u-a}{b-a}\left[P^{need}(b)-P^{need}(a)\right],\quad a\le u\le b. \tag{2}$$

其中，$P^{light}(u)$ 和 $P^{need}(u)$ 的单位均为 kW。

在线性插值假设下，第 t 个分钟区间 $[t,t+1)$ 内的功率随时间线性变化，因此该分钟的光伏发电量与负载用电量可表示为：

$$E_t^{light}=\frac{P^{light}(t)+P^{light}(t+1)}{120},\quad E_t^{need}=\frac{P^{need}(t)+P^{need}(t+1)}{120}. \tag{3}$$

其中，E_t^{light} 与 E_t^{need} 的单位均为 kWh。上述换算等价于以该分钟起、终点功率的平均值作为分钟平均功率，再乘以 $1/60$ h。由此，可将原始 10 min 功率采样数据转换为后续调度模型所需的分钟级电量序列。

5.2 居民用电周期性分析

对附件 2 中的小区负载数据进行周期性分析。选取每日 13:00 时刻的负载数据，分别计算其与滞后 1～14 日负载的 Pearson 相关系数，结果如图 2 所示。可以发现，滞后 7 日和 14 日的相关系数分别达到 0.979 和 0.975，显著高于其他滞后期，表明同一时刻的居民用电负载具有明显的 7 日周期特征。这一规律说明，相较于直接利用前一日负载信息，同星期位置的历史负载对当日用电水平具有更强的参考价值。因此，在后续负载预测中，本文以历史数据构造长期加权基准，并引入滞后 7 日同一时刻相对于其历史基准的偏差信息进行短期修正，以同时刻画负载的长期变化趋势与 7 日周期波动特征。

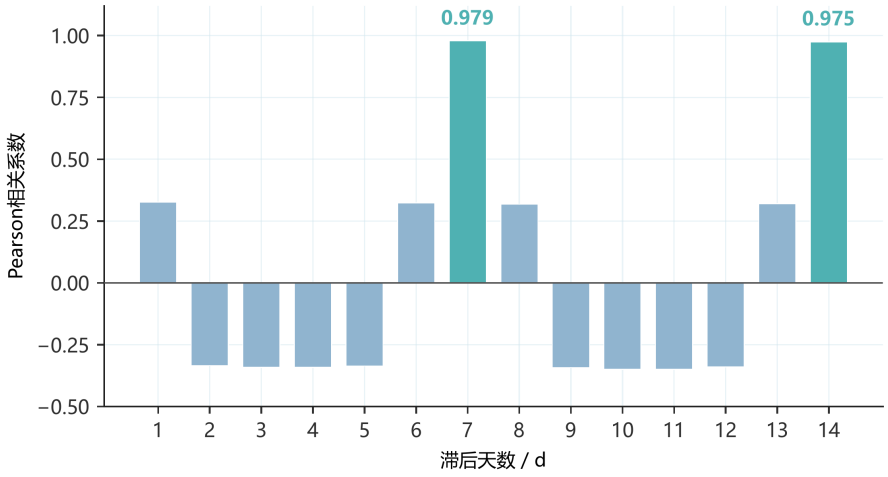


图 2: 居民用电周期性分析图

5.3 电价日内分析

对附件 1 中的分时电价进行可视化分析，可以发现外网电价具有明显的日内时序结构和峰谷差异。凌晨至清晨电价整体处于较低水平，早间开始明显上升；随后在中午附近出现阶段性回落，下午至晚间再次逐步抬升，并在晚间达到全天较高水平，22:00 后快速下降至低位。由图可见，全天存在多个相对低价购电窗口，而高、低价时段之间具有较明显的价格差异。

这一特征表明，储能设备具备较强的跨时段调节价值：在满足负载需求及储能运行约束的条件下，可在低价时段适当增加购电并储存电能，在后续高价时段通过储能放电减少外网购电，从而实现电能在不同价格时段之间的转移。因此，后续调度模型需要综合考虑电价的时间分布与储能状态变化，对全天购电及充放电行为进行统一优化。

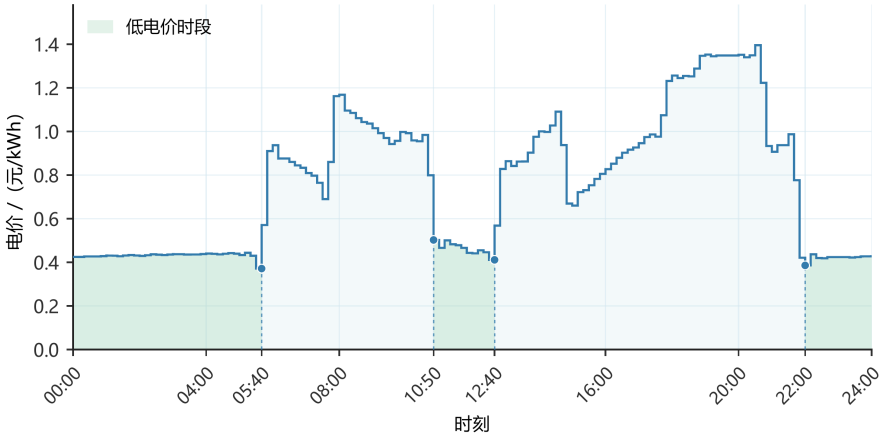
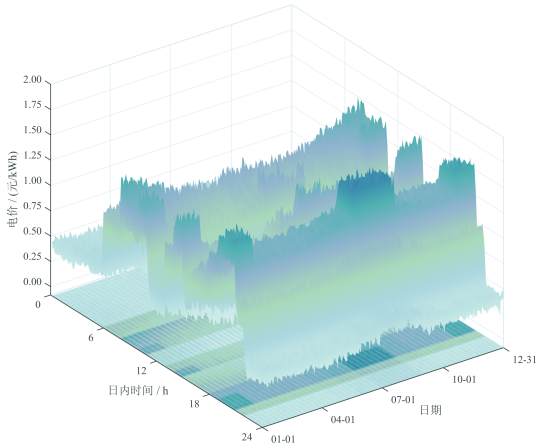


图 3: 电价分析图

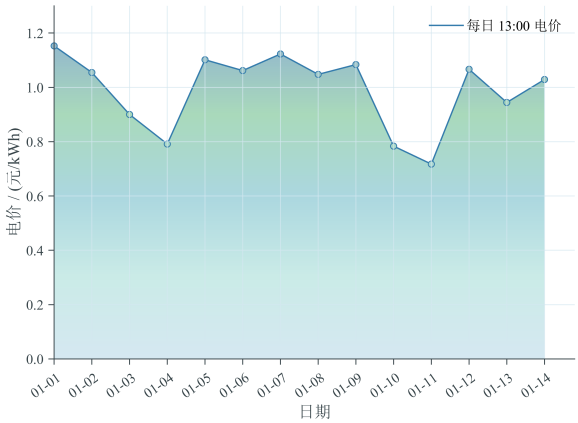
5.4 电价波动分析

对附件 4 中的全年电价数据进行时序分析，可以发现电价同时具有明显的日内分时特征与周期特征。从全年电价分布来看，虽然不同日期的价格水平存在一定波动，但各日内部仍呈现较稳定的峰谷结构，低价与高价时段在日内具有较明显的时间分布规律，说明第一问中观察到的分时电价特征在波动电价条件下仍然存在。

进一步对全年电价序列进行滞后相关性分析发现，滞后 7 日的 Pearson 相关系数达到 0.965，表明相隔一周的电价变化具有较强的一致性；从同一时刻的连续日期电价变化中也可以观察到较明显的周周期波动。因此，电价不仅与日内时刻有关，还表现出较强的星期周期特征。基于上述规律，后续电价预测可同时考虑“同星期”与“同时段”两个时间维度，利用历史同星期、同一时段的电价信息估计未来电价，为波动电价条件下的购电及储能调度提供依据。



(a) 全年电价的日内及日际波动特征



(b) 同一时刻电价的 7 日周期特征

图 4: 外网电价的时间变化特征

六、问题一的建立与求解

6.1 目标函数与决策变量

根据题意，微网需要在每日 0:00 制定当日计划购电与储能充放电策略。考虑前述时间尺度设置，以第 T 个 10 分钟区间的外网计划购电量 X_T 以及第 t 个分钟区间的储能放电量 Y_t、充电量 Z_t 为核心决策变量，其中 $T = 1, \ldots, 144$，$t = 0, \ldots, 1439$。此外，引入储电量 S_t 描述储能状态，并以 W_t 表示未被负载或储能消纳的光伏电量。

问题一要求在满足小区负载需求及储能运行约束的条件下尽可能降低全天外网购电费用。因此，以全天计划购电总费用 C 最小为目标，建立目标函数：

$$\min C = \sum_{T=1}^{144} p_T X_T. \tag{4}$$

其中，p_T 为第 T 个 10 分钟区间的外网购电单价，单位为元/kWh。

6.2 约束条件

为保证调度方案满足微网供需平衡及储能设备安全运行要求，模型需同时满足储能状态、容量与功率限制、分钟电量平衡及变量取值范围等约束，具体如下。

约束 1：为避免模型通过净消耗期初储能获得表面上的购电成本优势，并保证调度策略在日际运行中的可持续性，根据题意要求日末储电量与日初保持一致。因此有：

$$S_0 = S_{1440} = 6000. \tag{5}$$

约束 2：依据题目要求，为避免储能设备过度充电或过度放电，在运行过程中，储能设备的储电量应始终保持在允许范围内，即：

$$1200 \le S_t \le 10800, \qquad t = 0, \ldots, 1440. \tag{6}$$

约束 3：在任意一分钟区间 $[t, t+1)$ 内，光伏发电量、该分钟分配的外网购电量及储能实际放电量之和，应等于小区负载用电量、储能实际充电所需电量及弃光量之和。因此有：

$$E_t^{\text{light}} + \frac{X_{T(t)}}{10} + 0.9 Y_t = E_t^{\text{need}} + \frac{Z_t}{0.9} + W_t, \qquad t = 0, \ldots, 1439. \tag{7}$$

其中，$X_{T(t)}/10$ 为对应 10 分钟购电量均匀分配至该分钟的电量；$0.9Y_t$ 为储能放电后实际供给微网的电量；$Z_t/0.9$ 为使储能内部增加 Z_t 所需由微网输入的电量；W_t 为该分钟未被负载或储能消纳的光伏电量。

约束 4：依据题目要求，储能设备的最大放电功率为 5000 kW。由于 Y_t 表示第 t 个一分钟区间内储能内部释放的电量，故该分钟内的最大放电电量为 $5000/60$ kWh，因此有：

$$Y_t \le \frac{5000}{60}, \qquad t = 0, \ldots, 1439. \tag{8}$$

约束 5：同理，储能设备的最大充电功率为 5000 kW。考虑充电效率为 0.9，当储能内部增加电量 Z_t 时，微网侧实际需要输入的电量为 $Z_t/0.9$，因此有：

$$\frac{Z_t}{0.9} \le \frac{5000}{60}, \qquad t = 0, \ldots, 1439. \tag{9}$$

约束 6：储能设备的储电量随充放电行为逐分钟变化。由于 Y_t 和 Z_t 分别表示第 t 个分钟区间内储能内部的放电量和充电量，因此储能状态满足：

$$S_{t+1} = S_t - Y_t + Z_t, \quad t = 0, \ldots, 1439. \tag{10}$$

约束 7：外网购电量及储能充放电量均应为非负值，因此有：

$$X_T \ge 0, \qquad T = 1, \ldots, 144. \tag{11}$$

$$Y_t \ge 0, \qquad Z_t \ge 0, \qquad t = 0, \ldots, 1439. \tag{12}$$

约束 8：弃光量不能超过该分钟实际可获得的光伏发电量，因此有：

$$0 \le W_t \le E_t^{\text{light}}, \qquad t = 0, \ldots, 1439. \tag{13}$$

6.3 模型汇总

综上所述，本文建立了以全天购电费用最小为目标、以购电量和充放电量为决策变量的微网计划购电策略模型，汇总如下：

$$\min C = \sum_{T=1}^{144} p_T X_T \tag{14}$$

$$\textbf{s.t.} \begin{cases} S_0 = S_{1440} = 6000 \\ 1200 \le S_t \le 10800, \quad t = 0, \ldots, 1440 \\ E_t^{light} + \dfrac{X_{T(t)}}{10} + 0.9Y_t = E_t^{need} + \dfrac{Z_t}{0.9} + W_t, \quad t = 0, \ldots, 1439 \\ Y_t \le \dfrac{5000}{60}, \quad t = 0, \ldots, 1439 \\ \dfrac{Z_t}{0.9} \le \dfrac{5000}{60}, \quad t = 0, \ldots, 1439 \\ S_{t+1} = S_t - Y_t + Z_t, \quad t = 0, \ldots, 1439 \\ X_T \ge 0, \quad T = 1, \ldots, 144 \\ Y_t \ge 0, \quad Z_t \ge 0, \quad t = 0, \ldots, 1439 \\ 0 \le W_t \le E_t^{light}, \quad t = 0, \ldots, 1439 \end{cases} \tag{15}$$

6.4 模型求解

将负载、光伏预测及分时电价数据代入上述线性规划模型进行求解，得到微网全天计划购电及储能充放电方案。求解结果表明，全天计划购电量为 59,566.62kWh，购电总费用为 35,073.87 元，弃

光量仅为 0.3779 kWh。储能设备日初与日末储电量均为 6000kWh，满足首末储电量一致的要求。求解所得各时段具体购电量及储能充放电结果见表 1、表 2 和 result1。

表 1: 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量/KWh	时间段	购电量/KWh	时间段	购电量/KWh
10:00–10:10	0.0000	12:00–12:10	480.7120	14:00–14:10	0.0000
16:00–16:10	420.8898	18:00–18:10	589.3531	20:00–20:10	0.0000
全天购电量/KWh		59,566.6229	全天购电费/元		35,073.87

表 2: 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量/KWh	放电量/KWh	时间段	充电量/KWh	放电量/KWh
0:00–4:00	4499.2072	0.0000	4:00–8:00	834.2416	6466.9870
8:00–12:00	5012.0640	1907.2664	12:00–16:00	5414.2077	76.5720
16:00–20:00	73.3406	5836.1658	20:00–24:00	5347.2570	2869.1183
0:00 储电量/KWh		6000.0000	24:00 储电量/KWh		6000.0000

6.5 模型结果分析

为进一步分析最优方案的调度特征，图 5 给出了全天负载、光伏发电、计划购电、储能充放电及储电量的变化情况。由图可见，模型能够根据日内供需状态合理调整储能行为：在部分电价较低或电量较充裕的时段进行充电，在后续电价较高或光伏出力不足的时段释放储能，以减少高价时段的外网购电需求。

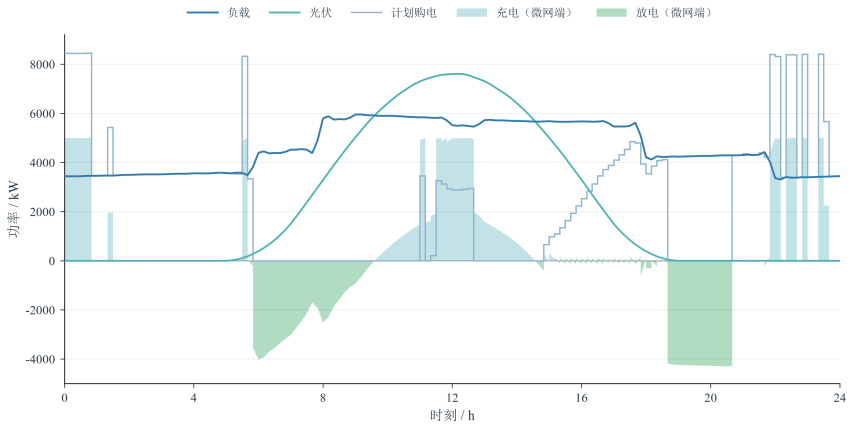


图 5: 全天负载、光伏、计划购电、充放电图

由图 6 可见，储能设备全天储电量均保持在 1200 ～ 10800 kWh 的允许范围内，并在 24:00 恢复至初始储电量 6000 kWh，满足容量及首末状态约束。结合求解结果可知，模型在满足各项运行约束的条件下，实现了给定模型下全天购电费用的最小化。

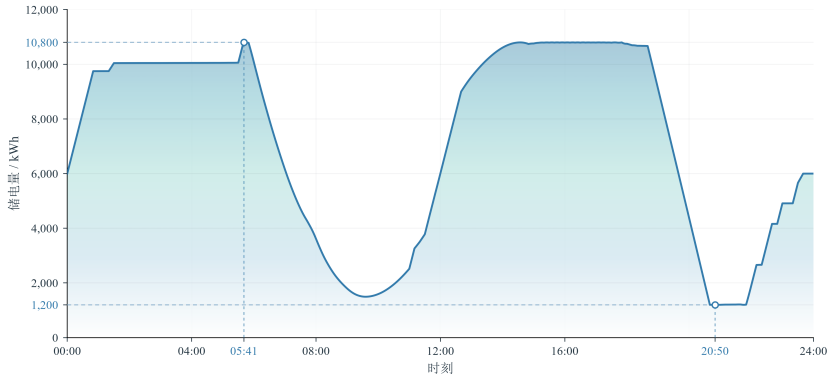


图 6: 日内储能变动图

七、问题二的建立与求解

7.1 模型总体框架

针对负载与光伏出力存在预测误差、供电不足将产生高额紧急购电费用的特点，本文在问题一确定性调度模型的基础上，构建“日前预测与风险修正–日前计划优化–日内实时校正–跨日状态更新”的动态调度框架。每日 0:00 首先利用历史负载与光伏数据形成当日预测，并结合历史净负载预测误差构造风险裕度；随后在满足储能运行约束的条件下制定全天计划购电与充放电方案。计划确定后，日内根据实际负载、光伏出力及当前储能状态逐分钟修正储能动作，若修正后仍存在供电缺口，则通过紧急购电补足。日末将实际储能状态及安全参考信息传递至次日，实现各日调度之间的连续衔接。

7.2 日前预测与风险裕度构造

每日 0:00 制定购电计划时，当日真实负载与光伏发电情况尚不可观测，因此需利用此前已观测的历史数据对当日供需情况进行预测。根据前文数据分析，小区负载具有较明显的周期特征，而光伏出力受近期气象条件影响较大。基于二者不同的时序特征，本文分别建立负载预测与光伏预测模型，得到日前优化所需的点预测结果。

7.2.1 小区负载预测

为综合利用历史负载信息，首先根据历史日期与预测日期之间的距离构造加权基准。设 d 为当前日序号，i 为历史日序号。考虑全年数据具有周期性，定义二者的环形日期距离为 $D_{d,i} = \min\{|d-i|, 365-|d-i|\}$, 进一步定义历史日期 i 对当前日期 d 的权重为：

$$w_{d,i} = 1 - \frac{2D_{d,i}}{365}. \tag{16}$$

由此，利用当前日期之前已实现的历史数据，构造第 d 日第 t 分钟的负载加权基准：

$$\bar{E}^{need}_{d,t} = \frac{\sum_{i<d} w_{d,i} E^{need}_{i,t}}{\sum_{i<d} w_{d,i}}. \tag{17}$$

其中，$E^{\text{need}}_{i,t}$ 表示历史日 i 第 t 分钟的实际负载电量，$\bar{E}^{\text{need}}_{d,t}$ 表示根据历史数据得到的负载基准值。

前文滞后相关性分析表明，小区负载存在明显的 7 日周期特征。因此，在长期历史基准的基础

上，引入七日前同一分钟相对于其当时历史基准的偏差信息。定义：

$$r_{d-7,t}^{need} = \frac{E_{d-7,t}^{need} - \bar{E}_{d-7,t}^{need}}{\bar{E}_{d-7,t}^{need}}. \tag{18}$$

则第 d 日第 t 分钟的负载点预测为：

$$\hat{E}_{d,t}^{\text{need}} = \max\left\{0,\ \bar{E}_{d,t}^{\text{need}}\left(1 + r_{d-7,t}^{\text{need}}\right)\right\}. \tag{19}$$

该方法利用历史加权基准刻画负载的长期水平，同时利用七日前同一时刻的相对偏差反映居民用电的周周期变化，从而兼顾长期趋势与短期周期特征。

7.2.2 光伏发电预测

与居民负载相比，光伏发电情况具有更明显的短期波动特征。本文采用预测日前连续 7 个已结束自然日同一分钟光伏发电量的平均值，作为当日对应分钟的光伏点预测，即：

$$\hat{E}_{d,t}^{light} = \frac{1}{7}\sum_{k=1}^{7} E_{d-k,t}^{light}. \tag{20}$$

其中，$E_{d-k,t}^{light}$ 表示第 $d-k$ 日第 t 分钟的实际光伏发电量。

该预测仅使用当前决策时点之前已观测的数据，不包含当日及未来信息，从而保证日前预测的信息边界。通过上述方法，分别得到当日分钟级负载点预测 $\hat{E}_{d,t}^{need}$ 与光伏点预测 $\hat{E}_{d,t}^{light}$，并将二者作为后续预测误差分析与日前计划优化的基础输入。

7.3 日前计划优化模型

每日 0:00，微网依据当日负载与光伏预测结果制定全天计划购电及储能充放电方案。由于预测误差可能导致实际净负载高于点预测值，本文将前述风险裕度加入负载预测，定义用于日前优化的决策负载为：

$$\tilde{E}_{d,t}^{\text{need}} = \hat{E}_{d,t}^{\text{need}} + R_{d,t}. \tag{21}$$

其中，$R_{d,t}$ 为根据历史净负载预测误差的 80% 分位数构造的非负风险裕度。

在此基础上，以当日实际初始储电量为状态起点，建立日前线性规划模型。

7.3.1 决策变量与基础约束

$X_{d,T}$ 为第 d 日第 T 个 10 分钟区间的计划购电量；$Y_{d,t}^{plan}$ 和 $Z_{d,t}^{plan}$ 分别表示第 t 分钟储能内部的放电量与充电量；$S_{d,t}^{plan}$ 为计划储电量；$W_{d,t}^{plan}$ 为该分钟未被负载或储能消纳的剩余电量。其中 $T = 1, \ldots, 144, \quad t = 0, \ldots, 1439$。每天计划以当日实际储能状态作为初始状态，因此有 $S_{d,0}^{plan} = S_{d,0}^{act}$，同时储能设备需满足问题一所述的容量约束。

第 t 分钟的电量平衡关系为：

$$\hat{E}_{d,t}^{light} + \frac{X_{d,T(t)}}{10} + 0.9Y_{d,t}^{plan} = \tilde{E}_{d,t}^{need} + \frac{Z_{d,t}^{plan}}{0.9} + W_{d,t}^{plan}. \tag{22}$$

其中，$T(t) = \left\lfloor \frac{t}{10} \right\rfloor + 1$。

同时储能状态按照分钟递推：

$$S_{d,t+1}^{plan} = S_{d,t}^{plan} - Y_{d,t}^{plan} + Z_{d,t}^{plan}. \tag{23}$$

充、放电功率约束和 X_d, T、$Y_{d,t}$、$Z_{d.t}$、$W_{d,t}$ 的非负约束依然满足第一问所述约束条件。由于 $W_{d,t}$ 于第二问中表示未利用电量，因此不再施加问题一中 $W_t \le E_t^{light}$ 约束。

7.3.2 分层线性规划求解

为兼顾计划购电经济性与日末储能安全水平，本文采用分层线性规划依次求解。

第一阶段：确定基准购电成本

在上述基础约束外，增加日初日末储电量一致性条件 $S_{d,1440}^{plan} = S_{d,0}^{actual}$, 并以全天计划购电费最小为目标，因此有：

$$C_d^* = min \sum_{T=1}^{144} p_T X_{d,T}. \tag{24}$$

其中，C_d^* 表示在当前风险调整预测下、不净消耗当日储能库存时的最低计划购电成本。

第二阶段：预算范围内优先控制日末储能偏差

取消日初日末储电量相等约束，将计划购电费用限制在基准成本的 1.05 倍以内，于是有：

$$C_d = \sum_{T=1}^{144} p_T X_{d,T} \leq 1.05 C_d^*. \tag{25}$$

为了控制日末储能偏差，本文引入 B_d 为第 d 日的安全储能参考水平，并引入辅助变量 A_d, 因此有：

$$A_d \geq S_{d,1440}^{plan} - B_d, \qquad A_d \geq B_d - S_{d,1440}^{plan}, \quad A_d \geq 0. \tag{26}$$

于是，$minA_d$ 就能得到最小日末储能偏差 A_d^*。

第三阶段：同等储能偏差下进一步降低费用

在第二阶段的基础上，增加约束，因此有：

$$A_d \leq A_d^* + 10^{-6}. \tag{27}$$

并再次求解 $minC_d$，所得结果即为当日最终全天计划购电及储能充放电方案，其求解逻辑可以按顺序概括为：一、确定基准成本；二、预算内最小化储能偏差；三、同等偏差下最小化购电费用。

7.4 日内实时修正模型

日前计划确定后，当日各 10 分钟区间的计划购电量 $X_{d,T}$ 保持不变。由于实际负载与光伏发电情况可能偏离日前预测，本文以当前真实储电量和日前计划充放电动作为参考，对储能行为进行逐分钟修正。需要说明的是，前述风险裕度仅用于日前计划阶段，实际运行时仍以真实负载和真实光伏出力进行电量平衡。

记当前真实储电量为 $s = S_{d,t}^{actual}$，日前计划放电量和充电量分别为 $Y_{d,t}^{plan}$ 和 $Z_{d,t}^{plan}$，则下一刻的储电量可以表示为：

$$q^{plan} = s - Y_{d,t}^{plan} + Z_{d,t}^{plan}. \tag{28}$$

若 q^{plan} 超出储能容量范围，则首先调整原计划动作，使其回到允许区间。具体地，当 $q^{plan} < 1200$ 时减少计划放电; 当 $q^{plan} > 10800$ 时减少计划充电; 若储能状态处于允许范围内，则暂时保持原计划动作。经容量及功率约束修正后，记当前待执行充、放电量分别为 $Z_{d,t}$ 和 $Y_{d,t}$。本文在此定义当前动作后的供电缺口为：

$$\Delta_{d,t} = E_{d,t}^{need} + \frac{Z_{d,t}}{0.9} - E_{d,t}^{light} - \frac{X_{d,T(x)}}{10} - 0.9 Y_{d,t}. \tag{29}$$

其中，$\Delta_{d,t} > 0$ 表示供电不足，反之则表示电量富裕。

当 $\Delta_{d,t} > 0$ 时，按照先减少充电、再增加放电、最后紧急购电的原则修正储能动作。若当前同时存在充、放电，则等量削减二者，在不改变储能净变化的条件下降低充放电效率损耗，最后仍无法弥补的缺口由紧急购电 $H_{d,t}$ 补足。相反，当 $\Delta_{d,t} < 0$ 时，则按照先减少放电、再增加充电、最后

记录剩余电量的原则修正。经日内修正后，实际储能逐分钟递推式为：

$$S_{d,t+1}^{actual} = S_{d,t}^{actual} - Y_{d,t}^{actual} + Z_{d,t}^{actual}, \qquad t = 0, \ldots, 1439. \tag{30}$$

并满足实际电量平衡关系：

$$E_{d,t}^{light} + \frac{X_{d,T(x)}}{10} + 0.9Y_{d,t}^{actual} + H_{d,t} = E_{d,t}^{need} + \frac{Z_{d,t}^{actual}}{0.9} + W_{d,t}^{actual}. \tag{31}$$

全天紧急购电量为 $H_d = \sum_{t=0}^{1439} H_{d,t}$，相应的紧急购电费用则为 $C_d^{emer} = \sum_{t=0}^{1439} 5p_{T(t)}H_{d,t}$，因此，第 d 日的实际总购电费用为：

$$C_d^{actual} = C_d + C_d^{emer}. \tag{32}$$

7.5 跨日储能状态与安全参考值更新

为保证实际运行过程在日期之间连续，本文不将每日初始储电量重置为6000，而是将前一日实际日末储电量直接作为次日的实际初始状态，即 $S_{d+1,0}^{actual} = S_{d,1440}^{actual}$，模拟初始时设 $S_{1,0}^{actual} = 6000, \quad B_1 = 6000, \quad \delta_1 = 0$，其中，$B_d$ 表示第 d 日日前优化所采用的日末安全储能参考值，δ_d 表示安全参考值的日回落量。

若第 d 日发生紧急购电，即 $H_d > 0$，说明当日实际供电出现较大缺口，为提高后续调度的储能备用水平，根据当日紧急购电量提高次日安全参考值，因此有：

$$B_{d+1} = \min\{10800,\, B_d + 5H_d\}. \tag{33}$$

同时重新确定后续日回落量 $\delta_{d+1} = \frac{B_{d+1}-6000}{180}$，若当日未发生紧急购电，则安全参考值按既定速度逐步回落 $B_{d+1} = \max\{6000,\, B_d - \delta_d\}, \quad \delta_{d+1} = \delta_d$。因此，当系统出现供电缺口时，安全参考值随紧急购电量提高，使后续日前计划保留更多储能；在连续未发生紧急购电的情况下，参考值则逐步回落至基准水平 6000。若回落过程中再次发生紧急购电，则依据新的参考值重新计算回落速度，从而形成具有反馈特征的跨日储能安全调节机制。

7.6 模型汇总

综合上述模型，问题二的最终优化模型为：

$$
\textbf{s.t.}\quad
\begin{cases}
D_{d,i}=\min\{|d-i|,365-|d-i|\},\quad w_{d,i}=1-\dfrac{2D_{d,i}}{365}\\
\bar{E}^{need}_{d,t}=\dfrac{\sum_{i<d}w_{d,i}E^{need}_{i,t}}{\sum_{i<d}w_{d,i}},\quad r^{need}_{d-7,t}=\dfrac{E^{need}_{d-7,t}-\bar{E}^{need}_{d-7,t}}{\bar{E}^{need}_{d-7,t}}\\
\hat{E}^{\text{need}}_{d,t}=\max\left\{0,\ \bar{E}^{\text{need}}_{d,t}\left(1+r^{\text{need}}_{d-7,t}\right)\right\},\quad \hat{E}^{light}_{d,t}=\dfrac{1}{7}\displaystyle\sum_{k=1}^{7}E^{light}_{d-k,t}\\
\tilde{E}^{\text{need}}_{d,t}=\hat{E}^{\text{need}}_{d,t}+R_{d,t}\\
\hat{E}^{light}_{d,t}+\dfrac{X_{d,T(t)}}{10}+0.9Y^{plan}_{d,t}=\tilde{E}^{need}_{d,t}+\dfrac{Z^{plan}_{d,t}}{0.9}+W^{plan}_{d,t}\\
C_d=\displaystyle\sum_{T=1}^{144}p_T X_{d,T}\le 1.05C^{*}_{d}\\
A_d\ge S^{plan}_{d,1440}-B_d,\quad A_d\ge B_d-S^{plan}_{d,1440},\quad A_d\ge 0\\
A_d\le A^{*}_{d}+10^{-6}\\
\Delta_{d,t}=E^{need}_{d,t}+\dfrac{Z_{d,t}}{0.9}-E^{light}_{d,t}-\dfrac{X_{d,T(t)}}{10}-0.9Y_{d,t}\\
H_{d,t}=\max\{0,\Delta_{d,t}\},\quad W^{actual}_{d,t}=\max\{0,-\Delta_{d,t}\}\\
E^{light}_{d,t}+\dfrac{X_{d,T(t)}}{10}+0.9Y^{actual}_{d,t}+H_{d,t}=E^{need}_{d,t}+\dfrac{Z^{actual}_{d,t}}{0.9}+W^{actual}_{d,t}\\
B_{d+1}=\min\{10800,B_d+5H_d\}\\
B_{d+1}=\max\{6000,B_d-\delta_d\}\\
\delta_{d+1}=\dfrac{B_{d+1}-6000}{180},\quad H_d>0\qquad \delta_{d+1}=\delta_d,\quad H_d=0
\end{cases}
\tag{34}
$$

7.7 模型求解流程

步骤 1：以 2 月 1 日为正式模拟起点，设置初始实际储电量及安全储能参考值，并读取此前已结束日期的历史负载、光伏数据和预测误差样本。

步骤 2：每日 0:00 利用历史数据计算当日负载与光伏点预测，并根据历史净负载预测误差构造风险裕度，得到日前优化所采用的决策负载。

步骤 3：以当前实际储电量为初始状态，在储能容量、充放电功率及电量平衡等约束下进行分层线性规划：首先计算循环储能条件下的最低基准购电成本；随后在规定预算范围内最小化日末储能偏差；最后在同等偏差水平下进一步最小化计划购电费用，得到当日最终计划购电量及储能充放电轨迹。

步骤 4：全日计划购电量确定后保持不变，按分钟读取实际负载、光伏出力与当前真实储能状态，并依据实时修正规则调整充放电动作；经储能调节后仍存在的供电缺口由紧急购电补足，同时记录未利用电量及实际储能轨迹。

步骤 5：汇总当日计划购电费用、紧急购电量及实际总费用，将实际日末储电量传递至次日，并根据当日紧急购电情况更新安全储能参考值及回落量。随后进入下一日，重复上述过程，直至完成全年模拟。

7.8 模型求解与结果分析

7.8.1 模型求解

根据前文建立的日前预测、风险修正、分层计划优化及日内实时校正模型，本文对 2025 年 2 月 1 日至 12 月 31 日共 334 天进行逐日连续模拟。1 月份数据用于负载、光伏预测及历史净负载预测误差的构造，2 月 1 日实际初始储电量设为 6000 kWh。每日 0:00 仅利用此前已实现的历史信息生成当日预测与风险裕度，并依次求解三阶段线性规划，得到全天计划购电量及储能充放电轨迹；随后按照分钟尺度输入实际负载和光伏数据，对计划充放电动作进行实时修正，并记录紧急购电量、未利用电量及实际储能状态。日末将实际储电量与安全参考信息传递至次日，重复上述过程直至 12 月 31 日。

结果为2月至12月总购电成本14,122,887.07元，紧急购电成本923,041.90，紧急购电量154,151.92 KWh，紧急购电日数为165日。

求解所得完整购电策略见 result2。表 3 所列 4 个指定日期的购电量及储能充放电量完整结果分别见附录表 7—表 14，正文仅列出紧急购电结果表 3。

表 3: 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
18:53–19:07	9.354	无紧急购电	0.000	06:10–06:11	0.028	07:59–08:00	0.154
20:09–20:14	0.877	—	—	20:10–20:30	42.815	18:50–18:51	0.000
20:20–20:21	0.137	—	—	20:38–20:40	95.387	19:58–20:00	1.265
—	—	—	—	21:39–21:42	1.416	—	—
合计	10.368	合计	0.000	合计	139.647	合计	1.419

7.8.2 结果分析

由图 7 可见，2025 年 6 月 1 日实际储能轨迹与日前计划偏离较大，并多次接近储能下限，导致部分时段无法继续通过储能调节弥补供电缺口，从而产生较高的紧急购电需求；相比之下，6 月 21 日实际储能虽与计划轨迹存在一定偏差，但剩余调节空间充足，全天未发生紧急购电。说明紧急购电不仅与预测偏差有关，还受到当时储能状态及剩余调节能力的共同影响。

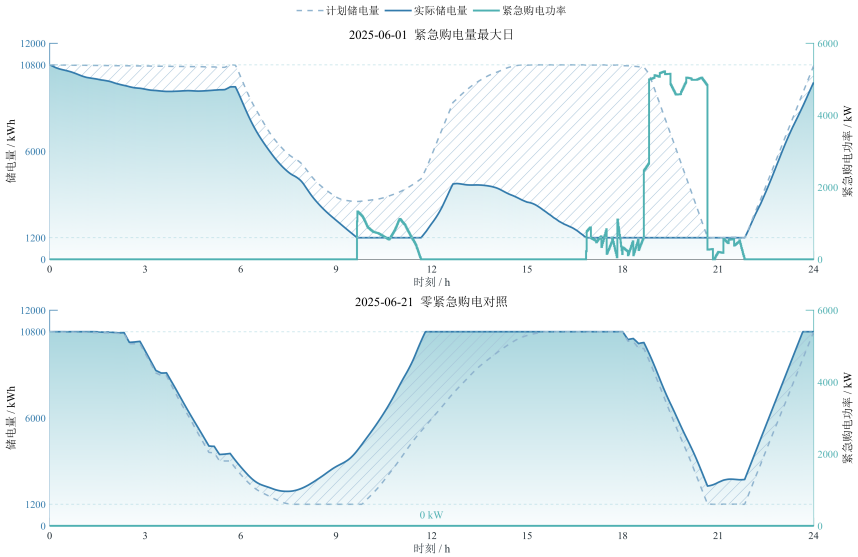


图 7: 典型日储能轨迹与紧急购电情况对比

由图 8 可见，全年紧急购电量在月份和日内时段上均呈现明显的聚集特征，并非均匀分布。部分月份及特定时段的累计紧急购电量显著较高，说明供电风险具有较强的时序差异。因此，在日前计划中保留适当风险裕度，并根据实际运行情况动态调整储能安全水平，有助于降低高风险时段发生大规模紧急购电的可能性。

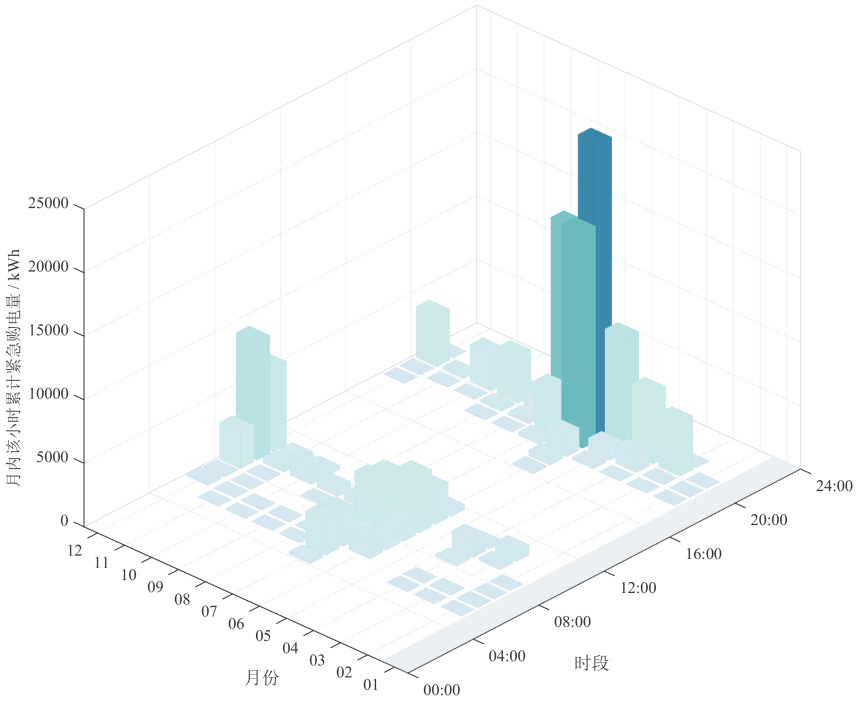


图 8: 全年紧急购电分布图

八、问题三的建立与求解

8.1 模型总体框架

问题三在问题二调度框架的基础上进一步引入日内多时刻光伏预报及购电计划调整机制。针对预报信息不断更新而计划调整又会产生额外费用的特点，本文构建“多时刻预测更新–初始计划制定–双路径评估–动态计划调整–实际运行修正”的日内动态调度框架。每日 0:00 依据当前可获得的负载与光伏预测制定当日初始购电及储能充放电计划；在 6:00、12:00 和 18:00 获得新一轮光伏预报后，分别构造保留当前计划与重新调整计划两条候选路径，综合考虑购电及调整费用与期末储能状态，对两条路径进行评价并决定是否更新当天尚未执行的购电计划。

实际运行过程中，购电计划按照当前有效订单执行，储能则依据实际负载、光伏出力及真实储能状态进行分钟级修正；若仍存在供电缺口，则通过紧急购电补足。日末按实际发生的初始购电、计划调整及紧急购电进行费用结算，并将真实储能状态传递至次日，从而形成随预报更新不断修正的动态闭环调度过程。

8.2 预测模型

8.2.1 光伏预测模型

问题三在每日 0:00、6:00、12:00 和 18:00 均可获得未来 24 h 整点光伏发电功率预报。随着预测目标时刻临近，新发布的预报通常包含更新的信息；与此同时，较远期预报的不确定性较大，仅依赖单一版本可能造成预测波动。因此，本文采用“短期直接使用最新预报、远期动态融合多版本信息”的分段预测方法。

设当前预报发布时间为 τ，提前小时数为 h，则目标时刻为 $\tau+h$。对于过去已发布且能够覆盖同一目标时刻的预报，记 $f(\tau-6k, h+6k)$ 为 $\tau-6k$ 时刻发布、针对目标时刻 $\tau+h$ 的预测值。为保证不同预报对应同一预测对象，所有预报均按照目标时刻而非表格列号进行对齐。对于可获得的多个历史版本，本文按照“预报越新，权重越高”的原则设置固定版本权重，并在可用版本减少时进行归一化，各提前期权重如下：

表 4: 不同提前小时下的历史预报版本权重

提前小时数/h	由旧到新的历史预报版本权重
$1 \sim 6$	$0.1,\ 0.2,\ 0.3,\ 0.4$
$7 \sim 12$	$0.2/0.9, 0.3/0.9, 0.4/0.9$
$13 \sim 18$	$0.3/0.7, 0.4/0.7$
$19 \sim 24$	1

据此得到多版本融合结果 $F(\tau, h)$。上述权重属于模型预设的版本融合规则，用于体现不同发布时间信息新旧程度的差异，并非通过后续样本重新估计。同时为进一步利用近期相同预测窗口的历史表现，取过去 7 天相同发布时间、相同提前量的预报值 f_j 及对应目标时刻的实际光伏功率 P_j，定义预测误差为 $e_j = f_j - P_j, \quad j = 1, \ldots, 7$，并以此构造历史预测分量：

$$G(\tau, h) = \frac{1}{7}\sum_{j=1}^{7} P_j + \frac{1}{7}\sum_{j=1}^{7} e_j. \tag{35}$$

由于 $e_j = f_j - p_j$，上述 $G(\tau, h)$ 在代数上等价于过去 7 个相同预测窗口历史预报值的均值。在 $F(\tau, h)$ 与 $G(\tau, h)$ 的基础上，进一步构造动态融合预测：

$$D(\tau, h) = \alpha(\tau, h)F(\tau, h) + [1 - \alpha(\tau, h)]G(\tau, h). \tag{36}$$

其中 $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$，与前述固定版本权重不同，$\alpha$ 由历史预测表现动态确定。针对每个发布时间和每个提前小时，分别利用过去 7 天相同预测窗口的已实现样本，计算各候选 α 下的平均绝对误差, 因此有：

$$MAE(\alpha) = \frac{1}{7}\sum_{j=1}^{7} |\hat{P}_j(\alpha) - P_j|. \tag{37}$$

并选取 MAE 最小的候选值作为当前预测窗口的融合权重。历史样本中的各项预测均按照其当时能够获得的信息重新构造，从而避免使用未来数据。最终，根据预测提前量采用分段规则：

$$\widehat{P}^{\text{light}}(\tau, h) = \begin{cases} f(\tau, h), & 1 \le h \le 6, \\ D(\tau, h), & 7 \le h \le 24. \end{cases} \tag{38}$$

即未来 1–6h 直接采用当前最新版原始预报，以充分利用近期预报的时效性; 未来 7–24h 则采用动态融合预测，以降低远期单一预报的不稳定性。所得未来 24h 整点预测功率再按照前文的线性插值及分钟电量换算方法转化为调度模型所需的分钟级光伏预测电量。图表示修正后光伏预报对最新版光伏预报的修正，提高了模型对未来光伏发电判断的准确性。

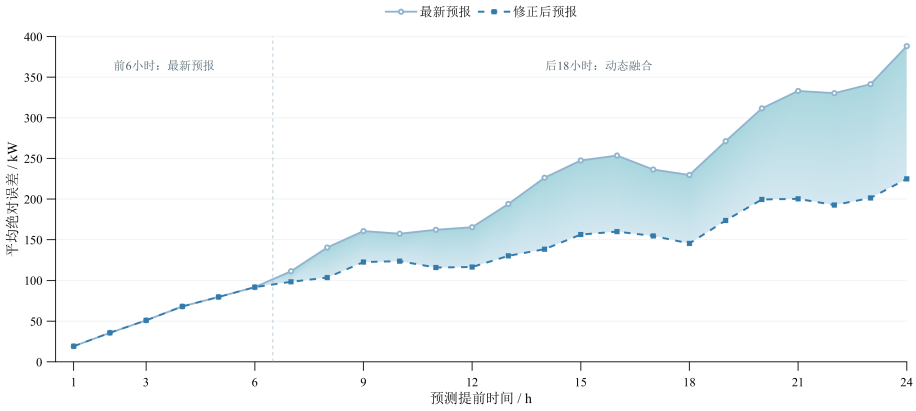


图 9: 修正预报示意图

8.2.2 小区负载预测

小区负载预测沿用问题二建立的“历史加权基准 + 七日前同一时刻偏差修正”方法。由于问题三在 6:00、12:00 和 18:00 的预测窗口可能跨越至次日，因此在预测目标日负载时，仅使用当前决策时点之前已经结束的完整历史日期构造加权基准，并采用目标日期对应的七日前同刻偏差进行修正。由此既保留居民用电的周周期特征，又避免在日内决策过程中引入尚未实现的负载信息。其余计算方法与问题二一致，此处不再赘述。

8.3 每日 0:00 初始计划模型

每日 0:00，微网以当前真实储电量 $S_{d,0}^{\text{actual}}$ 为初始状态，根据此时获得的光伏预测结果及负载预测结果，制定当天 0:00 ∼ 24:00 的初始购电与储能充放电计划。设 $X_{d,T}$ 为第 T 个 10 分钟区间的计划购电量，$Y_{d,t}^{\text{plan}}$、$Z_{d,t}^{\text{plan}}$ 分别为第 t 分钟储能内部的计划放电量与充电量，$S_{d,t}^{\text{plan}}$ 为计划储电量。

在沿用前述储能容量、充放电功率、状态递推及分钟电量平衡等物理约束的基础上，以当天计

划购电费用最小为目标，即 $min\sum_{T=1}^{144} p_T X_{d,T}$, 其中，分钟电量平衡满足：

$$\hat{E}_{d,t}^{light} + \frac{X_{d,T(x)}}{10} + 0.9Y_{d,t}^{plan} = \hat{E}_{d,t}^{need} + \frac{Z_{d,t}^{plan}}{0.9} + W_{d,t}^{plan}. \tag{39}$$

储能状态仍然按上述 (23) 式逐分钟递推，并满足储能设备容量约束及最大充放电功率限制。

为避免初始计划通过净消耗当日储能获得表面上的成本优势，要求预测环境下日末计划储电量恢复至当日实际初始储电量，即 $S_{d,1440}^{plan} = S_{d,0}^{actual}$。因此，0:00 初始计划是在预测供需条件下、保持日内储能净变化为零时的最低费用购电方案。需要指出的是，该约束仅针对计划轨迹; 由于日内实际负载与光伏出力可能偏离预测，实际日末储电量并不要求与日初严格相等，而是由真实运行过程决定并连续传递至次日。初始计划求解完成后，保存其完整副本，并以此初始化当天的当前有效计划与购电账本。此后若在 6:00、12:00 或 18:00 接受调整，仅更新尚未执行的有效计划，0:00 制定的初始计划仍保留作为题目要求的计划购电结果。

8.4 日内动态路径选择模型

每日 6:00、12:00 和 18:00 获得新的光伏预报后，微网需要判断是否调整当前尚未执行的购电计划。由于计划调整会产生额外交易成本，新的预报信息并不意味着必须修改原计划。因此，本文以当前真实储电量和最新预测信息为共同起点，分别构造保留当前计划与调整当前计划两条候选路径，并综合比较两条路径的预期现金费用与期末储能状态，从而决定是否更新当日剩余购电计划。

8.4.1 保留路径

设第 d 日 a:00(a=6,12,18) 为当前决策时刻，当前真实储电量记为 S_0。在新的光伏预报到达后，若不对现有购电计划进行调整，则形成保留路径。该路径以当前有效购电计划和储能充放电计划为基础，在最新负载与光伏预测条件下，对未来 24h 运行状态进行推演。对于当前时刻至当日 24:00 的剩余时段，保持当前有效购电计划不变，并以原计划充放电动作为参考，按照实际运行修正规则在预测环境下逐分钟推演，得到预测午夜储电量 M，同时记录该阶段可能产生的紧急购电量与未利用电量。

由于 24h 预测窗口会跨越至次日，为评价当前计划对后续储能状态的影响，本文从预测午夜储电量 M 出发，对次日 0:00 至次日 a:00 的辅助时段进行规划。设 $\rho = \frac{a}{24}$，则保留路径在 24h 窗口末端的目标储电量设为：

$$S_{target}^{keep} = (1-\rho)M + \rho S_0. \tag{40}$$

该设置的目的为使次日辅助时段的目标储电量逐步由预测午夜状态 M 向当前真实储电量 S_0 靠拢，从而避免短时评价中过度消耗或积累储能。在两条路径比较时，当前已经承诺且无法改变的购电费用对保留路径和调整路径均相同，因此可从比较指标中消去。保留路径的新增预计现金费用记为:

$$J_{keep} = C_{tomorrow}^{keep} + 5 \sum_{t\in[\tau,24:00)} p_{T(t)} H_t^{keep,pred}. \tag{41}$$

其中,$C_{tomorrow}^{keep}$ 为次日辅助时段的预计购电费用，$H_t^{keep,pred}$ 为当前日至 24:00 预测运行过程中产生的紧急购电量。保留路径最终根据该现金费用与期末储能偏差共同进行评价，具体加权方式将在 8.4.3 中统一给出。

8.4.2 修改路径

在同一决策时刻 a:00(a=6,12,18)，若允许根据最新光伏预报重新安排尚未执行的购电计划，则形成调整路径。该路径同样以当前真实储电量 S_0 为初始状态，并采用与保留路径完全一致的未来 24h 负载与光伏预测，从而保证两条路径具有相同的信息基础。

对于当前时刻至当日 24:00 的剩余时段，原有购电订单不再作为固定计划，而是允许重新确定计划购电量。由于调整已有订单会产生追加购电费用或取消订单成本，因此将各时段相对于当前有效订单的净调整费用记为 ΔC_T，其具体计算规则将在第 8.5 节中给出。对于预测窗口中跨入次日的部分，则不存在既有购电订单，按基础电价重新进行辅助规划。

与保留路径不同，调整路径直接从当前真实储电量 S_0 出发，对未来 24h 的购电与储能充放电进行联合优化，并将 24h 窗口末端的目标储电量设为当前储电水平，即 $S_{target} = S_0$。这样可以避免调整方案通过大量消耗当前储能而获得短期费用优势，并使不同决策时点之间的储能状态保持较好的连续性。在扣除两条路径共同的既有订单费用后，调整路径的预计现金费用可表示为：

$$J_{\text{change}} = \sum_{T \in \mathcal{T}_{\text{rem}}} \Delta C_T + C_{\text{tomorrow}}^{\text{change}}, \tag{42}$$

其中，$\mathcal{T}_{\text{rem}}$ 表示当日尚未执行的购电时段，$C_{\text{tomorrow}}^{\text{change}}$ 表示次日辅助时段在基础电价下的预计购电费用。

需要说明的是，调整路径在预测环境中通过重新优化购电量直接满足预测供需，因此规划阶段不额外设置紧急购电变量; 真实运行过程中若仍因预测误差出现供电缺口，则按照前述实际运行规则进行紧急购电。调整路径的预计现金费用与期末储能偏差将在第 8.4.3 节中与保留路径进行统一比较。

8.4.3 修改路径

为统一比较保留路径与调整路径，本文同时考虑预计现金费用和预测期末储能偏差。对于任一路径 $b \in \{\text{keep}, \text{change}\}$，定义评价函数：

$$F_b = J_b + \lambda_S A_b \tag{43}$$

其中，J_b 为该路径的预计现金费用，A_b 为 24h 预测窗口末端储电量与对应目标储电量之间的绝对偏差，即 $A_b = |S_{end}^b - S_{target}^b|$，本文取储能偏差惩罚系数 $\lambda_S = 0.5$，使路径评价能够在调整成本与期末储能状态之间进行权衡。仅当 $F_{change} < F_{keep} - 10^{-6}$ 时接受调整路径，否则继续保留当前有效计划。

若调整路径被接受，仅更新当日尚未执行的购电量及储能充放电计划; 预测窗口中的次日辅助方案仅用于当前决策评价，不实际锁定或结算，次日仍根据最新信息重新制定计划。

8.5 订单账本与调整成本

由于 6:00、12:00 和 18:00 均可能根据最新预报调整尚未执行的购电计划，且追加购电与取消原订单采用不同的结算规则，因此本文对各 10 分钟购电时段建立动态订单账本。设 O_T 和 U_T 分别表示时段 T 当前仍有效的原价订单量和加价订单量，则当前有效购电量为:$Q_T = O_T + U_T$。

每日 0:00 初始计划确定后，令 $O_T = X_{d,T}^{initial}$□$U_T = 0$。若某次调整得到新的候选购电量 x_T，则新增购电量与取消购电量分别为：

$$a_T = (x_T - Q_T)_+, \quad c_T = (Q_T - x_T)_+ \tag{44}$$

根据题意，新增部分按基础电价的 1.5 倍计费; 对于取消订单，退还原购入金额的 50%。当同一时段同时存在原价订单和加价订单时，本文优先取消加价订单，记 $c_{U,T} = min\{c_T, U_T\}$ ， $c_{O,T} =$

$c_T - c_{U,T}$。因此，第 T 个时段由本次计划调整产生的净费用变化为：

$$\Delta C_T = 1.5 p_T a_T - 0.75 p_T c_{U,T} - 0.5 p_T c_{O,T}. \tag{45}$$

其中，$0.75p_T$ 来源于加价订单按 $1.5p_T$ 购入后退还其中 50%，而原价订单取消时退还 $0.5p_T$。若调整方案被接受，则更新账本

$$O_T \leftarrow O_T - c_{O,T}, \quad U_T \leftarrow U_T - c_{U,T} + a_T;$$

反之，若调整方案未被接受，则订单账本保持不变，不产生实际调整费用。在路径比较过程中，两条候选路径已经共同承担的历史订单费用可以相互抵消，因此仅需考虑新增的净调整成本。上述调整费用关于候选购电量为凸分段线性函数，可通过线性约束进行等价表示，无需额外引入整数变量，从而保持整体模型的线性规划结构。

8.6 实际执行与费用结算

日内实际运行阶段沿用问题二的分钟级实时修正规则，即根据当前真实负载、光伏出力及实际储能状态，对计划充放电动作进行逐分钟校正; 若经储能调节后仍存在供电缺口，则通过紧急购电补足，电量富余时则优先由储能吸收。为避免重复，具体修正顺序不再赘述。

当 6:00、12:00 或 18:00 的调整路径被接受时，仅更新当天尚未执行时段的当前有效购电计划及储能充放电计划，并同步更新订单账本; 若调整方案未被接受，则继续执行原有效计划。预测窗口中涉及次日的辅助方案仅用于路径评价，不实际锁定或结算。实际储电量始终按照真实运行结果连续演化，并于日末传递至次日。

第 d 日的实际总费用由三部分构成:

$$C_d^{\text{actual}} = C_d^0 + C_d^{\text{adj}} + C_d^{\text{emer}}. \tag{46}$$

其中，C_d^0 为 0:00 初始计划购电费用，C_d^{adj} 为当日已接受调整产生的净费用变化，C_d^{emer} 为实际紧急购电费用。被拒绝的候选方案以及次日辅助规划费用均不计入实际账单。日末将真实储电量传递至次日，并于次日 0:00 依据最新信息重新制定初始计划。对于年末跨至次年的辅助预测窗口，仅使用当前时点已经获得的信息，真实运行仍截至 2025 年 12 月 31 日。

8.7 模型求解与结果分析

本文采用前述动态调度模型，对 2025 年 2 月 1 日至 12 月 31 日共 334 天进行连续模拟。每日 0:00 制定初始购电计划，并在 6:00、12:00 和 18:00 根据最新光伏预报重新评价保留路径与调整路径；若调整路径评价更优，则更新当天尚未执行的购电计划，否则继续执行当前有效计划。实际运行阶段按分钟修正储能充放电行为，并将真实日末储电量连续传递至次日。当前模型沿用储能偏差惩罚系数 $\lambda_S = 0.5$。最终全年实际总购电费用为 13,610,394.79 元，其中紧急购电费用为 809,155.62 元，紧急购电量为 147,172.71 kWh；全年共接受日内计划调整 819 次，发生紧急购电 296 天。

为检验日内多时刻预报的实际价值，将上述滚动调度方案与“仅每日 0:00 制定初始计划、日内不再调整”的方案进行比较。结果如图 10 所示，滚动调度使实际总费用由 1474.63 万元降至 1361.04 万元，共节省 113.59 万元，降幅为 7.70%。费用分解表明，初始计划费用仅增加约 0.35 万元，日内调整产生约 44.45 万元的净调整成本，但紧急购电费用减少约 158.39 万元，其节约额明显高于调整成本。由此说明，利用 6:00、12:00 和 18:00 更新的光伏预报动态修正购电计划，能够以一定的调整成本显著减少高价紧急购电支出，具有实际经济价值。

求解所得完整购电策略见 result3。表 3 所列 4 个指定日期的购电量及储能充放电量完整结果分别见附录 D 表 15—表 22，正文仅列出紧急购电结果表 5。

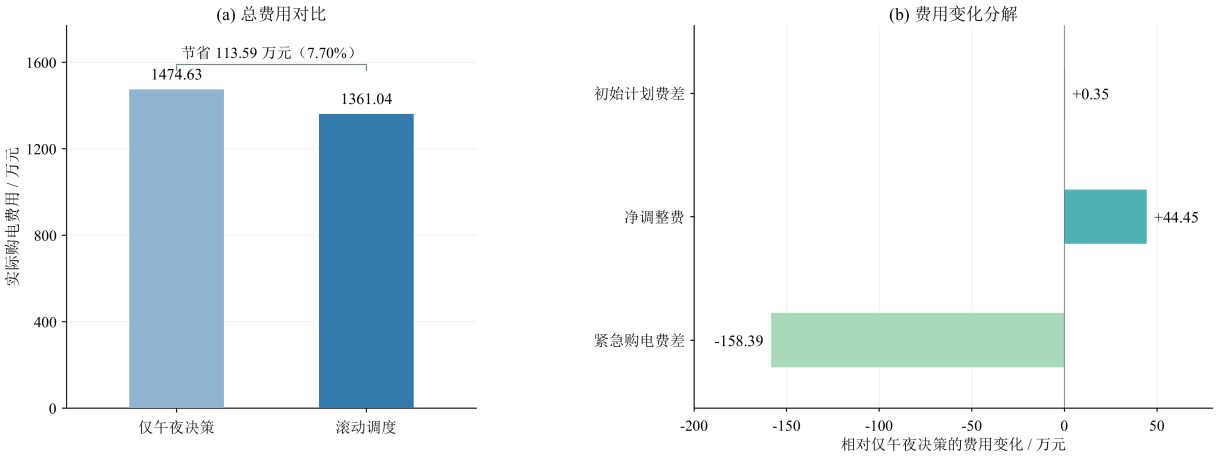


图 10: 方案修改前后对比

表 5: 微网在指定日期的紧急购电量

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh	时间段	购电量/kWh
05:59–06:02	0.867	06:57–07:27	141.721	05:59–06:16	19.030	07:57–08:08	21.584
06:10–06:13	4.890	—	—	09:10–09:57	212.090	18:50–18:51	0.000
09:00–10:07	687.824	—	—	18:50–18:52	0.332	19:54–20:06	15.644
18:53–19:08	10.067	—	—	19:30–19:33	1.349	20:38–20:50	138.779
20:06–20:10	2.697	—	—	19:59–20:33	127.452	—	—
20:20–20:21	0.137	—	—	20:39–20:40	67.203	—	—
—	—	—	—	20:58–21:47	161.962	—	—
合计	706.482	合计	141.721	合计	589.417	合计	176.007

九、问题四的建立与求解

9.1 模型总体框架

问题四在问题二和问题三的基础上进一步考虑外网电价随日期及日内时段波动的情形。由于未来实际电价在决策时点尚未实现，本文首先利用历史同星期、同时段电价信息构造日前电价预测，并结合周内价格变化规律设置分星期的日末储能参考水平，以引导储能在不同日期之间合理转移电量。在此基础上，问题四分别沿用问题二的负载预测、光伏预测、风险修正及日内实时校正机制，以及问题三的多时刻光伏预报、动态路径选择与订单调整机制，仅针对波动电价环境重新设计日前储能目标和费用评价方式，从而得到波动电价下对应问题二和问题三的购电调度策略。

9.2 电价预测与周阶梯日末储能参考

9.2.1 电价预测

在第 d 日制定购电计划时，未来实际电价尚未实现，因此不能直接作为优化输入。考虑电价具有明显的周周期特征，本文采用此前四周同星期、同一 10 分钟时段的实际电价均值作为第 d 日第 T 个时段的预测电价：

$$\hat{p}_{d,T} = \frac{1}{4}\sum_{k=1}^{4} p_{d-7k,T}. \tag{47}$$

其中，$p_{d-7k,T}$ 表示第 $d-7k$ 日第 T 个 10 分钟时段的实际电价，$\hat{p}_{d,T}$ 为当前决策所采用的预测电价。

该方法仅使用当前决策时点以前已经实现的历史价格信息，不读取当日及未来实际电价。规划阶段均采用预测电价进行决策，而实际费用则按照相应供电时段最终实现的真实电价进行结算。

为检验上述电价预测方法的有效性，选取典型日期比较预测电价与实际电价，结果如图 11 所示。可以看出，预测曲线能够较好跟随实际电价的日内峰谷变化，对主要价格跃迁及时段结构具有较好的刻画能力。该示例日预测的 MAE 为 0.0412 □/kWh，RMSE 为 0.0518 □/kWh，且大部分预测误差集中在 0 附近，表明基于历史同星期、同时段均价的预测方法能够较稳定地反映电价变化规律，可为后续日前购电优化提供有效的价格信息。

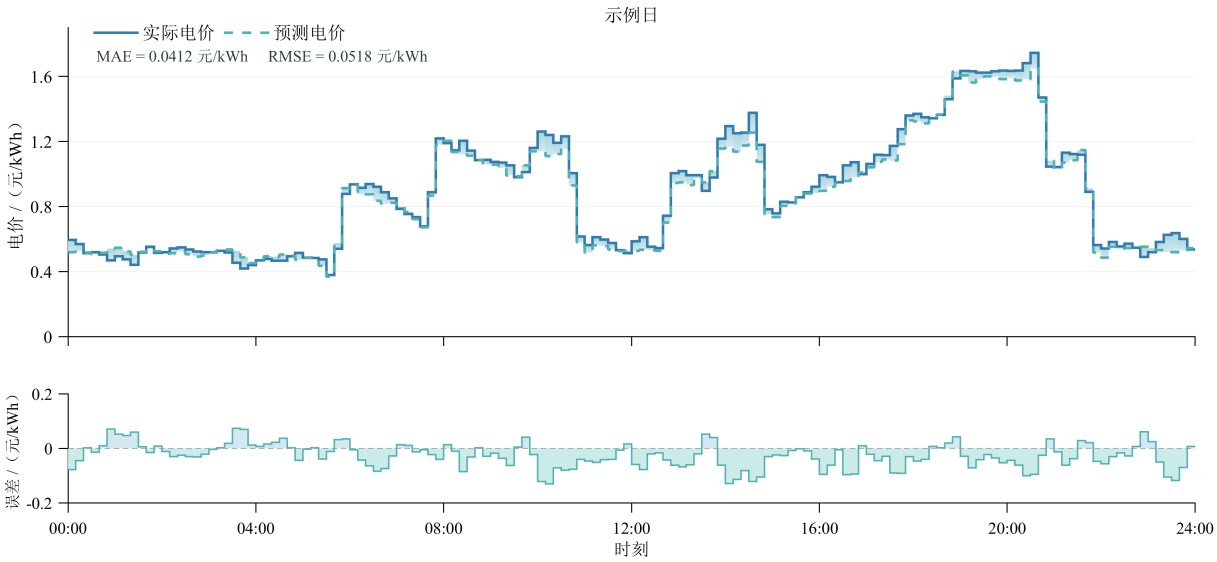


图 11: 电价预测图

9.2.2 周阶梯日末储能参考

由于储能设备可以实现电能的跨日转移，仅对单日购电费用进行优化可能无法充分利用周内电价差异。结合前述电价周期性分析，本文按照星期设置不同的日末储能参考值 B_d：

表 6: 不同星期的日末储能参考值

星期	周一	周二	周三	周四	周五	周六	周日
B_d/kWh	6960	5040	3120	1200	6000	10800	8880

9.3 波动电价下问题二的重构

问题四对应问题二时，负载预测、光伏预测、80% 分位风险裕度以及日内实时修正规则均沿用问题二，此处仅针对电价输入与跨日储能安排进行调整。由于未来实际电价在每日 0:00 尚未实现，日前规划采用第 3.2 节得到的预测电价 $\hat{p}_{d,T}$，而实际运行结束后的费用仍按附件 4 中的真实电价 $p_{d,T}$ 结算。

为利用周内电价差异，取消问题二中基于紧急购电动态上调的安全参考值及 $1.05C_d^*$ 费用上限，改为采用固定的周阶梯日末储能参考值 B_d。设计划日末储电量与参考值之间的绝对偏差为 A_d，有：

$$A_d \geq S_{d,1440}^{\text{plan}} - B_d, \quad A_d \geq B_d - S_{d,1440}^{\text{plan}}, \quad A_d \geq 0. \tag{48}$$

日前优化首先在原有物理约束下求解 $minA_d$，得到最小偏差 A_d^*，随后在 $A_d \leq A_d^* + 10^{-6}$ 的条件下进一步最小化预测购电费用 $min\hat{C}_d = \sum_{T=1}^{144} \hat{p}_{d,T} X_{d,T}$。因此，该模型形成“优先使日末储能接近周阶梯参考值，再在同等储能偏差下最小化预计购电费用”的分层优化结构。与问题二不同，本问不再设置首末储能相等条件，也不再采用 1.05 倍基准成本约束。计划确定后，当日各时段购电量保持锁定，实际运行仍按照问题二的分钟级修正规则处理真实负载与光伏偏差；日末实际储电量连续传递至次日。普通购电和紧急购电分别按照实际时段真实电价及其 5 倍进行结算，因此第 d 日实际总费用为计划购电费用与紧急购电费用之和。

9.4 波动电价下问题三的重构

问题四对应问题三时，负载预测、光伏动态融合预测、订单账本及分钟级实际运行规则均沿用问题三。主要变化在于：引入波动电价预测，并以周阶梯日末储能参考值重新设计 0:00 初始计划及日内路径评价机制。规划阶段采用第 9.2 节得到的预测电价 $\widehat{p}_{d,T}$，实际费用则按照供电时段最终实现的真实电价 $p_{d,T}$ 结算。每日 0:00，以当前真实储电量为初始状态，并以当日周阶梯参考值 B_d 为日末软目标。首先最小化日末储电量与 B_d 的偏差：

$$A_d \geq S_{d,1440}^{\text{plan}} - B_d, \quad A_d \geq B_d - S_{d,1440}^{\text{plan}}, \quad A_d \geq 0. \tag{49}$$

得到最小偏差 $A_d^{\min}$ 后，在 $A_d \leq A_d^{min}+10^{-6}$ 条件下进一步最小化预测购电费用 $min \sum_{T=1}^{144} \hat{p}_{d,T} X_{d,T}^0$。因此，本问不再采用问题三中“日初与日末储电量相等”的硬约束，而是通过周阶梯参考值引导储能进行跨日配置。

在 6:00、12:00 和 18:00 获得新预报后，仍分别构造保留路径与调整路径，但两条路径均以当日 24:00 的周阶梯参考值 B_d 作为评价目标。对于路径 $b \in \{\text{keep}, \text{change}\}$，定义

$$F_b = J_b + \lambda_S \left| S_{d,1440}^{b,\text{pred}} - B_d \right|. \tag{50}$$

其中最终取 $\lambda_S = 1$。仅当 $F_{\text{change}} < F_{\text{keep}} - 10^{-6}$ 时接受调整，否则继续执行当前有效计划。与问题三不同，本问日内候选方案仅优化至当天 24:00，次日辅助段不参与当天路径选择，而是在决策完成后单独计算，仅用于辅助评价，不反向改变当天调整结果。订单增加、取消及退款规则继续沿用问题三，预测阶段以 $\widehat{p}_{d,T}$ 计算调整成本，实际结算时则使用对应时段真实电价 $p_{d,T}$。由此形成适应实时波动电价的日内动态购电与储能调度策略。

9.5 模型求解与结果分析

在波动电价条件下，问题四对应问题二的实际总购电费用为 1472.04 万元，紧急购电费用为 96.45 万元，紧急购电量为 154528.26 kWh；对应问题三的实际总购电费用为 1429.90 万元，紧急购电费用为 84.04 万元，紧急购电量为 151219.83 kWh。从最终结果看，对应问题三的总费用较对应问题二减

少 42.14 万元，降幅约为 2.86%。但由于两者在预测方式、优化规则及期末储能状态等方面均存在差异，因此该差额反映的是两套完整调度策略的综合效果，不能全部归因于日内预报更新。求解所得具体购电策略见 result4-2.xlsx 和 result4-3.xlsx.

为进一步考察日内滚动调整本身的作用，本文在对应问题三模型内部，将滚动调度方案与仅在每日 0:00 制定计划、日内不再调整的方案进行对照。结果表明，滚动调度使实际总费用由 1545.89 万元降至 1429.90 万元，减少 115.99 万元，降幅为 7.50%；紧急购电量由 40.53 万 kWh 降至 15.12 万 kWh，减少 25.40 万 kWh，降幅达到 62.69%。由此可见，在外网电价与光伏预测均动态变化的条件下，利用日内更新信息及时修正尚未执行的购电计划，能够显著降低紧急购电需求及实际运行成本，体现了滚动调度机制在波动电价环境下的应用价值。

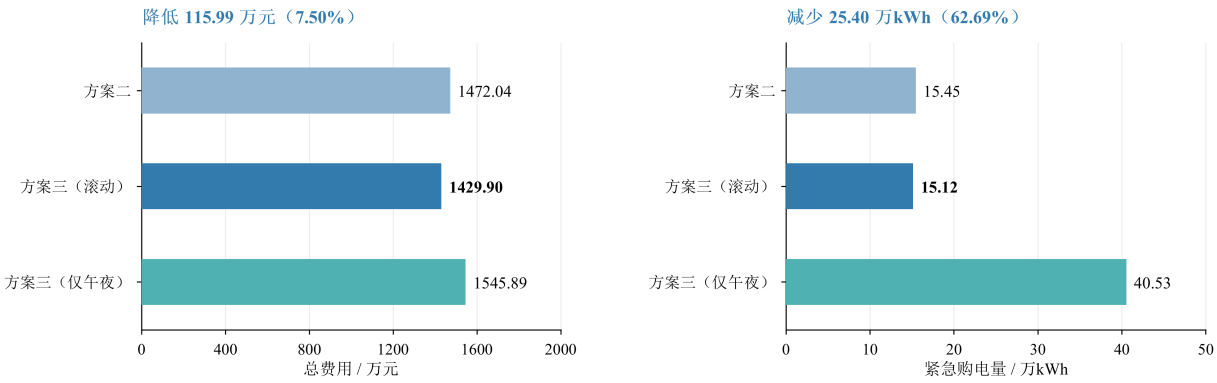


图 12: 波动电价下不同调度策略的费用与紧急购电量对比

十、模型的评价、改进与推广

10.1 模型的优点

优点 1：将预测误差显式纳入调度决策，增强模型对不确定性的适应能力

针对居民负载与光伏出力难以准确预知的问题，本文并未仅依赖单一点预测进行购电规划，而是结合历史数据刻画负载的周周期变化，并利用历史净负载预测误差构造风险裕度，将预测不确定性进一步传递至日前调度模型中。该方法能够在预测值与实际运行之间预留一定的供电缓冲空间，使模型不仅关注计划购电成本，还能够主动降低预测低估所引发的高价紧急购电风险，从而提高调度方案在随机供需环境下的稳健性。

优点 2：构建日前计划、日内调整与跨日状态衔接相结合的动态调度框架，具有较强的实际运行逻辑

本文将计划状态与实际运行状态进行区分：每日根据当前可获得的信息制定日前购电与储能计划，实际运行过程中再根据真实负载、光伏出力及储能状态逐分钟修正充放电行为；在获得新的光伏预报后，又通过比较保留路径与调整路径决定是否修改尚未执行的购电计划。同时，实际日末储电量连续传递至次日，使各日决策并非彼此独立。该设计能够较好模拟真实微网中“先制定计划、再根据新信息不断校正”的运行过程，避免采用全年实际信息进行一次性事后优化。

优点 3：兼顾经济性、储能安全与交易规则，同时保持模型较好的可解释性与求解效率

模型在考虑储能容量、充放电功率及效率等物理约束的基础上，通过分层优化协调购电费用与储能安全水平；对于日内计划调整，又进一步建立订单账本刻画追加购电、取消订单及退款成本。在波动电价条件下，本文利用历史电价的周周期特征设置周阶梯日末储能参考值，引导储能在不同日期之间合理转移电量。与此同时，各阶段的主要决策模型仍保持线性或凸分段线性结构，无需大量

引入整数变量，因而能够在保证模型可解释性的同时完成全年连续模拟，具有较好的计算可行性。

10.2 模型的缺点

缺点 1：部分关键参数仍具有一定经验性

本文在预测与调度过程中引入了风险分位水平、安全储能参考值调整系数、回落周期以及储能偏差惩罚系数等参数。这些参数能够使模型稳定运行并取得较好的仿真结果，但部分参数主要依据模型设计与有限范围内的数值实验确定，其在不同负载结构、光伏出力及电价环境下的最优性尚未得到充分验证。因此，当微网运行环境发生较大变化时，固定参数可能降低模型的适应能力。

缺点 2：预测模型与实时运行假设仍存在一定简化

本文主要依据历史周期规律预测负载、电价，并通过历史预报表现融合光伏预测，虽然具有较好的可解释性和计算效率，但对天气、节假日及突发负荷变化等外生因素考虑较少。此外，为进行分钟级仿真，模型对实际功率观测与储能控制过程进行了理想化离散处理，与真实微网中的测量延迟、通信误差及控制响应时间仍存在一定差异。因此，模型结果更多反映理想运行条件下的调度效果。

10.3 模型的改进

改进 1：引入数据驱动的参数自适应与不确定性优化方法

针对固定参数适应性不足的问题，可利用历史滚动样本对风险分位水平、储能安全参考值及目标函数权重进行动态更新，使参数能够随预测误差和运行环境变化自动调整。进一步地，可将负载、光伏及电价预测误差描述为概率分布或不确定集合，引入随机优化、鲁棒优化等方法，在控制运行风险的同时提高调度策略在不同场景下的稳定性。

改进 2：完善多源预测与实时控制机制

后续可进一步引入天气、节假日、温度等外生信息，并结合更灵活的时间序列或机器学习模型提高负载、光伏及电价预测能力。在实际运行层面，可采用滚动时域优化思想，根据最新观测和预测不断更新未来调度方案，使模型由理想化仿真逐步向实际微网在线调度场景拓展。

10.4 模型的推广

本文模型以“预测—优化—实时修正—跨日更新”为核心框架，各模块之间具有较强的独立性与可替换性。当负载规模、储能参数、电价机制或预测方法发生变化时，只需调整相应输入参数及局部决策模块，即可应用于工业园区、商业综合体、校园微网及充电站等含分布式新能源与储能设备的用能场景。此外，模型还可进一步扩展至多目标调度问题，在购电费用之外加入碳排放、储能寿命及供电可靠性等评价指标，并结合多区域微网之间的能量交换与更多不确定因素，形成更复杂的综合能源调度模型。

十一、AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、模型复核、程序完善与调试、图表绘制等辅助环节，详细使用情况见支撑材料。

十二、参考文献

[1] Jiang Y. Computation of monthly mean daily global solar radiation in China using artificial neural networks and comparison with other empirical models[J]. Energy, 2009, 34(9): 1276–1283.

[2] Parra D, Patel M K. Effect of tariffs on the performance and economic benefits of PV-coupled battery systems[J]. Applied Energy, 2016, 164: 175–187.

[3] Wu Z, Tazvinga H, Xia X. Demand side management of photovoltaic-battery hybrid system[J]. Applied Energy, 2015, 148: 294–304.

[4] Parisio A, Rikos E, Glielmo L. A model predictive control approach to microgrid operation optimization[J]. IEEE Transactions on Control Systems Technology, 2014, 22(5): 1813–1827.

附录

附录 A

文件名	功能描述
result1.xlsx	题目要求文件
result2.xlsx	题目要求文件
result3.xlsx	题目要求文件
result4-2.xlsx	题目要求文件
result4-3.xlsx	题目要求文件
problem1.py	问题一程序代码
problem2.py	问题二程序代码
problem3.py	问题三程序代码
problem4-2.py	问题四程序代码
problem4-3.py	问题四程序代码
AI 工具使用详情.pdf	题目要求文件
论文图表	论文所用图表文件夹

附录 B

附录 B. 问题一求解程序

Listing 1: 问题一求解程序

```python
# 问题一：先把 10 min 采样线性插值到分钟尺度，再用线性规划联合求解购电与储能调度。
from pathlib import Path
import argparse
```

```
from datetime import datetime,timedelta
import importlib.util
import linecache
import os
import sys
import types

ATTACHMENT_DIR = r""
OUTPUT_DIR = r""
TEMPLATE_DIR = r""
CASE = 'problem1'
CASE_NAME = '问题一'

os.environ.setdefault('OMP_NUM_THREADS','1')
os.environ.setdefault('OPENBLAS_NUM_THREADS','1')
_SCRIPT_PATH=Path(__file__).resolve()
_SCRIPT_DIR=_SCRIPT_PATH.parent

_MODULE_SOURCES = {
    'problem1.core': (
        'from __future__ import annotations\n'
        'from dataclasses import dataclass\n'
        'from datetime import time\n'
        'from typing import Any\n'
        'import numpy as np\n'
        'import openpyxl\n'
        'from scipy.optimize import linprog\n'
        'from scipy.sparse import lil_matrix\n'
        '\n'
        '\n'
        'MINUTES_PER_DAY = 1440\n'
        '\n'
        'SLOTS_PER_DAY = 144\n'
        '\n'
        'MINUTES_PER_SLOT = 10\n'
        '\n'
        'ETA_CHARGE = 0.9\n'
        '\n'
        'ETA_DISCHARGE = 0.9\n'
        '\n'
        'POWER_LIMIT_KW = 5000.0\n'
        '\n'
        'SOC_INITIAL_KWH = 6000.0\n'
        '\n'
        'SOC_MIN_KWH = 1200.0\n'
        '\n'
        'SOC_MAX_KWH = 10800.0\n'
        '\n'
        'TOL = 1e-07\n'
        '\n'
```

```
        '@dataclass(frozen=True)\n'
        'class InputData:\n'
        '    endpoint_minutes: np.ndarray\n'
        '    endpoint_labels: list[str]\n'
        '    price: np.ndarray\n'
        '    load_power: np.ndarray\n'
        '    pv_power: np.ndarray\n'
        '\n'
        '@dataclass(frozen=True)\n'
        'class MinuteData:\n'
        '    load_power: np.ndarray\n'
        '    pv_power: np.ndarray\n'
        '    load_energy: np.ndarray\n'
        '    pv_energy: np.ndarray\n'
        '\n'
        '@dataclass(frozen=True)\n'
        'class Solution:\n'
        '    purchase: np.ndarray\n'
        '    discharge_internal: np.ndarray\n'
        '    charge_internal: np.ndarray\n'
        '    curtailment: np.ndarray\n'
        '    soc: np.ndarray\n'
        '    objective: float\n'
        '    solver_message: str\n'
        '\n'
        'def _time_to_minutes(value: Any) -> int:\n'
        '    if isinstance(value, time):\n'
        '        return value.hour * 60 + value.minute\n'
        '    text = str(value).strip()\n'
        "    day_offset = 1440 if '+1' in text else 0\n"
        "    text = text.replace('+1', '')\n"
        "    hour, minute = (int(part) for part in text.split(':'))\n"
        '    return day_offset + hour * 60 + minute\n'
        '\n'
        'def _time_label(value: Any) -> str:\n'
        '    if isinstance(value, time):\n'
        "        return f'{value.hour:02d}:{value.minute:02d}'\n"
        '    return str(value).strip()\n'
        '\n'
        'def load_input(path: Path) -> InputData:\n'
        '    workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)\n'
        '    sheet = workbook.active\n'
        "    first_column = 2 if sheet.cell(1, 1).value == '序号' else 1\n"
        '    headers = [sheet.cell(1, column).value for column in range(first_column,
    first_column + 4)]\n'
        "    expected = ['时间', '电价', '小区负载', '光伏发电预测功率']\n"
        '    if headers != expected:\n'
        "        raise ValueError(f'附件1表头不匹配: {headers!r}')\n"
        '    rows = list(sheet.iter_rows(min_row=2, min_col=first_column, max_col=first_column
    + 3, values_only=True))\n'
```

```
        '    if len(rows) != SLOTS_PER_DAY:\n'
        "        raise ValueError(f'附件1应有144条数据，实际为{len(rows)}条')\n"
        '    endpoint_minutes = np.array([_time_to_minutes(row[0]) for row in rows], dtype=int)\n'
        '    expected_endpoints = np.arange(10, MINUTES_PER_DAY + 1, 10)\n'
        '    if not np.array_equal(endpoint_minutes, expected_endpoints):\n'
        "        raise ValueError('附件1时间列不是0:10、0:20、…、0:00+1的十分钟终点序列')\n"
        '    values = np.array([[float(row[i]) for i in range(1, 4)] for row in rows])\n'
        '    if not np.isfinite(values).all():\n'
        "        raise ValueError('附件1含空值或非有限数值')\n"
        '    price, load_power, pv_power = values.T\n'
        '    if np.any(price < 0) or np.any(load_power < 0) or np.any(pv_power < 0):\n'
        "        raise ValueError('电价、负载功率和光伏功率不得为负')\n"
        '    return InputData(endpoint_minutes=endpoint_minutes, endpoint_labels=[_time_label(row[0]) for row in rows], price=price, load_power=load_power, pv_power=pv_power)\n'
    '\n'
    'def build_minute_data(data: InputData) -> MinuteData:\n'
        '    sample_minutes = np.concatenate(([0], data.endpoint_minutes))\n'
        '    load_samples = np.concatenate(([data.load_power[-1]], data.load_power))\n'
        '    pv_samples = np.concatenate(([data.pv_power[-1]], data.pv_power))\n'
        '    minute_points = np.arange(MINUTES_PER_DAY + 1)\n'
        '    load_power = np.interp(minute_points, sample_minutes, load_samples)\n'
        '    pv_power = np.interp(minute_points, sample_minutes, pv_samples)\n'
        '    load_energy = (load_power[:-1] + load_power[1:]) / 120.0\n'
        '    pv_energy = (pv_power[:-1] + pv_power[1:]) / 120.0\n'
        '    return MinuteData(load_power, pv_power, load_energy, pv_energy)\n'
    '\n'
    'def solve(data: InputData, minute: MinuteData) -> Solution:\n'
        '    n_x = SLOTS_PER_DAY\n'
        '    n_y = n_z = n_w = MINUTES_PER_DAY\n'
        '    n_s = MINUTES_PER_DAY + 1\n'
        '    off_x = 0\n'
        '    off_y = off_x + n_x\n'
        '    off_z = off_y + n_y\n'
        '    off_w = off_z + n_z\n'
        '    off_s = off_w + n_w\n'
        '    n_vars = off_s + n_s\n'
        '    a_eq = lil_matrix((2 * MINUTES_PER_DAY, n_vars))\n'
        '    b_eq = np.zeros(2 * MINUTES_PER_DAY)\n'
        '    for minute_index in range(MINUTES_PER_DAY):\n'
        '        slot = minute_index // MINUTES_PER_SLOT\n'
        '        a_eq[minute_index, off_x + slot] = 1.0 / MINUTES_PER_SLOT\n'
        '        a_eq[minute_index, off_y + minute_index] = ETA_DISCHARGE\n'
        '        a_eq[minute_index, off_z + minute_index] = -1.0 / ETA_CHARGE\n'
        '        a_eq[minute_index, off_w + minute_index] = -1.0\n'
        '        b_eq[minute_index] = minute.load_energy[minute_index] - minute.pv_energy[minute_index]\n'
        '        state_row = MINUTES_PER_DAY + minute_index\n'
        '        a_eq[state_row, off_y + minute_index] = 1.0\n'
        '        a_eq[state_row, off_z + minute_index] = -1.0\n'
```

```
        '        a_eq[state_row, off_s + minute_index] = -1.0\n'
        '        a_eq[state_row, off_s + minute_index + 1] = 1.0\n'
        '    discharge_limit = POWER_LIMIT_KW / 60.0\n'
        '    charge_internal_limit = ETA_CHARGE * POWER_LIMIT_KW / 60.0\n'
        '    bounds: list[tuple[float, float | None]] = []\n'
        '    bounds.extend([(0.0, None)] * n_x)\n'
        '    bounds.extend([(0.0, discharge_limit)] * n_y)\n'
        '    bounds.extend([(0.0, charge_internal_limit)] * n_z)\n'
        '    bounds.extend([(0.0, float(value)) for value in minute.pv_energy])\n'
        '    bounds.extend([(SOC_MIN_KWH, SOC_MAX_KWH)] * n_s)\n'
        '    bounds[off_s] = (SOC_INITIAL_KWH, SOC_INITIAL_KWH)\n'
        '    bounds[off_s + MINUTES_PER_DAY] = (SOC_INITIAL_KWH, SOC_INITIAL_KWH)\n'
        '    objective = np.concatenate((data.price, np.zeros(n_vars - n_x)))\n'
        "    result = linprog(objective, A_eq=a_eq.tocsr(), b_eq=b_eq, bounds=bounds, method='highs')\n"
        '    if not result.success:\n'
        "        raise RuntimeError(f'问题一求解失败: {result.message}')\n"
        '    vector = result.x\n'
        '    return Solution(purchase=vector[off_x:off_y], discharge_internal=vector[off_y:off_z], charge_internal=vector[off_z:off_w], curtailment=vector[off_w:off_s], soc=vector[off_s:], objective=float(result.fun), solver_message=str(result.message))\n'
    '\n'
    'def validate(data: InputData, minute: MinuteData, solution: Solution) -> dict[str, Any]:\n'
        '    purchase_minute = np.repeat(solution.purchase / MINUTES_PER_SLOT, MINUTES_PER_SLOT)\n'
        '    balance = minute.pv_energy + purchase_minute + ETA_DISCHARGE * solution.discharge_internal - minute.load_energy - solution.charge_internal / ETA_CHARGE - solution.curtailment\n'
        '    state = solution.soc[1:] - solution.soc[:-1] + solution.discharge_internal - solution.charge_internal\n'
        '    simultaneous = (solution.discharge_internal > TOL) & (solution.charge_internal > TOL)\n'
        "    checks = {'max_balance_residual_kwh': float(np.max(np.abs(balance))), 'max_state_residual_kwh': float(np.max(np.abs(state))), 'soc_initial_kwh': float(solution.soc[0]), 'soc_terminal_kwh': float(solution.soc[-1]), 'soc_min_kwh': float(np.min(solution.soc)), 'soc_max_kwh': float(np.max(solution.soc)), 'max_discharge_internal_kwh_per_min': float(np.max(solution.discharge_internal)), 'max_charge_bus_kwh_per_min': float(np.max(solution.charge_internal / ETA_CHARGE)), 'max_curtailment_violation_kwh': float(np.max(np.maximum(solution.curtailment - minute.pv_energy, 0.0))), 'min_variable_kwh': float(min(np.min(solution.purchase), np.min(solution.discharge_internal), np.min(solution.charge_internal), np.min(solution.curtailment))), 'simultaneous_charge_discharge_minutes': int(np.sum(simultaneous)), 'objective_recalculated_yuan': float(np.dot(data.price, solution.purchase))}\n"
        "    checks['all_hard_constraints_pass'] = bool(checks['max_balance_residual_kwh'] <= TOL and checks['max_state_residual_kwh'] <= TOL and (abs(checks['soc_initial_kwh'] - SOC_INITIAL_KWH) <= TOL) and (abs(checks['soc_terminal_kwh'] - SOC_INITIAL_KWH) <= TOL) and (checks['soc_min_kwh'] >= SOC_MIN_KWH - TOL) and (checks['soc_max_kwh'] <= SOC_MAX_KWH + TOL) and (checks['max_discharge_internal_kwh_per_min'] <= POWER_LIMIT_KW / 60.0 + TOL) and (checks['max_charge_bus_kwh_per_min'] <= POWER_LIMIT_KW / 60.0 + TOL) and (checks['
```

```
    max_curtailment_violation_kwh'] <= TOL) and (checks['min_variable_kwh'] >= -TOL) and (abs(
    checks['objective_recalculated_yuan'] - solution.objective) <= TOL))\n"
        '    return checks\n'
    ),

    'output_common': (
        'import csv\n'
        'from pathlib import Path\n'
        'import numpy as np\n'
        '\n'
        'def write_csv(path, rows):\n'
        '    rows=list(rows)\n'
        '    if not rows:\n'
        '        return\n'
        "    with Path(path).open('w',encoding='utf-8-sig',newline='') as f:\n"
        '        w=csv.DictWriter(f,fieldnames=list(rows[0]))\n'
        '        w.writeheader(); w.writerows(rows)\n'
        '\n'
        'def save_case(out, arrays, daily):\n'
        '    out=Path(out); out.mkdir(parents=True,exist_ok=True)\n'
        "    np.savez_compressed(out/'all_results.npz',**arrays)\n"
        "    write_csv(out/'daily.csv',daily)\n"
    ),

    '_entry': (
        'import numpy as np\n'
        'from problem1 import core as model\n'
        'from output_common import save_case\n'
        '\n'
        'def run(out):\n'
        "    data=model.load_input(_DATA_DIR/'附件1.xlsx')\n"
        '    minute=model.build_minute_data(data)\n'
        '    sol=model.solve(data,minute)\n'
        '    checks=model.validate(data,minute,sol)\n'
        "    assert checks['all_hard_constraints_pass'], checks\n"
        '    arrays=dict(\n'
        "        dates=np.array(['2025-01-01']),\n"
        '        purchase_initial=sol.purchase[None,:],purchase_final=sol.purchase[None,:],\n'
        '        price=data.price[None,:],charge_internal=sol.charge_internal[None,:],\n'
        '        discharge_internal=sol.discharge_internal[None,:],soc=sol.soc[None,:],\n'
        '        emergency=np.zeros((1,1440)),surplus=sol.curtailment[None,:])\n'
        '    total=sol.objective\n'
        "    daily=[dict(date='2025-01-01',initial_cost=total,adjustment_cost=0.0,\n"
        '                emergency_cost=0.0,total_cost=total)]\n'
        '    save_case(out,arrays,daily)\n'
    ),

    'standalone_outputs': (
        'from __future__ import annotations\n'
        'import copy\n'
```

```
        'import csv\n'
        'import math\n'
        'import posixpath\n'
        'import re\n'
        'from datetime import date, timedelta\n'
        'from pathlib import Path\n'
        'from xml.etree import ElementTree as ET\n'
        'from xml.sax.saxutils import escape\n'
        'from zipfile import ZipFile, ZIP_DEFLATED\n'
        'import numpy as np\n'
        '\n'
        "S = 'http://schemas.openxmlformats.org/spreadsheetml/2006/main'\n"
        "R = 'http://schemas.openxmlformats.org/officeDocument/2006/relationships'\n"
        "P = 'http://schemas.openxmlformats.org/package/2006/relationships'\n"
        "SUPPLEMENT = '完整自然日购电量'\n"
        "FILES = {'problem1':'result1.xlsx','problem2':'result2.xlsx','problem3':'result3.xlsx','problem4-2':'result4-2.xlsx','problem4-3':'result4-3.xlsx'}\n"
        "ROW_RE = re.compile('<row\\\\b[^>]*?(?:/>|>.*?</row>)', re.S)\n"
        "CELL_RE = re.compile('<c\\\\b[^>]*?(?:/>|>.*?</c>)', re.S)\n"
        '\n'
        '\n'
        'def colname(number: int) -> str:\n'
        "    result = ''\n"
        '    while number:\n'
        '        number, rem = divmod(number - 1, 26)\n'
        '        result = chr(65 + rem) + result\n'
        '    return result\n'
        '\n'
        'def colnumber(name: str) -> int:\n'
        '    result = 0\n'
        '    for char in name:\n'
        '        result = 26 * result + ord(char) - 64\n'
        '    return result\n'
        '\n'
        'def address_key(ref: str) -> tuple[int, int]:\n'
        "    match = re.fullmatch('([A-Z]+)([0-9]+)', ref)\n"
        '    if match is None:\n'
        "        raise ValueError(f'Invalid cell reference {ref}')\n"
        '    return (int(match[2]), colnumber(match[1]))\n'
        '\n'
        'def attr(fragment: str, name: str, default: str | None=None) -> str | None:\n'
        '    match = re.search(\'\\\\b\' + re.escape(name) + \'="([^"]*)"\', fragment)\n'
        '    return match[1] if match else default\n'
        '\n'
        'def scalar_cell(ref: str, value, original: str | None=None, style: str | None=None) -> str:\n'
        '    """Replace only a cell value; retain its original attributes except type."""\n'
        '    if original:\n'
        "        opening = re.match('<c\\\\b([^>]*?)(?:/?>)', original)[1]\n"
        '        opening = re.sub(\'\\\\s+t="[^"]*"\', \'\', opening).rstrip(\'/ \')\n'
```

```
        '        opening = re.sub(\'\\\\br="[^"]*"\', f\'r="{ref}"\', opening)\n'
        '    else:\n'
        '        opening = f\' r="{ref}"\' + (f\' s="{style}"\' if style is not None else \'\')\n'
        '    if value is None:\n'
        "        return f'<c{opening}/>'\n"
        '    if isinstance(value, (str, np.str_)):\n'
        '        return f\'<c{opening} t="inlineStr"><is><t xml:space="preserve">{escape(str(value))}</t></is></c>\'\n'
        '    numeric = float(value)\n'
        '    if not math.isfinite(numeric):\n'
        "        raise ValueError(f'Non-finite numeric output at {ref}: {value}')\n"
        '    if abs(numeric) < 1e-10:\n'
        '        numeric = 0.0\n'
        '    return f"<c{opening}><v>{format(numeric, \'.15g\')}</v></c>"\n'
        '\n'
        'def rows_in(xml: str) -> dict[int, str]:\n'
        "    data = re.search('<sheetData>(.*?)</sheetData>', xml, re.S)\n"
        '    if data is None:\n'
        "        raise ValueError('Template has no sheetData element')\n"
        "    return {int(attr(row.split('>', 1)[0], 'r')): row for row in ROW_RE.findall(data[1])}\n"
        '\n'
        'def patch_row(row: str, updates: dict[str, object], new_number: int | None=None) -> str:\n'
        "    opening = re.match('<row\\\\b([^>]*?)(?:/?>)', row)[1].rstrip('/ ')\n"
        "    old_number = int(attr(opening, 'r'))\n"
        '    new_number = old_number if new_number is None else new_number\n'
        '    opening = re.sub(\'\\\\br="[^"]*"\', f\'r="{new_number}"\', opening)\n'
        '    cells = {}\n'
        '    for cell in CELL_RE.findall(row):\n'
        "        ref = attr(cell.split('>', 1)[0], 'r')\n"
        "        column = re.match('[A-Z]+', ref)[0]\n"
        "        new_ref = f'{column}{new_number}'\n"
        '        cells[new_ref] = re.sub(\'\\\\br="[^"]*"\', f\'r="{new_ref}"\', cell, count=1)\n'
        "    default_style = attr(opening, 's')\n"
        '    for column_or_ref, value in updates.items():\n'
        "        column = re.match('[A-Z]+', column_or_ref)[0]\n"
        "        ref = f'{column}{new_number}'\n"
        '        cells[ref] = scalar_cell(ref, value, cells.get(ref), default_style)\n'
        "    return f'<row{opening}>' + ''.join((cells[k] for k in sorted(cells, key=address_key))) + '</row>'\n"
        '\n'
        'def replace_rows(xml: str, rows: list[str], last_cell: str | None=None) -> str:\n'
        "    xml = re.sub('<sheetData>.*?</sheetData>', lambda _: '<sheetData>' + ''.join(rows) + '</sheetData>', xml, count=1, flags=re.S)\n"
        '    if last_cell:\n'
        '        xml = re.sub(\'<dimension\\\\b[^>]*/>\', f\'<dimension ref="A1:{last_cell}"/>\', xml, count=1)\n'
```

```
        '    return xml\n'
        '\n'
        'def excel_date(day: date) -> int:\n'
        '    return (day - date(1899, 12, 30)).days\n'
        '\n'
        'def clock(minutes: int) -> str:\n'
        "    return f'{minutes // 60:02d}:{minutes % 60:02d}'\n"
        '\n'
        'def intervals(energy: np.ndarray, tolerance: float=1e-08) -> list[tuple[int, int, float]]:\n'
        '    positive = np.asarray(energy) > tolerance\n'
        '    padded = np.r_[False, positive, False].astype(np.int8)\n'
        '    starts = np.flatnonzero(np.diff(padded) == 1)\n'
        '    ends = np.flatnonzero(np.diff(padded) == -1)\n'
        '    return [(int(a), int(b), float(np.sum(energy[a:b]))) for a, b in zip(starts, ends)]\n'
        '\n'
        'def load_results(folder: Path, case: str) -> dict:\n'
        '    folder = Path(folder).resolve()\n'
        "    with np.load(folder / 'all_results.npz', allow_pickle=False) as source:\n"
        '        data = {key: np.array(source[key]) for key in source.files}\n'
        "    required = {'dates', 'purchase_initial', 'purchase_final', 'price', 'charge_internal', 'discharge_internal', 'emergency', 'surplus', 'soc'}\n"
        '    if required - data.keys():\n'
        "        raise ValueError(f'{case}: missing arrays {sorted(required - data.keys())}')\n"
        "    dates = [date.fromisoformat(str(value)[:10]) for value in data['dates']]\n"
        '    n = len(dates)\n'
        "    if case == 'problem1':\n"
        '        if n != 1:\n'
        "            raise ValueError('problem1 must contain exactly one modeled day')\n"
        '    elif dates != [date(2025, 2, 1) + timedelta(days=i) for i in range(334)]:\n'
        "        raise ValueError(f'{case}: official export requires all 334 dates from February 1 to December 31')\n"
        "    for key in required - {'dates'}:\n"
        "        width = 1441 if key == 'soc' else 144 if key in {'purchase_initial', 'purchase_final', 'price'} else 1440\n"
        '        if data[key].shape != (n, width) or not np.all(np.isfinite(data[key])):\n'
        "            raise ValueError(f'{case}: invalid {key} shape or nonfinite values: {data[key].shape}')\n"
        "    data['dates'] = dates\n"
        "    daily_path = next((p for p in [folder / 'daily.csv', folder / 'summary' / 'daily.csv'] if p.exists()), None)\n"
        '    if daily_path is None:\n'
        "        raise FileNotFoundError(f'Missing daily settlement CSV under {folder}')\n"
        "    with daily_path.open(encoding='utf-8-sig', newline='') as stream:\n"
        '        records = list(csv.DictReader(stream))\n'
        '    if not records:\n'
        "        raise ValueError('Daily settlement CSV has no rows')\n"
        "    date_key = next((key for key in ['date', '日期', 'day'] if key in records[0]), None
```

```
    )\n"
    '    if date_key is None:\n'
    "        raise ValueError(f'{daily_path}: need date column')\n"
    '    by_date = {date.fromisoformat(row[date_key][:10]): row for row in records}\n'
    '    if len(by_date) != len(records) or set(by_date) != set(dates):\n'
    "        raise ValueError('Daily CSV dates must match NPZ dates exactly without duplicates')\n"
    "    aliases = {'initial_cost': ['initial_cost', 'initial_purchase_cost', 'initial_plan_cost', 'planned_cost', 'plan_cost', 'purchase_cost'], 'adjustment_cost': ['adjustment_cost', 'adjustment_net_cost', 'adjustment_net', 'adjust_cost'], 'emergency_cost': ['emergency_cost', 'emergency_purchase_cost'], 'total_cost': ['total_cost', 'actual_total_cost', 'actual_cost', 'cost']}\n"
    '    daily = []\n'
    '    for i, day in enumerate(dates):\n'
    '        if day not in by_date:\n'
    "            raise ValueError(f'{daily_path}: missing {day}')\n"
    '        row = by_date[day]\n'
    '        values = {}\n'
    '        for key, names in aliases.items():\n'
    "            found = next((row[name] for name in names if name in row and row[name] != ''), None)\n"
    '            if found is None:\n'
    "                if key == 'adjustment_cost' and case in {'problem1', 'problem2', 'problem4-2'}:\n"
    '                    values[key] = 0.0\n'
    '                else:\n'
    "                    raise ValueError(f'{daily_path}: need {key} for {day}; available columns={list(row)}')\n"
    '            else:\n'
    '                values[key] = float(found)\n'
    '        if not all((math.isfinite(v) for v in values.values())):\n'
    "            raise ValueError(f'Nonfinite daily cost for {day}')\n"
    "        computed_total = values['initial_cost'] + values['adjustment_cost'] + values['emergency_cost']\n"
    "        if not math.isclose(computed_total, values['total_cost'], abs_tol=0.01, rel_tol=1e-09):\n"
    "            raise ValueError(f'Daily settlement does not reconcile for {case}/{day}: {values}')\n"
    "        actual_emergency = float(np.dot(data['emergency'][i], np.repeat(data['price'][i], 10)) * 5)\n"
    "        if not math.isclose(actual_emergency, values['emergency_cost'], abs_tol=0.01, rel_tol=1e-08):\n"
    "            raise ValueError(f'Emergency cost disagrees with minute trace for {case}/{day}')\n"
    "        actual_initial = float(np.dot(data['purchase_initial'][i], data['price'][i]))\n"
    "        if not math.isclose(actual_initial, values['initial_cost'], abs_tol=0.01, rel_tol=1e-08):\n"
    "            raise ValueError(f'Initial plan cost disagrees with 144-slot trace for {case}/{day}')\n"
```

```
        '        daily.append(values)\n'
        "    data['daily'] = daily\n"
        "    data['daily_csv'] = daily_path.relative_to(folder).as_posix()\n"
        '    return data\n'
        '\n'
        'def sheet_paths(parts: dict[str, bytes]) -> dict[str, str]:\n'
        "    workbook = ET.fromstring(parts['xl/workbook.xml'])\n"
        "    rels = {r.attrib['Id']: r.attrib['Target'] for r in ET.fromstring(parts['xl/_rels/workbook.xml.rels'])}\n"
        "    return {s.attrib['name']: posixpath.normpath('xl/' + rels[s.attrib[f'{{{R}}}id']]) for s in workbook.find(f'{{{S}}}sheets')}\n"
        '\n'
        'def write_purchase(xml: str, data: dict, *, final: bool, single: bool) -> str:\n'
        '    rows = rows_in(xml)\n'
        "    array = data['purchase_final' if final else 'purchase_initial']\n"
        '    if single:\n'
        '        for row in range(2, 145):\n'
        "            rows[row] = patch_row(rows[row], {'B': array[0, row - 1]})\n"
        "        rows[145] = patch_row(rows[145], {'B': None})\n"
        '    else:\n'
        "        for i, day in enumerate(data['dates']):\n"
        '            row = i + 2\n'
        '            updates = {colname(j + 1): array[i, j] for j in range(1, 144)}\n'
        "            updates['EO'] = array[i + 1, 0] if i + 1 < len(array) else None\n"
        "            updates['EP'] = float(array[i].sum())\n"
        "            costs = data['daily'][i]\n"
        "            updates['EQ'] = costs['initial_cost'] + (costs['adjustment_cost'] if final else 0)\n"
        '            rows[row] = patch_row(rows[row], updates)\n'
        '    return replace_rows(xml, [rows[r] for r in sorted(rows)])\n'
        '\n'
        'def write_storage(xml: str, data: dict, *, single: bool) -> str:\n'
        '    original = rows_in(xml)\n'
        '    eta = 0.9\n'
        '    if single:\n'
        '        for block in range(6):\n'
        '            take = slice(240 * block, 240 * (block + 1))\n'
        "            updates = {'B': float(data['charge_internal'][0, take].sum() / eta), 'C': float(data['discharge_internal'][0, take].sum() * eta)}\n"
        '            if block < 2:\n'
        "                updates['E'] = data['soc'][0, 0 if block == 0 else -1]\n"
        '            original[block + 2] = patch_row(original[block + 2], updates)\n'
        '        return replace_rows(xml, [original[r] for r in sorted(original)])\n'
        '    output = [original[1]]\n'
        "    for i, day in enumerate(data['dates']):\n"
        '        for block in range(6):\n'
        '            row = 2 + 6 * i + block\n'
        '            take = slice(240 * block, 240 * (block + 1))\n'
        "            updates = {'A': excel_date(day) if block == 0 else None, 'C': float(data['charge_internal'][i, take].sum() / eta), 'D': float(data['discharge_internal'][i, take].sum
```

```
() * eta), 'F': data['soc'][i, 0 if block == 0 else -1] if block < 2 else None}\n"
    '            output.append(patch_row(original[block + 2], updates, row))\n'
    '    return replace_rows(xml, output, f"F{1 + 6 * len(data[\'dates\'])}")\n'
    '\n'
    'def write_emergency(xml: str, data: dict) -> tuple[str, int]:\n'
    '    original = rows_in(xml)\n'
    '    output = [original[1]]\n'
    '    count = 0\n'
    "    for i, day in enumerate(data['dates']):\n"
    "        events = intervals(data['emergency'][i])\n"
    '        count += len(events)\n'
    "        records = [(f'{clock(a)}-{clock(b)}', energy) for a, b, energy in events] or [('无', 0.0)]\n"
    '        for j, (label, energy) in enumerate(records):\n'
    '            prototype = original[2 if j == 0 else 4 if j == len(records) - 1 else 3]\n'
    "            output.append(patch_row(prototype, {'A': excel_date(day) if j == 0 else None, 'B': label, 'C': energy}, len(output) + 1))\n"
    "    return (replace_rows(xml, output, f'C{len(output)}'), count)\n"
    '\n'
    'def supplement_xml(parts: dict[str, bytes], data: dict, case: str) -> str:\n'
    '    """A compact additional matrix; preserve all original styles.xml bytes."""\n'
    '    if parts:\n'
    "        source = rows_in(parts[sheet_paths(parts)['计划购电量']].decode('utf-8'))\n"
    "        header_style = attr(CELL_RE.findall(source[1])[1].split('>', 1)[0], 's', '0')\n"
    "        value_style = attr(CELL_RE.findall(source[2])[-1].split('>', 1)[0], 's', '0')\n"
    '    else:\n'
    "        header_style, value_style = ('1', '2')\n"
    "    headings = ['日期', '计划类型'] + [f'{clock(10 * k)}-{clock(10 * (k + 1))}' for k in range(144)]\n"
    "    headings += ['全天购电量(kWh)', '初始计划款(元)', '调整净费用(元)', '紧急购电费(元)', '总购电费(元)', '紧急购电量(kWh)']\n"
    '    rows = [\'<row r="1" ht="28" customHeight="1">\' + \'\'.join((scalar_cell(f\'{colname(c)}1\', v, style=header_style) for c, v in enumerate(headings, 1))) + \'</row>\']\n'
    "    stage_rows = [('初始计划', 'purchase_initial'), ('最终有效计划', 'purchase_final')] if case in {'problem3', 'problem4-3'} else [('计划购电', 'purchase_initial')]\n"
    "    for i, day in enumerate(data['dates']):\n"
    "        costs = data['daily'][i]\n"
    '        for stage, key in stage_rows:\n'
    "            settlement = stage != '初始计划'\n"
    "            display_day = '附件1示例日' if case == 'problem1' else day.isoformat()\n"
    '            values = [display_day, stage] + data[key][i].tolist() + [float(data[key][i].sum())]\n'
    "            values += [costs['initial_cost'], costs['adjustment_cost'] if settlement else None, costs['emergency_cost'] if settlement else None, costs['total_cost'] if settlement else None, float(data['emergency'][i].sum()) if settlement else None]\n"
    '            r = len(rows) + 1\n'
```

```
    '            rows.append(f\'<row r="{r}" ht="20" customHeight="1">\' + \'\'.join((scalar_cell(f\'{colname(c)}{r}\', value, style=value_style) for c, value in enumerate(values, 1))) + \'</row>\')\n'
    '    return f\'<?xml version="1.0" encoding="UTF-8" standalone="yes"?><worksheet xmlns="{S}"><dimension ref="A1:EV{len(rows)}"/><sheetViews><sheetView showGridLines="0" workbookViewId="0"><pane xSplit="2" ySplit="1" topLeftCell="C2" activePane="bottomRight" state="frozen"/></sheetView></sheetViews><sheetFormatPr defaultRowHeight="20"/><cols><col min="1" max="2" width="18" customWidth="1"/><col min="3" max="146" width="16" customWidth="1"/><col min="147" max="152" width="22" customWidth="1"/></cols><sheetData>\' + \'\'.join(rows) + \'</sheetData><pageMargins left="0.3" right="0.3" top="0.5" bottom="0.5" header="0.2" footer="0.2"/></worksheet>\'\n'
    '\n'
    'def _display_day(day: date, case: str) -> str:\n'
    "    return '附件1示例日' if case == 'problem1' else day.isoformat()\n"
    '\n'
    'def _plain_sheet(headers: list[str], values: list[list], widths=None) -> str:\n'
    '    """Write an explicitly new sheet without any external authoring library."""\n'
    '    rows = []\n'
    '    for number, row in enumerate([headers] + values, 1):\n'
    "        style = '1' if number == 1 else '2'\n"
    "        cells = ''.join((scalar_cell(f'{colname(c)}{number}', value, style=style) for c, value in enumerate(row, 1)))\n"
    '        rows.append(f\'<row r="{number}" ht="30" customHeight="1">{cells}</row>\')\n'
    '    columns = \'\'.join((f\'<col min="{i}" max="{i}" width="{width}" customWidth="1"/>\' for i, width in enumerate(widths or [22] * len(headers), 1)))\n'
    '    return f\'\'\'<?xml version="1.0" encoding="UTF-8" standalone="yes"?><worksheet xmlns="{S}"><dimension ref="A1:{colname(len(headers))}{len(rows)}"/><sheetViews><sheetView showGridLines="0" workbookViewId="0"><pane ySplit="1" topLeftCell="A2" activePane="bottomLeft" state="frozen"/></sheetView></sheetViews><sheetFormatPr defaultRowHeight="30"/><cols>{columns}</cols><sheetData>{\'\'.join(rows)}</sheetData><pageMargins left="0.3" right="0.3" top="0.5" bottom="0.5" header="0.2" footer="0.2"/></worksheet>\'\'\'\n'
    '\n'
    'def _new_workbook_parts(sheets: list[tuple[str, str]]) -> dict[str, bytes]:\n'
    '    types = [\'<Default Extension="rels" ContentType="application/vnd.openxmlformats-package.relationships+xml"/>\', \'<Default Extension="xml" ContentType="application/xml"/>\', \'<Override PartName="/xl/workbook.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet.main+xml"/>\', \'<Override PartName="/xl/styles.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.styles+xml"/>\']\n'
    '    relations, entries = ([], [])\n'
    '    parts = {}\n'
    '    for index, (name, xml) in enumerate(sheets, 1):\n'
    "        member = f'xl/worksheets/sheet{index}.xml'\n"
    "        parts[member] = xml.encode('utf-8')\n"
    '        types.append(f\'<Override PartName="/{member}" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.worksheet+xml"/>\')\n'
    '        relations.append(f\'<Relationship Id="rId{index}" Type="{R}/worksheet" Target="worksheets/sheet{index}.xml"/>\')\n'
    '        entries.append(f\'<sheet name="{escape(name)}" sheetId="{index}" r:id="rId{index}"/>\')\n'
```

```
    '    relations.append(f\'<Relationship Id="rId{len(sheets) + 1}" Type="{R}/styles" Target="styles.xml"/>\')\n'
    '    parts[\'[Content_Types].xml\'] = f"""<Types xmlns="http://schemas.openxmlformats.org/package/2006/content-types">{\'\'.join(types)}</Types>""".encode()\n'
    '    parts[\'_rels/.rels\'] = f\'<Relationships xmlns="{P}"><Relationship Id="rId1" Type="{R}/officeDocument" Target="xl/workbook.xml"/></Relationships>\'.encode()\n'
    '    parts[\'xl/_rels/workbook.xml.rels\'] = f\'\'\'<Relationships xmlns="{P}">{\'\'.join(relations)}</Relationships>\'\'\'.encode()\n'
    '    parts[\'xl/workbook.xml\'] = f\'\'\'<workbook xmlns="{S}" xmlns:r="{R}"><bookViews><workbookView/></bookViews><sheets>{\'\'.join(entries)}</sheets></workbook>\'\'\'.encode()\n'
    '    parts[\'xl/styles.xml\'] = f\'<styleSheet xmlns="{S}"><fonts count="2"><font><sz val="11"/><name val="Microsoft YaHei"/></font><font><b/><sz val="11"/><color rgb="FFFFFFFF"/><name val="Microsoft YaHei"/></font></fonts><fills count="3"><fill><patternFill patternType="none"/></fill><fill><patternFill patternType="gray125"/></fill><fill><patternFill patternType="solid"><fgColor rgb="FF245A81"/><bgColor indexed="64"/></patternFill></fill></fills><borders count="1"><border/></borders><cellStyleXfs count="1"><xf numFmtId="0" fontId="0" fillId="0" borderId="0"/></cellStyleXfs><cellXfs count="3"><xf numFmtId="0" fontId="0" fillId="0" borderId="0" xfId="0"/><xf numFmtId="0" fontId="1" fillId="2" borderId="0" xfId="0" applyAlignment="1"><alignment vertical="center" wrapText="1"/></xf><xf numFmtId="0" fontId="0" fillId="0" borderId="0" xfId="0" applyAlignment="1"><alignment vertical="center" wrapText="1"/></xf></cellXfs><cellStyles count="1"><cellStyle name="Normal" xfId="0" builtinId="0"/></cellStyles></styleSheet>\'.encode()\n'
    '    return parts\n'
    '\n'
    'def _standalone_parts(data: dict, case: str) -> tuple[dict[str, bytes], int]:\n'
    '    storage, emergency, settlement = ([], [], [])\n'
    '    event_count = 0\n'
    "    for i, day in enumerate(data['dates']):\n"
    '        label = _display_day(day, case)\n'
    '        for block in range(6):\n'
    '            a, b = (240 * block, 240 * (block + 1))\n'
    "            storage.append([label, f'{clock(a)}-{clock(b)}', float(data['charge_internal'][i, a:b].sum() / 0.9), float(data['discharge_internal'][i, a:b].sum() * 0.9), float(data['soc'][i, a]), float(data['soc'][i, b])])\n"
    "        events = intervals(data['emergency'][i])\n"
    '        event_count += len(events)\n'
    "        emergency.extend([[label, f'{clock(a)}-{clock(b)}', energy] for a, b, energy in events] or [[label, '无', 0.0]])\n"
    "        costs = data['daily'][i]\n"
    "        settlement.append([label, float(data['purchase_initial'][i].sum()), float(data['purchase_final'][i].sum()), float(data['charge_internal'][i].sum() / 0.9), float(data['discharge_internal'][i].sum() * 0.9), float(data['emergency'][i].sum()), float(data['surplus'][i].sum()), float(data['soc'][i, 0]), float(data['soc'][i, -1]), costs['initial_cost'], costs['adjustment_cost'], costs['emergency_cost'], costs['total_cost']])\n"
    "    notes = [['文件来源', '未找到附件5对应模板；本文件为程序独立生成的完整结果表，不声称保留官方模板结构。'], ['问题', case], ['计算天数', len(data['dates'])], ['计算范围', '附件1未标日期的示例日' if case == 'problem1' else '2025-02-01至2025-12-31，共334个自然日'], ['购电区间', '完整自然日表逐项列出00:00-00:10至23:50-24:00的144段，保留初始及最终计划。'], ['充放电口径', '购电侧充电量=内部储能增量/0.9；供负载侧放电量=内部储能减少量×0.9；每4小时汇总。'], ['紧急购
```

```
    电', '按实际连续分钟区间逐事件列出；无事件的日期以‘无’和0列示，事件数不设上限。'], ['费用口径
    ', '总购电费=初始计划款+已接受调整净费用+紧急购电费；紧急单价为该分钟对应电价的5倍。'], ['数据
    来源', '同目录all_results.npz及daily.csv；数值保留为数值单元格。'], ['日期说明', 'Problem1显示
    ‘附件1示例日’，NPZ/CSV日期仅作内部关联键；其他问题显示真实2025年日期。']]\n"
516     "    sheets = [('导出说明', _plain_sheet(['项目', '说明'], notes, [22, 115])), (
    SUPPLEMENT, supplement_xml({}, data, case)), ('充放电量', _plain_sheet(['日期', '时段', '充
    电购电侧(kWh)', '放电供负载侧(kWh)', '时段起始SOC(kWh)', '时段结束SOC(kWh)'], storage)), ('紧
    急购电量', _plain_sheet(['日期', '实际连续分钟时段', '紧急购电量(kWh)'], emergency)), ('日电
    量与费用', _plain_sheet(['日期', '初始计划购电量(kWh)', '最终计划购电量(kWh)', '充电购电侧(kWh
    )', '放电供负载侧(kWh)', '紧急购电量(kWh)', '未利用电量(kWh)', '日初SOC(kWh)', '日末SOC(kWh)
    ', '初始计划款(元)', '调整净费用(元)', '紧急购电费(元)', '总购电费(元)'], settlement))]\n"
517     '    return (_new_workbook_parts(sheets), event_count)\n'
518     '\n'
519     'def beautify_excel(path: Path) -> None:\n'
520     '    """Apply the final workbook appearance without changing calculated values.\n'
521     '\n'
522     '    - Every non-empty cell in every sheet uses 宋体 10 pt.\n'
523     '    - Every non-empty cell is horizontally and vertically centered.\n'
524     '    - The last row of “紧急购电量” gets a black bottom border across A:C.\n'
525     '    Existing bold/italic/color/fill/number-format settings and all other border sides
    are preserved.\n'
526     '    """\n'
527     '    import openpyxl\n'
528     '    from copy import copy\n'
529     '    from openpyxl.styles import Side\n'
530     '\n'
531     '    workbook = openpyxl.load_workbook(path)\n'
532     '    try:\n'
533     '        for sheet in workbook.worksheets:\n'
534     '            for row in sheet.iter_rows():\n'
535     '                for cell in row:\n'
536     '                    if cell.value is None:\n'
537     '                        continue\n'
538     '\n'
539     '                    font = copy(cell.font)\n'
540     "                    font.name = '宋体'\n"
541     '                    font.sz = 10\n'
542     '                    font.scheme = None\n'
543     '                    cell.font = font\n'
544     '\n'
545     '                    alignment = copy(cell.alignment)\n'
546     "                    alignment.horizontal = 'center'\n"
547     "                    alignment.vertical = 'center'\n"
548     '                    cell.alignment = alignment\n'
549     '\n'
550     "        if '紧急购电量' in workbook.sheetnames:\n"
551     "            sheet = workbook['紧急购电量']\n"
552     '            last_row = sheet.max_row\n'
553     "            black = Side(style='thin', color='FF000000')\n"
554     '            for cell in sheet[last_row][:3]:\n'
```

```
        '                border = copy(cell.border)\n'
        '                border.bottom = black\n'
        '                cell.border = border\n'
        '\n'
        '        workbook.save(path)\n'
        '    finally:\n'
        '        workbook.close()\n'
        '\n'
        '\n'
        'def export_one(case, problem_output_dir, attachment_dir, template_dir=None):\n'
        '    folder = Path(problem_output_dir).resolve()\n'
        '    attachments = Path(attachment_dir).resolve()\n'
        "    templates = Path(template_dir).resolve() if template_dir is not None else attachments / '附件5'\n"
        '    source = templates / FILES[case]\n'
        '    destination = folder / FILES[case]\n'
        '    data = load_results(folder, case)\n'
        "    temporary = destination.with_suffix('.pending.xlsx')\n"
        '\n'
        '    if source.is_file():\n'
        '        with ZipFile(source) as archive:\n'
        '            infos = {info.filename: copy.copy(info) for info in archive.infolist()}\n'
        '            parts = {name: archive.read(name) for name in infos}\n'
        '        paths = sheet_paths(parts)\n'
        "        required = {'计划购电量', '充放电量'} | ({'紧急购电量'} if case != 'problem1' else set())\n"
        "        if case in {'problem3', 'problem4-3'}:\n"
        "            required.add('调整购电量')\n"
        '        if required - paths.keys():\n'
        "            raise ValueError('附件5中的结果模板工作表不完整')\n"
        '        for name, member in paths.items():\n'
        "            xml = parts[member].decode('utf-8')\n"
        "            if name in {'计划购电量', '调整购电量'}:\n"
        "                xml = write_purchase(xml, data, final=name == '调整购电量', single=case == 'problem1')\n"
        "            elif name == '充放电量':\n"
        "                xml = write_storage(xml, data, single=case == 'problem1')\n"
        "            elif name == '紧急购电量':\n"
        '                xml, _ = write_emergency(xml, data)\n'
        "            parts[member] = xml.encode('utf-8')\n"
        "        with ZipFile(temporary, 'w', compression=ZIP_DEFLATED, compresslevel=6) as output:\n"
        '            for name, content in parts.items():\n'
        '                output.writestr(infos.get(name, name), content)\n'
        '    else:\n'
        '        parts, _ = _standalone_parts(data, case)\n'
        "        with ZipFile(temporary, 'w', compression=ZIP_DEFLATED, compresslevel=6) as output:\n"
        '            for name, content in parts.items():\n'
        '                output.writestr(name, content)\n'
```

```
        '\n'
        '    temporary.replace(destination)\n'
        '    beautify_excel(destination)\n'
        '    return destination\n'
    ),
}

def _load_module(name,source,data_dir):
    parts=name.split('.')
    for i in range(1,len(parts)):
        parent='.'.join(parts[:i])
        if parent not in sys.modules:
            package=types.ModuleType(parent); package.__path__=[]; sys.modules[parent]=package
    filename=str(_SCRIPT_DIR/'__embedded__'/Path(*parts).with_suffix('.py'))
    module=types.ModuleType(name)
    module.__file__=filename
    module.__package__=name.rpartition('.')[0]
    module.__spec__=importlib.util.spec_from_loader(name,loader=None,origin=filename)
    module.__dict__.update(_SCRIPT_DIR=_SCRIPT_DIR,_DATA_DIR=data_dir)
    linecache.cache[filename]=(len(source),None,source.splitlines(keepends=True),filename)
    sys.modules[name]=module
    if '.' in name: setattr(sys.modules[name.rpartition('.')[0]],parts[-1],module)
    exec(compile(source,filename,'exec'),module.__dict__)
    return module

def main():
    parser=argparse.ArgumentParser()
    parser.add_argument('--input-dir',type=Path)
    parser.add_argument('--output-dir',type=Path)
    parser.add_argument('--templates-dir',type=Path)
    parser.add_argument('--max-days',type=int)
    args=parser.parse_args()

    data_dir=(args.input_dir or (Path(ATTACHMENT_DIR) if ATTACHMENT_DIR.strip() else
    _SCRIPT_DIR.parent/'附件')).resolve()
    if not (data_dir/'附件1.xlsx').exists() and (data_dir/'附件'/'附件1.xlsx').exists():
        data_dir=data_dir/'附件'
    required=[1]
    missing=[data_dir/f'附件{n}.xlsx' for n in required if not (data_dir/f'附件{n}.xlsx').
    is_file()]
    if missing: parser.error('缺少附件：\n'+'\n'.join(map(str,missing)))
    if args.max_days is not None and not 1<=args.max_days<=1:
        parser.error('--max-days 超出范围')

    out=(args.output_dir or (Path(OUTPUT_DIR) if OUTPUT_DIR.strip() else _SCRIPT_DIR.parent/'
    results'/_SCRIPT_PATH.stem)).resolve()
    out.mkdir(parents=True,exist_ok=True)
    template_dir=(args.templates_dir or (Path(TEMPLATE_DIR) if TEMPLATE_DIR.strip() else
    data_dir/'附件5')).resolve()

```

```
    try:
        import numpy as np
        import scipy
        import openpyxl
    except ImportError as e:
        parser.error('缺少依赖: '+str(e)+'; 请安装 numpy scipy openpyxl')

    modules={name:_load_module(name,source,data_dir) for name,source in _MODULE_SOURCES.items()
    }
    print(f'开始{CASE_NAME}; 结果: {out}',flush=True)
    modules['_entry'].run(out)
    with np.load(out/'all_results.npz') as z: days=len(z['dates'])
    if CASE=='problem1' or days==334:
        destination=modules['standalone_outputs'].export_one(CASE,out,data_dir,template_dir)
        print(f'Excel 已生成: {destination}',flush=True)
    else:
        print('快速检查仅计算数据; 完整 Excel 需运行完整评价期。',flush=True)

if __name__=='__main__':
    main()
```

附录 B. 问题二求解程序

Listing 2: 问题二求解程序

```
# 问题二: 滚动生成日前预测, 完成三层线性规划, 再按实际负载逐分钟修正调度。
from pathlib import Path
import argparse
from datetime import datetime,timedelta
import importlib.util
import linecache
import os
import sys
import types

ATTACHMENT_DIR = r""
OUTPUT_DIR = r""
TEMPLATE_DIR = r""
CASE = 'problem2'
CASE_NAME = '问题二'

os.environ.setdefault('OMP_NUM_THREADS','1')
os.environ.setdefault('OPENBLAS_NUM_THREADS','1')
_SCRIPT_PATH=Path(__file__).resolve()
_SCRIPT_DIR=_SCRIPT_PATH.parent

_MODULE_SOURCES = {
    'problem2.core': (
        'from __future__ import annotations\n'
```

```
        'from dataclasses import dataclass\n'
        'from datetime import date,datetime,time,timedelta\n'
        'from typing import Any\n'
        'import numpy as np\n'
        'import openpyxl\n'
        'from scipy.optimize import linprog\n'
        'from scipy.sparse import csr_matrix,lil_matrix\n'
        'MINUTES_PER_DAY = 1440\n'
        'SLOTS_PER_DAY = 144\n'
        'MINUTES_PER_SLOT = 10\n'
        'ETA = 0.9\n'
        'POWER_LIMIT_KW = 5000.0\n'
        'SOC_INITIAL_KWH = 6000.0\n'
        'SOC_MIN_KWH = 1200.0\n'
        'SOC_MAX_KWH = 10800.0\n'
        'LOAD_WEEKLY_DEVIATION_FACTOR = 1.0\n'
        'LOAD_LAG_DAYS = 7\n'
        'PV_HISTORY_DAYS = 7\n'
        'RISK_QUANTILE = 0.8\n'
        'RISK_ERROR_HISTORY_START_INDEX = 9\n'
        'BUDGET_FACTOR = 1.05\n'
        'REFERENCE_SHOCK_FACTOR = 5.0\n'
        'REFERENCE_DECAY_DAYS = 180.0\n'
        'TOL = 1e-08\n'
        'LP_TOL = 1e-07\n'
        'LEX_TOL = 1e-06\n'
        'DATA_ROOT = _SCRIPT_DIR\n'
        '\n'
        '@dataclass(frozen=True)\n'
        'class ActualData:\n'
        '    dates: list[date]\n'
        '    load_power_endpoints: np.ndarray\n'
        '    pv_power_endpoints: np.ndarray\n'
        '    load_energy: np.ndarray\n'
        '    pv_energy: np.ndarray\n'
        '\n'
        '@dataclass(frozen=True)\n'
        'class PlanSolution:\n'
        '    purchase: np.ndarray\n'
        '    discharge_internal: np.ndarray\n'
        '    charge_internal: np.ndarray\n'
        '    surplus: np.ndarray\n'
        '    soc: np.ndarray\n'
        '    baseline_cost: float\n'
        '    minimum_deviation: float\n'
        '    cost: float\n'
        '    solver_messages: tuple[str, str, str]\n'
        '\n'
        '@dataclass(frozen=True)\n'
        'class ActualDispatch:\n'
```

```
        '    discharge_internal: np.ndarray\n'
        '    charge_internal: np.ndarray\n'
        '    emergency: np.ndarray\n'
        '    surplus: np.ndarray\n'
        '    soc: np.ndarray\n'
        '    joint_adjustment: np.ndarray\n'
        '\n'
        '@dataclass(frozen=True)\n'
        'class DayResult:\n'
        '    day: date\n'
        '    load_forecast: np.ndarray\n'
        '    load_point_forecast: np.ndarray\n'
        '    pv_forecast: np.ndarray\n'
        '    risk_margin: np.ndarray\n'
        '    plan: PlanSolution\n'
        '    actual: ActualDispatch\n'
        '    reference_start: float\n'
        '    reference_next: float\n'
        '    decay_start: float\n'
        '    decay_next: float\n'
        '\n'
        'def _as_date(value: Any) -> date:\n'
        '    if isinstance(value, datetime):\n'
        '        return value.date()\n'
        '    if isinstance(value, date):\n'
        '        return value\n'
        '    return datetime.fromisoformat(str(value)).date()\n'
        '\n'
        'def _minutes(value: Any) -> int:\n'
        '    if isinstance(value, time):\n'
        '        return value.hour * 60 + value.minute\n'
        '    text = str(value).strip()\n'
        "    offset = 1440 if '+1' in text else 0\n"
        "    text = text.replace('+1', '')\n"
        "    hour, minute = (int(part) for part in text.split(':'))\n"
        '    return offset + 60 * hour + minute\n'
        '\n'
        'def load_price(path: Path) -> np.ndarray:\n'
        '    workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)\n'
        '    sheet = workbook.active\n'
        "    first_column = 2 if sheet.cell(1, 1).value == '序号' else 1\n"
        '    headers = [sheet.cell(1, column).value for column in range(first_column,
    first_column + 4)]\n'
        "    expected = ['时间', '电价', '小区负载', '光伏发电预测功率']\n"
        '    if headers != expected:\n'
        "        raise ValueError(f'附件1表头不匹配: {headers!r}')\n"
        '    rows = list(sheet.iter_rows(min_row=2, min_col=first_column, max_col=first_column
    + 3, values_only=True))\n'
        '    if len(rows) != SLOTS_PER_DAY:\n'
        "        raise ValueError(f'附件1应有144条数据, 实际为{len(rows)}条')\n"
```

```
        '    endpoints = np.array([_minutes(row[0]) for row in rows], dtype=int)\n'
        '    if not np.array_equal(endpoints, np.arange(10, 1441, 10)):\n'
        "        raise ValueError('附件1时间列不是0:10至次日0:00的十分钟序列')\n"
        '    price = np.array([float(row[1]) for row in rows], dtype=float)\n'
        '    if not np.isfinite(price).all() or np.any(price < 0):\n'
        "        raise ValueError('附件1电价含空值、非有限值或负值')\n"
        '    return price\n'
        '\n'
        'def _read_power_sheet(sheet: Any) -> tuple[list[date], np.ndarray]:\n'
        '    headers = [sheet.cell(1, column).value for column in range(1, 146)]\n'
        '    endpoints = np.array([_minutes(value) for value in headers[1:]], dtype=int)\n'
        '    if not np.array_equal(endpoints, np.arange(10, 1441, 10)):\n'
        "        raise ValueError(f'{sheet.title}时间表头不是0:10至次日0:00的序列')\n"
        '    rows = list(sheet.iter_rows(min_row=2, max_col=145, values_only=True))\n'
        '    dates = [_as_date(row[0]) for row in rows]\n'
        '    values = np.array([[float(value) for value in row[1:]] for row in rows])\n'
        '    if values.shape != (365, 144):\n'
        "        raise ValueError(f'{sheet.title}数据维度错误：{values.shape}')\n"
        '    if not np.isfinite(values).all() or np.any(values < 0):\n'
        "        raise ValueError(f'{sheet.title}含空值、非有限值或负值')\n"
        '    return (dates, values)\n'
        '\n'
        'def _minute_energy(power_endpoints: np.ndarray) -> np.ndarray:\n'
        '    days = power_endpoints.shape[0]\n'
        '    output = np.empty((days, MINUTES_PER_DAY), dtype=float)\n'
        '    sample_minutes = np.arange(0, MINUTES_PER_DAY + 1, MINUTES_PER_SLOT)\n'
        '    minute_points = np.arange(MINUTES_PER_DAY + 1)\n'
        '    for day_index in range(days):\n'
        '        if day_index == 0:\n'
        '            start_power = power_endpoints[day_index, 0]\n'
        '        else:\n'
        '            start_power = power_endpoints[day_index - 1, -1]\n'
        '        samples = np.concatenate(([start_power], power_endpoints[day_index]))\n'
        '        minute_power = np.interp(minute_points, sample_minutes, samples)\n'
        '        output[day_index] = (minute_power[:-1] + minute_power[1:]) / 120.0\n'
        '    output[0, :MINUTES_PER_SLOT] = np.nan\n'
        '    return output\n'
        '\n'
        'def load_actual(path: Path) -> ActualData:\n'
        '    workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)\n'
        "    expected_sheets = ['小区负载', '光伏发电实际功率']\n"
        '    if workbook.sheetnames != expected_sheets:\n'
        "        raise ValueError(f'附件2工作表不匹配：{workbook.sheetnames!r}')\n"
        '    load_dates, load_power = _read_power_sheet(workbook[expected_sheets[0]])\n'
        '    pv_dates, pv_power = _read_power_sheet(workbook[expected_sheets[1]])\n'
        '    if load_dates != pv_dates:\n'
        "        raise ValueError('附件2两个工作表的日期不一致')\n"
        '    expected_dates = [date(2025, 1, 1) + timedelta(days=i) for i in range(365)]\n'
        '    if load_dates != expected_dates:\n'
        "        raise ValueError('附件2日期不是2025-01-01至2025-12-31的连续序列')\n"
```

```
    '    return ActualData(dates=load_dates, load_power_endpoints=load_power, pv_power_endpoints=pv_power, load_energy=_minute_energy(load_power), pv_energy=_minute_energy(pv_power))\n'
    '\n'
    'def _weighted_baseline(values: np.ndarray, target_index: int, history_end: int) -> np.ndarray:\n'
    '    if history_end <= 0:\n'
    "        raise ValueError('历史集合为空')\n"
    '    history_indices = np.arange(history_end)\n'
    '    raw_distance = np.abs(target_index - history_indices)\n'
    '    ring_distance = np.minimum(raw_distance, 365 - raw_distance)\n'
    '    weights = 1.0 - 2.0 * ring_distance / 365.0\n'
    '    history = values[:history_end]\n'
    '    valid = np.isfinite(history)\n'
    '    weighted_sum = np.nansum(history * weights[:, None], axis=0)\n'
    '    weight_sum = np.sum(valid * weights[:, None], axis=0)\n'
    '    if np.any(weight_sum <= 0):\n'
    "        raise ValueError('历史加权基准存在零权重分母')\n"
    '    return weighted_sum / weight_sum\n'
    '\n'
    'def _point_forecast_day(actual: ActualData, day_index: int) -> tuple[np.ndarray, np.ndarray]:\n'
    '    if day_index < RISK_ERROR_HISTORY_START_INDEX:\n'
    "        raise ValueError('负荷周周期预测的历史数据不足')\n"
    '    load_baseline = _weighted_baseline(actual.load_energy, day_index, day_index)\n'
    '    week_ago_index = day_index - LOAD_LAG_DAYS\n'
    '    week_ago_load_baseline = _weighted_baseline(actual.load_energy, week_ago_index, week_ago_index)\n'
    '    week_ago_load = actual.load_energy[week_ago_index]\n'
    '    relative_deviation = np.divide(week_ago_load - week_ago_load_baseline, week_ago_load_baseline, out=np.zeros_like(week_ago_load), where=week_ago_load_baseline > TOL)\n'
    '    load_forecast = load_baseline * (1.0 + LOAD_WEEKLY_DEVIATION_FACTOR * relative_deviation)\n'
    '    pv_forecast = np.mean(actual.pv_energy[day_index - PV_HISTORY_DAYS:day_index], axis=0)\n'
    '    load_forecast = np.maximum(load_forecast, 0.0)\n'
    '    pv_forecast = np.maximum(pv_forecast, 0.0)\n'
    '    if not np.isfinite(load_forecast).all() or not np.isfinite(pv_forecast).all():\n'
    "        raise ValueError('预测结果含非有限值')\n"
    '    return (load_forecast, pv_forecast)\n'
    '\n'
    'def build_forecasts(actual: ActualData) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:\n'
    '    """构造点预测和只使用已发生误差的风险调整预测。"""\n'
    '    days = len(actual.dates)\n'
    '    point_load = np.full((days, MINUTES_PER_DAY), np.nan, dtype=float)\n'
    '    pv_forecast = np.full((days, MINUTES_PER_DAY), np.nan, dtype=float)\n'
    '    net_errors = np.full((days, MINUTES_PER_DAY), np.nan, dtype=float)\n'
    '    risk_margin = np.full((days, MINUTES_PER_DAY), np.nan, dtype=float)\n'
```

```
        '    for day_index in range(RISK_ERROR_HISTORY_START_INDEX, days):\n'
        '        load_value, pv_value = _point_forecast_day(actual, day_index)\n'
        '        point_load[day_index] = load_value\n'
        '        pv_forecast[day_index] = pv_value\n'
        '        net_errors[day_index] = actual.load_energy[day_index] - actual.pv_energy[
    day_index] - load_value + pv_value\n'
        '    start_index = actual.dates.index(date(2025, 2, 1))\n'
        '    for day_index in range(start_index, days):\n'
        '        historical_errors = net_errors[RISK_ERROR_HISTORY_START_INDEX:day_index]\n'
        '        margin = np.quantile(historical_errors, RISK_QUANTILE, axis=0)\n'
        '        risk_margin[day_index] = np.maximum(margin, 0.0)\n'
        '    decision_load = point_load + risk_margin\n'
        '    if not np.isfinite(decision_load[start_index:]).all():\n'
        "        raise ValueError('风险调整负荷预测含非有限值')\n"
        '    return (decision_load, point_load, pv_forecast, risk_margin)\n'
        '\n'
        'class DailyPlanner:\n'
        '\n'
        '    def __init__(self, price: np.ndarray) -> None:\n'
        '        self.price = np.asarray(price, dtype=float)\n'
        '        self.n_x = SLOTS_PER_DAY\n'
        '        self.n_y = self.n_z = self.n_w = MINUTES_PER_DAY\n'
        '        self.n_s = MINUTES_PER_DAY + 1\n'
        '        self.off_x = 0\n'
        '        self.off_y = self.off_x + self.n_x\n'
        '        self.off_z = self.off_y + self.n_y\n'
        '        self.off_w = self.off_z + self.n_z\n'
        '        self.off_s = self.off_w + self.n_w\n'
        '        self.off_a = self.off_s + self.n_s\n'
        '        self.n_vars = self.off_a + 1\n'
        '        self.a_eq = self._build_equalities()\n'
        '        self.a_ub = self._build_inequalities()\n'
        '\n'
        '    def _build_equalities(self) -> csr_matrix:\n'
        '        matrix = lil_matrix((2 * MINUTES_PER_DAY, self.n_vars))\n'
        '        for minute in range(MINUTES_PER_DAY):\n'
        '            slot = minute // MINUTES_PER_SLOT\n'
        '            matrix[minute, self.off_x + slot] = 1.0 / MINUTES_PER_SLOT\n'
        '            matrix[minute, self.off_y + minute] = ETA\n'
        '            matrix[minute, self.off_z + minute] = -1.0 / ETA\n'
        '            matrix[minute, self.off_w + minute] = -1.0\n'
        '            row = MINUTES_PER_DAY + minute\n'
        '            matrix[row, self.off_y + minute] = 1.0\n'
        '            matrix[row, self.off_z + minute] = -1.0\n'
        '            matrix[row, self.off_s + minute] = -1.0\n'
        '            matrix[row, self.off_s + minute + 1] = 1.0\n'
        '        return matrix.tocsr()\n'
        '\n'
        '    def _build_inequalities(self) -> csr_matrix:\n'
        '        matrix = lil_matrix((3, self.n_vars))\n'
```

```
        '        matrix[0, self.off_x:self.off_x + self.n_x] = self.price\n'
        '        matrix[1, self.off_s + MINUTES_PER_DAY] = 1.0\n'
        '        matrix[1, self.off_a] = -1.0\n'
        '        matrix[2, self.off_s + MINUTES_PER_DAY] = -1.0\n'
        '        matrix[2, self.off_a] = -1.0\n'
        '        return matrix.tocsr()\n'
        '\n'
        '    def _bounds(self, initial_soc: float, terminal_soc: float | None=None, max_abs_deviation: float | None=None) -> list[tuple[float, float | None]]:\n'
        '        discharge_limit = POWER_LIMIT_KW / 60.0\n'
        '        charge_internal_limit = ETA * POWER_LIMIT_KW / 60.0\n'
        '        bounds: list[tuple[float, float | None]] = []\n'
        '        bounds.extend([(0.0, None)] * self.n_x)\n'
        '        bounds.extend([(0.0, discharge_limit)] * self.n_y)\n'
        '        bounds.extend([(0.0, charge_internal_limit)] * self.n_z)\n'
        '        bounds.extend([(0.0, None)] * self.n_w)\n'
        '        bounds.extend([(SOC_MIN_KWH, SOC_MAX_KWH)] * self.n_s)\n'
        '        bounds.append((0.0, max_abs_deviation))\n'
        '        bounds[self.off_s] = (initial_soc, initial_soc)\n'
        '        if terminal_soc is not None:\n'
        '            bounds[self.off_s + MINUTES_PER_DAY] = (terminal_soc, terminal_soc)\n'
        '        return bounds\n'
        '\n'
        '    def _rhs(self, load: np.ndarray, pv: np.ndarray) -> np.ndarray:\n'
        '        rhs = np.zeros(2 * MINUTES_PER_DAY)\n'
        '        rhs[:MINUTES_PER_DAY] = load - pv\n'
        '        return rhs\n'
        '\n'
        '    def solve(self, load_forecast: np.ndarray, pv_forecast: np.ndarray, initial_soc: float, reference_soc: float) -> PlanSolution:\n'
        '        rhs = self._rhs(load_forecast, pv_forecast)\n'
        '        cost_objective = np.zeros(self.n_vars)\n'
        '        cost_objective[:self.n_x] = self.price\n'
        "        baseline = linprog(cost_objective, A_eq=self.a_eq, b_eq=rhs, bounds=self._bounds(initial_soc, terminal_soc=initial_soc), method='highs')\n"
        '        if not baseline.success:\n'
        "            raise RuntimeError(f'基准成本LP失败: {baseline.message}')\n"
        '        baseline_cost = float(baseline.fun)\n'
        '        deviation_objective = np.zeros(self.n_vars)\n'
        '        deviation_objective[self.off_a] = 1.0\n'
        '        inequality_rhs = np.array([BUDGET_FACTOR * baseline_cost, reference_soc, -reference_soc])\n'
        "        deviation = linprog(deviation_objective, A_ub=self.a_ub, b_ub=inequality_rhs, A_eq=self.a_eq, b_eq=rhs, bounds=self._bounds(initial_soc), method='highs')\n"
        '        if not deviation.success:\n'
        "            raise RuntimeError(f'日末偏差LP失败: {deviation.message}')\n"
        '        minimum_deviation = max(0.0, float(deviation.x[self.off_a]))\n'
        "        final = linprog(cost_objective, A_ub=self.a_ub, b_ub=inequality_rhs, A_eq=self.a_eq, b_eq=rhs, bounds=self._bounds(initial_soc, max_abs_deviation=minimum_deviation + LEX_TOL), method='highs')\n"
```

```
        '        if not final.success:\n'
        "            raise RuntimeError(f'同偏差最低费用LP失败: {final.message}')\n"
        '        vector = final.x\n'
        '        return PlanSolution(purchase=vector[self.off_x:self.off_y], discharge_internal=vector[self.off_y:self.off_z], charge_internal=vector[self.off_z:self.off_w], surplus=vector[self.off_w:self.off_s], soc=vector[self.off_s:self.off_a], baseline_cost=baseline_cost, minimum_deviation=minimum_deviation, cost=float(final.fun), solver_messages=(str(baseline.message), str(deviation.message), str(final.message)))\n'
        '\n'
        'def execute_day(initial_soc: float, purchase: np.ndarray, planned_discharge: np.ndarray, planned_charge: np.ndarray, load_actual: np.ndarray, pv_actual: np.ndarray) -> ActualDispatch:\n'
        '    purchase_minute = np.repeat(purchase / MINUTES_PER_SLOT, MINUTES_PER_SLOT)\n'
        '    discharge = np.zeros(MINUTES_PER_DAY)\n'
        '    charge = np.zeros(MINUTES_PER_DAY)\n'
        '    emergency = np.zeros(MINUTES_PER_DAY)\n'
        '    surplus = np.zeros(MINUTES_PER_DAY)\n'
        '    soc = np.empty(MINUTES_PER_DAY + 1)\n'
        '    soc[0] = initial_soc\n'
        '    joint = np.zeros(MINUTES_PER_DAY)\n'
        '    y_max = POWER_LIMIT_KW / 60.0\n'
        '    z_max = ETA * POWER_LIMIT_KW / 60.0\n'
        '    kappa = 1.0 / ETA - ETA\n'
        '    for minute in range(MINUTES_PER_DAY):\n'
        '        state = soc[minute]\n'
        '        y_plan = float(planned_discharge[minute])\n'
        '        z_plan = float(planned_charge[minute])\n'
        '        q_plan = state - y_plan + z_plan\n'
        '        if q_plan < SOC_MIN_KWH:\n'
        '            y = state + z_plan - SOC_MIN_KWH\n'
        '            z = z_plan\n'
        '        elif q_plan > SOC_MAX_KWH:\n'
        '            y = y_plan\n'
        '            z = SOC_MAX_KWH - state + y_plan\n'
        '        else:\n'
        '            y, z = (y_plan, z_plan)\n'
        '        y = min(max(y, 0.0), y_max)\n'
        '        z = min(max(z, 0.0), z_max)\n'
        '        q = state - y + z\n'
        '        residual = load_actual[minute] + z / ETA - pv_actual[minute] - purchase_minute[minute] - ETA * y\n'
        '        if residual > TOL:\n'
        '            amount = min(z, ETA * residual, max(q - SOC_MIN_KWH, 0.0))\n'
        '            z -= amount\n'
        '            residual -= amount / ETA\n'
        '            q = state - y + z\n'
        '            amount = min(y_max - y, residual / ETA, max(q - SOC_MIN_KWH, 0.0))\n'
        '            y += amount\n'
        '            residual -= ETA * amount\n'
        '            q = state - y + z\n'
```

```
        '            if residual > TOL and y > TOL and (z > TOL):\n'
        '                amount = min(y, z, residual / kappa)\n'
        '                y -= amount\n'
        '                z -= amount\n'
        '                residual -= kappa * amount\n'
        '                joint[minute] = amount\n'
        '            if residual > TOL:\n'
        '                emergency[minute] = residual\n'
        '            else:\n'
        '                residual = 0.0\n'
        '        elif residual < -TOL:\n'
        '            unused = -residual\n'
        '            amount = min(y, unused / ETA, max(SOC_MAX_KWH - q, 0.0))\n'
        '            y -= amount\n'
        '            unused -= ETA * amount\n'
        '            q = state - y + z\n'
        '            amount = min(z_max - z, ETA * unused, max(SOC_MAX_KWH - q, 0.0))\n'
        '            z += amount\n'
        '            unused -= amount / ETA\n'
        '            if unused > TOL:\n'
        '                surplus[minute] = unused\n'
        '        discharge[minute] = max(y, 0.0)\n'
        '        charge[minute] = max(z, 0.0)\n'
        '        soc[minute + 1] = state - discharge[minute] + charge[minute]\n'
        '    return ActualDispatch(discharge, charge, emergency, surplus, soc, joint)\n'
    '\n'
    'def validate_plan(price: np.ndarray, load: np.ndarray, pv: np.ndarray, initial_soc: float, reference_soc: float, plan: PlanSolution) -> dict[str, Any]:\n'
    '    purchase_minute = np.repeat(plan.purchase / MINUTES_PER_SLOT, MINUTES_PER_SLOT)\n'
    '    balance = pv + purchase_minute + ETA * plan.discharge_internal - load - plan.charge_internal / ETA - plan.surplus\n'
    '    state = plan.soc[1:] - plan.soc[:-1] + plan.discharge_internal - plan.charge_internal\n'
    '    cost = float(np.dot(price, plan.purchase))\n'
    '    deviation = abs(float(plan.soc[-1]) - reference_soc)\n'
    "    checks = {'max_balance_residual_kwh': float(np.max(np.abs(balance))), 'max_state_residual_kwh': float(np.max(np.abs(state))), 'initial_soc_residual_kwh': abs(float(plan.soc[0]) - initial_soc), 'soc_min_kwh': float(np.min(plan.soc)), 'soc_max_kwh': float(np.max(plan.soc)), 'cost_recalculated_yuan': cost, 'cost_residual_yuan': abs(cost - plan.cost), 'budget_slack_yuan': BUDGET_FACTOR * plan.baseline_cost - plan.cost, 'terminal_deviation_kwh': deviation, 'minimum_deviation_kwh': plan.minimum_deviation, 'max_discharge_internal_kwh': float(np.max(plan.discharge_internal)), 'max_charge_bus_kwh': float(np.max(plan.charge_internal / ETA)), 'min_variable_kwh': float(min(np.min(plan.purchase), np.min(plan.discharge_internal), np.min(plan.charge_internal), np.min(plan.surplus)))}\n"
    "    checks['pass'] = bool(checks['max_balance_residual_kwh'] <= LP_TOL and checks['max_state_residual_kwh'] <= LP_TOL and (checks['initial_soc_residual_kwh'] <= LP_TOL) and (checks['soc_min_kwh'] >= SOC_MIN_KWH - LP_TOL) and (checks['soc_max_kwh'] <= SOC_MAX_KWH + LP_TOL) and (checks['cost_residual_yuan'] <= 1e-05) and (checks['budget_slack_yuan'] >= -1e-05) and (deviation <= plan.minimum_deviation + 10 * LEX_TOL) and (checks['
```

```
    max_discharge_internal_kwh'] <= POWER_LIMIT_KW / 60 + LP_TOL) and (checks['
    max_charge_bus_kwh'] <= POWER_LIMIT_KW / 60 + LP_TOL) and (checks['min_variable_kwh'] >= -
    LP_TOL))\n"
382     '    return checks\n'
383     '\n'
384     'def validate_actual(purchase: np.ndarray, load: np.ndarray, pv: np.ndarray, actual:
    ActualDispatch) -> dict[str, Any]:\n'
385     '    purchase_minute = np.repeat(purchase / MINUTES_PER_SLOT, MINUTES_PER_SLOT)\n'
386     '    balance = pv + purchase_minute + ETA * actual.discharge_internal + actual.
    emergency - load - actual.charge_internal / ETA - actual.surplus\n'
387     '    state = actual.soc[1:] - actual.soc[:-1] + actual.discharge_internal - actual.
    charge_internal\n'
388     '    simultaneous = (actual.discharge_internal > TOL) & (actual.charge_internal > TOL)\
    n'
389     "    checks = {'max_balance_residual_kwh': float(np.max(np.abs(balance))), '
    max_state_residual_kwh': float(np.max(np.abs(state))), 'soc_min_kwh': float(np.min(actual.
    soc)), 'soc_max_kwh': float(np.max(actual.soc)), 'max_discharge_internal_kwh': float(np.max
    (actual.discharge_internal)), 'max_charge_bus_kwh': float(np.max(actual.charge_internal /
    ETA)), 'min_emergency_kwh': float(np.min(actual.emergency)), 'min_surplus_kwh': float(np.
    min(actual.surplus)), 'simultaneous_minutes': int(np.sum(simultaneous)), '
    joint_adjustment_minutes': int(np.sum(actual.joint_adjustment > TOL)), '
    joint_adjustment_internal_kwh': float(np.sum(actual.joint_adjustment))}\n"
390     "    checks['pass'] = bool(checks['max_balance_residual_kwh'] <= LP_TOL and checks['
    max_state_residual_kwh'] <= LP_TOL and (checks['soc_min_kwh'] >= SOC_MIN_KWH - LP_TOL) and
    (checks['soc_max_kwh'] <= SOC_MAX_KWH + LP_TOL) and (checks['max_discharge_internal_kwh']
    <= POWER_LIMIT_KW / 60 + LP_TOL) and (checks['max_charge_bus_kwh'] <= POWER_LIMIT_KW / 60 +
     LP_TOL) and (checks['min_emergency_kwh'] >= -LP_TOL) and (checks['min_surplus_kwh'] >= -
    LP_TOL))\n"
391     '    return checks\n'
392     '\n'
393     'def simulate(price: np.ndarray, actual: ActualData, max_days: int | None=None) ->
    tuple[list[DayResult], dict[str, Any]]:\n'
394     '    planner = DailyPlanner(price)\n'
395     '    decision_load, point_load, pv_forecasts, risk_margins = build_forecasts(actual)\n'
396     '    start_index = actual.dates.index(date(2025, 2, 1))\n'
397     '    end_index = len(actual.dates)\n'
398     '    if max_days is not None:\n'
399     '        end_index = min(end_index, start_index + max_days)\n'
400     '    state = SOC_INITIAL_KWH\n'
401     '    reference = SOC_INITIAL_KWH\n'
402     '    decay = 0.0\n'
403     '    results: list[DayResult] = []\n'
404     '    worst_plan_balance = 0.0\n'
405     '    worst_actual_balance = 0.0\n'
406     '    worst_state = 0.0\n'
407     '    for day_index in range(start_index, end_index):\n'
408     '        load_forecast = decision_load[day_index]\n'
409     '        load_point_forecast = point_load[day_index]\n'
410     '        pv_forecast = pv_forecasts[day_index]\n'
411     '        risk_margin = risk_margins[day_index]\n'
```

```
        '        plan = planner.solve(load_forecast, pv_forecast, state, reference)\n'
        '        plan_checks = validate_plan(price, load_forecast, pv_forecast, state,
    reference, plan)\n'
        "        if not plan_checks['pass']:\n"
        "            raise RuntimeError(f'{actual.dates[day_index]}日前计划校验失败: {
    plan_checks}')\n"
        '        dispatch = execute_day(state, plan.purchase, plan.discharge_internal, plan.
    charge_internal, actual.load_energy[day_index], actual.pv_energy[day_index])\n'
        '        actual_checks = validate_actual(plan.purchase, actual.load_energy[day_index],
    actual.pv_energy[day_index], dispatch)\n'
        "        if not actual_checks['pass']:\n"
        "            raise RuntimeError(f'{actual.dates[day_index]}实际调度校验失败: {
    actual_checks}')\n"
        '        emergency_total = float(np.sum(dispatch.emergency))\n'
        '        if emergency_total > TOL:\n'
        '            reference_next = min(SOC_MAX_KWH, reference + REFERENCE_SHOCK_FACTOR *
    emergency_total)\n'
        '            decay_next = (reference_next - SOC_INITIAL_KWH) / REFERENCE_DECAY_DAYS\n'
        '        else:\n'
        '            reference_next = max(SOC_INITIAL_KWH, reference - decay)\n'
        '            decay_next = decay\n'
        '        results.append(DayResult(day=actual.dates[day_index], load_forecast=
    load_forecast, load_point_forecast=load_point_forecast, pv_forecast=pv_forecast,
    risk_margin=risk_margin, plan=plan, actual=dispatch, reference_start=reference,
    reference_next=reference_next, decay_start=decay, decay_next=decay_next))\n'
        "        worst_plan_balance = max(worst_plan_balance, plan_checks['
    max_balance_residual_kwh'])\n"
        "        worst_actual_balance = max(worst_actual_balance, actual_checks['
    max_balance_residual_kwh'])\n"
        "        worst_state = max(worst_state, plan_checks['max_state_residual_kwh'],
    actual_checks['max_state_residual_kwh'])\n"
        '        state = float(dispatch.soc[-1])\n'
        '        reference, decay = (reference_next, decay_next)\n'
        '        if len(results) % 30 == 0:\n'
        "            print(f'Problem2 completed {len(results)} days', flush=True)\n"
        "    validation = {'days_simulated': len(results), 'first_date': results[0].day.
    isoformat() if results else None, 'last_date': results[-1].day.isoformat() if results else
    None, 'worst_plan_balance_residual_kwh': worst_plan_balance, '
    worst_actual_balance_residual_kwh': worst_actual_balance, 'worst_state_residual_kwh':
    worst_state, 'all_hard_constraints_pass': True}\n"
        '    return (results, validation)\n'
    ),

    'output_common': (
        'import csv\n'
        'from pathlib import Path\n'
        'import numpy as np\n'
        '\n'
        'def write_csv(path, rows):\n'
        '    rows=list(rows)\n'
```

```
        '    if not rows:\n'
        '        return\n'
        "    with Path(path).open('w',encoding='utf-8-sig',newline='') as f:\n"
        '        w=csv.DictWriter(f,fieldnames=list(rows[0]))\n'
        '        w.writeheader(); w.writerows(rows)\n'
        '\n'
        'def save_case(out, arrays, daily):\n'
        '    out=Path(out); out.mkdir(parents=True,exist_ok=True)\n'
        "    np.savez_compressed(out/'all_results.npz',**arrays)\n"
        "    write_csv(out/'daily.csv',daily)\n"
    ),

    '_entry': (
        'import numpy as np\n'
        'from problem2 import core as model\n'
        'from output_common import save_case\n'
        '\n'
        'def run(max_days,out):\n'
        "    price=model.load_price(_DATA_DIR/'附件1.xlsx')\n"
        "    data=model.load_actual(_DATA_DIR/'附件2.xlsx')\n"
        '    results,_=model.simulate(price,data,max_days=max_days)\n'
        '    n=len(results)\n'
        '    arrays=dict(\n'
        '        dates=np.array([str(r.day) for r in results]),\n'
        '        purchase_initial=np.array([r.plan.purchase for r in results]),\n'
        '        purchase_final=np.array([r.plan.purchase for r in results]),\n'
        '        price=np.tile(price,(n,1)),soc=np.array([r.actual.soc for r in results]),\n'
        '        charge_internal=np.array([r.actual.charge_internal for r in results]),\n'
        '        discharge_internal=np.array([r.actual.discharge_internal for r in results]),\n
'
        '        emergency=np.array([r.actual.emergency for r in results]),\n'
        '        surplus=np.array([r.actual.surplus for r in results]))\n'
        '    daily=[]\n'
        '    for r in results:\n'
        '        emergency_cost=float(5*np.repeat(price,10)@r.actual.emergency)\n'
        '        daily.append(dict(date=str(r.day),initial_cost=r.plan.cost,adjustment_cost
=0.0,\n'
        '                          emergency_cost=emergency_cost,total_cost=r.plan.cost+
emergency_cost))\n'
        '    save_case(out,arrays,daily)\n'
    ),

    'standalone_outputs': (
        'from __future__ import annotations\n'
        'import copy\n'
        'import csv\n'
        'import math\n'
        'import posixpath\n'
        'import re\n'
        'from datetime import date, timedelta\n'
```

```
        'from pathlib import Path\n'
        'from xml.etree import ElementTree as ET\n'
        'from xml.sax.saxutils import escape\n'
        'from zipfile import ZipFile, ZIP_DEFLATED\n'
        'import numpy as np\n'
        '\n'
        "S = 'http://schemas.openxmlformats.org/spreadsheetml/2006/main'\n"
        "R = 'http://schemas.openxmlformats.org/officeDocument/2006/relationships'\n"
        "P = 'http://schemas.openxmlformats.org/package/2006/relationships'\n"
        "SUPPLEMENT = '完整自然日购电量'\n"
        "FILES = {'problem1':'result1.xlsx','problem2':'result2.xlsx','problem3':'result3.xlsx','problem4-2':'result4-2.xlsx','problem4-3':'result4-3.xlsx'}\n"
        "ROW_RE = re.compile('<row\\\\b[^>]*?(?:/>|>.*?</row>)', re.S)\n"
        "CELL_RE = re.compile('<c\\\\b[^>]*?(?:/>|>.*?</c>)', re.S)\n"
        '\n'
        '\n'
        'def colname(number: int) -> str:\n'
        "    result = ''\n"
        '    while number:\n'
        '        number, rem = divmod(number - 1, 26)\n'
        '        result = chr(65 + rem) + result\n'
        '    return result\n'
        '\n'
        'def colnumber(name: str) -> int:\n'
        '    result = 0\n'
        '    for char in name:\n'
        '        result = 26 * result + ord(char) - 64\n'
        '    return result\n'
        '\n'
        'def address_key(ref: str) -> tuple[int, int]:\n'
        "    match = re.fullmatch('([A-Z]+)([0-9]+)', ref)\n"
        '    if match is None:\n'
        "        raise ValueError(f'Invalid cell reference {ref}')\n"
        '    return (int(match[2]), colnumber(match[1]))\n'
        '\n'
        'def attr(fragment: str, name: str, default: str | None=None) -> str | None:\n'
        '    match = re.search(\'\\\\b\' + re.escape(name) + \'="([^"]*)"\', fragment)\n'
        '    return match[1] if match else default\n'
        '\n'
        'def scalar_cell(ref: str, value, original: str | None=None, style: str | None=None) -> str:\n'
        '    """Replace only a cell value; retain its original attributes except type."""\n'
        '    if original:\n'
        "        opening = re.match('<c\\\\b([^>]*?)(?:/?>)', original)[1]\n"
        '        opening = re.sub(\'\\\\s+t="[^"]*"\', \'\', opening).rstrip(\'/ \')\n'
        '        opening = re.sub(\'\\\\br="[^"]*"\', f\'r="{ref}"\', opening)\n'
        '    else:\n'
        '        opening = f\' r="{ref}"\' + (f\' s="{style}"\' if style is not None else \'\')\n'
        '    if value is None:\n'
```

```
        "        return f'<c{opening}/>'\n"
        '    if isinstance(value, (str, np.str_)):\n'
        '        return f\'<c{opening} t="inlineStr"><is><t xml:space="preserve">{escape(str(value))}</t></is></c>\'\n'
        '    numeric = float(value)\n'
        '    if not math.isfinite(numeric):\n'
        "        raise ValueError(f'Non-finite numeric output at {ref}: {value}')\n"
        '    if abs(numeric) < 1e-10:\n'
        '        numeric = 0.0\n'
        '    return f"<c{opening}><v>{format(numeric, \'.15g\')}</v></c>"\n'
        '\n'
        'def rows_in(xml: str) -> dict[int, str]:\n'
        "    data = re.search('<sheetData>(.*?)</sheetData>', xml, re.S)\n"
        '    if data is None:\n'
        "        raise ValueError('Template has no sheetData element')\n"
        "    return {int(attr(row.split('>', 1)[0], 'r')): row for row in ROW_RE.findall(data[1])}\n"
        '\n'
        'def patch_row(row: str, updates: dict[str, object], new_number: int | None=None) -> str:\n'
        "    opening = re.match('<row\\\\b([^>]*?)(?:/?>)', row)[1].rstrip('/ ')\n"
        "    old_number = int(attr(opening, 'r'))\n"
        '    new_number = old_number if new_number is None else new_number\n'
        '    opening = re.sub(\'\\\\br="[^"]*"\', f\'r="{new_number}"\', opening)\n'
        '    cells = {}\n'
        '    for cell in CELL_RE.findall(row):\n'
        "        ref = attr(cell.split('>', 1)[0], 'r')\n"
        "        column = re.match('[A-Z]+', ref)[0]\n"
        "        new_ref = f'{column}{new_number}'\n"
        '        cells[new_ref] = re.sub(\'\\\\br="[^"]*"\', f\'r="{new_ref}"\', cell, count=1)\n'
        "    default_style = attr(opening, 's')\n"
        '    for column_or_ref, value in updates.items():\n'
        "        column = re.match('[A-Z]+', column_or_ref)[0]\n"
        "        ref = f'{column}{new_number}'\n"
        '        cells[ref] = scalar_cell(ref, value, cells.get(ref), default_style)\n'
        "    return f'<row{opening}>' + ''.join((cells[k] for k in sorted(cells, key=address_key))) + '</row>'\n"
        '\n'
        'def replace_rows(xml: str, rows: list[str], last_cell: str | None=None) -> str:\n'
        "    xml = re.sub('<sheetData>.*?</sheetData>', lambda _: '<sheetData>' + ''.join(rows) + '</sheetData>', xml, count=1, flags=re.S)\n"
        '    if last_cell:\n'
        '        xml = re.sub(\'<dimension\\\\b[^>]*/>\', f\'<dimension ref="A1:{last_cell}"/>\', xml, count=1)\n'
        '    return xml\n'
        '\n'
        'def excel_date(day: date) -> int:\n'
        '    return (day - date(1899, 12, 30)).days\n'
        '\n'
```

```
        'def clock(minutes: int) -> str:\n'
        "    return f'{minutes // 60:02d}:{minutes % 60:02d}'\n"
        '\n'
        'def intervals(energy: np.ndarray, tolerance: float=1e-08) -> list[tuple[int, int, float]]:\n'
        '    positive = np.asarray(energy) > tolerance\n'
        '    padded = np.r_[False, positive, False].astype(np.int8)\n'
        '    starts = np.flatnonzero(np.diff(padded) == 1)\n'
        '    ends = np.flatnonzero(np.diff(padded) == -1)\n'
        '    return [(int(a), int(b), float(np.sum(energy[a:b]))) for a, b in zip(starts, ends)]\n'
        '\n'
        'def load_results(folder: Path, case: str) -> dict:\n'
        '    folder = Path(folder).resolve()\n'
        "    with np.load(folder / 'all_results.npz', allow_pickle=False) as source:\n"
        '        data = {key: np.array(source[key]) for key in source.files}\n'
        "    required = {'dates', 'purchase_initial', 'purchase_final', 'price', 'charge_internal', 'discharge_internal', 'emergency', 'surplus', 'soc'}\n"
        '    if required - data.keys():\n'
        "        raise ValueError(f'{case}: missing arrays {sorted(required - data.keys())}')\n"
        "    dates = [date.fromisoformat(str(value)[:10]) for value in data['dates']]\n"
        '    n = len(dates)\n'
        "    if case == 'problem1':\n"
        '        if n != 1:\n'
        "            raise ValueError('problem1 must contain exactly one modeled day')\n"
        '    elif dates != [date(2025, 2, 1) + timedelta(days=i) for i in range(334)]:\n'
        "        raise ValueError(f'{case}: official export requires all 334 dates from February 1 to December 31')\n"
        "    for key in required - {'dates'}:\n"
        "        width = 1441 if key == 'soc' else 144 if key in {'purchase_initial', 'purchase_final', 'price'} else 1440\n"
        '        if data[key].shape != (n, width) or not np.all(np.isfinite(data[key])):\n'
        "            raise ValueError(f'{case}: invalid {key} shape or nonfinite values: {data[key].shape}')\n"
        "    data['dates'] = dates\n"
        "    daily_path = next((p for p in [folder / 'daily.csv', folder / 'summary' / 'daily.csv'] if p.exists()), None)\n"
        '    if daily_path is None:\n'
        "        raise FileNotFoundError(f'Missing daily settlement CSV under {folder}')\n"
        "    with daily_path.open(encoding='utf-8-sig', newline='') as stream:\n"
        '        records = list(csv.DictReader(stream))\n'
        '    if not records:\n'
        "        raise ValueError('Daily settlement CSV has no rows')\n"
        "    date_key = next((key for key in ['date', '日期', 'day'] if key in records[0]), None)\n"
        '    if date_key is None:\n'
        "        raise ValueError(f'{daily_path}: need date column')\n"
        '    by_date = {date.fromisoformat(row[date_key][:10]): row for row in records}\n'
        '    if len(by_date) != len(records) or set(by_date) != set(dates):\n'
```

```
    "        raise ValueError('Daily CSV dates must match NPZ dates exactly without duplicates')\n"
    "    aliases = {'initial_cost': ['initial_cost', 'initial_purchase_cost', 'initial_plan_cost', 'planned_cost', 'plan_cost', 'purchase_cost'], 'adjustment_cost': ['adjustment_cost', 'adjustment_net_cost', 'adjustment_net', 'adjust_cost'], 'emergency_cost': ['emergency_cost', 'emergency_purchase_cost'], 'total_cost': ['total_cost', 'actual_total_cost', 'actual_cost', 'cost']}\n"
    '    daily = []\n'
    '    for i, day in enumerate(dates):\n'
    '        if day not in by_date:\n'
    "            raise ValueError(f'{daily_path}: missing {day}')\n"
    '        row = by_date[day]\n'
    '        values = {}\n'
    '        for key, names in aliases.items():\n'
    "            found = next((row[name] for name in names if name in row and row[name] != ''), None)\n"
    '            if found is None:\n'
    "                if key == 'adjustment_cost' and case in {'problem1', 'problem2', 'problem4-2'}:\n"
    '                    values[key] = 0.0\n'
    '                else:\n'
    "                    raise ValueError(f'{daily_path}: need {key} for {day}; available columns={list(row)}')\n"
    '            else:\n'
    '                values[key] = float(found)\n'
    '        if not all((math.isfinite(v) for v in values.values())):\n'
    "            raise ValueError(f'Nonfinite daily cost for {day}')\n"
    "        computed_total = values['initial_cost'] + values['adjustment_cost'] + values['emergency_cost']\n"
    "        if not math.isclose(computed_total, values['total_cost'], abs_tol=0.01, rel_tol=1e-09):\n"
    "            raise ValueError(f'Daily settlement does not reconcile for {case}/{day}: {values}')\n"
    "        actual_emergency = float(np.dot(data['emergency'][i], np.repeat(data['price'][i], 10)) * 5)\n"
    "        if not math.isclose(actual_emergency, values['emergency_cost'], abs_tol=0.01, rel_tol=1e-08):\n"
    "            raise ValueError(f'Emergency cost disagrees with minute trace for {case}/{day}')\n"
    "        actual_initial = float(np.dot(data['purchase_initial'][i], data['price'][i]))\n"
    "        if not math.isclose(actual_initial, values['initial_cost'], abs_tol=0.01, rel_tol=1e-08):\n"
    "            raise ValueError(f'Initial plan cost disagrees with 144-slot trace for {case}/{day}')\n"
    '        daily.append(values)\n'
    "    data['daily'] = daily\n"
    "    data['daily_csv'] = daily_path.relative_to(folder).as_posix()\n"
    '    return data\n'
    '\n'
```

```
    'def sheet_paths(parts: dict[str, bytes]) -> dict[str, str]:\n'
    "    workbook = ET.fromstring(parts['xl/workbook.xml'])\n"
    "    rels = {r.attrib['Id']: r.attrib['Target'] for r in ET.fromstring(parts['xl/_rels/workbook.xml.rels'])}\n"
    "    return {s.attrib['name']: posixpath.normpath('xl/' + rels[s.attrib[f'{{{R}}}id']]) for s in workbook.find(f'{{{S}}}sheets')}\n"
    '\n'
    'def write_purchase(xml: str, data: dict, *, final: bool, single: bool) -> str:\n'
    '    rows = rows_in(xml)\n'
    "    array = data['purchase_final' if final else 'purchase_initial']\n"
    '    if single:\n'
    '        for row in range(2, 145):\n'
    "            rows[row] = patch_row(rows[row], {'B': array[0, row - 1]})\n"
    "        rows[145] = patch_row(rows[145], {'B': None})\n"
    '    else:\n'
    "        for i, day in enumerate(data['dates']):\n"
    '            row = i + 2\n'
    '            updates = {colname(j + 1): array[i, j] for j in range(1, 144)}\n'
    "            updates['EO'] = array[i + 1, 0] if i + 1 < len(array) else None\n"
    "            updates['EP'] = float(array[i].sum())\n"
    "            costs = data['daily'][i]\n"
    "            updates['EQ'] = costs['initial_cost'] + (costs['adjustment_cost'] if final else 0)\n"
    '            rows[row] = patch_row(rows[row], updates)\n'
    '    return replace_rows(xml, [rows[r] for r in sorted(rows)])\n'
    '\n'
    'def write_storage(xml: str, data: dict, *, single: bool) -> str:\n'
    '    original = rows_in(xml)\n'
    '    eta = 0.9\n'
    '    if single:\n'
    '        for block in range(6):\n'
    '            take = slice(240 * block, 240 * (block + 1))\n'
    "            updates = {'B': float(data['charge_internal'][0, take].sum() / eta), 'C': float(data['discharge_internal'][0, take].sum() * eta)}\n"
    '            if block < 2:\n'
    "                updates['E'] = data['soc'][0, 0 if block == 0 else -1]\n"
    '            original[block + 2] = patch_row(original[block + 2], updates)\n'
    '        return replace_rows(xml, [original[r] for r in sorted(original)])\n'
    '    output = [original[1]]\n'
    "    for i, day in enumerate(data['dates']):\n"
    '        for block in range(6):\n'
    '            row = 2 + 6 * i + block\n'
    '            take = slice(240 * block, 240 * (block + 1))\n'
    "            updates = {'A': excel_date(day) if block == 0 else None, 'C': float(data['charge_internal'][i, take].sum() / eta), 'D': float(data['discharge_internal'][i, take].sum() * eta), 'F': data['soc'][i, 0 if block == 0 else -1] if block < 2 else None}\n"
    '            output.append(patch_row(original[block + 2], updates, row))\n'
    '    return replace_rows(xml, output, f"F{1 + 6 * len(data[\'dates\'])}")\n'
    '\n'
    'def write_emergency(xml: str, data: dict) -> tuple[str, int]:\n'
```

```
    '    original = rows_in(xml)\n'
    '    output = [original[1]]\n'
    '    count = 0\n'
    "    for i, day in enumerate(data['dates']):\n"
    "        events = intervals(data['emergency'][i])\n"
    '        count += len(events)\n'
    "        records = [(f'{clock(a)}-{clock(b)}', energy) for a, b, energy in events] or [('无', 0.0)]\n"
    '        for j, (label, energy) in enumerate(records):\n'
    '            prototype = original[2 if j == 0 else 4 if j == len(records) - 1 else 3]\n'
    "            output.append(patch_row(prototype, {'A': excel_date(day) if j == 0 else None, 'B': label, 'C': energy}, len(output) + 1))\n"
    "    return (replace_rows(xml, output, f'C{len(output)}'), count)\n"
    '\n'
    'def supplement_xml(parts: dict[str, bytes], data: dict, case: str) -> str:\n'
    '    """A compact additional matrix; preserve all original styles.xml bytes."""\n'
    '    if parts:\n'
    "        source = rows_in(parts[sheet_paths(parts)['计划购电量']].decode('utf-8'))\n"
    "        header_style = attr(CELL_RE.findall(source[1])[1].split('>', 1)[0], 's', '0')\n"
    "        value_style = attr(CELL_RE.findall(source[2])[-1].split('>', 1)[0], 's', '0')\n"
    '    else:\n'
    "        header_style, value_style = ('1', '2')\n"
    "    headings = ['日期', '计划类型'] + [f'{clock(10 * k)}-{clock(10 * (k + 1))}' for k in range(144)]\n"
    "    headings += ['全天购电量(kWh)', '初始计划款(元)', '调整净费用(元)', '紧急购电费(元)', '总购电费(元)', '紧急购电量(kWh)']\n"
    '    rows = [\'<row r="1" ht="28" customHeight="1">\' + \'\'.join((scalar_cell(f\'{colname(c)}1\', v, style=header_style) for c, v in enumerate(headings, 1))) + \'</row>\']\n'
    "    stage_rows = [('初始计划', 'purchase_initial'), ('最终有效计划', 'purchase_final')] if case in {'problem3', 'problem4-3'} else [('计划购电', 'purchase_initial')]\n"
    "    for i, day in enumerate(data['dates']):\n"
    "        costs = data['daily'][i]\n"
    '        for stage, key in stage_rows:\n'
    "            settlement = stage != '初始计划'\n"
    "            display_day = '附件1示例日' if case == 'problem1' else day.isoformat()\n"
    '            values = [display_day, stage] + data[key][i].tolist() + [float(data[key][i].sum())]\n'
    "            values += [costs['initial_cost'], costs['adjustment_cost'] if settlement else None, costs['emergency_cost'] if settlement else None, costs['total_cost'] if settlement else None, float(data['emergency'][i].sum()) if settlement else None]\n"
    '            r = len(rows) + 1\n'
    '            rows.append(f\'<row r="{r}" ht="20" customHeight="1">\' + \'\'.join((scalar_cell(f\'{colname(c)}{r}\', value, style=value_style) for c, value in enumerate(values, 1))) + \'</row>\')\n'
    '    return f\'<?xml version="1.0" encoding="UTF-8" standalone="yes"?><worksheet xmlns="{S}"><dimension ref="A1:EV{len(rows)}"/><sheetViews><sheetView showGridLines="0"
```

```
    workbookViewId="0"><pane xSplit="2" ySplit="1" topLeftCell="C2" activePane="bottomRight" state="frozen"/></sheetView></sheetViews><sheetFormatPr defaultRowHeight="20"/><cols><col min="1" max="2" width="18" customWidth="1"/><col min="3" max="146" width="16" customWidth="1"/><col min="147" max="152" width="22" customWidth="1"/></cols><sheetData>\' + \'\'.join(rows) + \'</sheetData><pageMargins left="0.3" right="0.3" top="0.5" bottom="0.5" header="0.2" footer="0.2"/></worksheet>\'\n'
735     '\n'
736     'def _display_day(day: date, case: str) -> str:\n'
737     "    return '附件1示例日' if case == 'problem1' else day.isoformat()\n"
738     '\n'
739     'def _plain_sheet(headers: list[str], values: list[list], widths=None) -> str:\n'
740     '    """Write an explicitly new sheet without any external authoring library."""\n'
741     '    rows = []\n'
742     '    for number, row in enumerate([headers] + values, 1):\n'
743     "        style = '1' if number == 1 else '2'\n"
744     "        cells = ''.join((scalar_cell(f'{colname(c)}{number}', value, style=style) for c, value in enumerate(row, 1)))\n"
745     '        rows.append(f\'<row r="{number}" ht="30" customHeight="1">{cells}</row>\')\n'
746     '    columns = \'\'.join((f\'<col min="{i}" max="{i}" width="{width}" customWidth="1"/>\' for i, width in enumerate(widths or [22] * len(headers), 1)))\n'
747     '    return f\'\'\'<?xml version="1.0" encoding="UTF-8" standalone="yes"?><worksheet xmlns="{S}"><dimension ref="A1:{colname(len(headers))}{len(rows)}"/><sheetViews><sheetView showGridLines="0" workbookViewId="0"><pane ySplit="1" topLeftCell="A2" activePane="bottomLeft" state="frozen"/></sheetView></sheetViews><sheetFormatPr defaultRowHeight="30"/><cols>{columns}</cols><sheetData>{\'\'.join(rows)}</sheetData><pageMargins left="0.3" right="0.3" top="0.5" bottom="0.5" header="0.2" footer="0.2"/></worksheet>\'\'\'\n'
748     '\n'
749     'def _new_workbook_parts(sheets: list[tuple[str, str]]) -> dict[str, bytes]:\n'
750     '    types = [\'<Default Extension="rels" ContentType="application/vnd.openxmlformats-package.relationships+xml"/>\', \'<Default Extension="xml" ContentType="application/xml"/>\', \'<Override PartName="/xl/workbook.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet.main+xml"/>\', \'<Override PartName="/xl/styles.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.styles+xml"/>\']\n'
751     '    relations, entries = ([], [])\n'
752     '    parts = {}\n'
753     '    for index, (name, xml) in enumerate(sheets, 1):\n'
754     "        member = f'xl/worksheets/sheet{index}.xml'\n"
755     "        parts[member] = xml.encode('utf-8')\n"
756     '        types.append(f\'<Override PartName="/{member}" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.worksheet+xml"/>\')\n'
757     '        relations.append(f\'<Relationship Id="rId{index}" Type="{R}/worksheet" Target="worksheets/sheet{index}.xml"/>\')\n'
758     '        entries.append(f\'<sheet name="{escape(name)}" sheetId="{index}" r:id="rId{index}"/>\')\n'
759     '    relations.append(f\'<Relationship Id="rId{len(sheets) + 1}" Type="{R}/styles" Target="styles.xml"/>\')\n'
760     '    parts[\'[Content_Types].xml\'] = f"""<Types xmlns="http://schemas.openxmlformats.org/package/2006/content-types">{\'\'.join(types)}</Types>""".encode()\n'
761     '    parts[\'_rels/.rels\'] = f\'<Relationships xmlns="{P}"><Relationship Id="rId1"
```

```
    Type="{R}/officeDocument" Target="xl/workbook.xml"/></Relationships>\'.encode()\n'
762     '    parts[\'xl/_rels/workbook.xml.rels\'] = f\'\'\'<Relationships xmlns="{P}">{\'\'.
    join(relations)}</Relationships>\'\'\'.encode()\n'
763     '    parts[\'xl/workbook.xml\'] = f\'\'\'<workbook xmlns="{S}" xmlns:r="{R}"><bookViews
    ><workbookView/></bookViews><sheets>{\'\'.join(entries)}</sheets></workbook>\'\'\'.encode()
    \n'
764     '    parts[\'xl/styles.xml\'] = f\'<styleSheet xmlns="{S}"><fonts count="2"><font><sz
    val="11"/><name val="Microsoft YaHei"/></font><font><b/><sz val="11"/><color rgb="FFFFFFFF
    "/><name val="Microsoft YaHei"/></font></fonts><fills count="3"><fill><patternFill
    patternType="none"/></fill><fill><patternFill patternType="gray125"/></fill><fill><
    patternFill patternType="solid"><fgColor rgb="FF245A81"/><bgColor indexed="64"/></
    patternFill></fill></fills><borders count="1"><border/></borders><cellStyleXfs count="1"><
    xf numFmtId="0" fontId="0" fillId="0" borderId="0"/></cellStyleXfs><cellXfs count="3"><xf
    numFmtId="0" fontId="0" fillId="0" borderId="0" xfId="0"/><xf numFmtId="0" fontId="1"
    fillId="2" borderId="0" xfId="0" applyAlignment="1"><alignment vertical="center" wrapText
    ="1"/></xf><xf numFmtId="0" fontId="0" fillId="0" borderId="0" xfId="0" applyAlignment
    ="1"><alignment vertical="center" wrapText="1"/></xf></cellXfs><cellStyles count="1"><
    cellStyle name="Normal" xfId="0" builtinId="0"/></cellStyles></styleSheet>\'.encode()\n'
765     '    return parts\n'
766     '\n'
767     'def _standalone_parts(data: dict, case: str) -> tuple[dict[str, bytes], int]:\n'
768     '    storage, emergency, settlement = ([], [], [])\n'
769     '    event_count = 0\n'
770     "    for i, day in enumerate(data['dates']):\n"
771     '        label = _display_day(day, case)\n'
772     '        for block in range(6):\n'
773     '            a, b = (240 * block, 240 * (block + 1))\n'
774     "            storage.append([label, f'{clock(a)}-{clock(b)}', float(data['
    charge_internal'][i, a:b].sum() / 0.9), float(data['discharge_internal'][i, a:b].sum() *
    0.9), float(data['soc'][i, a]), float(data['soc'][i, b])])\n"
775     "        events = intervals(data['emergency'][i])\n"
776     '        event_count += len(events)\n'
777     "        emergency.extend([[label, f'{clock(a)}-{clock(b)}', energy] for a, b, energy
    in events] or [[label, '无', 0.0]])\n"
778     "        costs = data['daily'][i]\n"
779     "        settlement.append([label, float(data['purchase_initial'][i].sum()), float(data
    ['purchase_final'][i].sum()), float(data['charge_internal'][i].sum() / 0.9), float(data['
    discharge_internal'][i].sum() * 0.9), float(data['emergency'][i].sum()), float(data['
    surplus'][i].sum()), float(data['soc'][i, 0]), float(data['soc'][i, -1]), costs['
    initial_cost'], costs['adjustment_cost'], costs['emergency_cost'], costs['total_cost']])\n"
780     "    notes = [['文件来源', '未找到附件5对应模板；本文件为程序独立生成的完整结果表，不声称保
    留官方模板结构。'], ['问题', case], ['计算天数', len(data['dates'])], ['计算范围', '附件1未标
    日期的示例日' if case == 'problem1' else '2025-02-01至2025-12-31，共334个自然日'], ['购电区间
    ', '完整自然日表逐项列出00:00-00:10至23:50-24:00的144段，保留初始及最终计划。'], ['充放电口径
    ', '购电侧充电量=内部储能增量/0.9；供负载侧放电量=内部储能减少量×0.9；每4小时汇总。'], ['紧急购
    电', '按实际连续分钟区间逐事件列出；无事件的日期以‘无’和0列示，事件数不设上限。'], ['费用口径
    ', '总购电费=初始计划款+已接受调整净费用+紧急购电费；紧急单价为该分钟对应电价的5倍。'], ['数据
    来源', '同目录all_results.npz及daily.csv；数值保留为数值单元格。'], ['日期说明', 'Problem1显示
    ‘附件1示例日’，NPZ/CSV日期仅作内部关联键；其他问题显示真实2025年日期。']]\n"
781     "    sheets = [('导出说明', _plain_sheet(['项目', '说明'], notes, [22, 115])), (
```

```
    SUPPLEMENT, supplement_xml({}, data, case)), ('充放电量', _plain_sheet(['日期', '时段', '充电购电侧(kWh)', '放电供负载侧(kWh)', '时段起始SOC(kWh)', '时段结束SOC(kWh)'], storage)), ('紧急购电量', _plain_sheet(['日期', '实际连续分钟时段', '紧急购电量(kWh)'], emergency)), ('日电量与费用', _plain_sheet(['日期', '初始计划购电量(kWh)', '最终计划购电量(kWh)', '充电购电侧(kWh)', '放电供负载侧(kWh)', '紧急购电量(kWh)', '未利用电量(kWh)', '日初SOC(kWh)', '日末SOC(kWh)', '初始计划款(元)', '调整净费用(元)', '紧急购电费(元)', '总购电费(元)'], settlement))]\n"
782     '    return (_new_workbook_parts(sheets), event_count)\n'
783     '\n'
784     'def beautify_excel(path: Path) -> None:\n'
785     '    """Apply the final workbook appearance without changing calculated values.\n'
786     '\n'
787     '    - Every non-empty cell in every sheet uses 宋体 10 pt.\n'
788     '    - Every non-empty cell is horizontally and vertically centered.\n'
789     '    - The last row of “紧急购电量” gets a black bottom border across A:C.\n'
790     '    Existing bold/italic/color/fill/number-format settings and all other border sides are preserved.\n'
791     '    """\n'
792     '    import openpyxl\n'
793     '    from copy import copy\n'
794     '    from openpyxl.styles import Side\n'
795     '\n'
796     '    workbook = openpyxl.load_workbook(path)\n'
797     '    try:\n'
798     '        for sheet in workbook.worksheets:\n'
799     '            for row in sheet.iter_rows():\n'
800     '                for cell in row:\n'
801     '                    if cell.value is None:\n'
802     '                        continue\n'
803     '\n'
804     '                    font = copy(cell.font)\n'
805     "                    font.name = '宋体'\n"
806     '                    font.sz = 10\n'
807     '                    font.scheme = None\n'
808     '                    cell.font = font\n'
809     '\n'
810     '                    alignment = copy(cell.alignment)\n'
811     "                    alignment.horizontal = 'center'\n"
812     "                    alignment.vertical = 'center'\n"
813     '                    cell.alignment = alignment\n'
814     '\n'
815     "        if '紧急购电量' in workbook.sheetnames:\n"
816     "            sheet = workbook['紧急购电量']\n"
817     '            last_row = sheet.max_row\n'
818     "            black = Side(style='thin', color='FF000000')\n"
819     '            for cell in sheet[last_row][:3]:\n'
820     '                border = copy(cell.border)\n'
821     '                border.bottom = black\n'
822     '                cell.border = border\n'
823     '\n'
824     '        workbook.save(path)\n'
```

```
        '    finally:\n'
        '        workbook.close()\n'
        '\n'
        '\n'
        'def export_one(case, problem_output_dir, attachment_dir, template_dir=None):\n'
        '    folder = Path(problem_output_dir).resolve()\n'
        '    attachments = Path(attachment_dir).resolve()\n'
        "    templates = Path(template_dir).resolve() if template_dir is not None else attachments / '附件5'\n"
        '    source = templates / FILES[case]\n'
        '    destination = folder / FILES[case]\n'
        '    data = load_results(folder, case)\n'
        "    temporary = destination.with_suffix('.pending.xlsx')\n"
        '\n'
        '    if source.is_file():\n'
        '        with ZipFile(source) as archive:\n'
        '            infos = {info.filename: copy.copy(info) for info in archive.infolist()}\n'
        '            parts = {name: archive.read(name) for name in infos}\n'
        '        paths = sheet_paths(parts)\n'
        "        required = {'计划购电量', '充放电量'} | ({'紧急购电量'} if case != 'problem1' else set())\n"
        "        if case in {'problem3', 'problem4-3'}:\n"
        "            required.add('调整购电量')\n"
        '        if required - paths.keys():\n'
        "            raise ValueError('附件5中的结果模板工作表不完整')\n"
        '        for name, member in paths.items():\n'
        "            xml = parts[member].decode('utf-8')\n"
        "            if name in {'计划购电量', '调整购电量'}:\n"
        "                xml = write_purchase(xml, data, final=name == '调整购电量', single=case == 'problem1')\n"
        "            elif name == '充放电量':\n"
        "                xml = write_storage(xml, data, single=case == 'problem1')\n"
        "            elif name == '紧急购电量':\n"
        '                xml, _ = write_emergency(xml, data)\n'
        "            parts[member] = xml.encode('utf-8')\n"
        "        with ZipFile(temporary, 'w', compression=ZIP_DEFLATED, compresslevel=6) as output:\n"
        '            for name, content in parts.items():\n'
        '                output.writestr(infos.get(name, name), content)\n'
        '    else:\n'
        '        parts, _ = _standalone_parts(data, case)\n'
        "        with ZipFile(temporary, 'w', compression=ZIP_DEFLATED, compresslevel=6) as output:\n"
        '            for name, content in parts.items():\n'
        '                output.writestr(name, content)\n'
        '\n'
        '    temporary.replace(destination)\n'
        '    beautify_excel(destination)\n'
        '    return destination\n'
),
```

```
}

def _load_module(name,source,data_dir):
    parts=name.split('.')
    for i in range(1,len(parts)):
        parent='.'.join(parts[:i])
        if parent not in sys.modules:
            package=types.ModuleType(parent); package.__path__=[]; sys.modules[parent]=package
    filename=str(_SCRIPT_DIR/'__embedded__'/Path(*parts).with_suffix('.py'))
    module=types.ModuleType(name)
    module.__file__=filename
    module.__package__=name.rpartition('.')[0]
    module.__spec__=importlib.util.spec_from_loader(name,loader=None,origin=filename)
    module.__dict__.update(_SCRIPT_DIR=_SCRIPT_DIR,_DATA_DIR=data_dir)
    linecache.cache[filename]=(len(source),None,source.splitlines(keepends=True),filename)
    sys.modules[name]=module
    if '.' in name: setattr(sys.modules[name.rpartition('.')[0]],parts[-1],module)
    exec(compile(source,filename,'exec'),module.__dict__)
    return module

def main():
    parser=argparse.ArgumentParser()
    parser.add_argument('--input-dir',type=Path)
    parser.add_argument('--output-dir',type=Path)
    parser.add_argument('--templates-dir',type=Path)
    parser.add_argument('--max-days',type=int)
    args=parser.parse_args()

    data_dir=(args.input_dir or (Path(ATTACHMENT_DIR) if ATTACHMENT_DIR.strip() else
    _SCRIPT_DIR.parent/'附件')).resolve()
    if not (data_dir/'附件1.xlsx').exists() and (data_dir/'附件'/'附件1.xlsx').exists():
        data_dir=data_dir/'附件'
    required=[1, 2]
    missing=[data_dir/f'附件{n}.xlsx' for n in required if not (data_dir/f'附件{n}.xlsx').
    is_file()]
    if missing: parser.error('缺少附件: \n'+'\n'.join(map(str,missing)))
    if args.max_days is not None and not 1<=args.max_days<=334:
        parser.error('--max-days 超出范围')

    out=(args.output_dir or (Path(OUTPUT_DIR) if OUTPUT_DIR.strip() else _SCRIPT_DIR.parent/'
    results'/_SCRIPT_PATH.stem)).resolve()
    out.mkdir(parents=True,exist_ok=True)
    template_dir=(args.templates_dir or (Path(TEMPLATE_DIR) if TEMPLATE_DIR.strip() else
    data_dir/'附件5')).resolve()

    try:
        import numpy as np
        import scipy
        import openpyxl
    except ImportError as e:
```

```
        parser.error('缺少依赖：'+str(e)+'；请安装 numpy scipy openpyxl')

    modules={name:_load_module(name,source,data_dir) for name,source in _MODULE_SOURCES.items()
    }
    print(f'开始{CASE_NAME}；结果：{out}',flush=True)
    modules['_entry'].run(args.max_days,out)
    with np.load(out/'all_results.npz') as z: days=len(z['dates'])
    if CASE=='problem1' or days==334:
        destination=modules['standalone_outputs'].export_one(CASE,out,data_dir,template_dir)
        print(f'Excel 已生成：{destination}',flush=True)
    else:
        print('快速检查仅计算数据；完整 Excel 需运行完整评价期。',flush=True)

if __name__=='__main__':
    main()
```

附录 B. 问题三求解程序

Listing 3: 问题三求解程序

```
# 问题三：融合多版光伏预测与历史偏差，在 0/6/12/18 时判断是否重新购电。
from pathlib import Path
import argparse
from datetime import datetime,timedelta
import importlib.util
import linecache
import os
import sys
import types

ATTACHMENT_DIR = r""
OUTPUT_DIR = r""
TEMPLATE_DIR = r""
CASE = 'problem3'
CASE_NAME = '问题三'

os.environ.setdefault('OMP_NUM_THREADS','1')
os.environ.setdefault('OPENBLAS_NUM_THREADS','1')
_SCRIPT_PATH=Path(__file__).resolve()
_SCRIPT_DIR=_SCRIPT_PATH.parent

_MODULE_SOURCES = {
    'problem2_reference': (
        'from __future__ import annotations\n'
        'from dataclasses import dataclass\n'
        'from datetime import date,datetime,time,timedelta\n'
        'from typing import Any\n'
        'import numpy as np\n'
        'import openpyxl\n'
```

```
        '\n'
        '\n'
        'MINUTES_PER_DAY = 1440\n'
        '\n'
        'SLOTS_PER_DAY = 144\n'
        '\n'
        'MINUTES_PER_SLOT = 10\n'
        '\n'
        'ETA = 0.9\n'
        '\n'
        'POWER_LIMIT_KW = 5000.0\n'
        '\n'
        'SOC_INITIAL_KWH = 6000.0\n'
        '\n'
        'SOC_MIN_KWH = 1200.0\n'
        '\n'
        'SOC_MAX_KWH = 10800.0\n'
        '\n'
        'TOL = 1e-08\n'
        '\n'
        '@dataclass(frozen=True)\n'
        'class ActualData:\n'
        '    dates: list[date]\n'
        '    load_power_endpoints: np.ndarray\n'
        '    pv_power_endpoints: np.ndarray\n'
        '    load_energy: np.ndarray\n'
        '    pv_energy: np.ndarray\n'
        '\n'
        '@dataclass(frozen=True)\n'
        'class ActualDispatch:\n'
        '    discharge_internal: np.ndarray\n'
        '    charge_internal: np.ndarray\n'
        '    emergency: np.ndarray\n'
        '    surplus: np.ndarray\n'
        '    soc: np.ndarray\n'
        '    joint_adjustment: np.ndarray\n'
        '\n'
        'def _as_date(value: Any) -> date:\n'
        '    if isinstance(value, datetime):\n'
        '        return value.date()\n'
        '    if isinstance(value, date):\n'
        '        return value\n'
        '    return datetime.fromisoformat(str(value)).date()\n'
        '\n'
        'def _minutes(value: Any) -> int:\n'
        '    if isinstance(value, time):\n'
        '        return value.hour * 60 + value.minute\n'
        '    text = str(value).strip()\n'
        "    offset = 1440 if '+1' in text else 0\n"
        "    text = text.replace('+1', '')\n"
```

```
        "    hour, minute = (int(part) for part in text.split(':'))\n"
        '    return offset + 60 * hour + minute\n'
        '\n'
        'def load_price(path: Path) -> np.ndarray:\n'
        '    workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)\n'
        '    sheet = workbook.active\n'
        "    first_column = 2 if sheet.cell(1, 1).value == '序号' else 1\n"
        '    headers = [sheet.cell(1, column).value for column in range(first_column,
    first_column + 4)]\n'
        "    expected = ['时间', '电价', '小区负载', '光伏发电预测功率']\n"
        '    if headers != expected:\n'
        "        raise ValueError(f'附件1表头不匹配: {headers!r}')\n"
        '    rows = list(sheet.iter_rows(min_row=2, min_col=first_column, max_col=first_column
    + 3, values_only=True))\n'
        '    if len(rows) != SLOTS_PER_DAY:\n'
        "        raise ValueError(f'附件1应有144条数据, 实际为{len(rows)}条')\n"
        '    endpoints = np.array([_minutes(row[0]) for row in rows], dtype=int)\n'
        '    if not np.array_equal(endpoints, np.arange(10, 1441, 10)):\n'
        "        raise ValueError('附件1时间列不是0:10至次日0:00的十分钟序列')\n"
        '    price = np.array([float(row[1]) for row in rows], dtype=float)\n'
        '    if not np.isfinite(price).all() or np.any(price < 0):\n'
        "        raise ValueError('附件1电价含空值、非有限值或负值')\n"
        '    return price\n'
        '\n'
        'def _read_power_sheet(sheet: Any) -> tuple[list[date], np.ndarray]:\n'
        '    headers = [sheet.cell(1, column).value for column in range(1, 146)]\n'
        '    endpoints = np.array([_minutes(value) for value in headers[1:]], dtype=int)\n'
        '    if not np.array_equal(endpoints, np.arange(10, 1441, 10)):\n'
        "        raise ValueError(f'{sheet.title}时间表头不是0:10至次日0:00的序列')\n"
        '    rows = list(sheet.iter_rows(min_row=2, max_col=145, values_only=True))\n'
        '    dates = [_as_date(row[0]) for row in rows]\n'
        '    values = np.array([[float(value) for value in row[1:]] for row in rows])\n'
        '    if values.shape != (365, 144):\n'
        "        raise ValueError(f'{sheet.title}数据维度错误: {values.shape}')\n"
        '    if not np.isfinite(values).all() or np.any(values < 0):\n'
        "        raise ValueError(f'{sheet.title}含空值、非有限值或负值')\n"
        '    return (dates, values)\n'
        '\n'
        'def _minute_energy(power_endpoints: np.ndarray) -> np.ndarray:\n'
        '    days = power_endpoints.shape[0]\n'
        '    output = np.empty((days, MINUTES_PER_DAY), dtype=float)\n'
        '    sample_minutes = np.arange(0, MINUTES_PER_DAY + 1, MINUTES_PER_SLOT)\n'
        '    minute_points = np.arange(MINUTES_PER_DAY + 1)\n'
        '    for day_index in range(days):\n'
        '        if day_index == 0:\n'
        '            start_power = power_endpoints[day_index, 0]\n'
        '        else:\n'
        '            start_power = power_endpoints[day_index - 1, -1]\n'
        '        samples = np.concatenate(([start_power], power_endpoints[day_index]))\n'
        '        minute_power = np.interp(minute_points, sample_minutes, samples)\n'
```

```
        '        output[day_index] = (minute_power[:-1] + minute_power[1:]) / 120.0\n'
        '    output[0, :MINUTES_PER_SLOT] = np.nan\n'
        '    return output\n'
        '\n'
        'def load_actual(path: Path) -> ActualData:\n'
        '    workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)\n'
        "    expected_sheets = ['小区负载', '光伏发电实际功率']\n"
        '    if workbook.sheetnames != expected_sheets:\n'
        "        raise ValueError(f'附件2工作表不匹配: {workbook.sheetnames!r}')\n"
        '    load_dates, load_power = _read_power_sheet(workbook[expected_sheets[0]])\n'
        '    pv_dates, pv_power = _read_power_sheet(workbook[expected_sheets[1]])\n'
        '    if load_dates != pv_dates:\n'
        "        raise ValueError('附件2两个工作表的日期不一致')\n"
        '    expected_dates = [date(2025, 1, 1) + timedelta(days=i) for i in range(365)]\n'
        '    if load_dates != expected_dates:\n'
        "        raise ValueError('附件2日期不是2025-01-01至2025-12-31的连续序列')\n"
        '    return ActualData(dates=load_dates, load_power_endpoints=load_power,
    pv_power_endpoints=pv_power, load_energy=_minute_energy(load_power), pv_energy=
    _minute_energy(pv_power))\n'
        '\n'
        'def _weighted_baseline(values: np.ndarray, target_index: int, history_end: int) -> np.
    ndarray:\n'
        '    if history_end <= 0:\n'
        "        raise ValueError('历史集合为空')\n"
        '    history_indices = np.arange(history_end)\n'
        '    raw_distance = np.abs(target_index - history_indices)\n'
        '    ring_distance = np.minimum(raw_distance, 365 - raw_distance)\n'
        '    weights = 1.0 - 2.0 * ring_distance / 365.0\n'
        '    history = values[:history_end]\n'
        '    valid = np.isfinite(history)\n'
        '    weighted_sum = np.nansum(history * weights[:, None], axis=0)\n'
        '    weight_sum = np.sum(valid * weights[:, None], axis=0)\n'
        '    if np.any(weight_sum <= 0):\n'
        "        raise ValueError('历史加权基准存在零权重分母')\n"
        '    return weighted_sum / weight_sum\n'
        '\n'
        'def execute_day(initial_soc: float, purchase: np.ndarray, planned_discharge: np.
    ndarray, planned_charge: np.ndarray, load_actual: np.ndarray, pv_actual: np.ndarray) ->
    ActualDispatch:\n'
        '    purchase_minute = np.repeat(purchase / MINUTES_PER_SLOT, MINUTES_PER_SLOT)\n'
        '    discharge = np.zeros(MINUTES_PER_DAY)\n'
        '    charge = np.zeros(MINUTES_PER_DAY)\n'
        '    emergency = np.zeros(MINUTES_PER_DAY)\n'
        '    surplus = np.zeros(MINUTES_PER_DAY)\n'
        '    soc = np.empty(MINUTES_PER_DAY + 1)\n'
        '    soc[0] = initial_soc\n'
        '    joint = np.zeros(MINUTES_PER_DAY)\n'
        '    y_max = POWER_LIMIT_KW / 60.0\n'
        '    z_max = ETA * POWER_LIMIT_KW / 60.0\n'
        '    kappa = 1.0 / ETA - ETA\n'
```

```
        '    for minute in range(MINUTES_PER_DAY):\n'
        '        state = soc[minute]\n'
        '        y_plan = float(planned_discharge[minute])\n'
        '        z_plan = float(planned_charge[minute])\n'
        '        q_plan = state - y_plan + z_plan\n'
        '        if q_plan < SOC_MIN_KWH:\n'
        '            y = state + z_plan - SOC_MIN_KWH\n'
        '            z = z_plan\n'
        '        elif q_plan > SOC_MAX_KWH:\n'
        '            y = y_plan\n'
        '            z = SOC_MAX_KWH - state + y_plan\n'
        '        else:\n'
        '            y, z = (y_plan, z_plan)\n'
        '        y = min(max(y, 0.0), y_max)\n'
        '        z = min(max(z, 0.0), z_max)\n'
        '        q = state - y + z\n'
        '        residual = load_actual[minute] + z / ETA - pv_actual[minute] - purchase_minute
    [minute] - ETA * y\n'
        '        if residual > TOL:\n'
        '            amount = min(z, ETA * residual, max(q - SOC_MIN_KWH, 0.0))\n'
        '            z -= amount\n'
        '            residual -= amount / ETA\n'
        '            q = state - y + z\n'
        '            amount = min(y_max - y, residual / ETA, max(q - SOC_MIN_KWH, 0.0))\n'
        '            y += amount\n'
        '            residual -= ETA * amount\n'
        '            q = state - y + z\n'
        '            if residual > TOL and y > TOL and (z > TOL):\n'
        '                amount = min(y, z, residual / kappa)\n'
        '                y -= amount\n'
        '                z -= amount\n'
        '                residual -= kappa * amount\n'
        '                joint[minute] = amount\n'
        '            if residual > TOL:\n'
        '                emergency[minute] = residual\n'
        '            else:\n'
        '                residual = 0.0\n'
        '        elif residual < -TOL:\n'
        '            unused = -residual\n'
        '            amount = min(y, unused / ETA, max(SOC_MAX_KWH - q, 0.0))\n'
        '            y -= amount\n'
        '            unused -= ETA * amount\n'
        '            q = state - y + z\n'
        '            amount = min(z_max - z, ETA * unused, max(SOC_MAX_KWH - q, 0.0))\n'
        '            z += amount\n'
        '            unused -= amount / ETA\n'
        '            if unused > TOL:\n'
        '                surplus[minute] = unused\n'
        '        discharge[minute] = max(y, 0.0)\n'
        '        charge[minute] = max(z, 0.0)\n'
```

```
        '        soc[minute + 1] = state - discharge[minute] + charge[minute]\n'
        '    return ActualDispatch(discharge, charge, emergency, surplus, soc, joint)\n'
    ),

    'forecast': (
        '"""Causal hourly PV fusion. Does not implement or change dispatch rules."""\n'
        'from __future__ import annotations\n'
        'from datetime import datetime, timedelta\n'
        'import numpy as np\n'
        'import openpyxl\n'
        'ALPHAS = (0.0, 0.25, 0.5, 0.75, 1.0)\n'
        'ROOT = _SCRIPT_DIR\n'
        '\n'
        'def read_inputs(root=ROOT):\n'
        '    actual = {}\n'
        "    wb = openpyxl.load_workbook(_DATA_DIR / '附件2.xlsx', read_only=True, data_only=True)\n"
        "    for row in wb['光伏发电实际功率'].iter_rows(min_row=2, values_only=True):\n"
        '        if row[0] is None:\n'
        '            continue\n'
        '        day = row[0] if isinstance(row[0], datetime) else datetime.fromisoformat(str(row[0]))\n'
        '        for i, value in enumerate(row[1:145], 1):\n'
        '            if value is not None:\n'
        '                actual[day + timedelta(minutes=10 * i)] = float(value)\n'
        '    wb.close()\n'
        '    forecasts = {}\n'
        "    wb = openpyxl.load_workbook(_DATA_DIR / '附件3.xlsx', read_only=True, data_only=True)\n"
        '    day = None\n'
        '    for row in wb.active.iter_rows(min_row=2, values_only=True):\n'
        "        if row[0] not in (None, ''):\n"
        '            if isinstance(row[0], datetime):\n'
        '                day = row[0].replace(hour=0, minute=0, second=0)\n'
        '            else:\n'
        "                day = datetime.strptime(str(row[0]), '%Y-%m-%d')\n"
        "        if row[1] in (None, ''):\n"
        '            continue\n'
        '        clock = row[1]\n'
        "        hour = clock.hour if hasattr(clock, 'hour') else int(str(clock).split(':')[0])\n"
        '        issue = day + timedelta(hours=hour)\n'
        '        values = tuple((float(v) for v in row[2:26]))\n'
        '        if len(values) != 24 or not all(np.isfinite(values)):\n'
        "            raise ValueError(f'Invalid forecast at {issue}')\n"
        '        forecasts[issue] = values\n'
        '    wb.close()\n'
        '    return (forecasts, actual)\n'
        '\n'
        'class Fusion:\n'
```

```
        '\n'
        '    def __init__(self, forecasts, actual):\n'
        '        self.forecasts = forecasts\n'
        '        self.actual = actual\n'
        '        self.previous = {}\n'
        '        self.last_issue = None\n'
        '\n'
        '    def components(self, issue, lead):\n'
        '        if not 1 <= lead <= 24:\n'
        "            raise ValueError('lead must be 1..24')\n"
        '        weights, versions = ([], [])\n'
        '        for k in range((24 - lead) // 6, -1, -1):\n'
        '            old = issue - timedelta(hours=6 * k)\n'
        '            weights.append((4 - k) / 10)\n'
        '            versions.append(self.forecasts[old][lead + 6 * k - 1])\n'
        '        f = float(np.dot(weights, versions) / sum(weights))\n'
        '        values, errors = ([], [])\n'
        '        for j in range(1, 8):\n'
        '            old = issue - timedelta(days=j)\n'
        '            target = old + timedelta(hours=lead)\n'
        '            if target > issue:\n'
        "                raise AssertionError('Historical target not yet realized')\n"
        '            value = self.actual[target]\n'
        '            values.append(value)\n'
        '            errors.append(self.forecasts[old][lead - 1] - value)\n'
        '        mean_actual = float(np.mean(values))\n'
        '        mean_error = float(np.mean(errors))\n'
        '        return (f, mean_actual + mean_error, mean_actual, mean_error)\n'
        '\n'
        '    def predict(self, issue):\n'
        '        if self.last_issue is not None and issue <= self.last_issue:\n'
        "            raise ValueError('Process issue times in strictly increasing order')\n"
        '        output = []\n'
        '        next_previous = dict(self.previous)\n'
        '        for lead in range(1, 25):\n'
        '            f, g, real_mean, bias_mean = self.components(issue, lead)\n'
        '            samples = []\n'
        '            for j in range(1, 8):\n'
        '                old = issue - timedelta(days=j)\n'
        '                target = old + timedelta(hours=lead)\n'
        '                if target > issue:\n'
        "                    raise AssertionError('Unrealized training label')\n"
        '                old_f, old_g, _, _ = self.components(old, lead)\n'
        '                samples.append((old_f, old_g, self.actual[target]))\n'
        '            losses = [float(np.mean([abs(a * x + (1 - a) * y - z) for x, y, z in
    samples])) for a in ALPHAS]\n'
        '            best = min(losses)\n'
        '            ties = [a for a, loss in zip(ALPHAS, losses) if abs(loss - best) <= 1e
    -08]\n'
        '            previous = self.previous.get((issue.hour, lead))\n'
```

```
        '            alpha = previous if previous in ties else min(ties, key=lambda a: (abs(a - 0.5), -a))\n'
        '            next_previous[issue.hour, lead] = alpha\n'
        "            output.append(dict(issue=issue.isoformat(), target=(issue + timedelta(hours=lead)).isoformat(), lead=lead, alpha=alpha, multi_version=f, history_component=g, history_actual_mean=real_mean, history_signed_error_mean=bias_mean, prediction=alpha * f + (1 - alpha) * g, **{f'mae_{a:g}': loss for a, loss in zip(ALPHAS, losses)}))\n"
        '        self.previous = next_previous\n'
        '        self.last_issue = issue\n'
        '        return output\n'
        '\n'
        'def minute_energy(current_power, hourly_power):\n'
        '    endpoints = np.interp(np.arange(1441), np.arange(25) * 60, [current_power, *hourly_power])\n'
        '    if len(hourly_power) != 24 or not np.all(np.isfinite(endpoints)):\n'
        "        raise ValueError('Need 24 finite hourly endpoints and a current observation')\n"
        '    return (endpoints[:-1] + endpoints[1:]) / 120\n'
        '\n'
        "if __name__ == '__main__':\n"
        '    main()\n'
    ),

    'dispatch': (
        '"""Problem 3 deterministic two-path comparison and chronological execution."""\n'
        'from __future__ import annotations\n'
        'import inspect\n'
        'from datetime import datetime, timedelta\n'
        'from pathlib import Path\n'
        'from types import SimpleNamespace\n'
        'import numpy as np\n'
        'from scipy.optimize import linprog\n'
        'from scipy.sparse import coo_matrix\n'
        'from forecast import Fusion, read_inputs, minute_energy, ROOT\n'
        'HERE = Path(__file__).resolve().parent\n'
        'import problem2_reference as problem2\n'
        "_source = inspect.getsource(problem2.execute_day).replace('MINUTES_PER_DAY', 'len(load_actual)')\n"
        '_namespace = dict(vars(problem2))\n'
        'exec(_source, _namespace)\n'
        "execute = _namespace['execute_day']\n"
        '\n'
        'def net_adjustment(x, original, premium, price):\n'
        '    total = original + premium\n'
        '    added = np.maximum(x - total, 0)\n'
        '    cancelled = np.maximum(total - x, 0)\n'
        '    cancel_premium = np.minimum(cancelled, premium)\n'
        '    cancel_original = cancelled - cancel_premium\n'
        '    cost = price * (1.5 * added - 1.0 * cancel_premium - 0.5 * cancel_original)\n'
        '    return (cost, original - cancel_original, premium - cancel_premium + added)\n'
```

```
        '\n'
        'def solve(load, pv, initial, prices, target, hard=False, ledger=None, penalty=None):\n
    '
        '    """Minute Y/Z/S, ten-minute X, convex adjustment cost epigraph."""\n'
        '    n = len(load)\n'
        '    m = n // 10\n'
        '    assert n % 10 == 0 and len(prices) == m\n'
        '    iy = m\n'
        '    iz = iy + n\n'
        '    iw = iz + n\n'
        '    iss = iw + n\n'
        '    ia = iss + n + 1\n'
        '    ic = ia + 1\n'
        '    size = ic + m\n'
        '    t = np.arange(n)\n'
        '    slots = np.arange(m)\n'
        '    rows = []\n'
        '    cols = []\n'
        '    vals = []\n'
        '\n'
        '    def add(r, c, v):\n'
        '        rows.extend(np.asarray(r).tolist())\n'
        '        cols.extend(np.asarray(c).tolist())\n'
        '        vals.extend(np.broadcast_to(v, np.shape(r)).tolist())\n'
        '    add(t, t // 10, 0.1)\n'
        '    add(t, iy + t, 0.9)\n'
        '    add(t, iz + t, -1 / 0.9)\n'
        '    add(t, iw + t, -1.0)\n'
        '    add(n + t, iss + t + 1, 1.0)\n'
        '    add(n + t, iss + t, -1.0)\n'
        '    add(n + t, iy + t, 1.0)\n'
        '    add(n + t, iz + t, -1.0)\n'
        '    eq = coo_matrix((vals, (rows, cols)), shape=(2 * n, size)).tocsr()\n'
        '    rhs = np.r_[load - pv, np.zeros(n)]\n'
        '    bounds = [(0, None)] * m + [(0, 5000 / 60)] * n + [(0, 0.9 * 5000 / 60)] * n +
    [(0, None)] * n + [(1200, 10800)] * (n + 1) + [(0, None)] + [(None, None)] * m\n'
        '    bounds[iss] = (initial, initial)\n'
        '    if hard:\n'
        '        bounds[iss + n] = (target, target)\n'
        '    ur = []\n'
        '    uc = []\n'
        '    uv = []\n'
        '    ub = []\n'
        '\n'
        '    def constraint(entries, b):\n'
        '        row = len(ub)\n'
        '        for c, v in entries:\n'
        '            ur.append(row)\n'
        '            uc.append(c)\n'
        '            uv.append(v)\n'
```

```
        '        ub.append(b)\n'
        '    constraint([(iss + n, 1), (ia, -1)], target)\n'
        '    constraint([(iss + n, -1), (ia, -1)], -target)\n'
        '    for j, p in enumerate(prices):\n'
        '        if ledger is not None and j < len(ledger[0]):\n'
        '            o, u = (ledger[0][j], ledger[1][j])\n'
        '            for slope, intercept in ((0.5 * p, -0.5 * p * o - p * u), (p, -p * (o + u)
    ), (1.5 * p, -1.5 * p * (o + u))):\n'
        '                constraint([(j, slope), (ic + j, -1)], -intercept)\n'
        '        else:\n'
        '            constraint([(j, p), (ic + j, -1)], 0.0)\n'
        '    aub = coo_matrix((uv, (ur, uc)), shape=(len(ub), size)).tocsr()\n'
        '    objective = np.zeros(size)\n'
        '    if not hard and penalty is None:\n'
        '        objective[ia] = 1\n'
        "        first = linprog(objective, A_ub=aub, b_ub=ub, A_eq=eq, b_eq=rhs, bounds=bounds
    , method='highs')\n"
        '        if not first.success:\n'
        '            raise RuntimeError(first.message)\n'
        '        bounds[ia] = (0, max(0, float(first.fun)) + 1e-06)\n'
        '    objective[:] = 0\n'
        '    objective[ic:] = 1\n'
        '    if penalty is not None and (not hard):\n'
        '        assert penalty >= 0\n'
        '        objective[ia] = penalty\n'
        "    result = linprog(objective, A_ub=aub, b_ub=ub, A_eq=eq, b_eq=rhs, bounds=bounds,
    method='highs')\n"
        '    if not result.success:\n'
        '        raise RuntimeError(result.message)\n'
        '    v = result.x\n'
        '    if np.max(np.abs(eq @ v - rhs)) > 1e-05:\n'
        "        raise AssertionError('LP equality residual')\n"
        '    return SimpleNamespace(purchase=v[:m].copy(), y=v[iy:iz].copy(), z=v[iz:iw].copy()
    , soc=v[iss:ia].copy(), cost=float(result.fun) if penalty is None else float(v[ic:].sum()),
     weighted_cost=float(result.fun), deviation=abs(v[iss + n] - target))\n'
    '\n'
    'def load_prediction(data, target, cutoff):\n'
    '    base = problem2._weighted_baseline(data.load_energy, target, cutoff)\n'
    '    lag = target - 7\n'
    '    previous_base = problem2._weighted_baseline(data.load_energy, lag, lag)\n'
    '    return np.maximum(0, base * data.load_energy[lag] / previous_base)\n'
    '\n'
    'def check_actual(result, purchase, load, pv):\n'
    '    y, z, h, w, s = (result.discharge_internal, result.charge_internal, result.
    emergency, result.surplus, result.soc)\n'
    '    error = pv + np.repeat(purchase / 10, 10) + 0.9 * y + h - load - z / 0.9 - w\n'
    '    assert np.max(np.abs(error)) < 1e-05\n'
    '    assert np.min(s) >= 1200 - 1e-05 and np.max(s) <= 10800 + 1e-05\n'
    '    assert np.max(y) <= 5000 / 60 + 1e-05 and np.max(z) <= 0.9 * 5000 / 60 + 1e-05\n'
    '    assert min(np.min(y), np.min(z), np.min(h), np.min(w)) >= -1e-06\n'
```

```
        '    assert np.max(np.abs(np.diff(s) + y - z)) < 1e-05\n'
        '    return float(np.max(np.abs(error)))\n'
    ),

    'experiment_latest_first6': (
        'from forecast import Fusion\n'
        '\n'
        'class LatestFirstSix(Fusion):\n'
        '\n'
        '    def predict(self, issue):\n'
        '        rows = super().predict(issue)\n'
        '        for row in rows:\n'
        "            row['dynamic_prediction'] = row['prediction']\n"
        "            row['latest_prediction'] = self.forecasts[issue][row['lead'] - 1]\n"
        "            row['applied_method'] = 'latest' if row['lead'] <= 6 else 'dynamic'\n"
        "            if row['lead'] <= 6:\n"
        "                row['prediction'] = row['latest_prediction']\n"
        '        return rows\n'
    ),

    'price_reference': (
        '"""Causal four-week price forecast and frozen Monday--Sunday SOC targets."""\n'
        'import numpy as np\n'
        'import openpyxl\n'
        'import problem2_reference as problem2\n'
        'TARGETS = (6960.0, 5040.0, 3120.0, 1200.0, 6000.0, 10800.0, 8880.0)\n'
        '\n'
        'def load_prices(path, dates):\n'
        '    wb = openpyxl.load_workbook(path, data_only=True, read_only=True)\n'
        '    rows = list(wb.active.values)\n'
        '    wb.close()\n'
        '    assert len(rows[0]) == 145\n'
        '    assert [problem2._as_date(r[0]) for r in rows[1:]] == dates\n'
        '    assert [problem2._minutes(v) for v in rows[0][1:]] == list(range(10, 1441, 10))\n'
        '    values = np.asarray([r[1:] for r in rows[1:]], float)\n'
        '    assert np.isfinite(values).all() and np.min(values) >= 0\n'
        '    return values\n'
        '\n'
        'def forecast_price(prices, target):\n'
        '    assert target >= 28\n'
        '    return np.mean(prices[[target - 7, target - 14, target - 21, target - 28]], axis
    =0)\n'
    ),

    'runner': (
        'from datetime import datetime,timedelta\n'
        'import csv\n'
        'from pathlib import Path\n'
        'import numpy as np\n'
        'import dispatch as engine\n'
```

```
    'from forecast import read_inputs,minute_energy\n'
    'from experiment_latest_first6 import LatestFirstSix\n'
    'from price_reference import forecast_price,load_prices,TARGETS\n'
    '\n'
    'def write_csv(path,rows):\n'
    "    with Path(path).open('w',encoding='utf-8-sig',newline='') as f:\n"
    '        w=csv.DictWriter(f,fieldnames=list(rows[0])); w.writeheader(); w.writerows(rows)\n'
    '\n'
    'def problem3_decision(state,purchase,y,z,original,premium,hour,load,pv,prices):\n'
    '    minute=hour*60; slot=hour*6; remaining=1440-minute\n'
    '    keep=engine.execute(state,purchase[slot:],y[minute:],z[minute:],load[:remaining],pv[:remaining])\n'
    '    target=(1-hour/24)*float(keep.soc[-1])+hour/24*state\n'
    '    tail=engine.solve(load[remaining:],pv[remaining:],float(keep.soc[-1]),prices[:slot],target,penalty=0.5)\n'
    '    keep_score=tail.cost+float(5*np.repeat(prices[slot:],10)@keep.emergency)+0.5*tail.deviation\n'
    '    change=engine.solve(load,pv,state,np.r_[prices[slot:],prices[:slot]],state,\n'
    '                        ledger=(original[slot:],premium[slot:]),penalty=0.5)\n'
    '    return change.cost+0.5*change.deviation < keep_score-1e-6, change\n'
    '\n'
    'def problem4_3_decision(state,purchase,y,z,original,premium,hour,load,pv,prices,target):\n'
    '    slot=hour*6\n'
    '    keep=engine.execute(state,purchase[slot:],y[hour*60:],z[hour*60:],load,pv)\n'
    '    change=engine.solve(load,pv,state,prices[slot:],target,ledger=(original[slot:],premium[slot:]),penalty=1.0)\n'
    '    changed=engine.execute(state,change.purchase,change.y,change.z,load,pv)\n'
    '    keep_score=float(5*np.repeat(prices[slot:],10)@keep.emergency)+abs(float(keep.soc[-1])-target)\n'
    '    change_score=change.cost+float(5*np.repeat(prices[slot:],10)@changed.emergency)+abs(float(changed.soc[-1])-target)\n'
    '    return change_score < keep_score-1e-6, change\n'
    '\n'
    'def run(start,end,out,scenario):\n'
    '    out=Path(out); out.mkdir(parents=True,exist_ok=True)\n'
    "    data=engine.problem2.load_actual(_DATA_DIR/'附件2.xlsx')\n"
    "    fixed=engine.problem2.load_price(_DATA_DIR/'附件1.xlsx')\n"
    "    actual_prices=np.tile(fixed,(365,1)) if scenario=='problem3' else load_prices(_DATA_DIR/'附件4.xlsx',data.dates)\n"
    '    forecasts,actual_power=read_inputs()\n'
    '    fusion=LatestFirstSix(forecasts,actual_power)\n'
    '    state=6000.0; daily=[]\n'
    "    collected={k:[] for k in ('purchase_initial','purchase_final','price','soc','charge_internal','discharge_internal','emergency','surplus')}\n"
    '    dates=[]\n'
    '    for index,day in enumerate(data.dates):\n'
    '        midnight=datetime.combine(day,datetime.min.time())\n'
    '        if not start<=midnight<end: continue\n'
```

```
    "        target=state if scenario=='problem3' else TARGETS[day.weekday()]\n"
    "        prices=fixed.copy() if scenario=='problem3' else forecast_price(actual_prices,
index)\n"
    '        load_day=engine.load_prediction(data,index,index)\n'
    '        next_load=engine.load_prediction(data,index+1,index)\n'
    '        coeff=np.zeros(144); cancelled_o=np.zeros(144); cancelled_u=np.zeros(144);
added_all=np.zeros(144)\n'
    '        traces=[]; executed=[]\n'
    '        for hour in (0,6,12,18):\n'
    '            minute=hour*60; slot=hour*6; remaining=1440-minute\n'
    '            issue=midnight+timedelta(hours=hour)\n'
    '            prediction=fusion.predict(issue)\n'
    "            pv24=minute_energy(actual_power[issue],[row['prediction'] for row in
prediction])\n"
    '            load24=np.r_[load_day[minute:],next_load[:minute]]\n'
    '            if hour==0:\n'
    "                plan=engine.solve(load_day,pv24,state,prices,target,hard=scenario=='
problem3')\n"
    '                purchase,y,z=plan.purchase.copy(),plan.y.copy(),plan.z.copy()\n'
    '                initial_purchase=purchase.copy(); original=purchase.copy(); premium=np
.zeros(144)\n'
    '            else:\n'
    "                if scenario=='problem3':\n"
    '                    accepted,change=problem3_decision(state,purchase,y,z,original,
premium,hour,load24,pv24,prices)\n'
    '                else:\n'
    '                    accepted,change=problem4_3_decision(state,purchase,y,z,original,
premium,hour,\n'
    '                                                        load24[:remaining],pv24[:
remaining],prices,target)\n'
    '                if accepted:\n'
    '                    new_purchase=change.purchase[:144-slot]\n'
    '                    old_total=original[slot:]+premium[slot:]\n'
    '                    cancelled=np.maximum(old_total-new_purchase,0)\n'
    '                    cu=np.minimum(cancelled,premium[slot:]); co=cancelled-cu\n'
    '                    added=np.maximum(new_purchase-old_total,0)\n'
    '                    delta,new_o,new_u=engine.net_adjustment(new_purchase,original[slot
:],premium[slot:],np.ones(144-slot))\n'
    '                    coeff[slot:]+=delta; cancelled_o[slot:]+=co; cancelled_u[slot:]+=
cu; added_all[slot:]+=added\n'
    '                    original[slot:],premium[slot:]=new_o,new_u\n'
    '                    purchase[slot:]=new_purchase\n'
    '                    y[minute:],z[minute:]=change.y[:remaining],change.z[:remaining]\n'
    '            stop=minute+360; endslot=slot+36\n'
    '            executed.append(purchase[slot:endslot].copy())\n'
    '            ex=engine.execute(state,purchase[slot:endslot],y[minute:stop],z[minute:
stop],\n'
    '                              data.load_energy[index,minute:stop],data.pv_energy[index
,minute:stop])\n'
    '            engine.check_actual(ex,purchase[slot:endslot],data.load_energy[index,
```

```
    minute:stop],data.pv_energy[index,minute:stop])\n'
        '            state=float(ex.soc[-1]); traces.append(ex)\n'
        '        assert np.array_equal(purchase,np.concatenate(executed))\n'
        '        real_price=actual_prices[index]\n'
        '        initial_cost=float(real_price@initial_purchase)\n'
        '        adjustment_cost=float(real_price@coeff)\n'
        '        assert abs(initial_cost+adjustment_cost-float(real_price@(original+1.5*premium
    ))-float(0.5*real_price@(cancelled_o+cancelled_u)))<1e-5\n'
        '        h=np.concatenate([x.emergency for x in traces])\n'
        '        y_actual=np.concatenate([x.discharge_internal for x in traces])\n'
        '        z_actual=np.concatenate([x.charge_internal for x in traces])\n'
        '        surplus=np.concatenate([x.surplus for x in traces])\n'
        '        soc=np.r_[traces[0].soc,*[x.soc[1:] for x in traces[1:]]]\n'
        '        emergency_cost=float(5*np.repeat(real_price,10)@h)\n'
        '        dates.append(str(day))\n'
        '        daily.append(dict(date=str(day),initial_purchase_cost=initial_cost,
    adjustment_net_cost=adjustment_cost,\n'
        '                          emergency_cost=emergency_cost,total_cost=initial_cost+
    adjustment_cost+emergency_cost))\n'
        '        for key,value in {\n'
        "            'purchase_initial':initial_purchase,'purchase_final':purchase,'price':
    real_price,'soc':soc,\n"
        "            'charge_internal':z_actual,'discharge_internal':y_actual,'emergency':h,'
    surplus':surplus}.items():\n"
        '            collected[key].append(value.copy())\n'
        "        if len(daily)%30==0: print(f'Problem3 completed {len(daily)} days', flush=True
    )\n"
        "    if not daily: raise ValueError('没有可计算日期')\n"
        "    np.savez_compressed(out/'all_results.npz',dates=np.asarray(dates),**{k:np.stack(v)
     for k,v in collected.items()})\n"
        "    write_csv(out/'daily.csv',daily)\n"
    ),

    'standalone_outputs': (
        'from __future__ import annotations\n'
        'import copy\n'
        'import csv\n'
        'import math\n'
        'import posixpath\n'
        'import re\n'
        'from datetime import date, timedelta\n'
        'from pathlib import Path\n'
        'from xml.etree import ElementTree as ET\n'
        'from xml.sax.saxutils import escape\n'
        'from zipfile import ZipFile, ZIP_DEFLATED\n'
        'import numpy as np\n'
        '\n'
        "S = 'http://schemas.openxmlformats.org/spreadsheetml/2006/main'\n"
        "R = 'http://schemas.openxmlformats.org/officeDocument/2006/relationships'\n"
        "P = 'http://schemas.openxmlformats.org/package/2006/relationships'\n"
```

```
        "SUPPLEMENT = '完整自然日购电量'\n"
        "FILES = {'problem1':'result1.xlsx','problem2':'result2.xlsx','problem3':'result3.xlsx','problem4-2':'result4-2.xlsx','problem4-3':'result4-3.xlsx'}\n"
        "ROW_RE = re.compile('<row\\\\b[^>]*?(?:/>|>.*?</row>)', re.S)\n"
        "CELL_RE = re.compile('<c\\\\b[^>]*?(?:/>|>.*?</c>)', re.S)\n"
        '\n'
        '\n'
        'def colname(number: int) -> str:\n'
        "    result = ''\n"
        '    while number:\n'
        '        number, rem = divmod(number - 1, 26)\n'
        '        result = chr(65 + rem) + result\n'
        '    return result\n'
        '\n'
        'def colnumber(name: str) -> int:\n'
        '    result = 0\n'
        '    for char in name:\n'
        '        result = 26 * result + ord(char) - 64\n'
        '    return result\n'
        '\n'
        'def address_key(ref: str) -> tuple[int, int]:\n'
        "    match = re.fullmatch('([A-Z]+)([0-9]+)', ref)\n"
        '    if match is None:\n'
        "        raise ValueError(f'Invalid cell reference {ref}')\n"
        '    return (int(match[2]), colnumber(match[1]))\n'
        '\n'
        'def attr(fragment: str, name: str, default: str | None=None) -> str | None:\n'
        '    match = re.search(\'\\\\b\' + re.escape(name) + \'="([^"]*)"\', fragment)\n'
        '    return match[1] if match else default\n'
        '\n'
        'def scalar_cell(ref: str, value, original: str | None=None, style: str | None=None) -> str:\n'
        '    """Replace only a cell value; retain its original attributes except type."""\n'
        '    if original:\n'
        "        opening = re.match('<c\\\\b([^>]*?)(?:/?>)', original)[1]\n"
        '        opening = re.sub(\'\\\\s+t="[^"]*"\', \'\', opening).rstrip(\'/ \')\n'
        '        opening = re.sub(\'\\\\br="[^"]*"\', f\'r="{ref}"\', opening)\n'
        '    else:\n'
        '        opening = f\' r="{ref}"\' + (f\' s="{style}"\' if style is not None else \'\')\n'
        '    if value is None:\n'
        "        return f'<c{opening}/>'\n"
        '    if isinstance(value, (str, np.str_)):\n'
        '        return f\'<c{opening} t="inlineStr"><is><t xml:space="preserve">{escape(str(value))}</t></is></c>\'\n'
        '    numeric = float(value)\n'
        '    if not math.isfinite(numeric):\n'
        "        raise ValueError(f'Non-finite numeric output at {ref}: {value}')\n"
        '    if abs(numeric) < 1e-10:\n'
        '        numeric = 0.0\n'
```

```
        '    return f"<c{opening}><v>{format(numeric, \'.15g\')}</v></c>"\n'
        '\n'
        'def rows_in(xml: str) -> dict[int, str]:\n'
        "    data = re.search('<sheetData>(.*?)</sheetData>', xml, re.S)\n"
        '    if data is None:\n'
        "        raise ValueError('Template has no sheetData element')\n"
        "    return {int(attr(row.split('>', 1)[0], 'r')): row for row in ROW_RE.findall(data
    [1])}\n"
        '\n'
        'def patch_row(row: str, updates: dict[str, object], new_number: int | None=None) ->
    str:\n'
        "    opening = re.match('<row\\\\b([^>]*?)(?:/?>)', row)[1].rstrip('/ ')\n"
        "    old_number = int(attr(opening, 'r'))\n"
        '    new_number = old_number if new_number is None else new_number\n'
        '    opening = re.sub(\'\\\\br="[^"]*"\', f\'r="{new_number}"\', opening)\n'
        '    cells = {}\n'
        '    for cell in CELL_RE.findall(row):\n'
        "        ref = attr(cell.split('>', 1)[0], 'r')\n"
        "        column = re.match('[A-Z]+', ref)[0]\n"
        "        new_ref = f'{column}{new_number}'\n"
        '        cells[new_ref] = re.sub(\'\\\\br="[^"]*"\', f\'r="{new_ref}"\', cell, count=1)
    \n'
        "    default_style = attr(opening, 's')\n"
        '    for column_or_ref, value in updates.items():\n'
        "        column = re.match('[A-Z]+', column_or_ref)[0]\n"
        "        ref = f'{column}{new_number}'\n"
        '        cells[ref] = scalar_cell(ref, value, cells.get(ref), default_style)\n'
        "    return f'<row{opening}>' + ''.join((cells[k] for k in sorted(cells, key=
    address_key))) + '</row>'\n"
        '\n'
        'def replace_rows(xml: str, rows: list[str], last_cell: str | None=None) -> str:\n'
        "    xml = re.sub('<sheetData>.*?</sheetData>', lambda _: '<sheetData>' + ''.join(rows)
     + '</sheetData>', xml, count=1, flags=re.S)\n"
        '    if last_cell:\n'
        '        xml = re.sub(\'<dimension\\\\b[^>]*/>\', f\'<dimension ref="A1:{last_cell
    }"/>\', xml, count=1)\n'
        '    return xml\n'
        '\n'
        'def excel_date(day: date) -> int:\n'
        '    return (day - date(1899, 12, 30)).days\n'
        '\n'
        'def clock(minutes: int) -> str:\n'
        "    return f'{minutes // 60:02d}:{minutes % 60:02d}'\n"
        '\n'
        'def intervals(energy: np.ndarray, tolerance: float=1e-08) -> list[tuple[int, int,
    float]]:\n'
        '    positive = np.asarray(energy) > tolerance\n'
        '    padded = np.r_[False, positive, False].astype(np.int8)\n'
        '    starts = np.flatnonzero(np.diff(padded) == 1)\n'
        '    ends = np.flatnonzero(np.diff(padded) == -1)\n'
```

```
    '    return [(int(a), int(b), float(np.sum(energy[a:b]))) for a, b in zip(starts, ends)]\n'
    '\n'
    'def load_results(folder: Path, case: str) -> dict:\n'
    '    folder = Path(folder).resolve()\n'
    "    with np.load(folder / 'all_results.npz', allow_pickle=False) as source:\n"
    '        data = {key: np.array(source[key]) for key in source.files}\n'
    "    required = {'dates', 'purchase_initial', 'purchase_final', 'price', 'charge_internal', 'discharge_internal', 'emergency', 'surplus', 'soc'}\n"
    '    if required - data.keys():\n'
    "        raise ValueError(f'{case}: missing arrays {sorted(required - data.keys())}')\n"
    "    dates = [date.fromisoformat(str(value)[:10]) for value in data['dates']]\n"
    '    n = len(dates)\n'
    "    if case == 'problem1':\n"
    '        if n != 1:\n'
    "            raise ValueError('problem1 must contain exactly one modeled day')\n"
    '    elif dates != [date(2025, 2, 1) + timedelta(days=i) for i in range(334)]:\n'
    "        raise ValueError(f'{case}: official export requires all 334 dates from February 1 to December 31')\n"
    "    for key in required - {'dates'}:\n"
    "        width = 1441 if key == 'soc' else 144 if key in {'purchase_initial', 'purchase_final', 'price'} else 1440\n"
    '        if data[key].shape != (n, width) or not np.all(np.isfinite(data[key])):\n'
    "            raise ValueError(f'{case}: invalid {key} shape or nonfinite values: {data[key].shape}')\n"
    "    data['dates'] = dates\n"
    "    daily_path = next((p for p in [folder / 'daily.csv', folder / 'summary' / 'daily.csv'] if p.exists()), None)\n"
    '    if daily_path is None:\n'
    "        raise FileNotFoundError(f'Missing daily settlement CSV under {folder}')\n"
    "    with daily_path.open(encoding='utf-8-sig', newline='') as stream:\n"
    '        records = list(csv.DictReader(stream))\n'
    '    if not records:\n'
    "        raise ValueError('Daily settlement CSV has no rows')\n"
    "    date_key = next((key for key in ['date', '日期', 'day'] if key in records[0]), None)\n"
    '    if date_key is None:\n'
    "        raise ValueError(f'{daily_path}: need date column')\n"
    '    by_date = {date.fromisoformat(row[date_key][:10]): row for row in records}\n'
    '    if len(by_date) != len(records) or set(by_date) != set(dates):\n'
    "        raise ValueError('Daily CSV dates must match NPZ dates exactly without duplicates')\n"
    "    aliases = {'initial_cost': ['initial_cost', 'initial_purchase_cost', 'initial_plan_cost', 'planned_cost', 'plan_cost', 'purchase_cost'], 'adjustment_cost': ['adjustment_cost', 'adjustment_net_cost', 'adjustment_net', 'adjust_cost'], 'emergency_cost': ['emergency_cost', 'emergency_purchase_cost'], 'total_cost': ['total_cost', 'actual_total_cost', 'actual_cost', 'cost']}\n"
    '    daily = []\n'
    '    for i, day in enumerate(dates):\n'
```

```
        '        if day not in by_date:\n'
        "            raise ValueError(f'{daily_path}: missing {day}')\n"
        '        row = by_date[day]\n'
        '        values = {}\n'
        '        for key, names in aliases.items():\n'
        "            found = next((row[name] for name in names if name in row and row[name] != ''), None)\n"
        '            if found is None:\n'
        "                if key == 'adjustment_cost' and case in {'problem1', 'problem2', 'problem4-2'}:\n"
        '                    values[key] = 0.0\n'
        '                else:\n'
        "                    raise ValueError(f'{daily_path}: need {key} for {day}; available columns={list(row)}')\n"
        '            else:\n'
        '                values[key] = float(found)\n'
        '        if not all((math.isfinite(v) for v in values.values())):\n'
        "            raise ValueError(f'Nonfinite daily cost for {day}')\n"
        "        computed_total = values['initial_cost'] + values['adjustment_cost'] + values['emergency_cost']\n"
        "        if not math.isclose(computed_total, values['total_cost'], abs_tol=0.01, rel_tol=1e-09):\n"
        "            raise ValueError(f'Daily settlement does not reconcile for {case}/{day}: {values}')\n"
        "        actual_emergency = float(np.dot(data['emergency'][i], np.repeat(data['price'][i], 10)) * 5)\n"
        "        if not math.isclose(actual_emergency, values['emergency_cost'], abs_tol=0.01, rel_tol=1e-08):\n"
        "            raise ValueError(f'Emergency cost disagrees with minute trace for {case}/{day}')\n"
        "        actual_initial = float(np.dot(data['purchase_initial'][i], data['price'][i]))\n"
        "        if not math.isclose(actual_initial, values['initial_cost'], abs_tol=0.01, rel_tol=1e-08):\n"
        "            raise ValueError(f'Initial plan cost disagrees with 144-slot trace for {case}/{day}')\n"
        '        daily.append(values)\n'
        "    data['daily'] = daily\n"
        "    data['daily_csv'] = daily_path.relative_to(folder).as_posix()\n"
        '    return data\n'
        '\n'
        'def sheet_paths(parts: dict[str, bytes]) -> dict[str, str]:\n'
        "    workbook = ET.fromstring(parts['xl/workbook.xml'])\n"
        "    rels = {r.attrib['Id']: r.attrib['Target'] for r in ET.fromstring(parts['xl/_rels/workbook.xml.rels'])}\n"
        "    return {s.attrib['name']: posixpath.normpath('xl/' + rels[s.attrib[f'{{{R}}}id']]) for s in workbook.find(f'{{{S}}}sheets')}\n"
        '\n'
        'def write_purchase(xml: str, data: dict, *, final: bool, single: bool) -> str:\n'
        '    rows = rows_in(xml)\n'
```

```
        "    array = data['purchase_final' if final else 'purchase_initial']\n"
        '    if single:\n'
        '        for row in range(2, 145):\n'
        "            rows[row] = patch_row(rows[row], {'B': array[0, row - 1]})\n"
        "        rows[145] = patch_row(rows[145], {'B': None})\n"
        '    else:\n'
        "        for i, day in enumerate(data['dates']):\n"
        '            row = i + 2\n'
        '            updates = {colname(j + 1): array[i, j] for j in range(1, 144)}\n'
        "            updates['EO'] = array[i + 1, 0] if i + 1 < len(array) else None\n"
        "            updates['EP'] = float(array[i].sum())\n"
        "            costs = data['daily'][i]\n"
        "            updates['EQ'] = costs['initial_cost'] + (costs['adjustment_cost'] if final
     else 0)\n"
        '            rows[row] = patch_row(rows[row], updates)\n'
        '    return replace_rows(xml, [rows[r] for r in sorted(rows)])\n'
        '\n'
        'def write_storage(xml: str, data: dict, *, single: bool) -> str:\n'
        '    original = rows_in(xml)\n'
        '    eta = 0.9\n'
        '    if single:\n'
        '        for block in range(6):\n'
        '            take = slice(240 * block, 240 * (block + 1))\n'
        "            updates = {'B': float(data['charge_internal'][0, take].sum() / eta), 'C':
    float(data['discharge_internal'][0, take].sum() * eta)}\n"
        '            if block < 2:\n'
        "                updates['E'] = data['soc'][0, 0 if block == 0 else -1]\n"
        '            original[block + 2] = patch_row(original[block + 2], updates)\n'
        '        return replace_rows(xml, [original[r] for r in sorted(original)])\n'
        '    output = [original[1]]\n'
        "    for i, day in enumerate(data['dates']):\n"
        '        for block in range(6):\n'
        '            row = 2 + 6 * i + block\n'
        '            take = slice(240 * block, 240 * (block + 1))\n'
        "            updates = {'A': excel_date(day) if block == 0 else None, 'C': float(data['
    charge_internal'][i, take].sum() / eta), 'D': float(data['discharge_internal'][i, take].sum
    () * eta), 'F': data['soc'][i, 0 if block == 0 else -1] if block < 2 else None}\n"
        '            output.append(patch_row(original[block + 2], updates, row))\n'
        '    return replace_rows(xml, output, f"F{1 + 6 * len(data[\'dates\'])}")\n'
        '\n'
        'def write_emergency(xml: str, data: dict) -> tuple[str, int]:\n'
        '    original = rows_in(xml)\n'
        '    output = [original[1]]\n'
        '    count = 0\n'
        "    for i, day in enumerate(data['dates']):\n"
        "        events = intervals(data['emergency'][i])\n"
        '        count += len(events)\n'
        "        records = [(f'{clock(a)}-{clock(b)}', energy) for a, b, energy in events] or
    [('无', 0.0)]\n"
        '        for j, (label, energy) in enumerate(records):\n'
```

```
    '            prototype = original[2 if j == 0 else 4 if j == len(records) - 1 else 3]\n'
    "            output.append(patch_row(prototype, {'A': excel_date(day) if j == 0 else None, 'B': label, 'C': energy}, len(output) + 1))\n"
    "    return (replace_rows(xml, output, f'C{len(output)}'), count)\n"
    '\n'
    'def supplement_xml(parts: dict[str, bytes], data: dict, case: str) -> str:\n'
    '    """A compact additional matrix; preserve all original styles.xml bytes."""\n'
    '    if parts:\n'
    "        source = rows_in(parts[sheet_paths(parts)['计划购电量']].decode('utf-8'))\n"
    "        header_style = attr(CELL_RE.findall(source[1])[1].split('>', 1)[0], 's', '0')\n"
    "        value_style = attr(CELL_RE.findall(source[2])[-1].split('>', 1)[0], 's', '0')\n"
    '    else:\n'
    "        header_style, value_style = ('1', '2')\n"
    "    headings = ['日期', '计划类型'] + [f'{clock(10 * k)}-{clock(10 * (k + 1))}' for k in range(144)]\n"
    "    headings += ['全天购电量(kWh)', '初始计划款(元)', '调整净费用(元)', '紧急购电费(元)', '总购电费(元)', '紧急购电量(kWh)']\n"
    '    rows = [\'<row r="1" ht="28" customHeight="1">\' + \'\'.join((scalar_cell(f\'{colname(c)}1\', v, style=header_style) for c, v in enumerate(headings, 1))) + \'</row>\']\n'
    "    stage_rows = [('初始计划', 'purchase_initial'), ('最终有效计划', 'purchase_final')] if case in {'problem3', 'problem4-3'} else [('计划购电', 'purchase_initial')]\n"
    "    for i, day in enumerate(data['dates']):\n"
    "        costs = data['daily'][i]\n"
    '        for stage, key in stage_rows:\n'
    "            settlement = stage != '初始计划'\n"
    "            display_day = '附件1示例日' if case == 'problem1' else day.isoformat()\n"
    '            values = [display_day, stage] + data[key][i].tolist() + [float(data[key][i].sum())]\n'
    "            values += [costs['initial_cost'], costs['adjustment_cost'] if settlement else None, costs['emergency_cost'] if settlement else None, costs['total_cost'] if settlement else None, float(data['emergency'][i].sum()) if settlement else None]\n"
    '            r = len(rows) + 1\n'
    '            rows.append(f\'<row r="{r}" ht="20" customHeight="1">\' + \'\'.join((scalar_cell(f\'{colname(c)}{r}\', value, style=value_style) for c, value in enumerate(values, 1))) + \'</row>\')\n'
    '    return f\'<?xml version="1.0" encoding="UTF-8" standalone="yes"?><worksheet xmlns="{S}"><dimension ref="A1:EV{len(rows)}"/><sheetViews><sheetView showGridLines="0" workbookViewId="0"><pane xSplit="2" ySplit="1" topLeftCell="C2" activePane="bottomRight" state="frozen"/></sheetView></sheetViews><sheetFormatPr defaultRowHeight="20"/><cols><col min="1" max="2" width="18" customWidth="1"/><col min="3" max="146" width="16" customWidth="1"/><col min="147" max="152" width="22" customWidth="1"/></cols><sheetData>\' + \'\'.join(rows) + \'</sheetData><pageMargins left="0.3" right="0.3" top="0.5" bottom="0.5" header="0.2" footer="0.2"/></worksheet>\'\n'
    '\n'
    'def _display_day(day: date, case: str) -> str:\n'
    "    return '附件1示例日' if case == 'problem1' else day.isoformat()\n"
```

```
        '\n'
        'def _plain_sheet(headers: list[str], values: list[list], widths=None) -> str:\n'
        '    """Write an explicitly new sheet without any external authoring library."""\n'
        '    rows = []\n'
        '    for number, row in enumerate([headers] + values, 1):\n'
        "        style = '1' if number == 1 else '2'\n"
        "        cells = ''.join((scalar_cell(f'{colname(c)}{number}', value, style=style) for c, value in enumerate(row, 1)))\n"
        '        rows.append(f\'<row r="{number}" ht="30" customHeight="1">{cells}</row>\')\n'
        '    columns = \'\'.join((f\'<col min="{i}" max="{i}" width="{width}" customWidth="1"/>\' for i, width in enumerate(widths or [22] * len(headers), 1)))\n'
        '    return f\'\'\'<?xml version="1.0" encoding="UTF-8" standalone="yes"?><worksheet xmlns="{S}"><dimension ref="A1:{colname(len(headers))}{len(rows)}"/><sheetViews><sheetView showGridLines="0" workbookViewId="0"><pane ySplit="1" topLeftCell="A2" activePane="bottomLeft" state="frozen"/></sheetView></sheetViews><sheetFormatPr defaultRowHeight="30"/><cols>{columns}</cols><sheetData>{\'\'.join(rows)}</sheetData><pageMargins left="0.3" right="0.3" top="0.5" bottom="0.5" header="0.2" footer="0.2"/></worksheet>\'\'\'\n'
        '\n'
        'def _new_workbook_parts(sheets: list[tuple[str, str]]) -> dict[str, bytes]:\n'
        '    types = [\'<Default Extension="rels" ContentType="application/vnd.openxmlformats-package.relationships+xml"/>\', \'<Default Extension="xml" ContentType="application/xml"/>\', \'<Override PartName="/xl/workbook.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet.main+xml"/>\', \'<Override PartName="/xl/styles.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.styles+xml"/>\']\n'
        '    relations, entries = ([], [])\n'
        '    parts = {}\n'
        '    for index, (name, xml) in enumerate(sheets, 1):\n'
        "        member = f'xl/worksheets/sheet{index}.xml'\n"
        "        parts[member] = xml.encode('utf-8')\n"
        '        types.append(f\'<Override PartName="/{member}" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.worksheet+xml"/>\')\n'
        '        relations.append(f\'<Relationship Id="rId{index}" Type="{R}/worksheet" Target="worksheets/sheet{index}.xml"/>\')\n'
        '        entries.append(f\'<sheet name="{escape(name)}" sheetId="{index}" r:id="rId{index}"/>\')\n'
        '    relations.append(f\'<Relationship Id="rId{len(sheets) + 1}" Type="{R}/styles" Target="styles.xml"/>\')\n'
        '    parts[\'[Content_Types].xml\'] = f"""<Types xmlns="http://schemas.openxmlformats.org/package/2006/content-types">{\'\'.join(types)}</Types>""".encode()\n'
        '    parts[\'_rels/.rels\'] = f\'<Relationships xmlns="{P}"><Relationship Id="rId1" Type="{R}/officeDocument" Target="xl/workbook.xml"/></Relationships>\'.encode()\n'
        '    parts[\'xl/_rels/workbook.xml.rels\'] = f\'\'\'<Relationships xmlns="{P}">{\'\'.join(relations)}</Relationships>\'\'\'.encode()\n'
        '    parts[\'xl/workbook.xml\'] = f\'\'\'<workbook xmlns="{S}" xmlns:r="{R}"><bookViews><workbookView/></bookViews><sheets>{\'\'.join(entries)}</sheets></workbook>\'\'\'.encode()\n'
        '    parts[\'xl/styles.xml\'] = f\'<styleSheet xmlns="{S}"><fonts count="2"><font><sz val="11"/><name val="Microsoft YaHei"/></font><font><b/><sz val="11"/><color rgb="FFFFFFFF"/><name val="Microsoft YaHei"/></font></fonts><fills count="3"><fill><patternFill
```

```
    patternType="none"/></fill><fill><patternFill patternType="gray125"/></fill><fill><
    patternFill patternType="solid"><fgColor rgb="FF245A81"/><bgColor indexed="64"/></
    patternFill></fill></fills><borders count="1"><border/></borders><cellStyleXfs count="1"><
    xf numFmtId="0" fontId="0" fillId="0" borderId="0"/></cellStyleXfs><cellXfs count="3"><xf
    numFmtId="0" fontId="0" fillId="0" borderId="0" xfId="0"/><xf numFmtId="0" fontId="1"
    fillId="2" borderId="0" xfId="0" applyAlignment="1"><alignment vertical="center" wrapText
    ="1"/></xf><xf numFmtId="0" fontId="0" fillId="0" borderId="0" xfId="0" applyAlignment
    ="1"><alignment vertical="center" wrapText="1"/></xf></cellXfs><cellStyles count="1"><
    cellStyle name="Normal" xfId="0" builtinId="0"/></cellStyles></styleSheet>\'.encode()\n'
883     '    return parts\n'
884     '\n'
885     'def _standalone_parts(data: dict, case: str) -> tuple[dict[str, bytes], int]:\n'
886     '    storage, emergency, settlement = ([], [], [])\n'
887     '    event_count = 0\n'
888     "    for i, day in enumerate(data['dates']):\n"
889     '        label = _display_day(day, case)\n'
890     '        for block in range(6):\n'
891     '            a, b = (240 * block, 240 * (block + 1))\n'
892     "            storage.append([label, f'{clock(a)}-{clock(b)}', float(data['
    charge_internal'][i, a:b].sum() / 0.9), float(data['discharge_internal'][i, a:b].sum() *
    0.9), float(data['soc'][i, a]), float(data['soc'][i, b])])\n"
893     "        events = intervals(data['emergency'][i])\n"
894     '        event_count += len(events)\n'
895     "        emergency.extend([[label, f'{clock(a)}-{clock(b)}', energy] for a, b, energy
    in events] or [[label, '无', 0.0]])\n"
896     "        costs = data['daily'][i]\n"
897     "        settlement.append([label, float(data['purchase_initial'][i].sum()), float(data
    ['purchase_final'][i].sum()), float(data['charge_internal'][i].sum() / 0.9), float(data['
    discharge_internal'][i].sum() * 0.9), float(data['emergency'][i].sum()), float(data['
    surplus'][i].sum()), float(data['soc'][i, 0]), float(data['soc'][i, -1]), costs['
    initial_cost'], costs['adjustment_cost'], costs['emergency_cost'], costs['total_cost']])\n"
898     "    notes = [['文件来源', '未找到附件5对应模板；本文件为程序独立生成的完整结果表，不声称保
    留官方模板结构。'], ['问题', case], ['计算天数', len(data['dates'])], ['计算范围', '附件1未标
    日期的示例日' if case == 'problem1' else '2025-02-01至2025-12-31，共334个自然日'], ['购电区间
    ', '完整自然日表逐项列出00:00-00:10至23:50-24:00的144段，保留初始及最终计划。'], ['充放电口径
    ', '购电侧充电量=内部储能增量/0.9；供负载侧放电量=内部储能减少量×0.9；每4小时汇总。'], ['紧急购
    电', '按实际连续分钟区间逐事件列出；无事件的日期以‘无’和0列示，事件数不设上限。'], ['费用口径
    ', '总购电费=初始计划款+已接受调整净费用+紧急购电费；紧急单价为该分钟对应电价的5倍。'], ['数据
    来源', '同目录all_results.npz及daily.csv；数值保留为数值单元格。'], ['日期说明', 'Problem1显示
    ‘附件1示例日’，NPZ/CSV日期仅作内部关联键；其他问题显示真实2025年日期。']]\n"
899     "    sheets = [('导出说明', _plain_sheet(['项目', '说明'], notes, [22, 115])), (
    SUPPLEMENT, supplement_xml({}, data, case)), ('充放电量', _plain_sheet(['日期', '时段', '充
    电购电侧(kWh)', '放电供负载侧(kWh)', '时段起始SOC(kWh)', '时段结束SOC(kWh)'], storage)), ('紧
    急购电量', _plain_sheet(['日期', '实际连续分钟时段', '紧急购电量(kWh)'], emergency)), ('日电
    量与费用', _plain_sheet(['日期', '初始计划购电量(kWh)', '最终计划购电量(kWh)', '充电购电侧(kWh
    )', '放电供负载侧(kWh)', '紧急购电量(kWh)', '未利用电量(kWh)', '日初SOC(kWh)', '日末SOC(kWh)
    ', '初始计划款(元)', '调整净费用(元)', '紧急购电费(元)', '总购电费(元)'], settlement))]\n"
900     '    return (_new_workbook_parts(sheets), event_count)\n'
901     '\n'
902     'def beautify_excel(path: Path) -> None:\n'
```

```
        '    """Apply the final workbook appearance without changing calculated values.\n'
        '\n'
        '    - Every non-empty cell in every sheet uses 宋体 10 pt.\n'
        '    - Every non-empty cell is horizontally and vertically centered.\n'
        '    - The last row of “紧急购电量” gets a black bottom border across A:C.\n'
        '    Existing bold/italic/color/fill/number-format settings and all other border sides are preserved.\n'
        '    """\n'
        '    import openpyxl\n'
        '    from copy import copy\n'
        '    from openpyxl.styles import Side\n'
        '\n'
        '    workbook = openpyxl.load_workbook(path)\n'
        '    try:\n'
        '        for sheet in workbook.worksheets:\n'
        '            for row in sheet.iter_rows():\n'
        '                for cell in row:\n'
        '                    if cell.value is None:\n'
        '                        continue\n'
        '\n'
        '                    font = copy(cell.font)\n'
        "                    font.name = '宋体'\n"
        '                    font.sz = 10\n'
        '                    font.scheme = None\n'
        '                    cell.font = font\n'
        '\n'
        '                    alignment = copy(cell.alignment)\n'
        "                    alignment.horizontal = 'center'\n"
        "                    alignment.vertical = 'center'\n"
        '                    cell.alignment = alignment\n'
        '\n'
        "        if '紧急购电量' in workbook.sheetnames:\n"
        "            sheet = workbook['紧急购电量']\n"
        '            last_row = sheet.max_row\n'
        "            black = Side(style='thin', color='FF000000')\n"
        '            for cell in sheet[last_row][:3]:\n'
        '                border = copy(cell.border)\n'
        '                border.bottom = black\n'
        '                cell.border = border\n'
        '\n'
        '        workbook.save(path)\n'
        '    finally:\n'
        '        workbook.close()\n'
        '\n'
        '\n'
        'def export_one(case, problem_output_dir, attachment_dir, template_dir=None):\n'
        '    folder = Path(problem_output_dir).resolve()\n'
        '    attachments = Path(attachment_dir).resolve()\n'
        "    templates = Path(template_dir).resolve() if template_dir is not None else attachments / '附件5'\n"
```

```
        '    source = templates / FILES[case]\n'
        '    destination = folder / FILES[case]\n'
        '    data = load_results(folder, case)\n'
        "    temporary = destination.with_suffix('.pending.xlsx')\n"
        '\n'
        '    if source.is_file():\n'
        '        with ZipFile(source) as archive:\n'
        '            infos = {info.filename: copy.copy(info) for info in archive.infolist()}\n'
        '            parts = {name: archive.read(name) for name in infos}\n'
        '        paths = sheet_paths(parts)\n'
        "        required = {'计划购电量', '充放电量'} | ({'紧急购电量'} if case != 'problem1' else set())\n"
        "        if case in {'problem3', 'problem4-3'}:\n"
        "            required.add('调整购电量')\n"
        '        if required - paths.keys():\n'
        "            raise ValueError('附件5中的结果模板工作表不完整')\n"
        '        for name, member in paths.items():\n'
        "            xml = parts[member].decode('utf-8')\n"
        "            if name in {'计划购电量', '调整购电量'}:\n"
        "                xml = write_purchase(xml, data, final=name == '调整购电量', single=case == 'problem1')\n"
        "            elif name == '充放电量':\n"
        "                xml = write_storage(xml, data, single=case == 'problem1')\n"
        "            elif name == '紧急购电量':\n"
        '                xml, _ = write_emergency(xml, data)\n'
        "            parts[member] = xml.encode('utf-8')\n"
        "        with ZipFile(temporary, 'w', compression=ZIP_DEFLATED, compresslevel=6) as output:\n"
        '            for name, content in parts.items():\n'
        '                output.writestr(infos.get(name, name), content)\n'
        '    else:\n'
        '        parts, _ = _standalone_parts(data, case)\n'
        "        with ZipFile(temporary, 'w', compression=ZIP_DEFLATED, compresslevel=6) as output:\n"
        '            for name, content in parts.items():\n'
        '                output.writestr(name, content)\n'
        '\n'
        '    temporary.replace(destination)\n'
        '    beautify_excel(destination)\n'
        '    return destination\n'
    ),
}

def _load_module(name,source,data_dir):
    parts=name.split('.')
    for i in range(1,len(parts)):
        parent='.'.join(parts[:i])
        if parent not in sys.modules:
            package=types.ModuleType(parent); package.__path__=[]; sys.modules[parent]=package
    filename=str(_SCRIPT_DIR/'__embedded__'/Path(*parts).with_suffix('.py'))
```

```
    module=types.ModuleType(name)
    module.__file__=filename
    module.__package__=name.rpartition('.')[0]
    module.__spec__=importlib.util.spec_from_loader(name,loader=None,origin=filename)
    module.__dict__.update(_SCRIPT_DIR=_SCRIPT_DIR,_DATA_DIR=data_dir)
    linecache.cache[filename]=(len(source),None,source.splitlines(keepends=True),filename)
    sys.modules[name]=module
    if '.' in name: setattr(sys.modules[name.rpartition('.')[0]],parts[-1],module)
    exec(compile(source,filename,'exec'),module.__dict__)
    return module

def main():
    parser=argparse.ArgumentParser()
    parser.add_argument('--input-dir',type=Path)
    parser.add_argument('--output-dir',type=Path)
    parser.add_argument('--templates-dir',type=Path)
    parser.add_argument('--max-days',type=int)
    args=parser.parse_args()

    data_dir=(args.input_dir or (Path(ATTACHMENT_DIR) if ATTACHMENT_DIR.strip() else
    _SCRIPT_DIR.parent/'附件')).resolve()
    if not (data_dir/'附件1.xlsx').exists() and (data_dir/'附件'/'附件1.xlsx').exists():
        data_dir=data_dir/'附件'
    required=[1, 2, 3]
    missing=[data_dir/f'附件{n}.xlsx' for n in required if not (data_dir/f'附件{n}.xlsx').
    is_file()]
    if missing: parser.error('缺少附件: \n'+'\n'.join(map(str,missing)))
    if args.max_days is not None and not 1<=args.max_days<=334:
        parser.error('--max-days 超出范围')

    out=(args.output_dir or (Path(OUTPUT_DIR) if OUTPUT_DIR.strip() else _SCRIPT_DIR.parent/'
    results'/_SCRIPT_PATH.stem)).resolve()
    out.mkdir(parents=True,exist_ok=True)
    template_dir=(args.templates_dir or (Path(TEMPLATE_DIR) if TEMPLATE_DIR.strip() else
    data_dir/'附件5')).resolve()

    try:
        import numpy as np
        import scipy
        import openpyxl
    except ImportError as e:
        parser.error('缺少依赖: '+str(e)+'; 请安装 numpy scipy openpyxl')

    modules={name:_load_module(name,source,data_dir) for name,source in _MODULE_SOURCES.items()
    }
    print(f'开始{CASE_NAME}; 结果: {out}',flush=True)
    start=datetime(2025,2,1)
    end=datetime(2026,1,1) if args.max_days is None else start+timedelta(days=args.max_days)
    modules['runner'].run(start,end,out,CASE)
    with np.load(out/'all_results.npz') as z: days=len(z['dates'])
```

```
    if CASE=='problem1' or days==334:
        destination=modules['standalone_outputs'].export_one(CASE,out,data_dir,template_dir)
        print(f'Excel 已生成：{destination}',flush=True)
    else:
        print('快速检查仅计算数据；完整 Excel 需运行完整评价期。',flush=True)

if __name__=='__main__':
    main()
```

附录 B. 问题四-二求解程序

Listing 4: 问题四-二求解程序

```
# 问题四（2 口径）：以过去四个同星期数的分时电价均值作为次日价格预测。
from pathlib import Path
import argparse
from datetime import datetime,timedelta
import importlib.util
import linecache
import os
import sys
import types

ATTACHMENT_DIR = r""
OUTPUT_DIR = r""
TEMPLATE_DIR = r""
CASE = 'problem4-2'
CASE_NAME = '问题四对应问题二'

os.environ.setdefault('OMP_NUM_THREADS','1')
os.environ.setdefault('OPENBLAS_NUM_THREADS','1')
_SCRIPT_PATH=Path(__file__).resolve()
_SCRIPT_DIR=_SCRIPT_PATH.parent

_MODULE_SOURCES = {
    'problem2.core': (
        'from __future__ import annotations\n'
        'from dataclasses import dataclass\n'
        'from datetime import date,datetime,time,timedelta\n'
        'from typing import Any\n'
        'import numpy as np\n'
        'import openpyxl\n'
        'from scipy.optimize import linprog\n'
        'from scipy.sparse import csr_matrix,lil_matrix\n'
        '\n'
        '\n'
        'MINUTES_PER_DAY = 1440\n'
        '\n'
        'SLOTS_PER_DAY = 144\n'
```

```
        '\n'
        'MINUTES_PER_SLOT = 10\n'
        '\n'
        'ETA = 0.9\n'
        '\n'
        'POWER_LIMIT_KW = 5000.0\n'
        '\n'
        'SOC_INITIAL_KWH = 6000.0\n'
        '\n'
        'SOC_MIN_KWH = 1200.0\n'
        '\n'
        'SOC_MAX_KWH = 10800.0\n'
        '\n'
        'LOAD_WEEKLY_DEVIATION_FACTOR = 1.0\n'
        '\n'
        'LOAD_LAG_DAYS = 7\n'
        '\n'
        'PV_HISTORY_DAYS = 7\n'
        '\n'
        'RISK_QUANTILE = 0.8\n'
        '\n'
        'RISK_ERROR_HISTORY_START_INDEX = 9\n'
        '\n'
        'BUDGET_FACTOR = 1.05\n'
        '\n'
        'REFERENCE_SHOCK_FACTOR = 5.0\n'
        '\n'
        'REFERENCE_DECAY_DAYS = 180.0\n'
        '\n'
        'TOL = 1e-08\n'
        '\n'
        'LP_TOL = 1e-07\n'
        '\n'
        'LEX_TOL = 1e-06\n'
        '\n'
        '@dataclass(frozen=True)\n'
        'class ActualData:\n'
        '    dates: list[date]\n'
        '    load_power_endpoints: np.ndarray\n'
        '    pv_power_endpoints: np.ndarray\n'
        '    load_energy: np.ndarray\n'
        '    pv_energy: np.ndarray\n'
        '\n'
        '@dataclass(frozen=True)\n'
        'class PlanSolution:\n'
        '    purchase: np.ndarray\n'
        '    discharge_internal: np.ndarray\n'
        '    charge_internal: np.ndarray\n'
        '    surplus: np.ndarray\n'
        '    soc: np.ndarray\n'
```

```
        '    baseline_cost: float\n'
        '    minimum_deviation: float\n'
        '    cost: float\n'
        '    solver_messages: tuple[str, str, str]\n'
        '\n'
        '@dataclass(frozen=True)\n'
        'class ActualDispatch:\n'
        '    discharge_internal: np.ndarray\n'
        '    charge_internal: np.ndarray\n'
        '    emergency: np.ndarray\n'
        '    surplus: np.ndarray\n'
        '    soc: np.ndarray\n'
        '    joint_adjustment: np.ndarray\n'
        '\n'
        'def _as_date(value: Any) -> date:\n'
        '    if isinstance(value, datetime):\n'
        '        return value.date()\n'
        '    if isinstance(value, date):\n'
        '        return value\n'
        '    return datetime.fromisoformat(str(value)).date()\n'
        '\n'
        'def _minutes(value: Any) -> int:\n'
        '    if isinstance(value, time):\n'
        '        return value.hour * 60 + value.minute\n'
        '    text = str(value).strip()\n'
        "    offset = 1440 if '+1' in text else 0\n"
        "    text = text.replace('+1', '')\n"
        "    hour, minute = (int(part) for part in text.split(':'))\n"
        '    return offset + 60 * hour + minute\n'
        '\n'
        'def _read_power_sheet(sheet: Any) -> tuple[list[date], np.ndarray]:\n'
        '    headers = [sheet.cell(1, column).value for column in range(1, 146)]\n'
        '    endpoints = np.array([_minutes(value) for value in headers[1:]], dtype=int)\n'
        '    if not np.array_equal(endpoints, np.arange(10, 1441, 10)):\n'
        "        raise ValueError(f'{sheet.title}时间表头不是0:10至次日0:00的序列')\n"
        '    rows = list(sheet.iter_rows(min_row=2, max_col=145, values_only=True))\n'
        '    dates = [_as_date(row[0]) for row in rows]\n'
        '    values = np.array([[float(value) for value in row[1:]] for row in rows])\n'
        '    if values.shape != (365, 144):\n'
        "        raise ValueError(f'{sheet.title}数据维度错误: {values.shape}')\n"
        '    if not np.isfinite(values).all() or np.any(values < 0):\n'
        "        raise ValueError(f'{sheet.title}含空值、非有限值或负值')\n"
        '    return (dates, values)\n'
        '\n'
        'def _minute_energy(power_endpoints: np.ndarray) -> np.ndarray:\n'
        '    days = power_endpoints.shape[0]\n'
        '    output = np.empty((days, MINUTES_PER_DAY), dtype=float)\n'
        '    sample_minutes = np.arange(0, MINUTES_PER_DAY + 1, MINUTES_PER_SLOT)\n'
        '    minute_points = np.arange(MINUTES_PER_DAY + 1)\n'
        '    for day_index in range(days):\n'
```

```
        '        if day_index == 0:\n'
        '            start_power = power_endpoints[day_index, 0]\n'
        '        else:\n'
        '            start_power = power_endpoints[day_index - 1, -1]\n'
        '        samples = np.concatenate(([start_power], power_endpoints[day_index]))\n'
        '        minute_power = np.interp(minute_points, sample_minutes, samples)\n'
        '        output[day_index] = (minute_power[:-1] + minute_power[1:]) / 120.0\n'
        '    output[0, :MINUTES_PER_SLOT] = np.nan\n'
        '    return output\n'
        '\n'
        'def load_actual(path: Path) -> ActualData:\n'
        '    workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)\n'
        "    expected_sheets = ['小区负载', '光伏发电实际功率']\n"
        '    if workbook.sheetnames != expected_sheets:\n'
        "        raise ValueError(f'附件2工作表不匹配: {workbook.sheetnames!r}')\n"
        '    load_dates, load_power = _read_power_sheet(workbook[expected_sheets[0]])\n'
        '    pv_dates, pv_power = _read_power_sheet(workbook[expected_sheets[1]])\n'
        '    if load_dates != pv_dates:\n'
        "        raise ValueError('附件2两个工作表的日期不一致')\n"
        '    expected_dates = [date(2025, 1, 1) + timedelta(days=i) for i in range(365)]\n'
        '    if load_dates != expected_dates:\n'
        "        raise ValueError('附件2日期不是2025-01-01至2025-12-31的连续序列')\n"
        '    return ActualData(dates=load_dates, load_power_endpoints=load_power,
pv_power_endpoints=pv_power, load_energy=_minute_energy(load_power), pv_energy=
_minute_energy(pv_power))\n'
        '\n'
        'def _weighted_baseline(values: np.ndarray, target_index: int, history_end: int) -> np.
ndarray:\n'
        '    if history_end <= 0:\n'
        "        raise ValueError('历史集合为空')\n"
        '    history_indices = np.arange(history_end)\n'
        '    raw_distance = np.abs(target_index - history_indices)\n'
        '    ring_distance = np.minimum(raw_distance, 365 - raw_distance)\n'
        '    weights = 1.0 - 2.0 * ring_distance / 365.0\n'
        '    history = values[:history_end]\n'
        '    valid = np.isfinite(history)\n'
        '    weighted_sum = np.nansum(history * weights[:, None], axis=0)\n'
        '    weight_sum = np.sum(valid * weights[:, None], axis=0)\n'
        '    if np.any(weight_sum <= 0):\n'
        "        raise ValueError('历史加权基准存在零权重分母')\n"
        '    return weighted_sum / weight_sum\n'
        '\n'
        'def _point_forecast_day(actual: ActualData, day_index: int) -> tuple[np.ndarray, np.
ndarray]:\n'
        '    if day_index < RISK_ERROR_HISTORY_START_INDEX:\n'
        "        raise ValueError('负荷周周期预测的历史数据不足')\n"
        '    load_baseline = _weighted_baseline(actual.load_energy, day_index, day_index)\n'
        '    week_ago_index = day_index - LOAD_LAG_DAYS\n'
        '    week_ago_load_baseline = _weighted_baseline(actual.load_energy, week_ago_index,
week_ago_index)\n'
```

```
        '    week_ago_load = actual.load_energy[week_ago_index]\n'
        '    relative_deviation = np.divide(week_ago_load - week_ago_load_baseline,
    week_ago_load_baseline, out=np.zeros_like(week_ago_load), where=week_ago_load_baseline >
    TOL)\n'
        '    load_forecast = load_baseline * (1.0 + LOAD_WEEKLY_DEVIATION_FACTOR *
    relative_deviation)\n'
        '    pv_forecast = np.mean(actual.pv_energy[day_index - PV_HISTORY_DAYS:day_index],
    axis=0)\n'
        '    load_forecast = np.maximum(load_forecast, 0.0)\n'
        '    pv_forecast = np.maximum(pv_forecast, 0.0)\n'
        '    if not np.isfinite(load_forecast).all() or not np.isfinite(pv_forecast).all():\n'
        "        raise ValueError('预测结果含非有限值')\n"
        '    return (load_forecast, pv_forecast)\n'
        '\n'
        'def build_forecasts(actual: ActualData) -> tuple[np.ndarray, np.ndarray, np.ndarray,
    np.ndarray]:\n'
        '    """构造点预测和只使用已发生误差的风险调整预测。"""\n'
        '    days = len(actual.dates)\n'
        '    point_load = np.full((days, MINUTES_PER_DAY), np.nan, dtype=float)\n'
        '    pv_forecast = np.full((days, MINUTES_PER_DAY), np.nan, dtype=float)\n'
        '    net_errors = np.full((days, MINUTES_PER_DAY), np.nan, dtype=float)\n'
        '    risk_margin = np.full((days, MINUTES_PER_DAY), np.nan, dtype=float)\n'
        '    for day_index in range(RISK_ERROR_HISTORY_START_INDEX, days):\n'
        '        load_value, pv_value = _point_forecast_day(actual, day_index)\n'
        '        point_load[day_index] = load_value\n'
        '        pv_forecast[day_index] = pv_value\n'
        '        net_errors[day_index] = actual.load_energy[day_index] - actual.pv_energy[
    day_index] - load_value + pv_value\n'
        '    start_index = actual.dates.index(date(2025, 2, 1))\n'
        '    for day_index in range(start_index, days):\n'
        '        historical_errors = net_errors[RISK_ERROR_HISTORY_START_INDEX:day_index]\n'
        '        margin = np.quantile(historical_errors, RISK_QUANTILE, axis=0)\n'
        '        risk_margin[day_index] = np.maximum(margin, 0.0)\n'
        '    decision_load = point_load + risk_margin\n'
        '    if not np.isfinite(decision_load[start_index:]).all():\n'
        "        raise ValueError('风险调整负荷预测含非有限值')\n"
        '    return (decision_load, point_load, pv_forecast, risk_margin)\n'
        '\n'
        'class DailyPlanner:\n'
        '\n'
        '    def __init__(self, price: np.ndarray) -> None:\n'
        '        self.price = np.asarray(price, dtype=float)\n'
        '        self.n_x = SLOTS_PER_DAY\n'
        '        self.n_y = self.n_z = self.n_w = MINUTES_PER_DAY\n'
        '        self.n_s = MINUTES_PER_DAY + 1\n'
        '        self.off_x = 0\n'
        '        self.off_y = self.off_x + self.n_x\n'
        '        self.off_z = self.off_y + self.n_y\n'
        '        self.off_w = self.off_z + self.n_z\n'
        '        self.off_s = self.off_w + self.n_w\n'
```

```
        '        self.off_a = self.off_s + self.n_s\n'
        '        self.n_vars = self.off_a + 1\n'
        '        self.a_eq = self._build_equalities()\n'
        '        self.a_ub = self._build_inequalities()\n'
        '\n'
        '    def _build_equalities(self) -> csr_matrix:\n'
        '        matrix = lil_matrix((2 * MINUTES_PER_DAY, self.n_vars))\n'
        '        for minute in range(MINUTES_PER_DAY):\n'
        '            slot = minute // MINUTES_PER_SLOT\n'
        '            matrix[minute, self.off_x + slot] = 1.0 / MINUTES_PER_SLOT\n'
        '            matrix[minute, self.off_y + minute] = ETA\n'
        '            matrix[minute, self.off_z + minute] = -1.0 / ETA\n'
        '            matrix[minute, self.off_w + minute] = -1.0\n'
        '            row = MINUTES_PER_DAY + minute\n'
        '            matrix[row, self.off_y + minute] = 1.0\n'
        '            matrix[row, self.off_z + minute] = -1.0\n'
        '            matrix[row, self.off_s + minute] = -1.0\n'
        '            matrix[row, self.off_s + minute + 1] = 1.0\n'
        '        return matrix.tocsr()\n'
        '\n'
        '    def _build_inequalities(self) -> csr_matrix:\n'
        '        matrix = lil_matrix((3, self.n_vars))\n'
        '        matrix[0, self.off_x:self.off_x + self.n_x] = self.price\n'
        '        matrix[1, self.off_s + MINUTES_PER_DAY] = 1.0\n'
        '        matrix[1, self.off_a] = -1.0\n'
        '        matrix[2, self.off_s + MINUTES_PER_DAY] = -1.0\n'
        '        matrix[2, self.off_a] = -1.0\n'
        '        return matrix.tocsr()\n'
        '\n'
        '    def _bounds(self, initial_soc: float, terminal_soc: float | None=None,
    max_abs_deviation: float | None=None) -> list[tuple[float, float | None]]:\n'
        '        discharge_limit = POWER_LIMIT_KW / 60.0\n'
        '        charge_internal_limit = ETA * POWER_LIMIT_KW / 60.0\n'
        '        bounds: list[tuple[float, float | None]] = []\n'
        '        bounds.extend([(0.0, None)] * self.n_x)\n'
        '        bounds.extend([(0.0, discharge_limit)] * self.n_y)\n'
        '        bounds.extend([(0.0, charge_internal_limit)] * self.n_z)\n'
        '        bounds.extend([(0.0, None)] * self.n_w)\n'
        '        bounds.extend([(SOC_MIN_KWH, SOC_MAX_KWH)] * self.n_s)\n'
        '        bounds.append((0.0, max_abs_deviation))\n'
        '        bounds[self.off_s] = (initial_soc, initial_soc)\n'
        '        if terminal_soc is not None:\n'
        '            bounds[self.off_s + MINUTES_PER_DAY] = (terminal_soc, terminal_soc)\n'
        '        return bounds\n'
        '\n'
        '    def _rhs(self, load: np.ndarray, pv: np.ndarray) -> np.ndarray:\n'
        '        rhs = np.zeros(2 * MINUTES_PER_DAY)\n'
        '        rhs[:MINUTES_PER_DAY] = load - pv\n'
        '        return rhs\n'
        '\n'
```

```
    '    def solve(self, load_forecast: np.ndarray, pv_forecast: np.ndarray, initial_soc: float, reference_soc: float) -> PlanSolution:\n'
    '        rhs = self._rhs(load_forecast, pv_forecast)\n'
    '        cost_objective = np.zeros(self.n_vars)\n'
    '        cost_objective[:self.n_x] = self.price\n'
    "        baseline = linprog(cost_objective, A_eq=self.a_eq, b_eq=rhs, bounds=self._bounds(initial_soc, terminal_soc=initial_soc), method='highs')\n"
    '        if not baseline.success:\n'
    "            raise RuntimeError(f'基准成本LP失败: {baseline.message}')\n"
    '        baseline_cost = float(baseline.fun)\n'
    '        deviation_objective = np.zeros(self.n_vars)\n'
    '        deviation_objective[self.off_a] = 1.0\n'
    '        inequality_rhs = np.array([BUDGET_FACTOR * baseline_cost, reference_soc, -reference_soc])\n'
    "        deviation = linprog(deviation_objective, A_ub=self.a_ub, b_ub=inequality_rhs, A_eq=self.a_eq, b_eq=rhs, bounds=self._bounds(initial_soc), method='highs')\n"
    '        if not deviation.success:\n'
    "            raise RuntimeError(f'日末偏差LP失败: {deviation.message}')\n"
    '        minimum_deviation = max(0.0, float(deviation.x[self.off_a]))\n'
    "        final = linprog(cost_objective, A_ub=self.a_ub, b_ub=inequality_rhs, A_eq=self.a_eq, b_eq=rhs, bounds=self._bounds(initial_soc, max_abs_deviation=minimum_deviation + LEX_TOL), method='highs')\n"
    '        if not final.success:\n'
    "            raise RuntimeError(f'同偏差最低费用LP失败: {final.message}')\n"
    '        vector = final.x\n'
    '        return PlanSolution(purchase=vector[self.off_x:self.off_y], discharge_internal=vector[self.off_y:self.off_z], charge_internal=vector[self.off_z:self.off_w], surplus=vector[self.off_w:self.off_s], soc=vector[self.off_s:self.off_a], baseline_cost=baseline_cost, minimum_deviation=minimum_deviation, cost=float(final.fun), solver_messages=(str(baseline.message), str(deviation.message), str(final.message)))\n'
    '\n'
    'def execute_day(initial_soc: float, purchase: np.ndarray, planned_discharge: np.ndarray, planned_charge: np.ndarray, load_actual: np.ndarray, pv_actual: np.ndarray) -> ActualDispatch:\n'
    '    purchase_minute = np.repeat(purchase / MINUTES_PER_SLOT, MINUTES_PER_SLOT)\n'
    '    discharge = np.zeros(MINUTES_PER_DAY)\n'
    '    charge = np.zeros(MINUTES_PER_DAY)\n'
    '    emergency = np.zeros(MINUTES_PER_DAY)\n'
    '    surplus = np.zeros(MINUTES_PER_DAY)\n'
    '    soc = np.empty(MINUTES_PER_DAY + 1)\n'
    '    soc[0] = initial_soc\n'
    '    joint = np.zeros(MINUTES_PER_DAY)\n'
    '    y_max = POWER_LIMIT_KW / 60.0\n'
    '    z_max = ETA * POWER_LIMIT_KW / 60.0\n'
    '    kappa = 1.0 / ETA - ETA\n'
    '    for minute in range(MINUTES_PER_DAY):\n'
    '        state = soc[minute]\n'
    '        y_plan = float(planned_discharge[minute])\n'
    '        z_plan = float(planned_charge[minute])\n'
    '        q_plan = state - y_plan + z_plan\n'
```

```
        '        if q_plan < SOC_MIN_KWH:\n'
        '            y = state + z_plan - SOC_MIN_KWH\n'
        '            z = z_plan\n'
        '        elif q_plan > SOC_MAX_KWH:\n'
        '            y = y_plan\n'
        '            z = SOC_MAX_KWH - state + y_plan\n'
        '        else:\n'
        '            y, z = (y_plan, z_plan)\n'
        '        y = min(max(y, 0.0), y_max)\n'
        '        z = min(max(z, 0.0), z_max)\n'
        '        q = state - y + z\n'
        '        residual = load_actual[minute] + z / ETA - pv_actual[minute] - purchase_minute[minute] - ETA * y\n'
        '        if residual > TOL:\n'
        '            amount = min(z, ETA * residual, max(q - SOC_MIN_KWH, 0.0))\n'
        '            z -= amount\n'
        '            residual -= amount / ETA\n'
        '            q = state - y + z\n'
        '            amount = min(y_max - y, residual / ETA, max(q - SOC_MIN_KWH, 0.0))\n'
        '            y += amount\n'
        '            residual -= ETA * amount\n'
        '            q = state - y + z\n'
        '            if residual > TOL and y > TOL and (z > TOL):\n'
        '                amount = min(y, z, residual / kappa)\n'
        '                y -= amount\n'
        '                z -= amount\n'
        '                residual -= kappa * amount\n'
        '                joint[minute] = amount\n'
        '            if residual > TOL:\n'
        '                emergency[minute] = residual\n'
        '            else:\n'
        '                residual = 0.0\n'
        '        elif residual < -TOL:\n'
        '            unused = -residual\n'
        '            amount = min(y, unused / ETA, max(SOC_MAX_KWH - q, 0.0))\n'
        '            y -= amount\n'
        '            unused -= ETA * amount\n'
        '            q = state - y + z\n'
        '            amount = min(z_max - z, ETA * unused, max(SOC_MAX_KWH - q, 0.0))\n'
        '            z += amount\n'
        '            unused -= amount / ETA\n'
        '            if unused > TOL:\n'
        '                surplus[minute] = unused\n'
        '        discharge[minute] = max(y, 0.0)\n'
        '        charge[minute] = max(z, 0.0)\n'
        '        soc[minute + 1] = state - discharge[minute] + charge[minute]\n'
        '    return ActualDispatch(discharge, charge, emergency, surplus, soc, joint)\n'
        '\n'
        'def validate_actual(purchase: np.ndarray, load: np.ndarray, pv: np.ndarray, actual: ActualDispatch) -> dict[str, Any]:\n'
```

```
        '    purchase_minute = np.repeat(purchase / MINUTES_PER_SLOT, MINUTES_PER_SLOT)\n'
        '    balance = pv + purchase_minute + ETA * actual.discharge_internal + actual.emergency - load - actual.charge_internal / ETA - actual.surplus\n'
        '    state = actual.soc[1:] - actual.soc[:-1] + actual.discharge_internal - actual.charge_internal\n'
        '    simultaneous = (actual.discharge_internal > TOL) & (actual.charge_internal > TOL)\n'
        "    checks = {'max_balance_residual_kwh': float(np.max(np.abs(balance))), 'max_state_residual_kwh': float(np.max(np.abs(state))), 'soc_min_kwh': float(np.min(actual.soc)), 'soc_max_kwh': float(np.max(actual.soc)), 'max_discharge_internal_kwh': float(np.max(actual.discharge_internal)), 'max_charge_bus_kwh': float(np.max(actual.charge_internal / ETA)), 'min_emergency_kwh': float(np.min(actual.emergency)), 'min_surplus_kwh': float(np.min(actual.surplus)), 'simultaneous_minutes': int(np.sum(simultaneous)), 'joint_adjustment_minutes': int(np.sum(actual.joint_adjustment > TOL)), 'joint_adjustment_internal_kwh': float(np.sum(actual.joint_adjustment))}\n"
        "    checks['pass'] = bool(checks['max_balance_residual_kwh'] <= LP_TOL and checks['max_state_residual_kwh'] <= LP_TOL and (checks['soc_min_kwh'] >= SOC_MIN_KWH - LP_TOL) and (checks['soc_max_kwh'] <= SOC_MAX_KWH + LP_TOL) and (checks['max_discharge_internal_kwh'] <= POWER_LIMIT_KW / 60 + LP_TOL) and (checks['max_charge_bus_kwh'] <= POWER_LIMIT_KW / 60 + LP_TOL) and (checks['min_emergency_kwh'] >= -LP_TOL) and (checks['min_surplus_kwh'] >= -LP_TOL))\n"
        '    return checks\n'
    ),

    'problem4_2.core': (
        '"""Problem4-2: four-week price forecast, weekly SOC targets; Problem2 execution unchanged."""\n'
        'from pathlib import Path\n'
        'from types import SimpleNamespace\n'
        'import numpy as np\n'
        'import openpyxl\n'
        'from scipy.optimize import linprog\n'
        'from problem2 import core as problem2\n'
        'HERE = Path(__file__).resolve().parent\n'
        'TARGETS = (6960.0, 5040.0, 3120.0, 1200.0, 6000.0, 10800.0, 8880.0)\n'
        '\n'
        'def load_prices(path, dates):\n'
        '    rows = list(openpyxl.load_workbook(path, data_only=True, read_only=True).active.values)\n'
        '    assert len(rows[0]) == 145\n'
        '    assert [problem2._as_date(r[0]) for r in rows[1:]] == dates\n'
        '    assert [problem2._minutes(x) for x in rows[0][1:]] == list(range(10, 1441, 10))\n'
        '    price = np.asarray([r[1:] for r in rows[1:]], float)\n'
        '    assert np.isfinite(price).all() and np.min(price) >= 0\n'
        '    return price\n'
        '\n'
        'def forecast_price(price, i):\n'
        '    assert i >= 28\n'
        '    return np.mean(price[[i - 7, i - 14, i - 21, i - 28]], axis=0)\n'
        '\n'
```

```
        'class Planner(problem2.DailyPlanner):\n'
        '\n'
        '    def solve(self, load, pv, initial, target):\n'
        '        aub = self.a_ub[1:]\n'
        '        bub = np.array([target, -target])\n'
        '        rhs = self._rhs(load, pv)\n'
        '        obj = np.zeros(self.n_vars)\n'
        '        obj[self.off_a] = 1\n'
        "        first = linprog(obj, A_ub=aub, b_ub=bub, A_eq=self.a_eq, b_eq=rhs, bounds=self._bounds(initial), method='highs')\n"
        '        if not first.success:\n'
        '            raise RuntimeError(first.message)\n'
        '        dev = max(0.0, float(first.fun))\n'
        '        obj[:] = 0\n'
        '        obj[:self.n_x] = self.price\n'
        "        final = linprog(obj, A_ub=aub, b_ub=bub, A_eq=self.a_eq, b_eq=rhs, bounds=self._bounds(initial, max_abs_deviation=dev + problem2.LEX_TOL), method='highs')\n"
        '        if not final.success:\n'
        '            raise RuntimeError(final.message)\n'
        '        v = final.x\n'
        '        return SimpleNamespace(purchase=v[:self.off_y], discharge_internal=v[self.off_y:self.off_z], charge_internal=v[self.off_z:self.off_w], surplus=v[self.off_w:self.off_s], soc=v[self.off_s:self.off_a], minimum_deviation=dev, cost=float(final.fun))\n'
        '\n'
        'def validate_plan(plan, load, pv, initial, target, price):\n'
        '    proxy = SimpleNamespace(**vars(plan), emergency=np.zeros(1440), joint_adjustment=np.zeros(1440))\n'
        '    checks = problem2.validate_actual(plan.purchase, load, pv, proxy)\n'
        "    assert checks['pass'], checks\n"
        '    assert abs(plan.soc[0] - initial) < 1e-07\n'
        '    assert abs(plan.soc[-1] - target) <= plan.minimum_deviation + 10 * problem2.LEX_TOL\n'
        '    assert abs(plan.cost - price @ plan.purchase) < 1e-05\n'
        '    assert np.min(plan.purchase) >= -1e-07\n'
        '    return checks\n'
    ),

    'output_common': (
        'import csv\n'
        'from pathlib import Path\n'
        'import numpy as np\n'
        '\n'
        'def write_csv(path, rows):\n'
        '    rows=list(rows)\n'
        '    if not rows:\n'
        '        return\n'
        "    with Path(path).open('w',encoding='utf-8-sig',newline='') as f:\n"
        '        w=csv.DictWriter(f,fieldnames=list(rows[0]))\n'
        '        w.writeheader(); w.writerows(rows)\n'
        '\n'
```

```
        'def save_case(out, arrays, daily):\n'
        '    out=Path(out); out.mkdir(parents=True,exist_ok=True)\n'
        "    np.savez_compressed(out/'all_results.npz',**arrays)\n"
        "    write_csv(out/'daily.csv',daily)\n"
    ),

    '_entry': (
        'import numpy as np\n'
        'from problem4_2 import core as model\n'
        'from problem2 import core as problem2\n'
        'from output_common import save_case\n'
        '\n'
        'def run(max_days,out):\n'
        "    data=problem2.load_actual(_DATA_DIR/'附件2.xlsx')\n"
        "    prices=model.load_prices(_DATA_DIR/'附件4.xlsx',data.dates)\n"
        '    load,_,pv,_=problem2.build_forecasts(data)\n'
        '    start=31; end=365 if max_days is None else min(365,start+max_days)\n'
        '    state=6000.0; plans=[]; actuals=[]; daily=[]\n'
        '    for i in range(start,end):\n'
        '        day=data.dates[i]; target=model.TARGETS[day.weekday()]\n'
        '        forecast=model.forecast_price(prices,i)\n'
        '        plan=model.Planner(forecast).solve(load[i],pv[i],state,target)\n'
        '        model.validate_plan(plan,load[i],pv[i],state,target,forecast)\n'
        '        actual=problem2.execute_day(state,plan.purchase,plan.discharge_internal,plan.
    charge_internal,\n'
        '                                    data.load_energy[i],data.pv_energy[i])\n'
        "        assert problem2.validate_actual(plan.purchase,data.load_energy[i],data.
    pv_energy[i],actual)['pass']\n"
        '        initial=float(prices[i]@plan.purchase)\n'
        '        emergency_cost=float(5*np.repeat(prices[i],10)@actual.emergency)\n'
        '        daily.append(dict(date=str(day),initial_cost=initial,adjustment_cost=0.0,\n'
        '                          emergency_cost=emergency_cost,total_cost=initial+
    emergency_cost))\n'
        '        plans.append(plan); actuals.append(actual); state=float(actual.soc[-1])\n'
        "        if len(daily)%30==0: print(f'Problem4-2 completed {len(daily)} days',flush=
    True)\n"
        '    arrays=dict(\n'
        '        dates=np.array([str(x) for x in data.dates[start:end]]),\n'
        '        purchase_initial=np.array([x.purchase for x in plans]),\n'
        '        purchase_final=np.array([x.purchase for x in plans]),\n'
        '        price=prices[start:end],soc=np.array([x.soc for x in actuals]),\n'
        '        charge_internal=np.array([x.charge_internal for x in actuals]),\n'
        '        discharge_internal=np.array([x.discharge_internal for x in actuals]),\n'
        '        emergency=np.array([x.emergency for x in actuals]),\n'
        '        surplus=np.array([x.surplus for x in actuals]))\n'
        '    save_case(out,arrays,daily)\n'
    ),

    'standalone_outputs': (
        'from __future__ import annotations\n'
```

```
        'import copy\n'
        'import csv\n'
        'import math\n'
        'import posixpath\n'
        'import re\n'
        'from datetime import date, timedelta\n'
        'from pathlib import Path\n'
        'from xml.etree import ElementTree as ET\n'
        'from xml.sax.saxutils import escape\n'
        'from zipfile import ZipFile, ZIP_DEFLATED\n'
        'import numpy as np\n'
        '\n'
        "S = 'http://schemas.openxmlformats.org/spreadsheetml/2006/main'\n"
        "R = 'http://schemas.openxmlformats.org/officeDocument/2006/relationships'\n"
        "P = 'http://schemas.openxmlformats.org/package/2006/relationships'\n"
        "SUPPLEMENT = '完整自然日购电量'\n"
        "FILES = {'problem1':'result1.xlsx','problem2':'result2.xlsx','problem3':'result3.xlsx','problem4-2':'result4-2.xlsx','problem4-3':'result4-3.xlsx'}\n"
        "ROW_RE = re.compile('<row\\\\b[^>]*?(?:/>|>.*?</row>)', re.S)\n"
        "CELL_RE = re.compile('<c\\\\b[^>]*?(?:/>|>.*?</c>)', re.S)\n"
        '\n'
        '\n'
        'def colname(number: int) -> str:\n'
        "    result = ''\n"
        '    while number:\n'
        '        number, rem = divmod(number - 1, 26)\n'
        '        result = chr(65 + rem) + result\n'
        '    return result\n'
        '\n'
        'def colnumber(name: str) -> int:\n'
        '    result = 0\n'
        '    for char in name:\n'
        '        result = 26 * result + ord(char) - 64\n'
        '    return result\n'
        '\n'
        'def address_key(ref: str) -> tuple[int, int]:\n'
        "    match = re.fullmatch('([A-Z]+)([0-9]+)', ref)\n"
        '    if match is None:\n'
        "        raise ValueError(f'Invalid cell reference {ref}')\n"
        '    return (int(match[2]), colnumber(match[1]))\n'
        '\n'
        'def attr(fragment: str, name: str, default: str | None=None) -> str | None:\n'
        '    match = re.search(\'\\\\b\' + re.escape(name) + \'="([^"]*)"\', fragment)\n'
        '    return match[1] if match else default\n'
        '\n'
        'def scalar_cell(ref: str, value, original: str | None=None, style: str | None=None) -> str:\n'
        '    """Replace only a cell value; retain its original attributes except type."""\n'
        '    if original:\n'
        "        opening = re.match('<c\\\\b([^>]*?)(?:/?>)', original)[1]\n"
```

```
        '        opening = re.sub(\'\\\\s+t="[^"]*"\', \'\', opening).rstrip(\'/ \')\n'
        '        opening = re.sub(\'\\\\br="[^"]*"\', f\'r="{ref}"\', opening)\n'
        '    else:\n'
        '        opening = f\' r="{ref}"\' + (f\' s="{style}"\' if style is not None else \'\')\n'
        '    if value is None:\n'
        "        return f'<c{opening}/>'\n"
        '    if isinstance(value, (str, np.str_)):\n'
        '        return f\'<c{opening} t="inlineStr"><is><t xml:space="preserve">{escape(str(value))}</t></is></c>\'\n'
        '    numeric = float(value)\n'
        '    if not math.isfinite(numeric):\n'
        "        raise ValueError(f'Non-finite numeric output at {ref}: {value}')\n"
        '    if abs(numeric) < 1e-10:\n'
        '        numeric = 0.0\n'
        '    return f"<c{opening}><v>{format(numeric, \'.15g\')}</v></c>"\n'
        '\n'
        'def rows_in(xml: str) -> dict[int, str]:\n'
        "    data = re.search('<sheetData>(.*?)</sheetData>', xml, re.S)\n"
        '    if data is None:\n'
        "        raise ValueError('Template has no sheetData element')\n"
        "    return {int(attr(row.split('>', 1)[0], 'r')): row for row in ROW_RE.findall(data[1])}\n"
        '\n'
        'def patch_row(row: str, updates: dict[str, object], new_number: int | None=None) -> str:\n'
        "    opening = re.match('<row\\\\b([^>]*?)(?:/?>)', row)[1].rstrip('/ ')\n"
        "    old_number = int(attr(opening, 'r'))\n"
        '    new_number = old_number if new_number is None else new_number\n'
        '    opening = re.sub(\'\\\\br="[^"]*"\', f\'r="{new_number}"\', opening)\n'
        '    cells = {}\n'
        '    for cell in CELL_RE.findall(row):\n'
        "        ref = attr(cell.split('>', 1)[0], 'r')\n"
        "        column = re.match('[A-Z]+', ref)[0]\n"
        "        new_ref = f'{column}{new_number}'\n"
        '        cells[new_ref] = re.sub(\'\\\\br="[^"]*"\', f\'r="{new_ref}"\', cell, count=1)\n'
        "    default_style = attr(opening, 's')\n"
        '    for column_or_ref, value in updates.items():\n'
        "        column = re.match('[A-Z]+', column_or_ref)[0]\n"
        "        ref = f'{column}{new_number}'\n"
        '        cells[ref] = scalar_cell(ref, value, cells.get(ref), default_style)\n'
        "    return f'<row{opening}>' + ''.join((cells[k] for k in sorted(cells, key=address_key))) + '</row>'\n"
        '\n'
        'def replace_rows(xml: str, rows: list[str], last_cell: str | None=None) -> str:\n'
        "    xml = re.sub('<sheetData>.*?</sheetData>', lambda _: '<sheetData>' + ''.join(rows) + '</sheetData>', xml, count=1, flags=re.S)\n"
        '    if last_cell:\n'
        '        xml = re.sub(\'<dimension\\\\b[^>]*/>\', f\'<dimension ref="A1:{last_cell
```

```
    }"/>\', xml, count=1)\n'
        '    return xml\n'
        '\n'
        'def excel_date(day: date) -> int:\n'
        '    return (day - date(1899, 12, 30)).days\n'
        '\n'
        'def clock(minutes: int) -> str:\n'
        "    return f'{minutes // 60:02d}:{minutes % 60:02d}'\n"
        '\n'
        'def intervals(energy: np.ndarray, tolerance: float=1e-08) -> list[tuple[int, int, float]]:\n'
        '    positive = np.asarray(energy) > tolerance\n'
        '    padded = np.r_[False, positive, False].astype(np.int8)\n'
        '    starts = np.flatnonzero(np.diff(padded) == 1)\n'
        '    ends = np.flatnonzero(np.diff(padded) == -1)\n'
        '    return [(int(a), int(b), float(np.sum(energy[a:b]))) for a, b in zip(starts, ends)]\n'
        '\n'
        'def load_results(folder: Path, case: str) -> dict:\n'
        '    folder = Path(folder).resolve()\n'
        "    with np.load(folder / 'all_results.npz', allow_pickle=False) as source:\n"
        '        data = {key: np.array(source[key]) for key in source.files}\n'
        "    required = {'dates', 'purchase_initial', 'purchase_final', 'price', 'charge_internal', 'discharge_internal', 'emergency', 'surplus', 'soc'}\n"
        '    if required - data.keys():\n'
        "        raise ValueError(f'{case}: missing arrays {sorted(required - data.keys())}')\n"
        "    dates = [date.fromisoformat(str(value)[:10]) for value in data['dates']]\n"
        '    n = len(dates)\n'
        "    if case == 'problem1':\n"
        '        if n != 1:\n'
        "            raise ValueError('problem1 must contain exactly one modeled day')\n"
        '    elif dates != [date(2025, 2, 1) + timedelta(days=i) for i in range(334)]:\n'
        "        raise ValueError(f'{case}: official export requires all 334 dates from February 1 to December 31')\n"
        "    for key in required - {'dates'}:\n"
        "        width = 1441 if key == 'soc' else 144 if key in {'purchase_initial', 'purchase_final', 'price'} else 1440\n"
        '        if data[key].shape != (n, width) or not np.all(np.isfinite(data[key])):\n'
        "            raise ValueError(f'{case}: invalid {key} shape or nonfinite values: {data[key].shape}')\n"
        "    data['dates'] = dates\n"
        "    daily_path = next((p for p in [folder / 'daily.csv', folder / 'summary' / 'daily.csv'] if p.exists()), None)\n"
        '    if daily_path is None:\n'
        "        raise FileNotFoundError(f'Missing daily settlement CSV under {folder}')\n"
        "    with daily_path.open(encoding='utf-8-sig', newline='') as stream:\n"
        '        records = list(csv.DictReader(stream))\n'
        '    if not records:\n'
        "        raise ValueError('Daily settlement CSV has no rows')\n"
```

```
    "    date_key = next((key for key in ['date', '日期', 'day'] if key in records[0]), None)\n"
    '    if date_key is None:\n'
    "        raise ValueError(f'{daily_path}: need date column')\n"
    '    by_date = {date.fromisoformat(row[date_key][:10]): row for row in records}\n'
    '    if len(by_date) != len(records) or set(by_date) != set(dates):\n'
    "        raise ValueError('Daily CSV dates must match NPZ dates exactly without duplicates')\n"
    "    aliases = {'initial_cost': ['initial_cost', 'initial_purchase_cost', 'initial_plan_cost', 'planned_cost', 'plan_cost', 'purchase_cost'], 'adjustment_cost': ['adjustment_cost', 'adjustment_net_cost', 'adjustment_net', 'adjust_cost'], 'emergency_cost': ['emergency_cost', 'emergency_purchase_cost'], 'total_cost': ['total_cost', 'actual_total_cost', 'actual_cost', 'cost']}\n"
    '    daily = []\n'
    '    for i, day in enumerate(dates):\n'
    '        if day not in by_date:\n'
    "            raise ValueError(f'{daily_path}: missing {day}')\n"
    '        row = by_date[day]\n'
    '        values = {}\n'
    '        for key, names in aliases.items():\n'
    "            found = next((row[name] for name in names if name in row and row[name] != ''), None)\n"
    '            if found is None:\n'
    "                if key == 'adjustment_cost' and case in {'problem1', 'problem2', 'problem4-2'}:\n"
    '                    values[key] = 0.0\n'
    '                else:\n'
    "                    raise ValueError(f'{daily_path}: need {key} for {day}; available columns={list(row)}')\n"
    '            else:\n'
    '                values[key] = float(found)\n'
    '        if not all((math.isfinite(v) for v in values.values())):\n'
    "            raise ValueError(f'Nonfinite daily cost for {day}')\n"
    "        computed_total = values['initial_cost'] + values['adjustment_cost'] + values['emergency_cost']\n"
    "        if not math.isclose(computed_total, values['total_cost'], abs_tol=0.01, rel_tol=1e-09):\n"
    "            raise ValueError(f'Daily settlement does not reconcile for {case}/{day}: {values}')\n"
    "        actual_emergency = float(np.dot(data['emergency'][i], np.repeat(data['price'][i], 10)) * 5)\n"
    "        if not math.isclose(actual_emergency, values['emergency_cost'], abs_tol=0.01, rel_tol=1e-08):\n"
    "            raise ValueError(f'Emergency cost disagrees with minute trace for {case}/{day}')\n"
    "        actual_initial = float(np.dot(data['purchase_initial'][i], data['price'][i]))\n"
    "        if not math.isclose(actual_initial, values['initial_cost'], abs_tol=0.01, rel_tol=1e-08):\n"
    "            raise ValueError(f'Initial plan cost disagrees with 144-slot trace for {
```

```
    case}/{day}')\n"
        '        daily.append(values)\n'
        "    data['daily'] = daily\n"
        "    data['daily_csv'] = daily_path.relative_to(folder).as_posix()\n"
        '    return data\n'
        '\n'
        'def sheet_paths(parts: dict[str, bytes]) -> dict[str, str]:\n'
        "    workbook = ET.fromstring(parts['xl/workbook.xml'])\n"
        "    rels = {r.attrib['Id']: r.attrib['Target'] for r in ET.fromstring(parts['xl/_rels/
    workbook.xml.rels'])}\n"
        "    return {s.attrib['name']: posixpath.normpath('xl/' + rels[s.attrib[f'{{{R}}}id']])
     for s in workbook.find(f'{{{S}}}sheets')}\n"
        '\n'
        'def write_purchase(xml: str, data: dict, *, final: bool, single: bool) -> str:\n'
        '    rows = rows_in(xml)\n'
        "    array = data['purchase_final' if final else 'purchase_initial']\n"
        '    if single:\n'
        '        for row in range(2, 145):\n'
        "            rows[row] = patch_row(rows[row], {'B': array[0, row - 1]})\n"
        "        rows[145] = patch_row(rows[145], {'B': None})\n"
        '    else:\n'
        "        for i, day in enumerate(data['dates']):\n"
        '            row = i + 2\n'
        '            updates = {colname(j + 1): array[i, j] for j in range(1, 144)}\n'
        "            updates['EO'] = array[i + 1, 0] if i + 1 < len(array) else None\n"
        "            updates['EP'] = float(array[i].sum())\n"
        "            costs = data['daily'][i]\n"
        "            updates['EQ'] = costs['initial_cost'] + (costs['adjustment_cost'] if final
     else 0)\n"
        '            rows[row] = patch_row(rows[row], updates)\n'
        '    return replace_rows(xml, [rows[r] for r in sorted(rows)])\n'
        '\n'
        'def write_storage(xml: str, data: dict, *, single: bool) -> str:\n'
        '    original = rows_in(xml)\n'
        '    eta = 0.9\n'
        '    if single:\n'
        '        for block in range(6):\n'
        '            take = slice(240 * block, 240 * (block + 1))\n'
        "            updates = {'B': float(data['charge_internal'][0, take].sum() / eta), 'C':
    float(data['discharge_internal'][0, take].sum() * eta)}\n"
        '            if block < 2:\n'
        "                updates['E'] = data['soc'][0, 0 if block == 0 else -1]\n"
        '            original[block + 2] = patch_row(original[block + 2], updates)\n'
        '        return replace_rows(xml, [original[r] for r in sorted(original)])\n'
        '    output = [original[1]]\n'
        "    for i, day in enumerate(data['dates']):\n"
        '        for block in range(6):\n'
        '            row = 2 + 6 * i + block\n'
        '            take = slice(240 * block, 240 * (block + 1))\n'
        "            updates = {'A': excel_date(day) if block == 0 else None, 'C': float(data['
```

```
    charge_internal'][i, take].sum() / eta), 'D': float(data['discharge_internal'][i, take].sum() * eta), 'F': data['soc'][i, 0 if block == 0 else -1] if block < 2 else None}\n"
    '            output.append(patch_row(original[block + 2], updates, row))\n'
    '    return replace_rows(xml, output, f"F{1 + 6 * len(data[\'dates\'])}")\n'
    '\n'
    'def write_emergency(xml: str, data: dict) -> tuple[str, int]:\n'
    '    original = rows_in(xml)\n'
    '    output = [original[1]]\n'
    '    count = 0\n'
    "    for i, day in enumerate(data['dates']):\n"
    "        events = intervals(data['emergency'][i])\n"
    '        count += len(events)\n'
    "        records = [(f'{clock(a)}-{clock(b)}', energy) for a, b, energy in events] or [('无', 0.0)]\n"
    '        for j, (label, energy) in enumerate(records):\n'
    '            prototype = original[2 if j == 0 else 4 if j == len(records) - 1 else 3]\n'
    "            output.append(patch_row(prototype, {'A': excel_date(day) if j == 0 else None, 'B': label, 'C': energy}, len(output) + 1))\n"
    "    return (replace_rows(xml, output, f'C{len(output)}'), count)\n"
    '\n'
    'def supplement_xml(parts: dict[str, bytes], data: dict, case: str) -> str:\n'
    '    """A compact additional matrix; preserve all original styles.xml bytes."""\n'
    '    if parts:\n'
    "        source = rows_in(parts[sheet_paths(parts)['计划购电量']].decode('utf-8'))\n"
    "        header_style = attr(CELL_RE.findall(source[1])[1].split('>', 1)[0], 's', '0')\n"
    "        value_style = attr(CELL_RE.findall(source[2])[-1].split('>', 1)[0], 's', '0')\n"
    '    else:\n'
    "        header_style, value_style = ('1', '2')\n"
    "    headings = ['日期', '计划类型'] + [f'{clock(10 * k)}-{clock(10 * (k + 1))}' for k in range(144)]\n"
    "    headings += ['全天购电量(kWh)', '初始计划款(元)', '调整净费用(元)', '紧急购电费(元)', '总购电费(元)', '紧急购电量(kWh)']\n"
    '    rows = [\'<row r="1" ht="28" customHeight="1">\' + \'\'.join((scalar_cell(f\'{colname(c)}1\', v, style=header_style) for c, v in enumerate(headings, 1))) + \'</row>\']\n'
    "    stage_rows = [('初始计划', 'purchase_initial'), ('最终有效计划', 'purchase_final')] if case in {'problem3', 'problem4-3'} else [('计划购电', 'purchase_initial')]\n"
    "    for i, day in enumerate(data['dates']):\n"
    "        costs = data['daily'][i]\n"
    '        for stage, key in stage_rows:\n'
    "            settlement = stage != '初始计划'\n"
    "            display_day = '附件1示例日' if case == 'problem1' else day.isoformat()\n"
    '            values = [display_day, stage] + data[key][i].tolist() + [float(data[key][i].sum())]\n'
    "            values += [costs['initial_cost'], costs['adjustment_cost'] if settlement else None, costs['emergency_cost'] if settlement else None, costs['total_cost'] if settlement else None, float(data['emergency'][i].sum()) if settlement else None]\n"
```

```
    '            r = len(rows) + 1\n'
    '            rows.append(f\'<row r="{r}" ht="20" customHeight="1">\' + \'\'.join((scalar_cell(f\'{colname(c)}{r}\', value, style=value_style) for c, value in enumerate(values, 1))) + \'</row>\')\n'
    '    return f\'<?xml version="1.0" encoding="UTF-8" standalone="yes"?><worksheet xmlns="{S}"><dimension ref="A1:EV{len(rows)}"/><sheetViews><sheetView showGridLines="0" workbookViewId="0"><pane xSplit="2" ySplit="1" topLeftCell="C2" activePane="bottomRight" state="frozen"/></sheetView></sheetViews><sheetFormatPr defaultRowHeight="20"/><cols><col min="1" max="2" width="18" customWidth="1"/><col min="3" max="146" width="16" customWidth="1"/><col min="147" max="152" width="22" customWidth="1"/></cols><sheetData>\' + \'\'.join(rows) + \'</sheetData><pageMargins left="0.3" right="0.3" top="0.5" bottom="0.5" header="0.2" footer="0.2"/></worksheet>\'\n'
    '\n'
    'def _display_day(day: date, case: str) -> str:\n'
    "    return '附件1示例日' if case == 'problem1' else day.isoformat()\n"
    '\n'
    'def _plain_sheet(headers: list[str], values: list[list], widths=None) -> str:\n'
    '    """Write an explicitly new sheet without any external authoring library."""\n'
    '    rows = []\n'
    '    for number, row in enumerate([headers] + values, 1):\n'
    "        style = '1' if number == 1 else '2'\n"
    "        cells = ''.join((scalar_cell(f'{colname(c)}{number}', value, style=style) for c, value in enumerate(row, 1)))\n"
    '        rows.append(f\'<row r="{number}" ht="30" customHeight="1">{cells}</row>\')\n'
    '    columns = \'\'.join((f\'<col min="{i}" max="{i}" width="{width}" customWidth="1"/>\' for i, width in enumerate(widths or [22] * len(headers), 1)))\n'
    '    return f\'\'\'<?xml version="1.0" encoding="UTF-8" standalone="yes"?><worksheet xmlns="{S}"><dimension ref="A1:{colname(len(headers))}{len(rows)}"/><sheetViews><sheetView showGridLines="0" workbookViewId="0"><pane ySplit="1" topLeftCell="A2" activePane="bottomLeft" state="frozen"/></sheetView></sheetViews><sheetFormatPr defaultRowHeight="30"/><cols>{columns}</cols><sheetData>{\'\'.join(rows)}</sheetData><pageMargins left="0.3" right="0.3" top="0.5" bottom="0.5" header="0.2" footer="0.2"/></worksheet>\'\'\'\n'
    '\n'
    'def _new_workbook_parts(sheets: list[tuple[str, str]]) -> dict[str, bytes]:\n'
    '    types = [\'<Default Extension="rels" ContentType="application/vnd.openxmlformats-package.relationships+xml"/>\', \'<Default Extension="xml" ContentType="application/xml"/>\', \'<Override PartName="/xl/workbook.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet.main+xml"/>\', \'<Override PartName="/xl/styles.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.styles+xml"/>\']\n'
    '    relations, entries = ([], [])\n'
    '    parts = {}\n'
    '    for index, (name, xml) in enumerate(sheets, 1):\n'
    "        member = f'xl/worksheets/sheet{index}.xml'\n"
    "        parts[member] = xml.encode('utf-8')\n"
    '        types.append(f\'<Override PartName="/{member}" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.worksheet+xml"/>\')\n'
    '        relations.append(f\'<Relationship Id="rId{index}" Type="{R}/worksheet" Target="worksheets/sheet{index}.xml"/>\')\n'
    '        entries.append(f\'<sheet name="{escape(name)}" sheetId="{index}" r:id="rId{
```

```
    index}"/>\')\n'
    '    relations.append(f\'<Relationship Id="rId{len(sheets) + 1}" Type="{R}/styles" Target="styles.xml"/>\')\n'
    '    parts[\'[Content_Types].xml\'] = f"""<Types xmlns="http://schemas.openxmlformats.org/package/2006/content-types">{\'\'.join(types)}</Types>""".encode()\n'
    '    parts[\'_rels/.rels\'] = f\'<Relationships xmlns="{P}"><Relationship Id="rId1" Type="{R}/officeDocument" Target="xl/workbook.xml"/></Relationships>\'.encode()\n'
    '    parts[\'xl/_rels/workbook.xml.rels\'] = f\'\'\'<Relationships xmlns="{P}">{\'\'.join(relations)}</Relationships>\'\'\'.encode()\n'
    '    parts[\'xl/workbook.xml\'] = f\'\'\'<workbook xmlns="{S}" xmlns:r="{R}"><bookViews><workbookView/></bookViews><sheets>{\'\'.join(entries)}</sheets></workbook>\'\'\'.encode()\n'
    '    parts[\'xl/styles.xml\'] = f\'<styleSheet xmlns="{S}"><fonts count="2"><font><sz val="11"/><name val="Microsoft YaHei"/></font><font><b/><sz val="11"/><color rgb="FFFFFFFF"/><name val="Microsoft YaHei"/></font></fonts><fills count="3"><fill><patternFill patternType="none"/></fill><fill><patternFill patternType="gray125"/></fill><fill><patternFill patternType="solid"><fgColor rgb="FF245A81"/><bgColor indexed="64"/></patternFill></fill></fills><borders count="1"><border/></borders><cellStyleXfs count="1"><xf numFmtId="0" fontId="0" fillId="0" borderId="0"/></cellStyleXfs><cellXfs count="3"><xf numFmtId="0" fontId="0" fillId="0" borderId="0" xfId="0"/><xf numFmtId="0" fontId="1" fillId="2" borderId="0" xfId="0" applyAlignment="1"><alignment vertical="center" wrapText="1"/></xf><xf numFmtId="0" fontId="0" fillId="0" borderId="0" xfId="0" applyAlignment="1"><alignment vertical="center" wrapText="1"/></xf></cellXfs><cellStyles count="1"><cellStyle name="Normal" xfId="0" builtinId="0"/></cellStyles></styleSheet>\'.encode()\n'
    '    return parts\n'
    '\n'
    'def _standalone_parts(data: dict, case: str) -> tuple[dict[str, bytes], int]:\n'
    '    storage, emergency, settlement = ([], [], [])\n'
    '    event_count = 0\n'
    "    for i, day in enumerate(data['dates']):\n"
    '        label = _display_day(day, case)\n'
    '        for block in range(6):\n'
    '            a, b = (240 * block, 240 * (block + 1))\n'
    "            storage.append([label, f'{clock(a)}-{clock(b)}', float(data['charge_internal'][i, a:b].sum() / 0.9), float(data['discharge_internal'][i, a:b].sum() * 0.9), float(data['soc'][i, a]), float(data['soc'][i, b])])\n"
    "        events = intervals(data['emergency'][i])\n"
    '        event_count += len(events)\n'
    "        emergency.extend([[label, f'{clock(a)}-{clock(b)}', energy] for a, b, energy in events] or [[label, '无', 0.0]])\n"
    "        costs = data['daily'][i]\n"
    "        settlement.append([label, float(data['purchase_initial'][i].sum()), float(data['purchase_final'][i].sum()), float(data['charge_internal'][i].sum() / 0.9), float(data['discharge_internal'][i].sum() * 0.9), float(data['emergency'][i].sum()), float(data['surplus'][i].sum()), float(data['soc'][i, 0]), float(data['soc'][i, -1]), costs['initial_cost'], costs['adjustment_cost'], costs['emergency_cost'], costs['total_cost']])\n"
    "    notes = [['文件来源', '未找到附件5对应模板；本文件为程序独立生成的完整结果表，不声称保留官方模板结构。'], ['问题', case], ['计算天数', len(data['dates'])], ['计算范围', '附件1未标日期的示例日' if case == 'problem1' else '2025-02-01至2025-12-31，共334个自然日'], ['购电区间', '完整自然日表逐项列出00:00-00:10至23:50-24:00的144段，保留初始及最终计划。'], ['充放电口径
```

```
', '购电侧充电量=内部储能增量/0.9；供负载侧放电量=内部储能减少量×0.9；每4小时汇总。'], ['紧急购电', '按实际连续分钟区间逐事件列出；无事件的日期以‘无’和0列示，事件数不设上限。'], ['费用口径', '总购电费=初始计划款+已接受调整净费用+紧急购电费；紧急单价为该分钟对应电价的5倍。'], ['数据来源', '同目录all_results.npz及daily.csv；数值保留为数值单元格。'], ['日期说明', 'Problem1显示‘附件1示例日’，NPZ/CSV日期仅作内部关联键；其他问题显示真实2025年日期。']]\n"
    "    sheets = [('导出说明', _plain_sheet(['项目', '说明'], notes, [22, 115])), (SUPPLEMENT, supplement_xml({}, data, case)), ('充放电量', _plain_sheet(['日期', '时段', '充电购电侧(kWh)', '放电供负载侧(kWh)', '时段起始SOC(kWh)', '时段结束SOC(kWh)'], storage)), ('紧急购电量', _plain_sheet(['日期', '实际连续分钟时段', '紧急购电量(kWh)'], emergency)), ('日电量与费用', _plain_sheet(['日期', '初始计划购电量(kWh)', '最终计划购电量(kWh)', '充电购电侧(kWh)', '放电供负载侧(kWh)', '紧急购电量(kWh)', '未利用电量(kWh)', '日初SOC(kWh)', '日末SOC(kWh)', '初始计划款(元)', '调整净费用(元)', '紧急购电费(元)', '总购电费(元)'], settlement))]\n"
    '    return (_new_workbook_parts(sheets), event_count)\n'
    '\n'
    'def beautify_excel(path: Path) -> None:\n'
    '    """Apply the final workbook appearance without changing calculated values.\n'
    '\n'
    '    - Every non-empty cell in every sheet uses 宋体 10 pt.\n'
    '    - Every non-empty cell is horizontally and vertically centered.\n'
    '    - The last row of “紧急购电量” gets a black bottom border across A:C.\n'
    '    Existing bold/italic/color/fill/number-format settings and all other border sides are preserved.\n'
    '    """\n'
    '    import openpyxl\n'
    '    from copy import copy\n'
    '    from openpyxl.styles import Side\n'
    '\n'
    '    workbook = openpyxl.load_workbook(path)\n'
    '    try:\n'
    '        for sheet in workbook.worksheets:\n'
    '            for row in sheet.iter_rows():\n'
    '                for cell in row:\n'
    '                    if cell.value is None:\n'
    '                        continue\n'
    '\n'
    '                    font = copy(cell.font)\n'
    "                    font.name = '宋体'\n"
    '                    font.sz = 10\n'
    '                    font.scheme = None\n'
    '                    cell.font = font\n'
    '\n'
    '                    alignment = copy(cell.alignment)\n'
    "                    alignment.horizontal = 'center'\n"
    "                    alignment.vertical = 'center'\n"
    '                    cell.alignment = alignment\n'
    '\n'
    "        if '紧急购电量' in workbook.sheetnames:\n"
    "            sheet = workbook['紧急购电量']\n"
    '            last_row = sheet.max_row\n'
    "            black = Side(style='thin', color='FF000000')\n"
```

```
        '            for cell in sheet[last_row][:3]:\n'
        '                border = copy(cell.border)\n'
        '                border.bottom = black\n'
        '                cell.border = border\n'
        '\n'
        '        workbook.save(path)\n'
        '    finally:\n'
        '        workbook.close()\n'
        '\n'
        '\n'
        'def export_one(case, problem_output_dir, attachment_dir, template_dir=None):\n'
        '    folder = Path(problem_output_dir).resolve()\n'
        '    attachments = Path(attachment_dir).resolve()\n'
        "    templates = Path(template_dir).resolve() if template_dir is not None else attachments / '附件5'\n"
        '    source = templates / FILES[case]\n'
        '    destination = folder / FILES[case]\n'
        '    data = load_results(folder, case)\n'
        "    temporary = destination.with_suffix('.pending.xlsx')\n"
        '\n'
        '    if source.is_file():\n'
        '        with ZipFile(source) as archive:\n'
        '            infos = {info.filename: copy.copy(info) for info in archive.infolist()}\n'
        '            parts = {name: archive.read(name) for name in infos}\n'
        '        paths = sheet_paths(parts)\n'
        "        required = {'计划购电量', '充放电量'} | ({'紧急购电量'} if case != 'problem1' else set())\n"
        "        if case in {'problem3', 'problem4-3'}:\n"
        "            required.add('调整购电量')\n"
        '        if required - paths.keys():\n'
        "            raise ValueError('附件5中的结果模板工作表不完整')\n"
        '        for name, member in paths.items():\n'
        "            xml = parts[member].decode('utf-8')\n"
        "            if name in {'计划购电量', '调整购电量'}:\n"
        "                xml = write_purchase(xml, data, final=name == '调整购电量', single=case == 'problem1')\n"
        "            elif name == '充放电量':\n"
        "                xml = write_storage(xml, data, single=case == 'problem1')\n"
        "            elif name == '紧急购电量':\n"
        '                xml, _ = write_emergency(xml, data)\n'
        "            parts[member] = xml.encode('utf-8')\n"
        "        with ZipFile(temporary, 'w', compression=ZIP_DEFLATED, compresslevel=6) as output:\n"
        '            for name, content in parts.items():\n'
        '                output.writestr(infos.get(name, name), content)\n'
        '    else:\n'
        '        parts, _ = _standalone_parts(data, case)\n'
        "        with ZipFile(temporary, 'w', compression=ZIP_DEFLATED, compresslevel=6) as output:\n"
        '            for name, content in parts.items():\n'
```

```
        '                output.writestr(name, content)\n'
        '\n'
        '    temporary.replace(destination)\n'
        '    beautify_excel(destination)\n'
        '    return destination\n'
    ),
}

def _load_module(name,source,data_dir):
    parts=name.split('.')
    for i in range(1,len(parts)):
        parent='.'.join(parts[:i])
        if parent not in sys.modules:
            package=types.ModuleType(parent); package.__path__=[]; sys.modules[parent]=package
    filename=str(_SCRIPT_DIR/'__embedded__'/Path(*parts).with_suffix('.py'))
    module=types.ModuleType(name)
    module.__file__=filename
    module.__package__=name.rpartition('.')[0]
    module.__spec__=importlib.util.spec_from_loader(name,loader=None,origin=filename)
    module.__dict__.update(_SCRIPT_DIR=_SCRIPT_DIR,_DATA_DIR=data_dir)
    linecache.cache[filename]=(len(source),None,source.splitlines(keepends=True),filename)
    sys.modules[name]=module
    if '.' in name: setattr(sys.modules[name.rpartition('.')[0]],parts[-1],module)
    exec(compile(source,filename,'exec'),module.__dict__)
    return module

def main():
    parser=argparse.ArgumentParser()
    parser.add_argument('--input-dir',type=Path)
    parser.add_argument('--output-dir',type=Path)
    parser.add_argument('--templates-dir',type=Path)
    parser.add_argument('--max-days',type=int)
    args=parser.parse_args()

    data_dir=(args.input_dir or (Path(ATTACHMENT_DIR) if ATTACHMENT_DIR.strip() else
    _SCRIPT_DIR.parent/'附件')).resolve()
    if not (data_dir/'附件1.xlsx').exists() and (data_dir/'附件'/'附件1.xlsx').exists():
        data_dir=data_dir/'附件'
    required=[2, 4]
    missing=[data_dir/f'附件{n}.xlsx' for n in required if not (data_dir/f'附件{n}.xlsx').
    is_file()]
    if missing: parser.error('缺少附件: \n'+'\n'.join(map(str,missing)))
    if args.max_days is not None and not 1<=args.max_days<=334:
        parser.error('--max-days 超出范围')

    out=(args.output_dir or (Path(OUTPUT_DIR) if OUTPUT_DIR.strip() else _SCRIPT_DIR.parent/'
    results'/_SCRIPT_PATH.stem)).resolve()
    out.mkdir(parents=True,exist_ok=True)
    template_dir=(args.templates_dir or (Path(TEMPLATE_DIR) if TEMPLATE_DIR.strip() else
    data_dir/'附件5')).resolve()
```

```
    try:
        import numpy as np
        import scipy
        import openpyxl
    except ImportError as e:
        parser.error('缺少依赖: '+str(e)+'; 请安装 numpy scipy openpyxl')

    modules={name:_load_module(name,source,data_dir) for name,source in _MODULE_SOURCES.items()
    }
    print(f'开始{CASE_NAME}; 结果: {out}',flush=True)
    modules['_entry'].run(args.max_days,out)
    with np.load(out/'all_results.npz') as z: days=len(z['dates'])
    if CASE=='problem1' or days==334:
        destination=modules['standalone_outputs'].export_one(CASE,out,data_dir,template_dir)
        print(f'Excel 已生成: {destination}',flush=True)
    else:
        print('快速检查仅计算数据; 完整 Excel 需运行完整评价期。',flush=True)

if __name__=='__main__':
    main()
```

附录 B. 问题四-三求解程序

Listing 5: 问题四-三求解程序

```
# 问题四 (3 口径): 在波动电价下重做问题三的日内滚动决策。
from pathlib import Path
import argparse
from datetime import datetime,timedelta
import importlib.util
import linecache
import os
import sys
import types

ATTACHMENT_DIR = r""
OUTPUT_DIR = r""
TEMPLATE_DIR = r""
CASE = 'problem4-3'
CASE_NAME = '问题四对应问题三'

os.environ.setdefault('OMP_NUM_THREADS','1')
os.environ.setdefault('OPENBLAS_NUM_THREADS','1')
_SCRIPT_PATH=Path(__file__).resolve()
_SCRIPT_DIR=_SCRIPT_PATH.parent

_MODULE_SOURCES = {
    'problem2_reference': (
```

```
        'from __future__ import annotations\n'
        'from dataclasses import dataclass\n'
        'from datetime import date,datetime,time,timedelta\n'
        'from typing import Any\n'
        'import numpy as np\n'
        'import openpyxl\n'
        '\n'
        '\n'
        'MINUTES_PER_DAY = 1440\n'
        '\n'
        'SLOTS_PER_DAY = 144\n'
        '\n'
        'MINUTES_PER_SLOT = 10\n'
        '\n'
        'ETA = 0.9\n'
        '\n'
        'POWER_LIMIT_KW = 5000.0\n'
        '\n'
        'SOC_INITIAL_KWH = 6000.0\n'
        '\n'
        'SOC_MIN_KWH = 1200.0\n'
        '\n'
        'SOC_MAX_KWH = 10800.0\n'
        '\n'
        'TOL = 1e-08\n'
        '\n'
        '@dataclass(frozen=True)\n'
        'class ActualData:\n'
        '    dates: list[date]\n'
        '    load_power_endpoints: np.ndarray\n'
        '    pv_power_endpoints: np.ndarray\n'
        '    load_energy: np.ndarray\n'
        '    pv_energy: np.ndarray\n'
        '\n'
        '@dataclass(frozen=True)\n'
        'class ActualDispatch:\n'
        '    discharge_internal: np.ndarray\n'
        '    charge_internal: np.ndarray\n'
        '    emergency: np.ndarray\n'
        '    surplus: np.ndarray\n'
        '    soc: np.ndarray\n'
        '    joint_adjustment: np.ndarray\n'
        '\n'
        'def _as_date(value: Any) -> date:\n'
        '    if isinstance(value, datetime):\n'
        '        return value.date()\n'
        '    if isinstance(value, date):\n'
        '        return value\n'
        '    return datetime.fromisoformat(str(value)).date()\n'
        '\n'
```

```
        'def _minutes(value: Any) -> int:\n'
        '    if isinstance(value, time):\n'
        '        return value.hour * 60 + value.minute\n'
        '    text = str(value).strip()\n'
        "    offset = 1440 if '+1' in text else 0\n"
        "    text = text.replace('+1', '')\n"
        "    hour, minute = (int(part) for part in text.split(':'))\n"
        '    return offset + 60 * hour + minute\n'
        '\n'
        'def load_price(path: Path) -> np.ndarray:\n'
        '    workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)\n'
        '    sheet = workbook.active\n'
        "    first_column = 2 if sheet.cell(1, 1).value == '序号' else 1\n"
        '    headers = [sheet.cell(1, column).value for column in range(first_column,
    first_column + 4)]\n'
        "    expected = ['时间', '电价', '小区负载', '光伏发电预测功率']\n"
        '    if headers != expected:\n'
        "        raise ValueError(f'附件1表头不匹配：{headers!r}')\n"
        '    rows = list(sheet.iter_rows(min_row=2, min_col=first_column, max_col=first_column
    + 3, values_only=True))\n'
        '    if len(rows) != SLOTS_PER_DAY:\n'
        "        raise ValueError(f'附件1应有144条数据，实际为{len(rows)}条')\n"
        '    endpoints = np.array([_minutes(row[0]) for row in rows], dtype=int)\n'
        '    if not np.array_equal(endpoints, np.arange(10, 1441, 10)):\n'
        "        raise ValueError('附件1时间列不是0:10至次日0:00的十分钟序列')\n"
        '    price = np.array([float(row[1]) for row in rows], dtype=float)\n'
        '    if not np.isfinite(price).all() or np.any(price < 0):\n'
        "        raise ValueError('附件1电价含空值、非有限值或负值')\n"
        '    return price\n'
        '\n'
        'def _read_power_sheet(sheet: Any) -> tuple[list[date], np.ndarray]:\n'
        '    headers = [sheet.cell(1, column).value for column in range(1, 146)]\n'
        '    endpoints = np.array([_minutes(value) for value in headers[1:]], dtype=int)\n'
        '    if not np.array_equal(endpoints, np.arange(10, 1441, 10)):\n'
        "        raise ValueError(f'{sheet.title}时间表头不是0:10至次日0:00的序列')\n"
        '    rows = list(sheet.iter_rows(min_row=2, max_col=145, values_only=True))\n'
        '    dates = [_as_date(row[0]) for row in rows]\n'
        '    values = np.array([[float(value) for value in row[1:]] for row in rows])\n'
        '    if values.shape != (365, 144):\n'
        "        raise ValueError(f'{sheet.title}数据维度错误：{values.shape}')\n"
        '    if not np.isfinite(values).all() or np.any(values < 0):\n'
        "        raise ValueError(f'{sheet.title}含空值、非有限值或负值')\n"
        '    return (dates, values)\n'
        '\n'
        'def _minute_energy(power_endpoints: np.ndarray) -> np.ndarray:\n'
        '    days = power_endpoints.shape[0]\n'
        '    output = np.empty((days, MINUTES_PER_DAY), dtype=float)\n'
        '    sample_minutes = np.arange(0, MINUTES_PER_DAY + 1, MINUTES_PER_SLOT)\n'
        '    minute_points = np.arange(MINUTES_PER_DAY + 1)\n'
        '    for day_index in range(days):\n'
```

```
        '        if day_index == 0:\n'
        '            start_power = power_endpoints[day_index, 0]\n'
        '        else:\n'
        '            start_power = power_endpoints[day_index - 1, -1]\n'
        '        samples = np.concatenate(([start_power], power_endpoints[day_index]))\n'
        '        minute_power = np.interp(minute_points, sample_minutes, samples)\n'
        '        output[day_index] = (minute_power[:-1] + minute_power[1:]) / 120.0\n'
        '    output[0, :MINUTES_PER_SLOT] = np.nan\n'
        '    return output\n'
        '\n'
        'def load_actual(path: Path) -> ActualData:\n'
        '    workbook = openpyxl.load_workbook(path, read_only=True, data_only=True)\n'
        "    expected_sheets = ['小区负载', '光伏发电实际功率']\n"
        '    if workbook.sheetnames != expected_sheets:\n'
        "        raise ValueError(f'附件2工作表不匹配: {workbook.sheetnames!r}')\n"
        '    load_dates, load_power = _read_power_sheet(workbook[expected_sheets[0]])\n'
        '    pv_dates, pv_power = _read_power_sheet(workbook[expected_sheets[1]])\n'
        '    if load_dates != pv_dates:\n'
        "        raise ValueError('附件2两个工作表的日期不一致')\n"
        '    expected_dates = [date(2025, 1, 1) + timedelta(days=i) for i in range(365)]\n'
        '    if load_dates != expected_dates:\n'
        "        raise ValueError('附件2日期不是2025-01-01至2025-12-31的连续序列')\n"
        '    return ActualData(dates=load_dates, load_power_endpoints=load_power,
    pv_power_endpoints=pv_power, load_energy=_minute_energy(load_power), pv_energy=
    _minute_energy(pv_power))\n'
        '\n'
        'def _weighted_baseline(values: np.ndarray, target_index: int, history_end: int) -> np.
    ndarray:\n'
        '    if history_end <= 0:\n'
        "        raise ValueError('历史集合为空')\n"
        '    history_indices = np.arange(history_end)\n'
        '    raw_distance = np.abs(target_index - history_indices)\n'
        '    ring_distance = np.minimum(raw_distance, 365 - raw_distance)\n'
        '    weights = 1.0 - 2.0 * ring_distance / 365.0\n'
        '    history = values[:history_end]\n'
        '    valid = np.isfinite(history)\n'
        '    weighted_sum = np.nansum(history * weights[:, None], axis=0)\n'
        '    weight_sum = np.sum(valid * weights[:, None], axis=0)\n'
        '    if np.any(weight_sum <= 0):\n'
        "        raise ValueError('历史加权基准存在零权重分母')\n"
        '    return weighted_sum / weight_sum\n'
        '\n'
        'def execute_day(initial_soc: float, purchase: np.ndarray, planned_discharge: np.
    ndarray, planned_charge: np.ndarray, load_actual: np.ndarray, pv_actual: np.ndarray) ->
    ActualDispatch:\n'
        '    purchase_minute = np.repeat(purchase / MINUTES_PER_SLOT, MINUTES_PER_SLOT)\n'
        '    discharge = np.zeros(MINUTES_PER_DAY)\n'
        '    charge = np.zeros(MINUTES_PER_DAY)\n'
        '    emergency = np.zeros(MINUTES_PER_DAY)\n'
        '    surplus = np.zeros(MINUTES_PER_DAY)\n'
```

```
        '    soc = np.empty(MINUTES_PER_DAY + 1)\n'
        '    soc[0] = initial_soc\n'
        '    joint = np.zeros(MINUTES_PER_DAY)\n'
        '    y_max = POWER_LIMIT_KW / 60.0\n'
        '    z_max = ETA * POWER_LIMIT_KW / 60.0\n'
        '    kappa = 1.0 / ETA - ETA\n'
        '    for minute in range(MINUTES_PER_DAY):\n'
        '        state = soc[minute]\n'
        '        y_plan = float(planned_discharge[minute])\n'
        '        z_plan = float(planned_charge[minute])\n'
        '        q_plan = state - y_plan + z_plan\n'
        '        if q_plan < SOC_MIN_KWH:\n'
        '            y = state + z_plan - SOC_MIN_KWH\n'
        '            z = z_plan\n'
        '        elif q_plan > SOC_MAX_KWH:\n'
        '            y = y_plan\n'
        '            z = SOC_MAX_KWH - state + y_plan\n'
        '        else:\n'
        '            y, z = (y_plan, z_plan)\n'
        '        y = min(max(y, 0.0), y_max)\n'
        '        z = min(max(z, 0.0), z_max)\n'
        '        q = state - y + z\n'
        '        residual = load_actual[minute] + z / ETA - pv_actual[minute] - purchase_minute
    [minute] - ETA * y\n'
        '        if residual > TOL:\n'
        '            amount = min(z, ETA * residual, max(q - SOC_MIN_KWH, 0.0))\n'
        '            z -= amount\n'
        '            residual -= amount / ETA\n'
        '            q = state - y + z\n'
        '            amount = min(y_max - y, residual / ETA, max(q - SOC_MIN_KWH, 0.0))\n'
        '            y += amount\n'
        '            residual -= ETA * amount\n'
        '            q = state - y + z\n'
        '            if residual > TOL and y > TOL and (z > TOL):\n'
        '                amount = min(y, z, residual / kappa)\n'
        '                y -= amount\n'
        '                z -= amount\n'
        '                residual -= kappa * amount\n'
        '                joint[minute] = amount\n'
        '            if residual > TOL:\n'
        '                emergency[minute] = residual\n'
        '            else:\n'
        '                residual = 0.0\n'
        '        elif residual < -TOL:\n'
        '            unused = -residual\n'
        '            amount = min(y, unused / ETA, max(SOC_MAX_KWH - q, 0.0))\n'
        '            y -= amount\n'
        '            unused -= ETA * amount\n'
        '            q = state - y + z\n'
        '            amount = min(z_max - z, ETA * unused, max(SOC_MAX_KWH - q, 0.0))\n'
```

```
        '            z += amount\n'
        '            unused -= amount / ETA\n'
        '            if unused > TOL:\n'
        '                surplus[minute] = unused\n'
        '        discharge[minute] = max(y, 0.0)\n'
        '        charge[minute] = max(z, 0.0)\n'
        '        soc[minute + 1] = state - discharge[minute] + charge[minute]\n'
        '    return ActualDispatch(discharge, charge, emergency, surplus, soc, joint)\n'
    ),

    'forecast': (
        '"""Causal hourly PV fusion. Does not implement or change dispatch rules."""\n'
        'from __future__ import annotations\n'
        'from datetime import datetime, timedelta\n'
        'import numpy as np\n'
        'import openpyxl\n'
        'ALPHAS = (0.0, 0.25, 0.5, 0.75, 1.0)\n'
        'ROOT = _SCRIPT_DIR\n'
        '\n'
        'def read_inputs(root=ROOT):\n'
        '    actual = {}\n'
        "    wb = openpyxl.load_workbook(_DATA_DIR / '附件2.xlsx', read_only=True, data_only=True)\n"
        "    for row in wb['光伏发电实际功率'].iter_rows(min_row=2, values_only=True):\n"
        '        if row[0] is None:\n'
        '            continue\n'
        '        day = row[0] if isinstance(row[0], datetime) else datetime.fromisoformat(str(row[0]))\n'
        '        for i, value in enumerate(row[1:145], 1):\n'
        '            if value is not None:\n'
        '                actual[day + timedelta(minutes=10 * i)] = float(value)\n'
        '    wb.close()\n'
        '    forecasts = {}\n'
        "    wb = openpyxl.load_workbook(_DATA_DIR / '附件3.xlsx', read_only=True, data_only=True)\n"
        '    day = None\n'
        '    for row in wb.active.iter_rows(min_row=2, values_only=True):\n'
        "        if row[0] not in (None, ''):\n"
        '            if isinstance(row[0], datetime):\n'
        '                day = row[0].replace(hour=0, minute=0, second=0)\n'
        '            else:\n'
        "                day = datetime.strptime(str(row[0]), '%Y-%m-%d')\n"
        "        if row[1] in (None, ''):\n"
        '            continue\n'
        '        clock = row[1]\n'
        "        hour = clock.hour if hasattr(clock, 'hour') else int(str(clock).split(':')[0])\n"
        '        issue = day + timedelta(hours=hour)\n'
        '        values = tuple((float(v) for v in row[2:26]))\n'
        '        if len(values) != 24 or not all(np.isfinite(values)):\n'
```

```
        "            raise ValueError(f'Invalid forecast at {issue}')\n"
        '        forecasts[issue] = values\n'
        '    wb.close()\n'
        '    return (forecasts, actual)\n'
        '\n'
        'class Fusion:\n'
        '\n'
        '    def __init__(self, forecasts, actual):\n'
        '        self.forecasts = forecasts\n'
        '        self.actual = actual\n'
        '        self.previous = {}\n'
        '        self.last_issue = None\n'
        '\n'
        '    def components(self, issue, lead):\n'
        '        if not 1 <= lead <= 24:\n'
        "            raise ValueError('lead must be 1..24')\n"
        '        weights, versions = ([], [])\n'
        '        for k in range((24 - lead) // 6, -1, -1):\n'
        '            old = issue - timedelta(hours=6 * k)\n'
        '            weights.append((4 - k) / 10)\n'
        '            versions.append(self.forecasts[old][lead + 6 * k - 1])\n'
        '        f = float(np.dot(weights, versions) / sum(weights))\n'
        '        values, errors = ([], [])\n'
        '        for j in range(1, 8):\n'
        '            old = issue - timedelta(days=j)\n'
        '            target = old + timedelta(hours=lead)\n'
        '            if target > issue:\n'
        "                raise AssertionError('Historical target not yet realized')\n"
        '            value = self.actual[target]\n'
        '            values.append(value)\n'
        '            errors.append(self.forecasts[old][lead - 1] - value)\n'
        '        mean_actual = float(np.mean(values))\n'
        '        mean_error = float(np.mean(errors))\n'
        '        return (f, mean_actual + mean_error, mean_actual, mean_error)\n'
        '\n'
        '    def predict(self, issue):\n'
        '        if self.last_issue is not None and issue <= self.last_issue:\n'
        "            raise ValueError('Process issue times in strictly increasing order')\n"
        '        output = []\n'
        '        next_previous = dict(self.previous)\n'
        '        for lead in range(1, 25):\n'
        '            f, g, real_mean, bias_mean = self.components(issue, lead)\n'
        '            samples = []\n'
        '            for j in range(1, 8):\n'
        '                old = issue - timedelta(days=j)\n'
        '                target = old + timedelta(hours=lead)\n'
        '                if target > issue:\n'
        "                    raise AssertionError('Unrealized training label')\n"
        '                old_f, old_g, _, _ = self.components(old, lead)\n'
        '                samples.append((old_f, old_g, self.actual[target]))\n'
```

```
        '            losses = [float(np.mean([abs(a * x + (1 - a) * y - z) for x, y, z in samples])) for a in ALPHAS]\n'
        '            best = min(losses)\n'
        '            ties = [a for a, loss in zip(ALPHAS, losses) if abs(loss - best) <= 1e-08]\n'
        '            previous = self.previous.get((issue.hour, lead))\n'
        '            alpha = previous if previous in ties else min(ties, key=lambda a: (abs(a - 0.5), -a))\n'
        '            next_previous[issue.hour, lead] = alpha\n'
        "            output.append(dict(issue=issue.isoformat(), target=(issue + timedelta(hours=lead)).isoformat(), lead=lead, alpha=alpha, multi_version=f, history_component=g, history_actual_mean=real_mean, history_signed_error_mean=bias_mean, prediction=alpha * f + (1 - alpha) * g, **{f'mae_{a:g}': loss for a, loss in zip(ALPHAS, losses)}))\n"
        '        self.previous = next_previous\n'
        '        self.last_issue = issue\n'
        '        return output\n'
        '\n'
        'def minute_energy(current_power, hourly_power):\n'
        '    endpoints = np.interp(np.arange(1441), np.arange(25) * 60, [current_power, *hourly_power])\n'
        '    if len(hourly_power) != 24 or not np.all(np.isfinite(endpoints)):\n'
        "        raise ValueError('Need 24 finite hourly endpoints and a current observation')\n"
        '    return (endpoints[:-1] + endpoints[1:]) / 120\n'
        '\n'
        "if __name__ == '__main__':\n"
        '    main()\n'
    ),

    'dispatch': (
        '"""Problem 3 deterministic two-path comparison and chronological execution."""\n'
        'from __future__ import annotations\n'
        'import inspect\n'
        'from datetime import datetime, timedelta\n'
        'from pathlib import Path\n'
        'from types import SimpleNamespace\n'
        'import numpy as np\n'
        'from scipy.optimize import linprog\n'
        'from scipy.sparse import coo_matrix\n'
        'from forecast import Fusion, read_inputs, minute_energy, ROOT\n'
        'HERE = Path(__file__).resolve().parent\n'
        'import problem2_reference as problem2\n'
        "_source = inspect.getsource(problem2.execute_day).replace('MINUTES_PER_DAY', 'len(load_actual)')\n"
        '_namespace = dict(vars(problem2))\n'
        'exec(_source, _namespace)\n'
        "execute = _namespace['execute_day']\n"
        '\n'
        'def net_adjustment(x, original, premium, price):\n'
        '    total = original + premium\n'
```

```
        '    added = np.maximum(x - total, 0)\n'
        '    cancelled = np.maximum(total - x, 0)\n'
        '    cancel_premium = np.minimum(cancelled, premium)\n'
        '    cancel_original = cancelled - cancel_premium\n'
        '    cost = price * (1.5 * added - 1.0 * cancel_premium - 0.5 * cancel_original)\n'
        '    return (cost, original - cancel_original, premium - cancel_premium + added)\n'
        '\n'
        'def solve(load, pv, initial, prices, target, hard=False, ledger=None, penalty=None):\n
    '
        '    """Minute Y/Z/S, ten-minute X, convex adjustment cost epigraph."""\n'
        '    n = len(load)\n'
        '    m = n // 10\n'
        '    assert n % 10 == 0 and len(prices) == m\n'
        '    iy = m\n'
        '    iz = iy + n\n'
        '    iw = iz + n\n'
        '    iss = iw + n\n'
        '    ia = iss + n + 1\n'
        '    ic = ia + 1\n'
        '    size = ic + m\n'
        '    t = np.arange(n)\n'
        '    slots = np.arange(m)\n'
        '    rows = []\n'
        '    cols = []\n'
        '    vals = []\n'
        '\n'
        '    def add(r, c, v):\n'
        '        rows.extend(np.asarray(r).tolist())\n'
        '        cols.extend(np.asarray(c).tolist())\n'
        '        vals.extend(np.broadcast_to(v, np.shape(r)).tolist())\n'
        '    add(t, t // 10, 0.1)\n'
        '    add(t, iy + t, 0.9)\n'
        '    add(t, iz + t, -1 / 0.9)\n'
        '    add(t, iw + t, -1.0)\n'
        '    add(n + t, iss + t + 1, 1.0)\n'
        '    add(n + t, iss + t, -1.0)\n'
        '    add(n + t, iy + t, 1.0)\n'
        '    add(n + t, iz + t, -1.0)\n'
        '    eq = coo_matrix((vals, (rows, cols)), shape=(2 * n, size)).tocsr()\n'
        '    rhs = np.r_[load - pv, np.zeros(n)]\n'
        '    bounds = [(0, None)] * m + [(0, 5000 / 60)] * n + [(0, 0.9 * 5000 / 60)] * n +
    [(0, None)] * n + [(1200, 10800)] * (n + 1) + [(0, None)] + [(None, None)] * m\n'
        '    bounds[iss] = (initial, initial)\n'
        '    if hard:\n'
        '        bounds[iss + n] = (target, target)\n'
        '    ur = []\n'
        '    uc = []\n'
        '    uv = []\n'
        '    ub = []\n'
        '\n'
```

```
        '    def constraint(entries, b):\n'
        '        row = len(ub)\n'
        '        for c, v in entries:\n'
        '            ur.append(row)\n'
        '            uc.append(c)\n'
        '            uv.append(v)\n'
        '        ub.append(b)\n'
        '    constraint([(iss + n, 1), (ia, -1)], target)\n'
        '    constraint([(iss + n, -1), (ia, -1)], -target)\n'
        '    for j, p in enumerate(prices):\n'
        '        if ledger is not None and j < len(ledger[0]):\n'
        '            o, u = (ledger[0][j], ledger[1][j])\n'
        '            for slope, intercept in ((0.5 * p, -0.5 * p * o - p * u), (p, -p * (o + u)
    ), (1.5 * p, -1.5 * p * (o + u))):\n'
        '                constraint([(j, slope), (ic + j, -1)], -intercept)\n'
        '        else:\n'
        '            constraint([(j, p), (ic + j, -1)], 0.0)\n'
        '    aub = coo_matrix((uv, (ur, uc)), shape=(len(ub), size)).tocsr()\n'
        '    objective = np.zeros(size)\n'
        '    if not hard and penalty is None:\n'
        '        objective[ia] = 1\n'
        "        first = linprog(objective, A_ub=aub, b_ub=ub, A_eq=eq, b_eq=rhs, bounds=bounds
    , method='highs')\n"
        '        if not first.success:\n'
        '            raise RuntimeError(first.message)\n'
        '        bounds[ia] = (0, max(0, float(first.fun)) + 1e-06)\n'
        '    objective[:] = 0\n'
        '    objective[ic:] = 1\n'
        '    if penalty is not None and (not hard):\n'
        '        assert penalty >= 0\n'
        '        objective[ia] = penalty\n'
        "    result = linprog(objective, A_ub=aub, b_ub=ub, A_eq=eq, b_eq=rhs, bounds=bounds,
    method='highs')\n"
        '    if not result.success:\n'
        '        raise RuntimeError(result.message)\n'
        '    v = result.x\n'
        '    if np.max(np.abs(eq @ v - rhs)) > 1e-05:\n'
        "        raise AssertionError('LP equality residual')\n"
        '    return SimpleNamespace(purchase=v[:m].copy(), y=v[iy:iz].copy(), z=v[iz:iw].copy()
    , soc=v[iss:ia].copy(), cost=float(result.fun) if penalty is None else float(v[ic:].sum()),
     weighted_cost=float(result.fun), deviation=abs(v[iss + n] - target))\n'
        '\n'
        'def load_prediction(data, target, cutoff):\n'
        '    base = problem2._weighted_baseline(data.load_energy, target, cutoff)\n'
        '    lag = target - 7\n'
        '    previous_base = problem2._weighted_baseline(data.load_energy, lag, lag)\n'
        '    return np.maximum(0, base * data.load_energy[lag] / previous_base)\n'
        '\n'
        'def check_actual(result, purchase, load, pv):\n'
        '    y, z, h, w, s = (result.discharge_internal, result.charge_internal, result.
```

```
    emergency, result.surplus, result.soc)\n'
        '    error = pv + np.repeat(purchase / 10, 10) + 0.9 * y + h - load - z / 0.9 - w\n'
        '    assert np.max(np.abs(error)) < 1e-05\n'
        '    assert np.min(s) >= 1200 - 1e-05 and np.max(s) <= 10800 + 1e-05\n'
        '    assert np.max(y) <= 5000 / 60 + 1e-05 and np.max(z) <= 0.9 * 5000 / 60 + 1e-05\n'
        '    assert min(np.min(y), np.min(z), np.min(h), np.min(w)) >= -1e-06\n'
        '    assert np.max(np.abs(np.diff(s) + y - z)) < 1e-05\n'
        '    return float(np.max(np.abs(error)))\n'
    ),

    'experiment_latest_first6': (
        'from forecast import Fusion\n'
        '\n'
        'class LatestFirstSix(Fusion):\n'
        '\n'
        '    def predict(self, issue):\n'
        '        rows = super().predict(issue)\n'
        '        for row in rows:\n'
        "            row['dynamic_prediction'] = row['prediction']\n"
        "            row['latest_prediction'] = self.forecasts[issue][row['lead'] - 1]\n"
        "            row['applied_method'] = 'latest' if row['lead'] <= 6 else 'dynamic'\n"
        "            if row['lead'] <= 6:\n"
        "                row['prediction'] = row['latest_prediction']\n"
        '        return rows\n'
    ),

    'price_reference': (
        '"""Causal four-week price forecast and frozen Monday--Sunday SOC targets."""\n'
        'import numpy as np\n'
        'import openpyxl\n'
        'import problem2_reference as problem2\n'
        'TARGETS = (6960.0, 5040.0, 3120.0, 1200.0, 6000.0, 10800.0, 8880.0)\n'
        '\n'
        'def load_prices(path, dates):\n'
        '    wb = openpyxl.load_workbook(path, data_only=True, read_only=True)\n'
        '    rows = list(wb.active.values)\n'
        '    wb.close()\n'
        '    assert len(rows[0]) == 145\n'
        '    assert [problem2._as_date(r[0]) for r in rows[1:]] == dates\n'
        '    assert [problem2._minutes(v) for v in rows[0][1:]] == list(range(10, 1441, 10))\n'
        '    values = np.asarray([r[1:] for r in rows[1:]], float)\n'
        '    assert np.isfinite(values).all() and np.min(values) >= 0\n'
        '    return values\n'
        '\n'
        'def forecast_price(prices, target):\n'
        '    assert target >= 28\n'
        '    return np.mean(prices[[target - 7, target - 14, target - 21, target - 28]], axis
    =0)\n'
    ),

```

```
    'runner': (
        'from datetime import datetime,timedelta\n'
        'import csv\n'
        'from pathlib import Path\n'
        'import numpy as np\n'
        'import dispatch as engine\n'
        'from forecast import read_inputs,minute_energy\n'
        'from experiment_latest_first6 import LatestFirstSix\n'
        'from price_reference import forecast_price,load_prices,TARGETS\n'
        '\n'
        'def write_csv(path,rows):\n'
        "    with Path(path).open('w',encoding='utf-8-sig',newline='') as f:\n"
        '        w=csv.DictWriter(f,fieldnames=list(rows[0])); w.writeheader(); w.writerows(rows)\n'
        '\n'
        'def problem3_decision(state,purchase,y,z,original,premium,hour,load,pv,prices):\n'
        '    minute=hour*60; slot=hour*6; remaining=1440-minute\n'
        '    keep=engine.execute(state,purchase[slot:],y[minute:],z[minute:],load[:remaining],pv[:remaining])\n'
        '    target=(1-hour/24)*float(keep.soc[-1])+hour/24*state\n'
        '    tail=engine.solve(load[remaining:],pv[remaining:],float(keep.soc[-1]),prices[:slot],target,penalty=0.5)\n'
        '    keep_score=tail.cost+float(5*np.repeat(prices[slot:],10)@keep.emergency)+0.5*tail.deviation\n'
        '    change=engine.solve(load,pv,state,np.r_[prices[slot:],prices[:slot]],state,\n'
        '                        ledger=(original[slot:],premium[slot:]),penalty=0.5)\n'
        '    return change.cost+0.5*change.deviation < keep_score-1e-6, change\n'
        '\n'
        'def problem4_3_decision(state,purchase,y,z,original,premium,hour,load,pv,prices,target):\n'
        '    slot=hour*6\n'
        '    keep=engine.execute(state,purchase[slot:],y[hour*60:],z[hour*60:],load,pv)\n'
        '    change=engine.solve(load,pv,state,prices[slot:],target,ledger=(original[slot:],premium[slot:]),penalty=1.0)\n'
        '    changed=engine.execute(state,change.purchase,change.y,change.z,load,pv)\n'
        '    keep_score=float(5*np.repeat(prices[slot:],10)@keep.emergency)+abs(float(keep.soc[-1])-target)\n'
        '    change_score=change.cost+float(5*np.repeat(prices[slot:],10)@changed.emergency)+abs(float(changed.soc[-1])-target)\n'
        '    return change_score < keep_score-1e-6, change\n'
        '\n'
        'def run(start,end,out,scenario):\n'
        '    out=Path(out); out.mkdir(parents=True,exist_ok=True)\n'
        "    data=engine.problem2.load_actual(_DATA_DIR/'附件2.xlsx')\n"
        "    fixed=engine.problem2.load_price(_DATA_DIR/'附件1.xlsx')\n"
        "    actual_prices=np.tile(fixed,(365,1)) if scenario=='problem3' else load_prices(_DATA_DIR/'附件4.xlsx',data.dates)\n"
        '    forecasts,actual_power=read_inputs()\n'
        '    fusion=LatestFirstSix(forecasts,actual_power)\n'
        '    state=6000.0; daily=[]\n'
```

```
    "    collected={k:[] for k in ('purchase_initial','purchase_final','price','soc','charge_internal','discharge_internal','emergency','surplus')}\n"
    '    dates=[]\n'
    '    for index,day in enumerate(data.dates):\n'
    '        midnight=datetime.combine(day,datetime.min.time())\n'
    '        if not start<=midnight<end: continue\n'
    "        target=state if scenario=='problem3' else TARGETS[day.weekday()]\n"
    "        prices=fixed.copy() if scenario=='problem3' else forecast_price(actual_prices,index)\n"
    '        load_day=engine.load_prediction(data,index,index)\n'
    '        next_load=engine.load_prediction(data,index+1,index)\n'
    '        coeff=np.zeros(144); cancelled_o=np.zeros(144); cancelled_u=np.zeros(144); added_all=np.zeros(144)\n'
    '        traces=[]; executed=[]\n'
    '        for hour in (0,6,12,18):\n'
    '            minute=hour*60; slot=hour*6; remaining=1440-minute\n'
    '            issue=midnight+timedelta(hours=hour)\n'
    '            prediction=fusion.predict(issue)\n'
    "            pv24=minute_energy(actual_power[issue],[row['prediction'] for row in prediction])\n"
    '            load24=np.r_[load_day[minute:],next_load[:minute]]\n'
    '            if hour==0:\n'
    "                plan=engine.solve(load_day,pv24,state,prices,target,hard=scenario=='problem3')\n"
    '                purchase,y,z=plan.purchase.copy(),plan.y.copy(),plan.z.copy()\n'
    '                initial_purchase=purchase.copy(); original=purchase.copy(); premium=np.zeros(144)\n'
    '            else:\n'
    "                if scenario=='problem3':\n"
    '                    accepted,change=problem3_decision(state,purchase,y,z,original,premium,hour,load24,pv24,prices)\n'
    '                else:\n'
    '                    accepted,change=problem4_3_decision(state,purchase,y,z,original,premium,hour,\n'
    '                                                        load24[:remaining],pv24[:remaining],prices,target)\n'
    '                if accepted:\n'
    '                    new_purchase=change.purchase[:144-slot]\n'
    '                    old_total=original[slot:]+premium[slot:]\n'
    '                    cancelled=np.maximum(old_total-new_purchase,0)\n'
    '                    cu=np.minimum(cancelled,premium[slot:]); co=cancelled-cu\n'
    '                    added=np.maximum(new_purchase-old_total,0)\n'
    '                    delta,new_o,new_u=engine.net_adjustment(new_purchase,original[slot:],premium[slot:],np.ones(144-slot))\n'
    '                    coeff[slot:]+=delta; cancelled_o[slot:]+=co; cancelled_u[slot:]+=cu; added_all[slot:]+=added\n'
    '                    original[slot:],premium[slot:]=new_o,new_u\n'
    '                    purchase[slot:]=new_purchase\n'
    '                    y[minute:],z[minute:]=change.y[:remaining],change.z[:remaining]\n'
    '            stop=minute+360; endslot=slot+36\n'
```

```
        '            executed.append(purchase[slot:endslot].copy())\n'
        '            ex=engine.execute(state,purchase[slot:endslot],y[minute:stop],z[minute:
    stop],\n'
        '                              data.load_energy[index,minute:stop],data.pv_energy[index
    ,minute:stop])\n'
        '            engine.check_actual(ex,purchase[slot:endslot],data.load_energy[index,
    minute:stop],data.pv_energy[index,minute:stop])\n'
        '            state=float(ex.soc[-1]); traces.append(ex)\n'
        '        assert np.array_equal(purchase,np.concatenate(executed))\n'
        '        real_price=actual_prices[index]\n'
        '        initial_cost=float(real_price@initial_purchase)\n'
        '        adjustment_cost=float(real_price@coeff)\n'
        '        assert abs(initial_cost+adjustment_cost-float(real_price@(original+1.5*premium
    ))-float(0.5*real_price@(cancelled_o+cancelled_u)))<1e-5\n'
        '        h=np.concatenate([x.emergency for x in traces])\n'
        '        y_actual=np.concatenate([x.discharge_internal for x in traces])\n'
        '        z_actual=np.concatenate([x.charge_internal for x in traces])\n'
        '        surplus=np.concatenate([x.surplus for x in traces])\n'
        '        soc=np.r_[traces[0].soc,*[x.soc[1:] for x in traces[1:]]]\n'
        '        emergency_cost=float(5*np.repeat(real_price,10)@h)\n'
        '        dates.append(str(day))\n'
        '        daily.append(dict(date=str(day),initial_purchase_cost=initial_cost,
    adjustment_net_cost=adjustment_cost,\n'
        '                          emergency_cost=emergency_cost,total_cost=initial_cost+
    adjustment_cost+emergency_cost))\n'
        '        for key,value in {\n'
        "            'purchase_initial':initial_purchase,'purchase_final':purchase,'price':
    real_price,'soc':soc,\n"
        "            'charge_internal':z_actual,'discharge_internal':y_actual,'emergency':h,'
    surplus':surplus}.items():\n"
        '            collected[key].append(value.copy())\n'
        "        if len(daily)%30==0: print(f'Problem4-3 completed {len(daily)} days',flush=
    True)\n"
        "    if not daily: raise ValueError('没有可计算日期')\n"
        "    np.savez_compressed(out/'all_results.npz',dates=np.asarray(dates),**{k:np.stack(v)
     for k,v in collected.items()})\n"
        "    write_csv(out/'daily.csv',daily)\n"
    ),

    'standalone_outputs': (
        'from __future__ import annotations\n'
        'import copy\n'
        'import csv\n'
        'import math\n'
        'import posixpath\n'
        'import re\n'
        'from datetime import date, timedelta\n'
        'from pathlib import Path\n'
        'from xml.etree import ElementTree as ET\n'
        'from xml.sax.saxutils import escape\n'
```

```
        'from zipfile import ZipFile, ZIP_DEFLATED\n'
        'import numpy as np\n'
        '\n'
        "S = 'http://schemas.openxmlformats.org/spreadsheetml/2006/main'\n"
        "R = 'http://schemas.openxmlformats.org/officeDocument/2006/relationships'\n"
        "P = 'http://schemas.openxmlformats.org/package/2006/relationships'\n"
        "SUPPLEMENT = '完整自然日购电量'\n"
        "FILES = {'problem1':'result1.xlsx','problem2':'result2.xlsx','problem3':'result3.xlsx','problem4-2':'result4-2.xlsx','problem4-3':'result4-3.xlsx'}\n"
        "ROW_RE = re.compile('<row\\\\b[^>]*?(?:/>|>.*?</row>)', re.S)\n"
        "CELL_RE = re.compile('<c\\\\b[^>]*?(?:/>|>.*?</c>)', re.S)\n"
        '\n'
        '\n'
        'def colname(number: int) -> str:\n'
        "    result = ''\n"
        '    while number:\n'
        '        number, rem = divmod(number - 1, 26)\n'
        '        result = chr(65 + rem) + result\n'
        '    return result\n'
        '\n'
        'def colnumber(name: str) -> int:\n'
        '    result = 0\n'
        '    for char in name:\n'
        '        result = 26 * result + ord(char) - 64\n'
        '    return result\n'
        '\n'
        'def address_key(ref: str) -> tuple[int, int]:\n'
        "    match = re.fullmatch('([A-Z]+)([0-9]+)', ref)\n"
        '    if match is None:\n'
        "        raise ValueError(f'Invalid cell reference {ref}')\n"
        '    return (int(match[2]), colnumber(match[1]))\n'
        '\n'
        'def attr(fragment: str, name: str, default: str | None=None) -> str | None:\n'
        '    match = re.search(\'\\\\b\' + re.escape(name) + \'="([^"]*)"\', fragment)\n'
        '    return match[1] if match else default\n'
        '\n'
        'def scalar_cell(ref: str, value, original: str | None=None, style: str | None=None) -> str:\n'
        '    """Replace only a cell value; retain its original attributes except type."""\n'
        '    if original:\n'
        "        opening = re.match('<c\\\\b([^>]*?)(?:/?>)', original)[1]\n"
        '        opening = re.sub(\'\\\\s+t="[^"]*"\', \'\', opening).rstrip(\'/ \')\n'
        '        opening = re.sub(\'\\\\br="[^"]*"\', f\'r="{ref}"\', opening)\n'
        '    else:\n'
        '        opening = f\' r="{ref}"\' + (f\' s="{style}"\' if style is not None else \'\')\n'
        '    if value is None:\n'
        "        return f'<c{opening}/>'\n"
        '    if isinstance(value, (str, np.str_)):\n'
        '        return f\'<c{opening} t="inlineStr"><is><t xml:space="preserve">{escape(str(
```

```
    value))}</t></is></c>\'\n'
        '    numeric = float(value)\n'
        '    if not math.isfinite(numeric):\n'
        "        raise ValueError(f'Non-finite numeric output at {ref}: {value}')\n"
        '    if abs(numeric) < 1e-10:\n'
        '        numeric = 0.0\n'
        '    return f"<c{opening}><v>{format(numeric, \'.15g\')}</v></c>"\n'
        '\n'
        'def rows_in(xml: str) -> dict[int, str]:\n'
        "    data = re.search('<sheetData>(.*?)</sheetData>', xml, re.S)\n"
        '    if data is None:\n'
        "        raise ValueError('Template has no sheetData element')\n"
        "    return {int(attr(row.split('>', 1)[0], 'r')): row for row in ROW_RE.findall(data
    [1])}\n"
        '\n'
        'def patch_row(row: str, updates: dict[str, object], new_number: int | None=None) ->
    str:\n'
        "    opening = re.match('<row\\\\b([^>]*?)(?:/?>)', row)[1].rstrip('/ ')\n"
        "    old_number = int(attr(opening, 'r'))\n"
        '    new_number = old_number if new_number is None else new_number\n'
        '    opening = re.sub(\'\\\\br="[^"]*"\', f\'r="{new_number}"\', opening)\n'
        '    cells = {}\n'
        '    for cell in CELL_RE.findall(row):\n'
        "        ref = attr(cell.split('>', 1)[0], 'r')\n"
        "        column = re.match('[A-Z]+', ref)[0]\n"
        "        new_ref = f'{column}{new_number}'\n"
        '        cells[new_ref] = re.sub(\'\\\\br="[^"]*"\', f\'r="{new_ref}"\', cell, count=1)
    \n'
        "    default_style = attr(opening, 's')\n"
        '    for column_or_ref, value in updates.items():\n'
        "        column = re.match('[A-Z]+', column_or_ref)[0]\n"
        "        ref = f'{column}{new_number}'\n"
        '        cells[ref] = scalar_cell(ref, value, cells.get(ref), default_style)\n'
        "    return f'<row{opening}>' + ''.join((cells[k] for k in sorted(cells, key=
    address_key))) + '</row>'\n"
        '\n'
        'def replace_rows(xml: str, rows: list[str], last_cell: str | None=None) -> str:\n'
        "    xml = re.sub('<sheetData>.*?</sheetData>', lambda _: '<sheetData>' + ''.join(rows)
     + '</sheetData>', xml, count=1, flags=re.S)\n"
        '    if last_cell:\n'
        '        xml = re.sub(\'<dimension\\\\b[^>]*/>\', f\'<dimension ref="A1:{last_cell
    }"/>\', xml, count=1)\n'
        '    return xml\n'
        '\n'
        'def excel_date(day: date) -> int:\n'
        '    return (day - date(1899, 12, 30)).days\n'
        '\n'
        'def clock(minutes: int) -> str:\n'
        "    return f'{minutes // 60:02d}:{minutes % 60:02d}'\n"
        '\n'
```

```
        'def intervals(energy: np.ndarray, tolerance: float=1e-08) -> list[tuple[int, int, float]]:\n'
        '    positive = np.asarray(energy) > tolerance\n'
        '    padded = np.r_[False, positive, False].astype(np.int8)\n'
        '    starts = np.flatnonzero(np.diff(padded) == 1)\n'
        '    ends = np.flatnonzero(np.diff(padded) == -1)\n'
        '    return [(int(a), int(b), float(np.sum(energy[a:b]))) for a, b in zip(starts, ends)]\n'
        '\n'
        'def load_results(folder: Path, case: str) -> dict:\n'
        '    folder = Path(folder).resolve()\n'
        "    with np.load(folder / 'all_results.npz', allow_pickle=False) as source:\n"
        '        data = {key: np.array(source[key]) for key in source.files}\n'
        "    required = {'dates', 'purchase_initial', 'purchase_final', 'price', 'charge_internal', 'discharge_internal', 'emergency', 'surplus', 'soc'}\n"
        '    if required - data.keys():\n'
        "        raise ValueError(f'{case}: missing arrays {sorted(required - data.keys())}')\n"
        "    dates = [date.fromisoformat(str(value)[:10]) for value in data['dates']]\n"
        '    n = len(dates)\n'
        "    if case == 'problem1':\n"
        '        if n != 1:\n'
        "            raise ValueError('problem1 must contain exactly one modeled day')\n"
        '    elif dates != [date(2025, 2, 1) + timedelta(days=i) for i in range(334)]:\n'
        "        raise ValueError(f'{case}: official export requires all 334 dates from February 1 to December 31')\n"
        "    for key in required - {'dates'}:\n"
        "        width = 1441 if key == 'soc' else 144 if key in {'purchase_initial', 'purchase_final', 'price'} else 1440\n"
        '        if data[key].shape != (n, width) or not np.all(np.isfinite(data[key])):\n'
        "            raise ValueError(f'{case}: invalid {key} shape or nonfinite values: {data[key].shape}')\n"
        "    data['dates'] = dates\n"
        "    daily_path = next((p for p in [folder / 'daily.csv', folder / 'summary' / 'daily.csv'] if p.exists()), None)\n"
        '    if daily_path is None:\n'
        "        raise FileNotFoundError(f'Missing daily settlement CSV under {folder}')\n"
        "    with daily_path.open(encoding='utf-8-sig', newline='') as stream:\n"
        '        records = list(csv.DictReader(stream))\n'
        '    if not records:\n'
        "        raise ValueError('Daily settlement CSV has no rows')\n"
        "    date_key = next((key for key in ['date', '日期', 'day'] if key in records[0]), None)\n"
        '    if date_key is None:\n'
        "        raise ValueError(f'{daily_path}: need date column')\n"
        '    by_date = {date.fromisoformat(row[date_key][:10]): row for row in records}\n'
        '    if len(by_date) != len(records) or set(by_date) != set(dates):\n'
        "        raise ValueError('Daily CSV dates must match NPZ dates exactly without duplicates')\n"
        "    aliases = {'initial_cost': ['initial_cost', 'initial_purchase_cost', '
```

```
    initial_plan_cost', 'planned_cost', 'plan_cost', 'purchase_cost'], 'adjustment_cost': ['
    adjustment_cost', 'adjustment_net_cost', 'adjustment_net', 'adjust_cost'], 'emergency_cost
    ': ['emergency_cost', 'emergency_purchase_cost'], 'total_cost': ['total_cost', '
    actual_total_cost', 'actual_cost', 'cost']}\n"
744     '    daily = []\n'
745     '    for i, day in enumerate(dates):\n'
746     '        if day not in by_date:\n'
747     "            raise ValueError(f'{daily_path}: missing {day}')\n"
748     '        row = by_date[day]\n'
749     '        values = {}\n'
750     '        for key, names in aliases.items():\n'
751     "            found = next((row[name] for name in names if name in row and row[name] !=
    ''), None)\n"
752     '            if found is None:\n'
753     "                if key == 'adjustment_cost' and case in {'problem1', 'problem2', '
    problem4-2'}:\n"
754     '                    values[key] = 0.0\n'
755     '                else:\n'
756     "                    raise ValueError(f'{daily_path}: need {key} for {day}; available
    columns={list(row)}')\n"
757     '            else:\n'
758     '                values[key] = float(found)\n'
759     '        if not all((math.isfinite(v) for v in values.values())):\n'
760     "            raise ValueError(f'Nonfinite daily cost for {day}')\n"
761     "        computed_total = values['initial_cost'] + values['adjustment_cost'] + values['
    emergency_cost']\n"
762     "        if not math.isclose(computed_total, values['total_cost'], abs_tol=0.01,
    rel_tol=1e-09):\n"
763     "            raise ValueError(f'Daily settlement does not reconcile for {case}/{day}: {
    values}')\n"
764     "        actual_emergency = float(np.dot(data['emergency'][i], np.repeat(data['price'][
    i], 10)) * 5)\n"
765     "        if not math.isclose(actual_emergency, values['emergency_cost'], abs_tol=0.01,
    rel_tol=1e-08):\n"
766     "            raise ValueError(f'Emergency cost disagrees with minute trace for {case}/{
    day}')\n"
767     "        actual_initial = float(np.dot(data['purchase_initial'][i], data['price'][i]))\
    n"
768     "        if not math.isclose(actual_initial, values['initial_cost'], abs_tol=0.01,
    rel_tol=1e-08):\n"
769     "            raise ValueError(f'Initial plan cost disagrees with 144-slot trace for {
    case}/{day}')\n"
770     '        daily.append(values)\n'
771     "    data['daily'] = daily\n"
772     "    data['daily_csv'] = daily_path.relative_to(folder).as_posix()\n"
773     '    return data\n'
774     '\n'
775     'def sheet_paths(parts: dict[str, bytes]) -> dict[str, str]:\n'
776     "    workbook = ET.fromstring(parts['xl/workbook.xml'])\n"
777     "    rels = {r.attrib['Id']: r.attrib['Target'] for r in ET.fromstring(parts['xl/_rels/
```

```
    workbook.xml.rels'])}\n"
778     "    return {s.attrib['name']: posixpath.normpath('xl/' + rels[s.attrib[f'{{{R}}}id']])
     for s in workbook.find(f'{{{S}}}sheets')}\n"
779     '\n'
780     'def write_purchase(xml: str, data: dict, *, final: bool, single: bool) -> str:\n'
781     '    rows = rows_in(xml)\n'
782     "    array = data['purchase_final' if final else 'purchase_initial']\n"
783     '    if single:\n'
784     '        for row in range(2, 145):\n'
785     "            rows[row] = patch_row(rows[row], {'B': array[0, row - 1]})\n"
786     "        rows[145] = patch_row(rows[145], {'B': None})\n"
787     '    else:\n'
788     "        for i, day in enumerate(data['dates']):\n"
789     '            row = i + 2\n'
790     '            updates = {colname(j + 1): array[i, j] for j in range(1, 144)}\n'
791     "            updates['EO'] = array[i + 1, 0] if i + 1 < len(array) else None\n"
792     "            updates['EP'] = float(array[i].sum())\n"
793     "            costs = data['daily'][i]\n"
794     "            updates['EQ'] = costs['initial_cost'] + (costs['adjustment_cost'] if final
     else 0)\n"
795     '            rows[row] = patch_row(rows[row], updates)\n'
796     '    return replace_rows(xml, [rows[r] for r in sorted(rows)])\n'
797     '\n'
798     'def write_storage(xml: str, data: dict, *, single: bool) -> str:\n'
799     '    original = rows_in(xml)\n'
800     '    eta = 0.9\n'
801     '    if single:\n'
802     '        for block in range(6):\n'
803     '            take = slice(240 * block, 240 * (block + 1))\n'
804     "            updates = {'B': float(data['charge_internal'][0, take].sum() / eta), 'C':
    float(data['discharge_internal'][0, take].sum() * eta)}\n"
805     '            if block < 2:\n'
806     "                updates['E'] = data['soc'][0, 0 if block == 0 else -1]\n"
807     '            original[block + 2] = patch_row(original[block + 2], updates)\n'
808     '        return replace_rows(xml, [original[r] for r in sorted(original)])\n'
809     '    output = [original[1]]\n'
810     "    for i, day in enumerate(data['dates']):\n"
811     '        for block in range(6):\n'
812     '            row = 2 + 6 * i + block\n'
813     '            take = slice(240 * block, 240 * (block + 1))\n'
814     "            updates = {'A': excel_date(day) if block == 0 else None, 'C': float(data['
    charge_internal'][i, take].sum() / eta), 'D': float(data['discharge_internal'][i, take].sum
    () * eta), 'F': data['soc'][i, 0 if block == 0 else -1] if block < 2 else None}\n"
815     '            output.append(patch_row(original[block + 2], updates, row))\n'
816     '    return replace_rows(xml, output, f"F{1 + 6 * len(data[\'dates\'])}")\n'
817     '\n'
818     'def write_emergency(xml: str, data: dict) -> tuple[str, int]:\n'
819     '    original = rows_in(xml)\n'
820     '    output = [original[1]]\n'
821     '    count = 0\n'
```

```
    "    for i, day in enumerate(data['dates']):\n"
    "        events = intervals(data['emergency'][i])\n"
    '        count += len(events)\n'
    "        records = [(f'{clock(a)}-{clock(b)}', energy) for a, b, energy in events] or [('无', 0.0)]\n"
    '        for j, (label, energy) in enumerate(records):\n'
    '            prototype = original[2 if j == 0 else 4 if j == len(records) - 1 else 3]\n'
    "            output.append(patch_row(prototype, {'A': excel_date(day) if j == 0 else None, 'B': label, 'C': energy}, len(output) + 1))\n"
    "    return (replace_rows(xml, output, f'C{len(output)}'), count)\n"
    '\n'
    'def supplement_xml(parts: dict[str, bytes], data: dict, case: str) -> str:\n'
    '    """A compact additional matrix; preserve all original styles.xml bytes."""\n'
    '    if parts:\n'
    "        source = rows_in(parts[sheet_paths(parts)['计划购电量']].decode('utf-8'))\n"
    "        header_style = attr(CELL_RE.findall(source[1])[1].split('>', 1)[0], 's', '0')\n"
    "        value_style = attr(CELL_RE.findall(source[2])[-1].split('>', 1)[0], 's', '0')\n"
    '    else:\n'
    "        header_style, value_style = ('1', '2')\n"
    "    headings = ['日期', '计划类型'] + [f'{clock(10 * k)}-{clock(10 * (k + 1))}' for k in range(144)]\n"
    "    headings += ['全天购电量(kWh)', '初始计划款(元)', '调整净费用(元)', '紧急购电费(元)', '总购电费(元)', '紧急购电量(kWh)']\n"
    '    rows = [\'<row r="1" ht="28" customHeight="1">\' + \'\'.join((scalar_cell(f\'{colname(c)}1\', v, style=header_style) for c, v in enumerate(headings, 1))) + \'</row>\']\n'
    "    stage_rows = [('初始计划', 'purchase_initial'), ('最终有效计划', 'purchase_final')] if case in {'problem3', 'problem4-3'} else [('计划购电', 'purchase_initial')]\n"
    "    for i, day in enumerate(data['dates']):\n"
    "        costs = data['daily'][i]\n"
    '        for stage, key in stage_rows:\n'
    "            settlement = stage != '初始计划'\n"
    "            display_day = '附件1示例日' if case == 'problem1' else day.isoformat()\n"
    '            values = [display_day, stage] + data[key][i].tolist() + [float(data[key][i].sum())]\n'
    "            values += [costs['initial_cost'], costs['adjustment_cost'] if settlement else None, costs['emergency_cost'] if settlement else None, costs['total_cost'] if settlement else None, float(data['emergency'][i].sum()) if settlement else None]\n"
    '            r = len(rows) + 1\n'
    '            rows.append(f\'<row r="{r}" ht="20" customHeight="1">\' + \'\'.join((scalar_cell(f\'{colname(c)}{r}\', value, style=value_style) for c, value in enumerate(values, 1))) + \'</row>\')\n'
    '    return f\'<?xml version="1.0" encoding="UTF-8" standalone="yes"?><worksheet xmlns="{S}"><dimension ref="A1:EV{len(rows)}"/><sheetViews><sheetView showGridLines="0" workbookViewId="0"><pane xSplit="2" ySplit="1" topLeftCell="C2" activePane="bottomRight" state="frozen"/></sheetView></sheetViews><sheetFormatPr defaultRowHeight="20"/><cols><col min="1" max="2" width="18" customWidth="1"/><col min="3" max="146" width="16" customWidth
```

```
    ="1"/><col min="147" max="152" width="22" customWidth="1"/></cols><sheetData>\' + \'\'.join
    (rows) + \'</sheetData><pageMargins left="0.3" right="0.3" top="0.5" bottom="0.5" header
    ="0.2" footer="0.2"/></worksheet>\'\n'
853     '\n'
854     'def _display_day(day: date, case: str) -> str:\n'
855     "    return '附件1示例日' if case == 'problem1' else day.isoformat()\n"
856     '\n'
857     'def _plain_sheet(headers: list[str], values: list[list], widths=None) -> str:\n'
858     '    """Write an explicitly new sheet without any external authoring library."""\n'
859     '    rows = []\n'
860     '    for number, row in enumerate([headers] + values, 1):\n'
861     "        style = '1' if number == 1 else '2'\n"
862     "        cells = ''.join((scalar_cell(f'{colname(c)}{number}', value, style=style) for
    c, value in enumerate(row, 1)))\n"
863     '        rows.append(f\'<row r="{number}" ht="30" customHeight="1">{cells}</row>\')\n'
864     '    columns = \'\'.join((f\'<col min="{i}" max="{i}" width="{width}" customWidth
    ="1"/>\' for i, width in enumerate(widths or [22] * len(headers), 1)))\n'
865     '    return f\'\'\'<?xml version="1.0" encoding="UTF-8" standalone="yes"?><worksheet
    xmlns="{S}"><dimension ref="A1:{colname(len(headers))}{len(rows)}"/><sheetViews><sheetView
    showGridLines="0" workbookViewId="0"><pane ySplit="1" topLeftCell="A2" activePane="
    bottomLeft" state="frozen"/></sheetView></sheetViews><sheetFormatPr defaultRowHeight
    ="30"/><cols>{columns}</cols><sheetData>{\'\'.join(rows)}</sheetData><pageMargins left
    ="0.3" right="0.3" top="0.5" bottom="0.5" header="0.2" footer="0.2"/></worksheet>\'\'\'\n'
866     '\n'
867     'def _new_workbook_parts(sheets: list[tuple[str, str]]) -> dict[str, bytes]:\n'
868     '    types = [\'<Default Extension="rels" ContentType="application/vnd.openxmlformats-
    package.relationships+xml"/>\', \'<Default Extension="xml" ContentType="application/xml
    "/>\', \'<Override PartName="/xl/workbook.xml" ContentType="application/vnd.openxmlformats-
    officedocument.spreadsheetml.sheet.main+xml"/>\', \'<Override PartName="/xl/styles.xml"
    ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.styles+xml"/>\']\n
    '
869     '    relations, entries = ([], [])\n'
870     '    parts = {}\n'
871     '    for index, (name, xml) in enumerate(sheets, 1):\n'
872     "        member = f'xl/worksheets/sheet{index}.xml'\n"
873     "        parts[member] = xml.encode('utf-8')\n"
874     '        types.append(f\'<Override PartName="/{member}" ContentType="application/vnd.
    openxmlformats-officedocument.spreadsheetml.worksheet+xml"/>\')\n'
875     '        relations.append(f\'<Relationship Id="rId{index}" Type="{R}/worksheet" Target
    ="worksheets/sheet{index}.xml"/>\')\n'
876     '        entries.append(f\'<sheet name="{escape(name)}" sheetId="{index}" r:id="rId{
    index}"/>\')\n'
877     '    relations.append(f\'<Relationship Id="rId{len(sheets) + 1}" Type="{R}/styles"
    Target="styles.xml"/>\')\n'
878     '    parts[\'[Content_Types].xml\'] = f"""<Types xmlns="http://schemas.openxmlformats.
    org/package/2006/content-types">{\'\'.join(types)}</Types>""".encode()\n'
879     '    parts[\'_rels/.rels\'] = f\'<Relationships xmlns="{P}"><Relationship Id="rId1"
    Type="{R}/officeDocument" Target="xl/workbook.xml"/></Relationships>\'.encode()\n'
880     '    parts[\'xl/_rels/workbook.xml.rels\'] = f\'\'\'<Relationships xmlns="{P}">{\'\'.
    join(relations)}</Relationships>\'\'\'.encode()\n'
```

```
    '    parts[\'xl/workbook.xml\'] = f\'\'\'<workbook xmlns="{S}" xmlns:r="{R}"><bookViews><workbookView/></bookViews><sheets>{\'\'.join(entries)}</sheets></workbook>\'\'\'.encode()\n'
    '    parts[\'xl/styles.xml\'] = f\'<styleSheet xmlns="{S}"><fonts count="2"><font><sz val="11"/><name val="Microsoft YaHei"/></font><font><b/><sz val="11"/><color rgb="FFFFFFFF"/><name val="Microsoft YaHei"/></font></fonts><fills count="3"><fill><patternFill patternType="none"/></fill><fill><patternFill patternType="gray125"/></fill><fill><patternFill patternType="solid"><fgColor rgb="FF245A81"/><bgColor indexed="64"/></patternFill></fill></fills><borders count="1"><border/></borders><cellStyleXfs count="1"><xf numFmtId="0" fontId="0" fillId="0" borderId="0"/></cellStyleXfs><cellXfs count="3"><xf numFmtId="0" fontId="0" fillId="0" borderId="0" xfId="0"/><xf numFmtId="0" fontId="1" fillId="2" borderId="0" xfId="0" applyAlignment="1"><alignment vertical="center" wrapText="1"/></xf><xf numFmtId="0" fontId="0" fillId="0" borderId="0" xfId="0" applyAlignment="1"><alignment vertical="center" wrapText="1"/></xf></cellXfs><cellStyles count="1"><cellStyle name="Normal" xfId="0" builtinId="0"/></cellStyles></styleSheet>\'.encode()\n'
    '    return parts\n'
    '\n'
    'def _standalone_parts(data: dict, case: str) -> tuple[dict[str, bytes], int]:\n'
    '    storage, emergency, settlement = ([], [], [])\n'
    '    event_count = 0\n'
    "    for i, day in enumerate(data['dates']):\n"
    '        label = _display_day(day, case)\n'
    '        for block in range(6):\n'
    '            a, b = (240 * block, 240 * (block + 1))\n'
    "            storage.append([label, f'{clock(a)}-{clock(b)}', float(data['charge_internal'][i, a:b].sum() / 0.9), float(data['discharge_internal'][i, a:b].sum() * 0.9), float(data['soc'][i, a]), float(data['soc'][i, b])])\n"
    "        events = intervals(data['emergency'][i])\n"
    '        event_count += len(events)\n'
    "        emergency.extend([[label, f'{clock(a)}-{clock(b)}', energy] for a, b, energy in events] or [[label, '无', 0.0]])\n"
    "        costs = data['daily'][i]\n"
    "        settlement.append([label, float(data['purchase_initial'][i].sum()), float(data['purchase_final'][i].sum()), float(data['charge_internal'][i].sum() / 0.9), float(data['discharge_internal'][i].sum() * 0.9), float(data['emergency'][i].sum()), float(data['surplus'][i].sum()), float(data['soc'][i, 0]), float(data['soc'][i, -1]), costs['initial_cost'], costs['adjustment_cost'], costs['emergency_cost'], costs['total_cost']])\n"
    "    notes = [['文件来源', '未找到附件5对应模板；本文件为程序独立生成的完整结果表，不声称保留官方模板结构。'], ['问题', case], ['计算天数', len(data['dates'])], ['计算范围', '附件1未标日期的示例日' if case == 'problem1' else '2025-02-01至2025-12-31，共334个自然日'], ['购电区间', '完整自然日表逐项列出00:00-00:10至23:50-24:00的144段，保留初始及最终计划。'], ['充放电口径', '购电侧充电量=内部储能增量/0.9；供负载侧放电量=内部储能减少量×0.9；每4小时汇总。'], ['紧急购电', '按实际连续分钟区间逐事件列出；无事件的日期以‘无’和0列示，事件数不设上限。'], ['费用口径', '总购电费=初始计划款+已接受调整净费用+紧急购电费；紧急单价为该分钟对应电价的5倍。'], ['数据来源', '同目录all_results.npz及daily.csv；数值保留为数值单元格。'], ['日期说明', 'Problem1显示‘附件1示例日’，NPZ/CSV日期仅作内部关联键；其他问题显示真实2025年日期。']]\n"
    "    sheets = [('导出说明', _plain_sheet(['项目', '说明'], notes, [22, 115])), (SUPPLEMENT, supplement_xml({}, data, case)), ('充放电量', _plain_sheet(['日期', '时段', '充电购电侧(kWh)', '放电供负载侧(kWh)', '时段起始SOC(kWh)', '时段结束SOC(kWh)'], storage)), ('紧急购电量', _plain_sheet(['日期', '实际连续分钟时段', '紧急购电量(kWh)'], emergency)), ('日电
```

```
    量与费用', _plain_sheet(['日期', '初始计划购电量(kWh)', '最终计划购电量(kWh)', '充电购电侧(kWh)', '放电供负载侧(kWh)', '紧急购电量(kWh)', '未利用电量(kWh)', '日初SOC(kWh)', '日末SOC(kWh)', '初始计划款(元)', '调整净费用(元)', '紧急购电费(元)', '总购电费(元)'], settlement))]\n"
900     '    return (_new_workbook_parts(sheets), event_count)\n'
901     '\n'
902     'def beautify_excel(path: Path) -> None:\n'
903     '    """Apply the final workbook appearance without changing calculated values.\n'
904     '\n'
905     '    - Every non-empty cell in every sheet uses 宋体 10 pt.\n'
906     '    - Every non-empty cell is horizontally and vertically centered.\n'
907     '    - The last row of “紧急购电量” gets a black bottom border across A:C.\n'
908     '    Existing bold/italic/color/fill/number-format settings and all other border sides are preserved.\n'
909     '    """\n'
910     '    import openpyxl\n'
911     '    from copy import copy\n'
912     '    from openpyxl.styles import Side\n'
913     '\n'
914     '    workbook = openpyxl.load_workbook(path)\n'
915     '    try:\n'
916     '        for sheet in workbook.worksheets:\n'
917     '            for row in sheet.iter_rows():\n'
918     '                for cell in row:\n'
919     '                    if cell.value is None:\n'
920     '                        continue\n'
921     '\n'
922     '                    font = copy(cell.font)\n'
923     "                    font.name = '宋体'\n"
924     '                    font.sz = 10\n'
925     '                    font.scheme = None\n'
926     '                    cell.font = font\n'
927     '\n'
928     '                    alignment = copy(cell.alignment)\n'
929     "                    alignment.horizontal = 'center'\n"
930     "                    alignment.vertical = 'center'\n"
931     '                    cell.alignment = alignment\n'
932     '\n'
933     "        if '紧急购电量' in workbook.sheetnames:\n"
934     "            sheet = workbook['紧急购电量']\n"
935     '            last_row = sheet.max_row\n'
936     "            black = Side(style='thin', color='FF000000')\n"
937     '            for cell in sheet[last_row][:3]:\n'
938     '                border = copy(cell.border)\n'
939     '                border.bottom = black\n'
940     '                cell.border = border\n'
941     '\n'
942     '        workbook.save(path)\n'
943     '    finally:\n'
944     '        workbook.close()\n'
945     '\n'
```

```
        '\n'
        'def export_one(case, problem_output_dir, attachment_dir, template_dir=None):\n'
        '    folder = Path(problem_output_dir).resolve()\n'
        '    attachments = Path(attachment_dir).resolve()\n'
        "    templates = Path(template_dir).resolve() if template_dir is not None else attachments / '附件5'\n"
        '    source = templates / FILES[case]\n'
        '    destination = folder / FILES[case]\n'
        '    data = load_results(folder, case)\n'
        "    temporary = destination.with_suffix('.pending.xlsx')\n"
        '\n'
        '    if source.is_file():\n'
        '        with ZipFile(source) as archive:\n'
        '            infos = {info.filename: copy.copy(info) for info in archive.infolist()}\n'
        '            parts = {name: archive.read(name) for name in infos}\n'
        '        paths = sheet_paths(parts)\n'
        "        required = {'计划购电量', '充放电量'} | ({'紧急购电量'} if case != 'problem1' else set())\n"
        "        if case in {'problem3', 'problem4-3'}:\n"
        "            required.add('调整购电量')\n"
        '        if required - paths.keys():\n'
        "            raise ValueError('附件5中的结果模板工作表不完整')\n"
        '        for name, member in paths.items():\n'
        "            xml = parts[member].decode('utf-8')\n"
        "            if name in {'计划购电量', '调整购电量'}:\n"
        "                xml = write_purchase(xml, data, final=name == '调整购电量', single=case == 'problem1')\n"
        "            elif name == '充放电量':\n"
        "                xml = write_storage(xml, data, single=case == 'problem1')\n"
        "            elif name == '紧急购电量':\n"
        '                xml, _ = write_emergency(xml, data)\n'
        "            parts[member] = xml.encode('utf-8')\n"
        "        with ZipFile(temporary, 'w', compression=ZIP_DEFLATED, compresslevel=6) as output:\n"
        '            for name, content in parts.items():\n'
        '                output.writestr(infos.get(name, name), content)\n'
        '    else:\n'
        '        parts, _ = _standalone_parts(data, case)\n'
        "        with ZipFile(temporary, 'w', compression=ZIP_DEFLATED, compresslevel=6) as output:\n"
        '            for name, content in parts.items():\n'
        '                output.writestr(name, content)\n'
        '\n'
        '    temporary.replace(destination)\n'
        '    beautify_excel(destination)\n'
        '    return destination\n'
    ),
}

def _load_module(name,source,data_dir):
```

```
    parts=name.split('.')
    for i in range(1,len(parts)):
        parent='.'.join(parts[:i])
        if parent not in sys.modules:
            package=types.ModuleType(parent); package.__path__=[]; sys.modules[parent]=package
    filename=str(_SCRIPT_DIR/'__embedded__'/Path(*parts).with_suffix('.py'))
    module=types.ModuleType(name)
    module.__file__=filename
    module.__package__=name.rpartition('.')[0]
    module.__spec__=importlib.util.spec_from_loader(name,loader=None,origin=filename)
    module.__dict__.update(_SCRIPT_DIR=_SCRIPT_DIR,_DATA_DIR=data_dir)
    linecache.cache[filename]=(len(source),None,source.splitlines(keepends=True),filename)
    sys.modules[name]=module
    if '.' in name: setattr(sys.modules[name.rpartition('.')[0]],parts[-1],module)
    exec(compile(source,filename,'exec'),module.__dict__)
    return module

def main():
    parser=argparse.ArgumentParser()
    parser.add_argument('--input-dir',type=Path)
    parser.add_argument('--output-dir',type=Path)
    parser.add_argument('--templates-dir',type=Path)
    parser.add_argument('--max-days',type=int)
    args=parser.parse_args()

    data_dir=(args.input_dir or (Path(ATTACHMENT_DIR) if ATTACHMENT_DIR.strip() else
    _SCRIPT_DIR.parent/'附件')).resolve()
    if not (data_dir/'附件1.xlsx').exists() and (data_dir/'附件'/'附件1.xlsx').exists():
        data_dir=data_dir/'附件'
    required=[1, 2, 3, 4]
    missing=[data_dir/f'附件{n}.xlsx' for n in required if not (data_dir/f'附件{n}.xlsx').
    is_file()]
    if missing: parser.error('缺少附件: \n'+'\n'.join(map(str,missing)))
    if args.max_days is not None and not 1<=args.max_days<=334:
        parser.error('--max-days 超出范围')

    out=(args.output_dir or (Path(OUTPUT_DIR) if OUTPUT_DIR.strip() else _SCRIPT_DIR.parent/'
    results'/_SCRIPT_PATH.stem)).resolve()
    out.mkdir(parents=True,exist_ok=True)
    template_dir=(args.templates_dir or (Path(TEMPLATE_DIR) if TEMPLATE_DIR.strip() else
    data_dir/'附件5')).resolve()

    try:
        import numpy as np
        import scipy
        import openpyxl
    except ImportError as e:
        parser.error('缺少依赖: '+str(e)+'; 请安装 numpy scipy openpyxl')

    modules={name:_load_module(name,source,data_dir) for name,source in _MODULE_SOURCES.items()
```

```
    }
    print(f'开始{CASE_NAME}；结果：{out}',flush=True)
    start=datetime(2025,2,1)
    end=datetime(2026,1,1) if args.max_days is None else start+timedelta(days=args.max_days)
    modules['runner'].run(start,end,out,CASE)
    with np.load(out/'all_results.npz') as z: days=len(z['dates'])
    if CASE=='problem1' or days==334:
        destination=modules['standalone_outputs'].export_one(CASE,out,data_dir,template_dir)
        print(f'Excel 已生成：{destination}',flush=True)
    else:
        print('快速检查仅计算数据；完整 Excel 需运行完整评价期。',flush=True)

if __name__=='__main__':
    main()
```

附录 C

表 7: 问题二 2025-03-20 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量/KWh	时间段	购电量/KWh	时间段	购电量/KWh
10:00–10:10	737.8750	12:00–12:10	613.9420	14:00–14:10	0.0000
16:00–16:10	453.6781	18:00–18:10	716.0660	20:00–20:10	10.4996
全天购电量/KWh		69862.5802	全天购电费/元		42489.3910

表 8: 问题二 2025-06-21 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量/KWh	时间段	购电量/KWh	时间段	购电量/KWh
10:00–10:10	0.0000	12:00–12:10	0.0000	14:00–14:10	0.0000
16:00–16:10	173.1037	18:00–18:10	0.0000	20:00–20:10	0.0000
全天购电量/KWh		38151.4245	全天购电费/元		21609.5370

表 9: 问题二 2025-09-23 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量/KWh	时间段	购电量/KWh	时间段	购电量/KWh
10:00–10:10	0.0000	12:00–12:10	512.9526	14:00–14:10	0.0000
16:00–16:10	492.8186	18:00–18:10	784.0005	20:00–20:10	78.8523
全天购电量/KWh		70312.1337	全天购电费/元		44456.7519

表 10: 问题二 2025-12-21 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量/KWh	时间段	购电量/KWh	时间段	购电量/KWh
10:00–10:10	0.0000	12:00–12:10	1026.1325	14:00–14:10	0.0000
16:00–16:10	829.3505	18:00–18:10	737.9892	20:00–20:10	37.2615
全天购电量/KWh		99539.6808	全天购电费/元		64413.0126

表 11: 问题二 2025-03-20 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量/KWh	放电量/KWh	时间段	充电量/KWh	放电量/KWh
0:00–4:00	152.4716	259.5536	4:00–8:00	340.2795	5764.1201
8:00–12:00	5484.4807	3350.5128	12:00–16:00	3606.2252	332.0810
16:00–20:00	256.6123	6178.7938	20:00–24:00	9326.5494	3281.5574
0:00 储电量/KWh		10800.0000	24:00 储电量/KWh		10800.0000

表 12: 问题二 2025-06-21 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量/KWh	放电量/KWh	时间段	充电量/KWh	放电量/KWh
0:00–4:00	90.4747	3405.3542	4:00–8:00	292.3618	5621.5800
8:00–12:00	8644.0977	0.0000	12:00–16:00	0.0000	0.0000
16:00–20:00	113.4571	6000.0754	20:00–24:00	8616.7593	2730.1410
0:00 储电量/KWh		10800.0000	24:00 储电量/KWh		10800.0000

表 13: 问题二 2025-09-23 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量/KWh	放电量/KWh	时间段	充电量/KWh	放电量/KWh
0:00–4:00	61.9433	27.8822	4:00–8:00	89.5317	6657.3247
8:00–12:00	4960.1329	2375.2380	12:00–16:00	4182.6115	463.7916
16:00–20:00	122.5287	6280.5900	20:00–24:00	9657.6989	3235.5594
0:00 储电量/KWh		10765.9389	24:00 储电量/KWh		10800.0000

表 14: 问题二 2025-12-21 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量/KWh	放电量/KWh	时间段	充电量/KWh	放电量/KWh
0:00–4:00	50.9670	174.2388	4:00–8:00	906.5409	1532.4333
8:00–12:00	5251.4015	7586.0297	12:00–16:00	5418.6869	2334.8946
16:00–20:00	41.0347	6238.5457	20:00–24:00	9289.7590	3092.2480
0:00 储电量/KWh		10800.0000	24:00 储电量/KWh		10800.0000

附录 D

表 15: 问题三 2025-03-20 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量/KWh	时间段	购电量/KWh	时间段	购电量/KWh
10:00–10:10	0.0000	12:00–12:10	529.2842	14:00–14:10	0.0000
16:00–16:10	477.1323	18:00–18:10	705.5368	20:00–20:10	0.0000
全天购电量/KWh		66730.0326	全天购电费/元		44906.6381

表 16: 问题三 2025-06-21 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量/KWh	时间段	购电量/KWh	时间段	购电量/KWh
10:00–10:10	0.0000	12:00–12:10	0.0000	14:00–14:10	0.0000
16:00–16:10	105.9637	18:00–18:10	0.0000	20:00–20:10	0.0000
全天购电量/KWh		34621.2686	全天购电费/元		19917.8863

表 17: 问题三 2025-09-23 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量/KWh	时间段	购电量/KWh	时间段	购电量/KWh
10:00–10:10	0.0000	12:00–12:10	368.1360	14:00–14:10	0.0000
16:00–16:10	507.9955	18:00–18:10	764.0430	20:00–20:10	45.0228
全天购电量/KWh		67156.3540	全天购电费/元		45055.7582

表 18: 问题三 2025-12-21 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量/KWh	时间段	购电量/KWh	时间段	购电量/KWh
10:00–10:10	0.0000	12:00–12:10	925.1087	14:00–14:10	0.0000
16:00–16:10	814.5896	18:00–18:10	713.0323	20:00–20:10	10.5139
全天购电量/KWh		94952.9654	全天购电费/元		62812.5313

表 19: 问题三 2025-03-20 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量/KWh	放电量/KWh	时间段	充电量/KWh	放电量/KWh
0:00–4:00	2134.3001	341.7674	4:00–8:00	877.2047	6810.2274
8:00–12:00	4237.9306	2586.2644	12:00–16:00	5026.7173	254.7422
16:00–20:00	103.1057	5823.4816	20:00–24:00	6840.5148	3325.1140
0:00 储电量/KWh		7926.7544	24:00 储电量/KWh		8004.9306

表 20: 问题三 2025-06-21 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量/KWh	放电量/KWh	时间段	充电量/KWh	放电量/KWh
0:00–4:00	220.5474	1046.6383	4:00–8:00	265.2034	5001.4446
8:00–12:00	9352.8118	0.0000	12:00–16:00	10.4862	0.0000
16:00–20:00	36.6183	6441.7398	20:00–24:00	6330.0364	2802.9712
0:00 储电量/KWh		6999.0341	24:00 储电量/KWh		7921.9436

表 21: 问题三 2025-09-23 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量/KWh	放电量/KWh	时间段	充电量/KWh	放电量/KWh
0:00–4:00	445.6464	147.4785	4:00–8:00	597.6937	7551.1006
8:00–12:00	4572.6637	2139.5830	12:00–16:00	5367.3316	400.9030
16:00–20:00	90.6341	6385.9113	20:00–24:00	8914.2342	3264.2984
0:00 储电量/KWh		9993.9855	24:00 储电量/KWh		10092.9143

表 22: 问题三 2025-12-21 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量/KWh	放电量/KWh	时间段	充电量/KWh	放电量/KWh
0:00–4:00	6085.7027	301.2086	4:00–8:00	2111.1339	1760.1820
8:00–12:00	4568.3255	8321.2145	12:00–16:00	7122.1108	2273.6896
16:00–20:00	44.0313	6620.3261	20:00–24:00	2464.5121	3085.8044
0:00 储电量/KWh		3569.0217	24:00 储电量/KWh		3602.4130